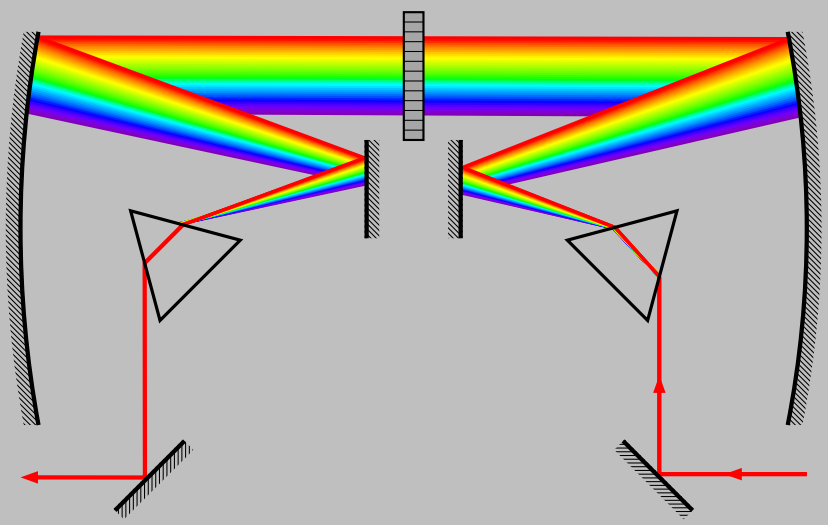
PSTricks

pst-optexp

Optische Versuchsaufbauten

v6.1

2022/02/26



Paketautor:
Christoph Bersch

Inhaltsverzeichnis

1. Einführung **6**
- 1.1. Über das Paket . . . 6
- 1.2. Anforderungen . . . 6
- 1.3. Verbreitung und Installation . . . 6
- 1.4. Lizenz . . . 7
- 1.5. Abwärtskompatibilität . . . 7
- 1.6. Notation . . . 7
- 1.7. Danksagung . . . 8

2. Grundlegende Ideen **9**
- 2.1. Die Komponenten . . . 9
- 2.2. Beschriftungen . . . 10
- 2.3. Freistrahl-Verbindungen . . . 10
- 2.4. Faserverbindungen . . . 12
- 2.5. Schritt-für-Schritt-Beispiele . . . 13
  - 2.5.1. Freistrahlaufbau . . . 13
  - 2.5.2. Galilei-Fernrohr . . . 13
  - 2.5.3. Faseraufbau . . . 15
  - 2.5.4. Faseraufbau (Alternative) . . . 17
  - 2.5.5. Michelson-Interferometer . . . 18

3. Allgemeine Komponentenparameter **19**
- 3.1. Beschriftungen . . . 19
- 3.2. Positionierung . . . 22
- 3.3. Drehen und Verschieben . . . 23
- 3.4. Verwendung von Stilen . . . 25
- 3.5. Aussehen der Komponenten . . . 25

4. Freistrahlkomponenten **27**
- 4.1. Linse . . . 27
- 4.2. Asphärische Linse . . . 29
- 4.3. Optisches Plättchen . . . 31

4.4. Verzögerungsplättchen . . . 32
4.5. Lochblende . . . 32
4.6. Box . . . 33
4.7. Pfeilkomponente . . . 33
4.8. Balkenkomponente . . . 34
4.9. Optische Quelle . . . 36
4.10. Kristall . . . 37
4.11. Optische Diode . . . 38
4.12. Doveprisma . . . 39
4.13. Glan-Thompson-Prisma . . . 39
4.14. Polarisation . . . 40
4.15. Keil . . . 41
4.16. Axikon . . . 42
4.17. Spiegel . . . 43
4.18. Parabolspiegel . . . 46
4.19. Off-axis-Parabolspiegel . . . 47
4.20. Strahlteiler . . . 48
4.21. Optisches Gitter . . . 49
4.22. Transmissionsgitter . . . 51
4.23. Akustooptischer Modulator . . . 51
4.24. Prisma . . . 55
4.25. Umkehrprisma . . . 56
4.26. Pentaprisma . . . 57

5. Faserkomponenten 58

5.1. Optische Faser . . . 58
5.2. Optischer Verstärker . . . 59
5.3. Mach-Zehnder-Modulator . . . 60
5.4. Polarisationssteller . . . 60
5.5. Isolator . . . 60
5.6. Optischer Schalter . . . 61
5.7. Faserverzögerungstrecke . . . 62
5.8. Polarisator . . . 62
5.9. Optischer Zirkulator . . . 62
5.10. Faserkoppler . . . 64
5.10.1. Variable Knotenanzahl . . . 66
5.10.2. Eingangs- und Ausgangsknoten . . . 67
5.10.3. Referenzknoten . . . 67
5.11. Faserbox . . . 68

6. Elektrische Komponenten 70
6.1. Koppler . . . 70
6.2. Synthesizer . . . 71
6.3. Mixer . . . 72

7. Hybridkomponenten 74
7.1. Optischer Filter . . . 74
7.2. Faserkollimator . . . 75
7.3. Detektor . . . 76

8. Spezielle Knoten 78
8.1. Komponenten-Bezeichner . . . 79
8.2. Referenzknoten . . . 80
8.3. Mittelpunktknoten . . . 81
8.4. Beschriftungsknoten . . . 81
8.5. Externe Knoten . . . 82
8.6. Grenzflächenknoten . . . 84
8.7. Referenzknoten für die Rotation . . . 85
8.8. Strahlknoten . . . 85
8.9. Strahlvektor . . . 87
8.10. Knotenübersicht . . . 89

9. Verbinden von Komponenten 91
9.1. Zugriff auf Komponenten . . . 91
9.2. Strahlen zeichnen . . . 93
9.2.1. Raytracing . . . 93
9.2.2. Brechungsindex . . . 95
9.2.3. Anfangsbedingungen . . . 98
9.2.4. Interner Strahlengang . . . 101
9.2.5. Verbinden mit Knoten . . . 104
9.2.6. Automatische Verbindungen . . . 106
9.2.7. Strahlaussehen . . . 106
9.3. Aufgeweitete Strahlen . . . 108
9.3.1. Strahlaussehen . . . 109
9.4. Fehlerbehandlung . . . 110
9.5. Angepasste Strahlen . . . 112
9.6. Faserverbindungen . . . 115
9.6.1. Faserwinkel . . . 116
9.6.2. Faseraussehen . . . 120
9.7. Elektrische Verbindungen . . . 121

9.8. Automatische Faser- und Drahtverbindungen 123
9.8.1. Aussehen der automatischen Verbindungen 126
9.8.2. Bekannte Einschränkungen 127
9.9. Zeichenebenen 127

10. Benutzerdefinierte Komponenten 129
10.1. Benutzerdefinierte Version existierender Komponenten 131
10.2. Neue Objekte definieren 132
10.2.1. Die Zeichnung der Komponente 133
10.2.2. Unterstützung von Raytracing 134
10.2.3. Unterstützung von externen Knoten 135
10.2.4. Beispiele für benutzerdefinierte Komponenten 136

11. Beispiele 138

12. Zusatzinformationen 165
12.1. Interne Struktur der Komponenten 165
12.2. Übersicht der Spezialknoten 167
12.2.1. Externe und Rotationsreferenzknoten 167
12.2.2. Grenzflächenknoten 169
12.3. Abwärtskompatibilität 171
12.3.1. Version 4.3a 171
12.3.2. Version 4.3 172
12.3.3. Version 4.2 172
12.3.4. Version 4.0 172
12.3.5. Version 3.3 173
12.3.6. Version 3.0 174

Dokumentationsindex 177

A. Versionsgeschichte 200

1. Einführung

1.1. Über das Paket

pst-optexp ist ein PSTricks-Paket zum Skizzieren optischer Versuchsaufbauten. Dafür werden viele unterschiedliche Freistrahl- und Faserkomponenten bereitgestellt, deren Ausrichtung, Positionierung und Beschriftung einfach und flexibel eingestellt werden kann. Die Komponenten können dann mit Fasern oder Lichtstrahlen verbunden werden, wobei auch realistische Strahlengänge mit Raytracing möglich sind.

1.2. Anforderungen

pst-optexp benötigt LaTeX und aktuelle Versionen der Pakete pstricks (≥ 2.53), pst-plot ($\geq 1.6.1$), pst-node, pstricks-add, multido, pst-eucl, pst-intersect (≥ 0.4) und environ.

Alle PSTricks-Pakete machen regen Gebrauch von der Postscript-Sprache, so dass der typische Arbeitsfluss latex, dvips und ggf. ps2pdf umfasst. Es gibt viele alternative Methoden um die Dokumente zu kompilieren.[1]

1.3. Verbreitung und Installation

Dieses Paket ist auf CTAN[2] erhältlich und in TeXLive and MiKTeX enthalten.

Das pst-optexp-Paket umfasst die zwei Hauptdateien pst-optexp.ins und pst-optexp.dtx. Durch Aufrufen von tex pst-optexp.ins werden die beiden folgenden Dateien erzeugt:

- pst-optexp.pro: die Postscript Prologdatei

[1] http://tug.org/PSTricks/main.cgi?file=pdf/pdfoutput
[2] http://mirror.ctan.org/help/Catalogue/entries/pst-optexp.html

- `pst-optexp.sty`: die LaTeX Stildatei

Speichern Sie diese Dateien in einem Verzeichnis der Teil Ihres lokalen TeX-Baums ist.

Vergessen Sie nicht `texhash` aufzurufen um den Baum zu aktualisieren. MiKTeX-Benutzer müssen die Dateinamen-Datenbank (FNDB) aktualisieren.

Detailliertere Information finden Sie in der Dokumentation Ihrer LaTeX-Distribution über die Installation in den lokalen TeX-Baum.

1.4. Lizenz

Es wird die Erlaubnis gewährt, dieses Dokument zu kopieren, zu verteilen und/oder zu modifizieren, unter den Bestimmungen der LaTeX Project Public License, Version 1.3c.[3]. Dieses Paket wird vom Autor betreut (author-maintained).

1.5. Abwärtskompatibilität

In Version 3.0 wurde dem Paket erheblich fortgeschrittenere Funktionalität hinzugefügt, die es zu schwierig machte zur Vorgängerversion 2.1 vollständig abwärtskompatibel zu bleiben. Insbesondere war die Funktionsweise des `\drawbeam` Makros grundlegend falsch konzipiert, so dass ein Wechsel zur neuen Funktionalität so schnell wie möglich erfolgen sollte.

Eine genauere Zusammenstellung der nicht-kompatiblen Änderungen und Hinweise zur Migration von älteren Dokumenten finden Sie in Kap. 12.3.

1.6. Notation

Die meisten Parametertypen, die im Laufe des Dokuments verwendet werden, sind selbsterklärend, andere bedürfen einer genaueren Erläuterung (z.B. der Unterschied zwischen ⟨*num*⟩ und ⟨*psnum*⟩). Tab. 1.1 erklärt die geläufigsten Typen.

[3] http://www.latex-project.org/lppl.txt

Name	Beschreibung
num	Gleitkommazahl
psnum	Postscript-Kode der zu einer Zahl auswertet
int	Ganzzahl
dimen	Länge
psstyle	Benutzerdefinierte Parameterkonfiguration definiert mit `\newpsstyle`
refpoint	Analog zum Referenzpunkt von `\rput`, kann jede Kombination von `c` (mittig), `t` (oben), `b` (unten), `l` (links) und `r` (rechts) sein.

Tabelle 1.1.: Parametertypen die in den Parameterdefinitionen verwendet werden.

1.7. Danksagung

Ich danke allen Aktiven auf der PSTricks-Mailingliste für ihre Hilfe, insbesondere Herbert Voß. Mein Dank gilt ebenfalls unterschiedlichen Paketautoren, von denen ich Code kopiert und viel gelernt habe: David Carlisle, Florent Chervet, Enrico Gregorio, Rolf Niepraschk und Heiko Oberdiek. Christine Römer hat mich mit ihrem Artikel in der DTK[4] überzeugt, ebenfalls eine deutsche Dokumentation bereitzustellen. Der Dokumentationsstil ist eine Mischung aus der `pst-doc` Klasse (Herbert Voß) und dem `ltxdockit` Paket für die `biblatex` Dokumentation (Philipp Lehmann).

[4] Pakete in Deutsch dokumentieren. in Die TeXnische Komödie. Heft 2/2011, S. 28-35.

2. Grundlegende Ideen

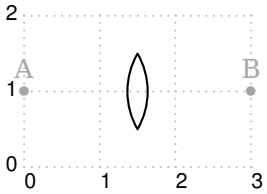
Dieses Kapitel zeigt die grundlegenden Ideen und Konzepte, die in diesem Paket stecken. Anhand von elementaren Beispielen werden die Grundfunktionen wie Ausrichtung, Positionierung (2.1) und Beschriftung (2.2) von Komponenten beschrieben. Anschließen wird deren Verbindung mit Strahlen (2.3) oder Fasern (2.4) vorgestellt.

In Kap. 2.5 wird Schritt für Schritt die angedachte Vorgehensweise für das Erstellen von umfassenden Experimentskizzen erläutert. Eine vollständige Referenz aller Makros und deren Parameter finden Sie in Kap. 3–10.

Ein Versuchsaufbau besteht aus Komponenten, die anhand ihrer Referenzknoten positioniert, verschoben und gedreht werden können und optional eine Beschriftung erhalten. Anschließen werden die Komponenten mit Lichtstrahlen oder Fasern verbunden.

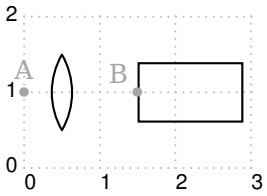
2.1. Die Komponenten

Eine Komponente wird anhand der Knoten, die bei ihrere Definition angegeben werden – den Referenzknoten – platziert. Im folgenden Beispiel wird eine Linse mittig auf die Verbindungslinie zwischen den Knoten $\langle A \rangle$ und $\langle B \rangle$ gesetzt:




```
\begin{pspicture}[showgrid](0,-0.3)(3,2.3)
  \pnodes(0,1){A}(3,1){B}
  \lens(A)(B)
\end{pspicture}
```

Diese Positionierung kann anhand von unterschiedlichen Parametern, wie z.B. `abspos` beinflusst werden (siehe dazu Kap. 3.2 und Kap. 3.3). Im folgenden Beispiel wird die Linse `0.5` Einheiten vom Knoten $\langle A \rangle$ entfernt platziert, die Box sitzt am Ende der Referenzlinie $\overline{AB}$.

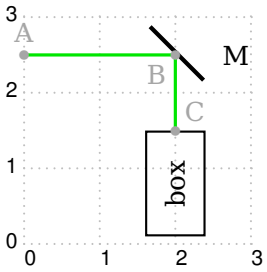



```
\begin{pspicture}[showgrid](0,-0.3)(3,2.3)
  \pnode(0,1){A}\pnode(1.5,1){B}
  \lens[abspos=0.5](A)(B)
  \optbox[position=end](A)(B)
\end{pspicture}
```

Die Komponenten lassen sich allgemein in zwei Kategorien unterteilen: Faser- und Freistrahlkomponenten. Diese unterscheiden sich in erster Linie bezüglich ihrer Verbindungsmöglichkeiten: Freistrahlkomponenten können sowohl mit Lichtstrahlen als auch mit Fasern verbunden werden, wohingegen Faserkomponenten nur Faserverbindungen erlauben.

2.2. Beschriftungen

Jede Komponente kann optional eine Beschriftung erhalten, die relativ zur Komponente platziert wird.



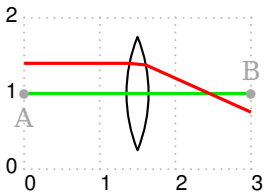

```
\begin{pspicture}[showgrid](0,-0.3)(3,3.3)
  \pnode(0,2.5){A}\pnode(2,2.5){B}\pnode(2,1.5){C}
  \mirror[labelangle=-45](A)(B)(C){M}
  \optbox[position=start, labeloffset=0, labelref=relative](C)(B){box}
  \drawbeam(A){1}{2}
\end{pspicture}
```

2.3. Freistrahl-Verbindungen

Freistrahlkomponenten können mit Lichtstrahlen verbunden werden, deren Weg mit Raytracing berechnet werden kann. Da es sich um ein Paket zum *Skizzieren* experimenteller Aufbauten handelt, werden hier trotz der realistischen Berechnungen viele Möglichkeiten offen gehalten den Strahlengang zu manipulieren.

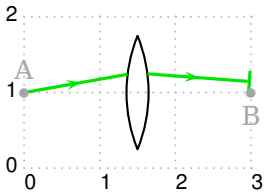
Im einfachsten Fall wird eine einzelne Linie gezeichnet, deren optischer Weg über den Brechungsindex n und das Snelliussche Brechungsgesetz bestimmt wird.

Das Beispiel zeigt zwei Strahlen, die mit unterschiedlichen Anfangsbedingungen auf die Linse treffen, der rote Strahl startet mit einem Abstand von `0.2` von der optischen Achse (`beampos`) und wird entsprechend von der Linse abgelenkt.



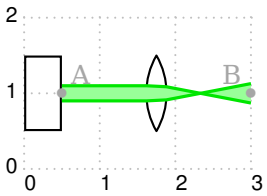

```
\begin{pspicture}[showgrid](0,-0.3)(3,2.3)
  \pnode(0,1){A}\pnode(3,1){B}
  \lens[n=2, lensradius=2 2, lensheight=1.5](A)(B)
  \drawbeam(A){1}(B)
  \drawbeam[beampos=0.4, linecolor=red](A){1}(B)
\end{pspicture}
```

Die Strahlen können mit beliebigen Strichlierungen und Pfeilen dekoriert werden.



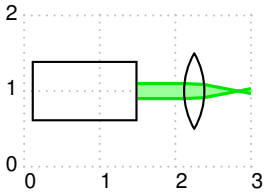

```
\begin{pspicture}[showgrid](0,-0.3)(3,2.3)
  \pnode(0,1){A}\pnode(3,1){B}
  \lens[n=2, lens=2 2 1.5](A)(B)
  \drawbeam[beamangle=10, ArrowInside=->, arrows=-|,
      beaminside=false](A){1}(B)
\end{pspicture}
```

Ebenfalls können ausgedehnte Strahlen gezeichnet werden deren Randstrahlen mit `linestyle` etc. und deren Füllung über `fillstyle` geändert werden können.




```
\begin{pspicture}[showgrid](0,-0.3)(3,2.3)
  \pnode(0.5,1){A}\pnode(3,1){B}
  \optbox[position=start, optboxsize=0.5 1](A)(B)
  \lens[n=2](A)(B)
  \drawwidebeam[beamwidth=0.2, fillstyle=solid,
      fillcolor=green!40]{1-2}(B)
\end{pspicture}
```

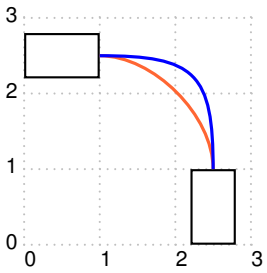
Die Strahlen können auch hinter die Komponenten gelegt werden. Dafür müssen alle beteiligten Komponenten und Strahlen in eine `optexp`-Umgebung gepackt werden (vergleiche das folgende Beispiel mit dem vorangegangenen).




```
\begin{pspicture}[showgrid](0,-0.3)(3,2.3)
  \pnode(1.5,1){A}\pnode(3,1){B}
  \begin{optexp}
    \optbox[position=start](A)(B)
    \lens[n=2](A)(B)
    \drawwidebeam[beamwidth=0.2, fillstyle=solid,
        fillcolor=green!40]{1}{2}(B)
  \end{optexp}
\end{pspicture}
```

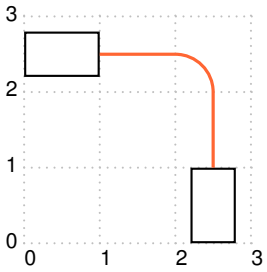
2.4. Faserverbindungen

Eine andere Möglichkeit, Komponenten zu verbinden, sind Fasern (\drawfiber). Hiermit bestehen weitreichende Möglichkeiten die Fasern automatisch an die Komponentenausrichtung anzupassen.




```
\begin{pspicture}[showgrid](0,-0.3)(3,3.3)
  \pnode(1,2.5){A}\pnode(2.5,1){B}
  \psset{optboxsize=1 0.6}
  \optbox[position=start](A)(B|A)
  \optbox[position=end](B|A)(B)
  \drawfiber{1}{2}
  \drawfiber[linecolor=blue, ncurv=1.2]{1}{2}
\end{pspicture}
```

Üblicherweise werden Fasern mit einer \nccurve-Verbindung gezeichnet, jede andere Knotenverbindung ist aber ebenfalls möglich (fiberstyle).




```
\begin{pspicture}[showgrid](0,-0.3)(3,3.3)
  \pnode(1,2.5){A}\pnode(2.5,1){B}
  \psset{optboxsize=1 0.6}
  \optbox[position=start](A)(B|A)
  \optbox[position=end](B|A)(B)
  \drawfiber[fiberstyle=angle, linearc=0.5]{1}{2}
\end{pspicture}
```

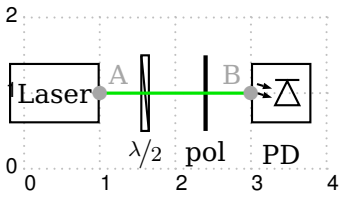
2.5. Schritt-für-Schritt-Beispiele

2.5.1. Freistrahlaufbau

In diesem ersten Beispiel werden alle Komponenten auf eine Verbindungslinie verteilt und mit einem Strahl verbunden.

Zuerst werden Anfangsknoten $\langle A \rangle$ und Endknoten $\langle B \rangle$ definiert. Die erste `\optbox` wird an den Anfang platziert, die Wellenplatte und der Polarisator werden innerhalb der Verbindungslinie positioniert. Der Parameter `position` erwartet Werte zwischen `0` und `1`, die relativ zur Länge der Verbindungslinie sind, oder `start`/`end`. Der Detektor wird immer an das Ende der Linie gesetzt.

Zuletzt werden alle Komponenten mit einem einfachen Lichtstrahl verbunden.



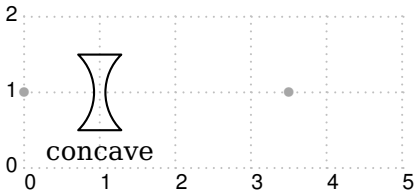

```
\begin{pspicture}[showgrid](4,2)
  \pnode(1,1){A}\pnode(3,1){B}
  \optbox[position=start, optboxwidth=1.2, ↵
     labeloffset=0](A)(B){Laser}
  \optretplate[position=0.3](A)(B){$\nicefrac{\lambda}{2}$}
  \optplate[position=0.7](A)(B){pol}
  \optdetector[dettype=diode](A)(B){PD}
  \drawbeam{1}{2}{3}{4}
\end{pspicture}
```

2.5.2. Galilei-Fernrohr

Im Folgenden wird ein sehr einfacher Aufbau mit einer konkaven und einer konvexen Linse schrittweise erläutert.

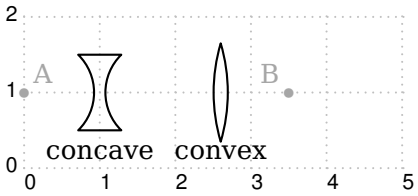
Zuerst werden die Referenzknoten $\langle A \rangle$ und $\langle B \rangle$ definiert, die für die Ausrichtung der Komponenten verwendet werden. Zur besseren Übersicht werden beide Knoten gekennzeichnet, desweiteren dient ein Koordinatengitter der Orientierung.

Die erste Linse soll konkav gekrümmte Grenzflächen haben. Mit `lensradius` werden die Radien der linken und rechten Fläche auf negative Werte gesetzt, was eine konkave Krümmung ergibt. Die Linse wird mit `abspos` eine Einheit vom $\langle A \rangle$-Knoten entfernt positioniert.



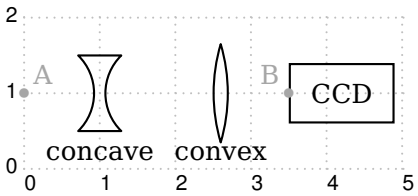

```
\begin{pspicture}[showgrid](0,-0.3)(5,2.3)
  \pnode(0,1){A}\pnode(3.5,1){B}
  \lens[lensradius=-0.7,
        abspos=1](A)(B){concave}
\end{pspicture}
```

Die zweite Linse bekommt eine konvexe Krümmung (`lensradius` ist positiv) und wird `2.6` Einheiten von ⟨*A*⟩ positioniert. Die Linsenhöhe wird mit `lensheight` eingestellt.



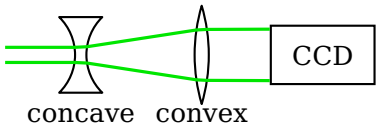

```
\begin{pspicture}[showgrid](0,-0.3)(5,2.3)
  \pnode(0,1){A}\pnode(3.5,1){B}
  \lens[lensradius=-0.7,
        abspos=1](A)(B){concave}
  \lens[lensradius=2.3, lensheight=1.3,
        abspos=2.6](A)(B){convex}
\end{pspicture}
```

Die Lichtstrahlen sollen auf einer CCD-Kamera enden, die mit einer `\optbox` gezeichnet wird. Mit `position=end` wird die Box ans Ende der Referenzlinie von ⟨*A*⟩ nach ⟨*B*⟩ gesetzt, die Beschriftung wird mit `labeloffset` in die Mitte der Komponente gelegt.




```
\begin{pspicture}[showgrid](0,-0.3)(5,2.3)
  \pnode(0,1){A}\pnode(3.5,1){B}
  \lens[lensradius=-0.7,
        abspos=1](A)(B){concave}
  \lens[lensradius=2.3, lensheight=1.3,
        abspos=2.6](A)(B){convex}
  \optbox[position=end, labeloffset=0](A)(B){CCD}
\end{pspicture}
```

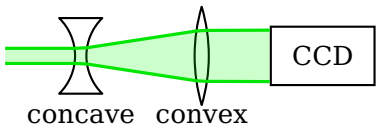
Zum Schluß wird ein aufgeweiteter Strahl mit einer Strahlbreite von `0.2` und ohne anfängliche Divergenz durch die Komponenten geschickt. Der Strahlengang wird mittels Raytracing berechnet. Die Linsenparameter in diesem Beispiel wurden so gewählt das bei kollimiertem Eingangsstrahl der Ausgangsstrahl ebenfalls kollimiert ist.




```
\begin{pspicture}(5,2)
  \pnode(0,1){A}\pnode(3.5,1){B}
  \lens[lensradius=-0.7,
        abspos=1](A)(B){concave}
  \lens[lensradius=2.3, lensheight=1.3,
        abspos=2.6](A)(B){convex}
  \optbox[position=end, labeloffset=0](A)(B){CCD}
  \drawwidebeam[beamwidth=0.2](A){1}{2}{3}
\end{pspicture}
```

Das Aussehen des Lichtstrahls wird über den Beam-Stil eingestellt, und kann z.B. mit einer halbtransparenten Füllung versehen werden.

Bsp. 2.1



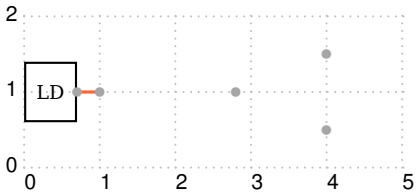

```
\begin{pspicture}(5,2)
  \pnode(0,1){A}\pnode(3.5,1){B}
  \lens[lensradius=-0.7,
        abspos=1](A)(B){concave}
  \lens[lensradius=2.3, lensheight=1.3,
        abspos=2.6](A)(B){convex}
  \optbox[position=end, labeloffset=0](A)(B){CCD}
  \addtopsstyle{Beam}{fillstyle=solid,
    fillcolor=green, opacity=0.2}
  \drawwidebeam[beamwidth=0.2](A){1}{2}{3}
\end{pspicture}
```

2.5.3. Faseraufbau

Dieses Beispiel zeigt, wie ein faseroptischer Aufbau mit automatisch gezeichneten Faserverbindungen konstruiert werden kann.

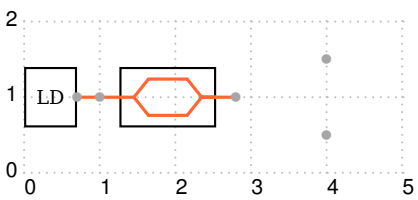
Auch hier werden die Referenzknoten zur besseren Orientierung eingezeichnet. Im Gegensatz zum vorhergehenden Beispiel benötigen wir mehr Referenzknoten, da die Faserkomponenten direkt mit diesen verbunden werden.

Der Laser wird wieder an den Anfang gesetzt. Der Abstand der Beschriftung wird mit dem Parameter `label` eingestellt, mit dem alle Beschriftungsparameter gesammelt eingestellt werden können. Da eine `\optbox` sowohl als Freistrahl- als auch als Faserkomponente verwendet werden kann, wird in der Standardeinstellung keine Faser gezeichnet, was mit `fiber` geändert werden kann. Die Breite dieser und aller folgenden `\optbox` wird auf `0.7` geändert.



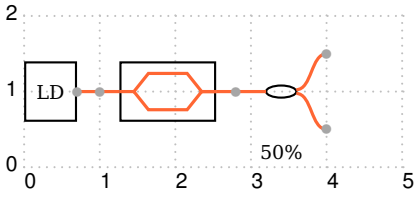

```
\begin{pspicture}[showgrid](4.7,2)
  \pnodes(0.7,1){A}(1,1){B}(2.8,1){C}(4,1.5){D}(4,0.5){E}
  \psset{optboxwidth=0.7, labelstyle=\scriptsize}
  \optbox[position=start, label=0, fiber](A)(B){LD}
\end{pspicture}
```

Als nächstes wird ein Mach-Zehnder-Modulator ohne Beschriftung gezeichnet, der automatisch mit Fasern mit seinen Referenzknoten verbunden wird.




```
\begin{pspicture}[showgrid](4.7,2)
  \pnodes(0.7,1){A}(1,1){B}(2.8,1){C}(4,1.5){D}(4,0.5){E}
  \psset{optboxwidth=0.7, labelstyle=\scriptsize}
  \optbox[position=start, label=0, fiber](A)(B){LD}
  \optmzm(B)(C)
\end{pspicture}
```

Danach wird ein Faserkoppler mit einem Eingang und zwei Ausgängen gezeichnet. Der Koppler wird zwischen dem Eingangsknoten und dem Mittelpunkt der beiden Ausgangsknoten ausgerichtet.

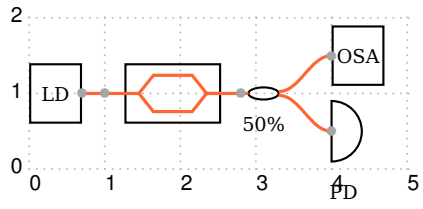



```
\begin{pspicture}[showgrid](4.7,2)
  \pnodes(0.7,1){A}(1,1){B}(2.8,1){C}(4,1.5){D}(4,0.5){E}
  \psset{optboxwidth=0.7, labelstyle=\scriptsize}
  \optbox[position=start, label=0, fiber](A)(B){LD}
  \optmzm(B)(C)
  \wdmsplitter(C)(D)(E){50\%}
\end{pspicture}
```

An die Enden des Kopplers wird ein optischer Spektrumanalysator (OSA) und eine Leistungsmessgerät (PD) gesetzt. Der `\optdetector` wird nach Voreinstellung ans Ende der Referenzlinie gesetzt (`position=end`). Diese beiden Komponenten werden ohne weitere Einstellungen ohne Faserverbindungen gezeichnet. Das passt, da ihre jeweiligen Endpunkte mit den Ausgängen des Kopplers zusammenfallen.

Die Schreibweise `(A|E)` nimmt die x-Koordinate von $\langle A \rangle$ und die y-Koordinate von $\langle E \rangle$.

Bsp. 2.2




```
\begin{pspicture}[showgrid](4.7,2)
  \pnodes(0.7,1){A}(1,1){B}(2.8,1){C}(4,1.5){D}(4,0.5){E}
  \psset{optboxwidth=0.7, labelstyle=\scriptsize}
  \optbox[position=start, label=0, fiber](A)(B){LD}
  \optmzm(B)(C)
  \wdmsplitter[abspos=0.3, label=0.4](C)(D)(E){50\%}
  \optbox[position=end, label=0](A|D)(D){OSA}
  \optdetector(A|E)(E){PD}
\end{pspicture}
```

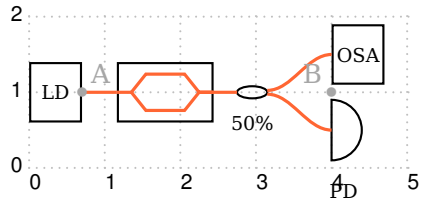
2.5.4. Faseraufbau (Alternative)

Dieses Beispiel ist vom Ergebnis her identisch mit dem vorangegangenen (Bsp. 2.5.3), wird aber anders angegangen.

Hier verwenden wir nicht die automatischen Faserverbindungen sondern zeichnen, wie bei Freistrahlaufbauten, zuerst die Komponenten und verbinden diese zum Schluß explizit mit Fasern (`\drawfiber`). Daher benötigen wir weniger Referenzknoten, die Faserverbindungen werden mit `fiber=none` global unterdrückt.

Welche Herangehensweise verwendet wird ist meistens Geschmackssache, bei Ringstrukturen wie z.B. Faserlasern kann das nachträgliche Verbinden mit `\drawfiber` von Vorteil sein.

Bsp. 2.3

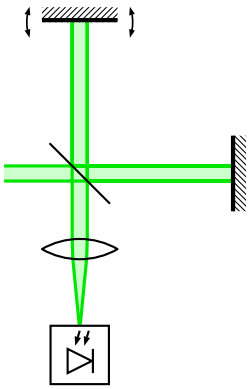



```
\begin{pspicture}[showgrid](4.9,2)
  \pnode(0.7,1){A}\pnode(4,1){B}
  \psset{optboxwidth=0.7, labelstyle=\scriptsize,
      fiber=none}
  \optbox[position=start, labeloffset=0](A)(B){LD}
  \optmzm[abspos=1.1](A)(B)
  \wdmsplitter[abspos=2.25, label=0.4](A)(B)(B){50\%}
  \optbox[position=end,label=0, compshift=0.5](A)(B){OSA}
  \optdetector[compshift=-0.5](A)(B){PD}
  \drawfiber{1-4}
  \drawfiber{3}{5}
\end{pspicture}
```

2.5.5. Michelson-Interferometer

In diesem Beispiel werden neben Linsen auch reflektierende Elemente (Spiegel) eingesetzt sowie Elemente, die sowohl transmittieren als auch reflektieren (Strahlteiler).

Bsp. 2.4



```
1  \begin{pspicture}(3.2,5)
2    \pnodes(0,3){A}(1,3){BS}(3,3){M1}(1,5){M2}(1,1){PD}
3    \psset{mirrortype=extended, mirrordepth=0.2}
4    \begin{optexp}
5      \beamsplitter[bsstyle=plate, compname=BS](A)(BS)(PD)
6      \mirror[compname=M1](BS)(M1)(BS)
7      \mirror[compname=M2, variable](BS)(M2)(BS)
8      \lens[compname=L](BS)(PD)
9      \optdetector[compname=Det, dettype=diode](BS)(PD)
10     \addtopsstyle{Beam}{beamwidth=0.2, fillstyle=solid,
           fillcolor=green!20!white}
11     \drawwidebeam(A){BS}{M1}{BS}{M2}{BS}{L}{Det}
12   \end{optexp}
13 \end{pspicture}
```

Wir beginnen wieder, indem wir die Referenzknoten definieren (Zeile 2) und ein paar grundlegende Einstellungen setzen (Zeile 3).

Dann werden der Reihe nach alle Komponenten platziert (Zeile 5–9). Die reflektierenden benötigen drei Referenzknoten: den Eingangsknoten, den Knoten an dem die reflektierende Grenzfläche ist und den Ausgangsknoten. Anschließend wird noch das Aussehen des Lichtstrahls festgelegt (Zeile 10) und der Strahl gezeichnet (Zeile 11).

Diese Mal haben wir den Komponenten Namen gegeben (`compname`) die beim Raytracing anstelle der IDs – der automatisch vergebenen Nummern – verwendet werden können. Dies kann die Nachverfolgung des Strahlengangs erleichtern.

Alle Komponenten und Strahlen wurden zusätzlich in eine `optexp`-Umgebung gekapselt. Innerhalb dieser werden alle Lichtstrahlen hinter die Komponenten gelegt, sodass diese nicht verdeckt werden.

3. Allgemeine Komponentenparameter

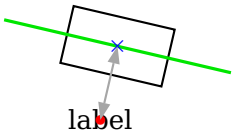
Dieses Kapitel enthält alle Informationen zu allgemeinen Parametern und Konzepten. Das beinhaltet das Setzen von Beschriftungen (3.1), Positionierung (3.2), Drehen und Verschieben von Komponenten (3.3) und Anpassen der Aussehens (3.4 und 3.5).

3.1. Beschriftungen

Alle Komponenten können mit einer Beschriftung versehen werden, deren Position und Ausrichtung genau angepasst werden kann. Die Beschriftung ist für jede Komponente optional. Sollten die vielfältigen Parameter nicht ausreichend sein, können Sie den Beschriftungsknoten – in einigen Beispielen mit einem roten Punkt gekennzeichnet – auch direkt verwenden (siehe auch Kap. 8.4).

`labeloffset`=⟨*num*⟩ default: `0.8`

Der Abstand des Referenzknotens für die Beschriftung (roter Punkt) zur Komponentenmitte (blaues Kreuz).



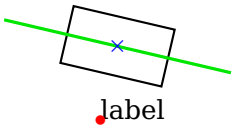

```
\begin{pspicture}(3,1.8)
  \optbox[beam, labeloffset=1](0,1.5)(3,0.8){label}
\end{pspicture}
```

`labelstyle`=⟨*macros*⟩

Der Schriftstil der Beschriftungen.

`labelalign`=⟨*refpoint*⟩ default: `c`

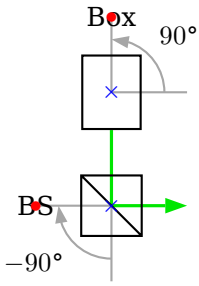
Definiert die Ausrichtung der Beschriftung bezüglich des Beschriftungsknotens.




```
\begin{pspicture}(3,1.8)
  \psset{beam, labeloffset=1}
  \optbox[labelalign=bl](0,1.5)(3,0.8){label}
\end{pspicture}
```

`labelangle`=⟨*num*⟩ default: `0`

Der Rotationswinkel des Beschriftungsknotens um die Mitte. Der Nullpunkt hängt sowohl von der jeweiligen Komponenten und dem gewählten Referenzsystem ab (siehe `labelref`).

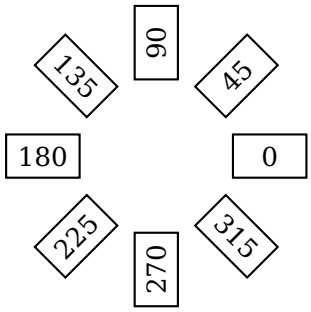



```
\begin{pspicture}(-0.3,0)(2,3.7)
  \psset{labeloffset=1, optboxwidth=1, arrowscale=1.5, arrowinset=0}
  \optbox[position=end, labelangle=90](1,1)(1,2){Box}
  \beamsplitter[labelangle=-90](1,2)(1,1)(2,1){BS}
  \drawbeam[arrows=->]{1}{2}(2,1)
\end{pspicture}
```

`labelref`=`relative`, `relgrav`, `global`, `absolute` default: `relgrav`

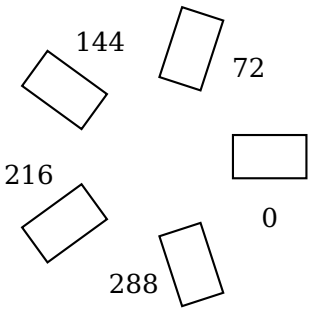
Wählt das Bezugssystem für `labelangle` und die Orientierung der Beschriftung.

`relative` Die Beschriftung ist parallel zu der Referenzlinie der Komponente und der Winkel wird immer auf den Bereich zwischen $-90°$ und $+90°$ zurückgerechnet.



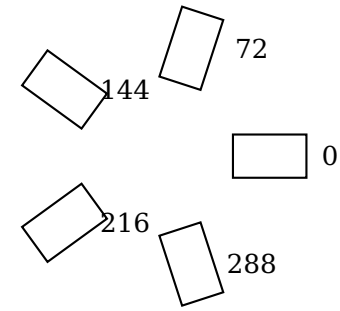

```
\begin{pspicture}(0,-2.2)(2,2.2)
  \psset{position=end, optboxsize=1 0.6, labeloffset=0,
      labelref=relative}
  \multido{\i=0+45}{8}{%
    \optbox(0,0)(1;\i){\i}
  }
\end{pspicture}
```

`relgrav` Die Beschriftung ist immer horizontal, lediglich der Beschriftungsknoten wird zusammen mit der Komponente gedreht.




```
\begin{pspicture}(0,-2.2)(2,2.2)
  \psset{position=end, optboxsize=1 0.6, labelref=relgrav}
  \multido{\i=0+72}{5}{%
    \optbox(0,0)(1;\i){\i}
  }
\end{pspicture}
```

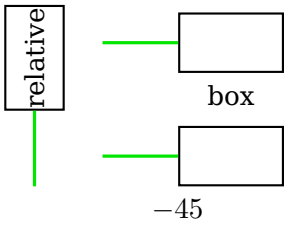
`global, absolute` Die Beschriftung ist unabhängig von der Komponentenausrichtung.




```
\begin{pspicture}(0,-2.2)(2,2.2)
  \psset{position=end, optboxsize=1 0.6, labelref=global}
  \multido{\i=0+72}{5}{%
    \optbox(0,0)(1;\i){\i}
  }
\end{pspicture}
```

`label`=⟨*offset*⟩[⟨*angle*⟩[⟨*refpoint*⟩[⟨*labelref*⟩]]]

Diese Option erlaubt die kompakte Definition mehrerer Beschriftungsparameter. Es können bis zu vier Leerzeichen-getrennte Argumente übergeben werden (`labeloffset`, `labelangle`, `labelalign` und `labelref`). Mit einem Punkt kann eine Option übersprungen werden.




```
\begin{pspicture}(0,0)(4,3)
  \psset{endbox, beam}
  \optbox[label=0.7](1.5,2.5)(2.5,2.5){box}
  \optbox[label=1 -45](1.5,1)(2.5,1){$-45$}
  \optbox[label=0 . . relative]%
        (0.6,0.6)(0.6,1.6){relative}
\end{pspicture}
```

`innerlabel=true`

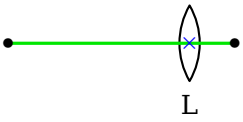
Das ist ein Alias für `label=0 . c relative`

3.2. Positionierung

Alle Komponenten außer den Freistrahl-Dreipolen können zwischen ihren beiden Referenzknoten \oenodeRefA und \oenodeRefB positioniert werden.

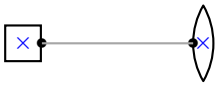
position=⟨*num*⟩, start, end

Ist der Wert eine Zahl, so wird damit die relative Position der Mitte eines Objektes (blaues Kreuz) bezüglich seiner beiden Referenzknoten (schwarze Punkte) gesetzt. Das ist äquivalent zum npos Parameter von \ncput, d.h. eine Zahl im Intervall $[0, 1]$ (siehe pst-node Dokumentation[1]).



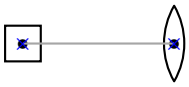

```
\begin{pspicture}(3,1.5)
  \lens[beam, position=0.8](0,1)(3,1){L}
\end{pspicture}
```

Der Wert start setzt ein Objekt an den Anfang, end ans Ende der Referenzlinie, so dass die entsprechende Grenzfläche auf dem Referenzknoten (schwarzer Punkt) liegt, und nicht die Mitte des Objektes (blaues Kreuz), wie es mit position=1 bzw. position=0 der Fall wäre. Unter Verwendung von Zahlen wäre dieses Verhalten deutlich schwieriger und weniger flexibel zu erreichen, da die Größe und Form des Objektes berücksichtigt werden muss.



```
\begin{pspicture}(3,2)
  \pnodes(0.5,1){A}(2.5,1){B}\psdot(A)\psdot(B)
  \optbox[optboxsize=0.5 0.5, position=start](A)(B)
  \lens[position=end](A)(B)
\end{pspicture}
```

Hier ist im Vergleich dazu die Positionierung mit position=0/1.

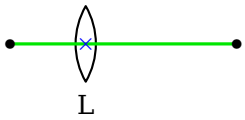


```
\begin{pspicture}(3,2)
  \pnodes(0.5,1){A}(2.5,1){B}\psdot(A)\psdot(B)
  \optbox[optboxsize=0.5 0.5, position=0](A)(B)
  \lens[position=1](A)(B)
\end{pspicture}
```

abspos=⟨*num*⟩, start, end

Eine Zahl setzt die absolute Position eines Objektes auf der Referenzlinie. Die Werte start und end werden identisch wie bei position behandelt.

[1] http://mirror.ctan.org/help/Catalogue/entries/pst-node.html




```
\begin{pspicture}(3,1.5)
  \lens[beam, abspos=1](0,1)(3,1){L}
\end{pspicture}
```

endbox=true, false

Dieser Parameter sollte nicht mehr verwendet werden, er ist äquivalent zu position=end.

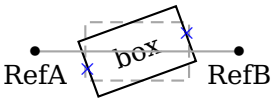
3.3. Drehen und Verschieben

Alle Komponenten können versetzt und um verschiedene Bezugspunkte gedreht werden.

Beachten Sie, dass mit den Komponenten auch die Grenzflächenknoten (Kap. 8.6, in den folgenden Beispielen mit blauen Kreuzen markiert) transformiert werden, wodurch Verbindungen beinflußt werden. Die transformierten Referenzknoten sind als eigene Knoten verfügbar (Kap. 8.2).

angle=⟨*psnum*⟩ default: 0

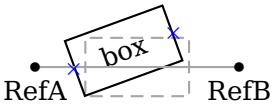
Dreht eine Komponente um den Winkel ⟨*num*⟩ (in Grad). Die Ausgangsposition ist grau gestrichelt eingezeichnet.




```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(2.7,1){B}
  \optbox[angle=20, innerlabel](A)(B){box}
\end{pspicture}
```

rotateref=⟨*refpoint*⟩ default: c

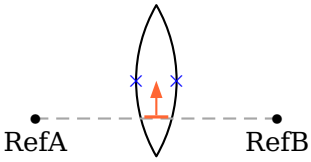
Setzt den Bezugspunkt für die Drehung der Komponente. Bitte lesen Sie Kap. 8.7 für eine genauere Beschreibung und entnehmen Sie Kap. 12.2.1 die möglichen Bezugspunkte für alle Komponenten. Die Ausgangsposition ist grau gestrichelt eingezeichnet.



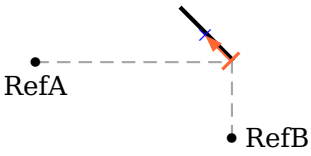

```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(2.7,1){B}
  \optbox[angle=20, rotateref=bl, innerlabel](A)(B){box}
\end{pspicture}
```

`compshift`=⟨*num*⟩ default: `0`

Verschiebt eine Komponente vertikal bezüglich ihrer Referenzlinie (siehe Kap. 8.2). Reflektive Komponenten werden entlang der reflektiven Grenzfläche verschoben.



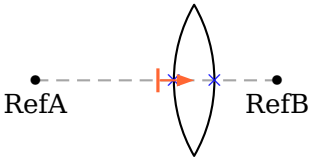

```
\begin{pspicture}(4,2)
  \pnodes(0.4, 0.5){A}(3.6,0.5){B}
  \lens[lens=2 2 2, compshift=0.5](A)(B)
\end{pspicture}
```



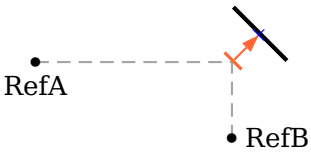

```
\begin{pspicture}(4,2)
  \pnodes(0.4,1.2){A}(3,0.2){B}
  \mirror[compshift=0.5](A)(B|A)(B)
\end{pspicture}
```

`compoffset`=⟨*num*⟩ default: `0`

Verschiebt eine Komponente vertikal bezüglich ihrer Referenzlinie (siehe Kap. 8.2). Reflektierende Komponenten werden entlang der reflektiven Grenzfläche verschoben. Die Verschiebung erfolgt also senkrecht zu einer Verschiebung mit `compshift`. Ein Anwendungsbeispiel ist Bsp. 11.



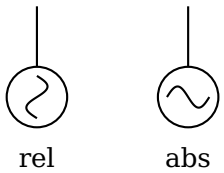

```
\begin{pspicture}(4,2)
  \pnodes(0.4, 1){A}(3.6, 1){B}
  \lens[lens=2 2 2, compoffset=0.5](A)(B)
\end{pspicture}
```




```
\begin{pspicture}(4,2)
  \pnodes(0.4,1.2){A}(3,0.2){B}
  \mirror[compoffset=0.5](A)(B|A)(B)
\end{pspicture}
```

`innercompalign`=`rel`, `relative`, `abs`, `absolute`

Ist dieser Parameter auf `absolute` gesetzt, so wird das „Innere“ einer Komponente nicht mitgedreht. Diese Option wird derzeit von `\optfilter`, `\elecsynthesizer` und `\optdipole` verwendet.



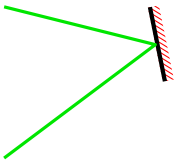

```
\begin{pspicture}(3,2)
\psset[optexp]{labelangle=-90}
\elecsynthesizer[innercompalign=rel](0.5,1.2)(0.5,2){rel}
\elecsynthesizer[innercompalign=abs](2.5,1.2)(2.5,2){abs}
\end{pspicture}
```

3.4. Verwendung von Stilen

Stile sind benutzerdefinierte Grafik-Konfigurationen für PSTricks-Objekte. Das `pst-optexp`-Paket macht umfangreichen Gebrauch von Stilen um einzelne, logische Bereiche von Komponenten flexibel gestalten zu können, was über das optionale Argument meistens nicht möglich ist.

Vorhandene Stile können mit `\addtopsstyle` erweitert und mit `\newpsstyle` überschrieben werden.

Als Beispiel nehmen wir einen `extended` Spiegel und möchten nur den „extended"-Teil verändern. Hierfür wird der Stil `ExtendedMirror` bereitgestellt:



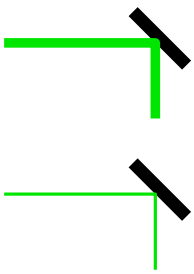
```
\begin{pspicture}(3,2)
  \addtopsstyle{ExtendedMirror}{hatchcolor=red}
  \mirror[mirrortype=extended, beam](0,0)(2,1.5)(0,2)
\end{pspicture}
```

Die veränderbaren Unterbereiche einer Komponente sind in der jeweiligen Referenz zu finden.

3.5. Aussehen der Komponenten

`OptComp` ⟨*psstyle*⟩

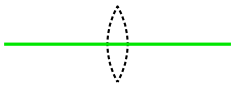
Bestimmt das grundlegende Aussehen aller optischen Komponenten. Die üblichen Parameter, wie z.B. `linestyle`, würden auch die Verbindungen beeinflussen, die mit der Komponente zusammen gezeichnet werden (`beam`, `fiber` usw.). Mit `newOptComp` und `addtoOptComp` kann der Stil für einzelne Komponenten oder zusammen mit `\newpsobject` im optionalen Argument verwendet werden.



```
\begin{pspicture}(2.5,4)
  \psset{beam}
  % falsch, die Strahlbreite wird auch geaendert
  \mirror[linewidth=3\pslinewidth](0,3)(2,3)(2,2)
  % korrektes Ergebnis
  \addtopsstyle{OptComp}{linewidth=3\pslinewidth}
  \mirror(0,1)(2,1)(2,0)
\end{pspicture}
```

`OptionalStyle` ⟨*psstyle*⟩ default: `linestyle=dashed,dash=1.5pt 1pt`

Der Stil wird zusätzlich zu `OptComp` auf Komponenten angewendet, die als `optional` markiert sind.



```
\begin{pspicture}(3,1.5)
\lens[optional, beam](0,1)(3,1)
\end{pspicture}
```

`OptionalStyle` wird nach `OptComp` angewendet, so dass einzelne Einstellungen überschrieben werden können.

```
\begin{pspicture}(3,1.5)
\addtopsstyle{OptComp}{linewidth=3\pslinewidth, linecolor=red}
\newpsstyle{OptionalStyle}{linecolor=blue}
\lens(0,1)(2,1)
% Linienfarbe wird ueberschrieben, die Linienbreite nicht.
\lens[optional](0,1)(3,1)
\end{pspicture}
```

`VariableStyle` ⟨*psstyle*⟩ default: `linewidth=0.8\pslinewidth, arrowinset=0, arrowscale=0.8, arrows=<->`

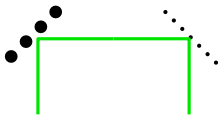
Der Stil der Pfeile für verstellbare Komponenten (`\mirror` und `\optgrating`).

`addtoOptComp`=⟨*list*⟩

Der vorhandene `OptComp`-Stil wird lokal um die Parameter in ⟨*list*⟩ erweitert, ⟨*list*⟩ muss mit geschweiften Klammern gekapselt werden.

`newOptComp`=⟨*list*⟩

Ähnlich wie `addtoOptComp`, nur wird der `OptComp`-Stil mit den neuen Parametern überschrieben.



```
\begin{pspicture}(3,1.5)
  \psset{beam}
  \newpsstyle{OptComp}{linewidth=3\pslinewidth}
  \mirror[addtoOptComp={linestyle=dotted}](0.5,0)(0.5,1)(1.5,1)
  % Ueberschreibt linewidth-Einstellungen
  \mirror[newOptComp={linestyle=dotted}](1.5,1)(2.5,1)(2.5,0)
\end{pspicture}
```

`optional`=`true`, `false` default: `false`

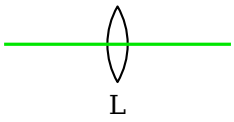
Markiert eine Komponente als optional indem der Stil `OptionalStyle` angewendet wird.

4. Freistrahlkomponenten

In diesem Kapitel werden alle Freistrahlkomponenten und deren Parameter beschrieben. Freistrahlkomponenten können, im Gegensatz zu Faserkomponenten, transmittive und reflektive Grenzflächen haben, die für Raytracing von Lichtstrahlen verwendet werden können. Es gibt zwei unterschiedliche Komponententypen: manche benötigen nur zwei Knoten um positioniert zu werden (4.1–4.14), wohingegen z.B. Spiegel drei Knoten benötigen um korrekt ausgerichtet zu werden (4.17–4.26).

4.1. Linse

`\lens[`⟨*options*⟩`](`⟨*in*⟩`)(`⟨*out*⟩`){`⟨*label*⟩`}`

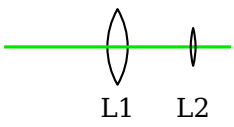



```
\begin{pspicture}(3,1.5)
  \lens[beam](0,1)(3,1){L}
\end{pspicture}
```

Eine Linse wird durch ihre Höhe und die beiden Radien der Eingangs- und Ausgangsflächen definiert.

`lensheight=`⟨*num*⟩ default: 1

Setzt die Höhe der Linse.



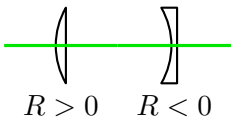

```
\begin{pspicture}(3,1.5)
  \lens[beam](0,1)(3,1){L1}
  \lens[lensheight=0.5](2,1)(3,1){L2}
\end{pspicture}
```

`lensradiusleft=`⟨*num*⟩ default: 1

Setzt den Radius der linken Linsengrenzfläche. Ein positiver Wert ⟨*num*⟩ ist für eine konvexe, ein negativer für eine konkave Krümmung. Null ergibt eine ebene Fläche.

lensradiusright=⟨*num*⟩ default: 1

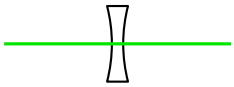
Das Gleiche wie lensradiusleft, nur für die rechte Grenzfläche.




```
\begin{pspicture}(3,1.5)
  \psset{lensradiusright=0, beam}
  \lens[lensradiusleft=1](0,1)(1.5,1){$R > 0$}
  \lens[lensradiusleft=-1](1.5,1)(3,1){$R < 0$}
\end{pspicture}
```

lensradius=⟨*left*⟩[⟨*right*⟩]

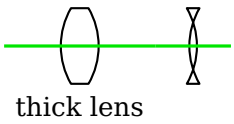
Setzt die linke und rechte Linsenkrümmung. Wird nur ein Wert angegeben, so wird dieser für beide Krümmungen verwendet.



```
\begin{pspicture}(3,1.5)
  \lens[lensradius=-2, beam](0,1)(3,1)
\end{pspicture}
```

lenswidth=⟨*num*⟩ default: 0

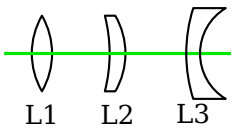
Üblicherweise werden nur die Höhe und die beiden Radien für die Konstruktion der Linse verwendet, die Mittenbreite wird automatisch berechnet. Wenn lenswidth auf einen Wert größer Null gesetzt wird, wird diese Breite verwendet. Das ist nur sinnvoll, wenn dicke Linsen gezeichnet werden sollen, da es andernfalls für zu kleine Werte zu unschönen Ergebnissen führt.




```
\begin{pspicture}(3,2)
  \lens[lenswidth=0.5, beam](0,1)(2,1){thick lens}
  \lens[lenswidth=0.1, beam](2,1)(3,1)
\end{pspicture}
```

lens=⟨*radiusleft*⟩[⟨*radiusright*⟩[⟨*height*⟩[⟨*width*⟩]]]

Eine Option zum gleichzeitigen Setzen mehrerer Linsenparameter. Werte mittendrin können mit einem Punkt übersprungen und Werte am Ende können weggelassen werden (z.B. `lens=. 1 1`).




```
\begin{pspicture}(3,1.5)
  \lens[lens=1](0,1)(1,1){L1}
  \lens[lensradius=-2 1, beam](0,1)(3,1){L2}
  \lens[lens=2 -0.7 1.2](2,1)(3,1){L3}
\end{pspicture}
```

thicklens=true, false default: false

Diese Option was bis Version 2.1 notwendig um dicke Linsen der Breite lenswidth zu zeichnen, anstatt die berechnete Breite zu verwenden. Seit Version 3.0 ist diese Option überflüssig, da dicke Linsen gezeichnet werden, sobald lenswidth auf einen positiven Wert gesetzt wird.

4.2. Asphärische Linse

\asphericlens[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}

Eine asphärische Linse besitzt eine Oberfläche, deren Form durch die Gleichung

$$z(y) = \frac{y^2}{R(1+\sqrt{1-(1+k)y^2/R^2})} + A_4 y^4 + A_6 y^6 + A_8 y^8 + A_{10} y^{10}$$

gegeben ist. z ist der Sagittalschnitt parallel zur optischen Achse, y der Abstand zur optischen Achse und R der Krümmungsradius. Desweiteren ist k die konische Konstante und A_4, A_6, A_8 und A_{10} die Asphärenkoeffizienten der 4. bis 10. Ordnung.

asphereheight=⟨*num*⟩ default: 1

Die Höhe der Linse.

aspherewidth=⟨*num*⟩ default: 0

Die Breite der Linse. Wird nur verwendet, wenn der angegebene Wert über dem minimal berechneten liegt.

asphereradiusleft=⟨*num*⟩ default: 1

Der Krümmungsradius der linken, asphärischen Oberfläche.

asphereradiusright=⟨*num*⟩ default: 0

Der Krümmungsradius der rechten Oberfläche. Ist dieser gleich Null, dann ist die Oberfläche plan.

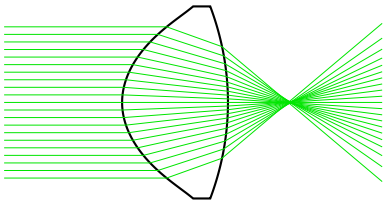
asphereconstant=⟨*num*⟩

Die konische Konstante k der linken Oberfläche.

`aspherecoefficients`=⟨A_4⟩ [⟨A_6⟩ [⟨A_8⟩ [⟨A_{10}⟩]]]

Die Asphärenkoeffizienten A_4 bis A_{10}. Es können ein bis vier Zahlen angegeben werden.

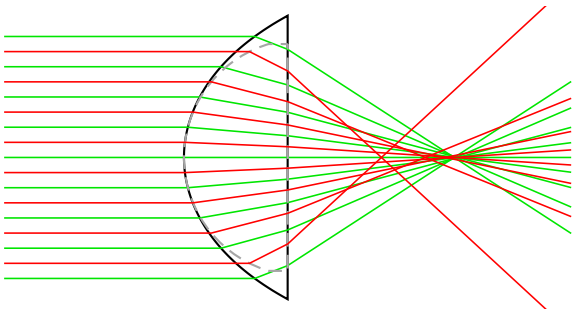
Als Beispiel nehmen wir die Parameter einer kommerziell erhältlichen asphärische Kondensorlinse[1]:



```
\begin{pspicture}(0,-0.3)(5,2.3)
  \pnodes(0,1){A}(5,1){B}
  \asphericlens[%
    plotstyle=cspline,
    aspherеheight=2.54,
    asphereWidth=1.4,
    asphereradiusleft=0.8818197,
    aspherecoefficients=8.682e-3 6.376e-3 2.4073e-2
        -1.719e-2,
    asphereconstant=-0.9992,
    asphereradiusright=6.99995](A)(B)
  \multido{\r=-1+0.1}{21}{%
    \drawbeam[beampos=\r,
        linewidth=0.3\pslinewidth](A){}(B)}
\end{pspicture}
```

Ein weiteres Beispiel zeigt anschaulich den Unterschied einer asphärischen Linsen zu einer sphärischen Linse mit gleichem Krümmungsradius:

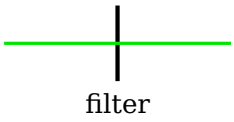
[1] http://www.thorlabs.de/thorproduct.cfm?partnumber=ACL25416U



```
\begin{pspicture*}(15,8)
  \pnodes(0,4){A}(15,4){B}
  \asphericlens[aspherehеight=7.5, asphereradiusleft=3.12848,
    aspherеconstant=-1.991145, aspherecoefficients=4.98e-3](A)(B)
  \multido{\r=-3.2+0.8}{9}{%
    \drawbeam[beampos=\r, linewidth=0.5\pslinewidth](A){}(B)}
  \lens[lens=3.12848 0 6 5.49, abspos=6.13, style=Refline, linestyle=dashed](A)(B)
  \multido{\r=-2.8+0.8}{8}{%
    \drawbeam[beampos=\r, linewidth=0.5\pslinewidth, linecolor=red](A){}(B)}%
\end{pspicture*}
```

4.3. Optisches Plättchen

`\optplate`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.5)
  \optplate[beam](0,1)(3,1){filter}
\end{pspicture}
```

`plateheight`=⟨*num*⟩ default: 1

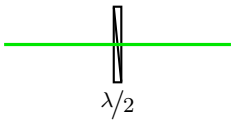
Die Höhe des Plättchens.

`platelinewidth`=⟨*num*⟩ or ⟨*dimen*⟩ default: `2\pslinewidth`

Die Linienbreite des Plättchens. Diese könnte ebenfalls über den `linewidth` Parameter gesetzt werden. Mit diesem Parameter kann aber die Linienbreite aller Plättchen global eingestellt werden.

4.4. Verzögerungsplättchen

`\optretplate`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.5)
  \optretplate[beam](0,1)(3,1){$\nicefrac{\lambda}{2}$}
\end{pspicture}
```

`plateheight`=⟨*num*⟩ default: `1`

Die Höhe des Plättchens.

`platewidth`=⟨*num*⟩ default: `0.1`

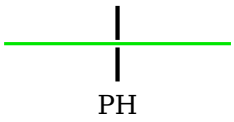
Die Breite des Plättchens.

`platesize`=⟨*width*⟩ ⟨*height*⟩

Die Breite und Höhe des Plättchens, äquivalent zum Aufruf von `platewidth` und `plateheight`.

4.5. Lochblende

`\pinhole`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.5)
  \pinhole[beam](0,1)(3,1){PH}
\end{pspicture}
```

`outerheight`=⟨*num*⟩ default: `1`

Die Gesamthöhe der Lochblende.

`innerheight`=⟨*num*⟩ default: `0.1`

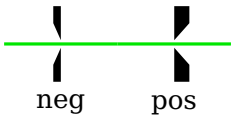
Die Höhe des Lochs.

`phlinewidth`=⟨*num*⟩ or ⟨*dimen*⟩ default: `2\pslinewidth`

Die Linienbreite der Lochblende. Diese könnte ebenfalls über den `linewidth` Parameter gesetzt werden. Mit diesem Parameter kann aber die Linienbreite aller Lochblenden global eingestellt werden.

phwidth=⟨*num*⟩ default: 0

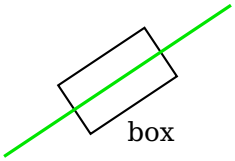
Die Lochblende wird plastischer gezeichnet, falls phwidth ungleich Null ist. In diesem Fall wird phlinewidth ignoriert. Bei einem negativen Wert wird die Form gespiegelt.




```
\begin{pspicture}(3,1.5)
  \pinhole[beam, phwidth=-0.1](0,1)(1.5,1){neg}
  \pinhole[beam, phwidth=0.2](1.5,1)(3,1){pos}
\end{pspicture}
```

4.6. Box

\optbox[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \optbox[beam](0,0)(3,2){box}
\end{pspicture}
```

optboxwidth=⟨*num*⟩ default: 1.4

Die Breite der Box.

optboxheight=⟨*num*⟩ default: 0.8

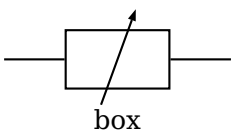
Die Höhe der Box.

optboxsize=⟨*width*⟩ ⟨*height*⟩

Die Breite und Höhe der Box, äquivalent zum Aufruf von optboxwidth und optboxheight.

4.7. Pfeilkomponente

\optarrowcomp[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.7)
  \optarrowcomp[wire](0,1)(3,1){box}
\end{pspicture}
```

Eine Komponente für den allgemeinen, flexiblen Gebrauch ähnlich zu \fiberdelayline, aber deutlich variabler.

arrowcompwidth=⟨num⟩ default: 1.4

Die Breite der Box.

arrowcompheight=⟨num⟩ default: 0.8

Die Höhe der Komponente.

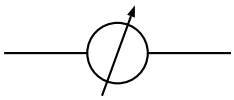
arrowcompsize=⟨size⟩ or ⟨width⟩ ⟨height⟩

Die Breite und Höhe der Komponente, äquivalent zum Aufruf von arrowcompwidth und arrowcompheight.

arrowcompangle=⟨num⟩ default: 70

Der Winkel des Pfeils bezüglich der x-Achse.

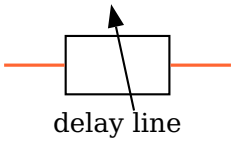
arrowcompshape=rectangle, circle default: rectangle



```
\begin{pspicture}(3,1)
  \optarrowcomp[arrowcompshape=circle, wire](0,0.5)(3,0.5)
\end{pspicture}
```

ArrowCompStyle ⟨psstyle⟩ default: arrowinset=0, arrows=->

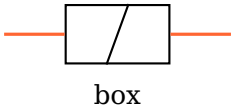
Der Stil für den Pfeil. Das kann insbesonders nützlich sein um die Länge und Ausrichtung des Pfeils zu verändern.




```
\begin{pspicture}(3,1.5)
  \addtopsstyle{ArrowCompStyle}{xunit=-1, arrowscale=2}
  \optarrowcomp[fiber](0,1)(3,1){delay line}
\end{pspicture}
```

4.8. Balkenkomponente

\optbarcomp[⟨options⟩](⟨in⟩)(⟨out⟩){⟨label⟩}




```
\begin{pspicture}(3,1.5)
  \optbarcomp[fiber](0,1)(3,1){box}
\end{pspicture}
```

Eine Komponente für den allgemeinen, flexiblen Gebrauch ähnlich zu `\optfiberpolarizer`, aber deutlich variabler.

`barcompwidth`=⟨*num*⟩ default: `1.4`

Die Breite der Box.

`barcompheight`=⟨*num*⟩ default: `0.8`

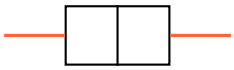
Die Höhe der Box.

`barcompsize`=⟨*size*⟩ or ⟨*width*⟩ ⟨*height*⟩

Die Breite und Höhe der Box, äquivalent zum Aufruf von `barcompwidth` und `barcompheight`.

`barcompangle`=⟨*num*⟩ default: `70`

Der Winkel des Balkens bezüglich der x-Achse.



```
\begin{pspicture}(3,1)
  \optbarcomp[barcompangle=90, fiber](0,0.5)(3,0.5)
\end{pspicture}
```

`barcompshape`=`rectangle`, `circle` default: `rectangle`

```
\begin{pspicture}(3,1)
  \optbarcomp[barcompshape=circle, fiber](0,0.5)(3,0.5)
\end{pspicture}
```

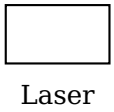
`BarCompStyle` ⟨*psstyle*⟩

Der Stil für den Balken.

```
\begin{pspicture}(3,1)
  \newpsstyle{BarCompStyle}{unit=0.7, arrows=->}
  \optbarcomp[barcompshape=circle, barcompangle=45, ↩
      fiber](0,0.5)(3,0.5)
\end{pspicture}
```

4.9. Optische Quelle

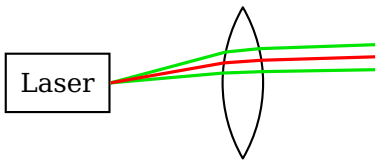
\optsource[⟨options⟩](⟨in⟩)(⟨out⟩){⟨label⟩}




```
\begin{pspicture}(3,2)
  \optsource(1.5,1)(3,1){Laser}
\end{pspicture}
```

Eine optische Quelle gibt ein paar Eigenschaften für alle von ihr ausgehenden Strahlen vor, falls es die erste Komponente in einem Strahlengang ist. Die verwendbaren Eigenschaften sind beamangle, beamwidth, beamdiv und beamalign. Alle Eigenschaften, die von einer optischen Quelle definiert werden, können nicht später durch Strahlparameter überschrieben werden. Wird ein Parameter nicht explizit von der Quelle definiert, so werden die Parameter von dem Strahl-Stil verwendet.

Die Voreinstellung für die Positionierung ist position=start.




```
\begin{pspicture}(5,2)
  \pnodes(1.5,1){A}(5,1){B}
  \optsource[innerlabel, beamdiv=10,
     beamangle=10](A)(B){Laser}
  \lens[lens=2 2 2](A)(B)
  \optplane(B)
  \drawwidebeam{-}
  \drawbeam[linecolor=red]{-}
\end{pspicture}
```

sourcewidth=⟨num⟩ default: 1.4

Die Breite der Box.

sourceheight=⟨num⟩ default: 0.8

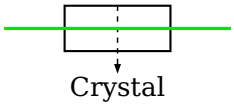
Die Höhe der Box.

sourcesize=⟨width⟩ ⟨height⟩

Die Breite und Höhe der Box, äquivalent zum Aufruf von sourcewidth und sourceheight.

4.10. Kristall

`\crystal`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.3)
  \crystal[beam](0,1)(3,1){Crystal}
\end{pspicture}
```

`crystalwidth`=⟨*num*⟩ default: `1.4`

Die Breite des Kristalls.

`crystalheight`=⟨*num*⟩ default: `0.6`

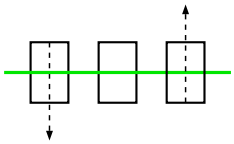
Die Höhe des Kristalls.

`crystalsize`=⟨*width*⟩ ⟨*height*⟩

Die Breite und Höhe des Kristalls, äquivalent zum Aufruf von `crystalwidth` und `crystalheight`.

`caxislength`=⟨*num*⟩ default: `0.3`

Die zusätzliche Länge des Pfeils für die c-Achse, wird weggelassen falls die Länge `0` ist und gespiegelt falls der Wert negativ ist. Die Gesamtlänge ist `caxislength` plus `crystalheight`.



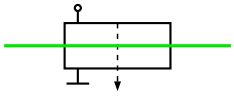
```
\begin{pspicture}(0,0.4)(3,1.5)
  \psset{crystalsize=0.5 0.8, label=-45 . l}
  \crystal[position=0.2, caxislength=0.5](0,1)(3,1){pos}
  \crystal[beam, caxislength=0](0,1)(3,1){0}
  \crystal[position=0.8, caxislength=-0.5](0,1)(3,1){neg}
\end{pspicture}
```

`caxisinv`=`true`, `false` default: `false`

Invertiert die Richtung der c-Achse, ist äquivalent zu einem Vorzeichenwechsel von `caxislength`.

`voltage`=`true`, `false` default: `false`

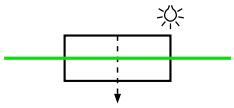
Zeichnet einen Spannungsanschluss und ein Erdungszeichen.



```
\begin{pspicture}(0,0.4)(3,1.5)
  \crystal[voltage, beam](0,1)(3,1)
\end{pspicture}
```

lamp=true, false default: false

Zeichnet eine Lampe neben den Kristall.



```
\begin{pspicture}(0,0.4)(3,1.7)
  \crystal[lamp, beam](0,1)(3,1)
\end{pspicture}
```

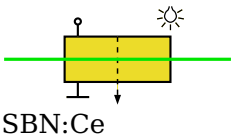
CrystalCaxis ⟨*psstyle*⟩ default: linewidth=0.7\pslinewidth, linestyle=dashed, dash=2pt 2pt, arrowinset=0, arrows=->

Der Stil für die c-Achse inklusive der Pfeilart.

```
\begin{pspicture}(0,0.5)(3,1.6)
  \newpsstyle{CrystalCaxis}{linecolor=red, arrows=->>}
  \crystal[beam](0,1)(3,1)
\end{pspicture}
```

CrystalLamp ⟨*psstyle*⟩ default: linewidth=0.6\pslinewidth

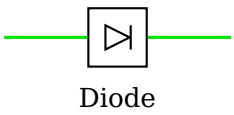
Der Stil für die Hintergrundlampe.




```
\begin{pspicture}(3,1.7)
  \crystal[fillstyle=solid, fillcolor=yellow!90!black,
           label=1.2 -45, voltage,
           lamp, beam](0,1)(3,1){SBN:Ce}
\end{pspicture}
```

4.11. Optische Diode

\optdiode[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \optdiode[beam](0,1)(3,1){Diode}
\end{pspicture}
```

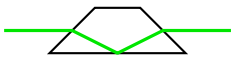
Die optische Diode hat die Voreinstellung allowbeaminside=false.

optdiodesize=⟨*num*⟩ default: 0.8

Die Seitenlänge der optischen Diode.

4.12. Doveprisma

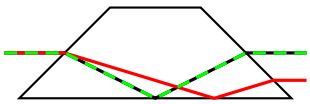
\doveprism[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.5)
  \doveprism[beam](0,1)(3,1){Dove}
\end{pspicture}
```

doveprismsize=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: 0.6

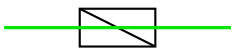
Wenn eine Zahl angegeben ist, dann ist das die Höhe des Prismas, die Gesamtbreite beträgt das dreifache. Zwei Zahlen geben Breite und Höhe des Doveprisma an. Die Winkel an der Eingangs- und Ausgangsfläche betragen immer 45°.



```
\begin{pspicture}(4,1.5)
  \optplane(0,1)
  \doveprism[doveprismsize=1.2](0,1)(4,1)
  \optplane(4,1)
  \drawbeam[raytrace=false, linecolor=black]{-}
  \drawbeam[n=1.5, linecolor=red]{-}
  \drawbeam[n=*sqrt(5), linecolor=green, linestyle=dashed]{-}
\end{pspicture}
```

4.13. Glan-Thompson-Prisma

\glanthompson[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.5)
  \glanthompson[beam](0,1)(3,1){Glan-Thompson}
\end{pspicture}
```

glanthompsonwidth=⟨*num*⟩ default: 1.0

Die Breite des Prisma.

glanthompsonheight=⟨*num*⟩ default: 0.5

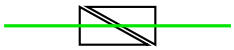
Die Höhe des Prisma.

glanthompsonsize=⟨*width*⟩ ⟨*height*⟩

Die Breite und Höhe des Prisma, äquivalent zum Aufruf von glanthompsonwidth und glanthompsonheight.

glanthompsongap=⟨*num*⟩ default: 0

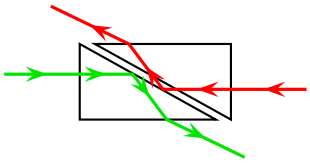
Der Abstand der beiden Teilprismen in x-Richtung, die Linienbreiten werden nicht berücksichtigt. Die Gesamtbreite ist von diesem Wert unabhängig, aber der Winkel der reflektierenden Fläche wird davon beeinflusst.



```
\begin{pspicture}(3,1)
  \glanthompson[glanthompsongap=0.1, beam](0,0.5)(3,0.5)
\end{pspicture}
```

Von der Implementierung entspricht das Glan-Thompson-Prisma für glanthompsongap=0 einem Strahlteiler, der nur zwei Knoten zur Positionierung benötigt und nicht quadratisch ist.

Beachten Sie, dass das Prisma für glanthompsongap>0 nicht von beiden Seiten gleich verwendet werden kann, da in der derzeitigen Implementierung nur eine uneindeutige Grenzfläche (reflektierend oder transmittierend) zugelassen ist. In dem folgenden Beispiel sieht man, dass der rote Strahl an der falsche Fläche reflektiert wird.



```
\begin{pspicture}(4,2)
  \pnodes(0,1){A}(4,1){B}
  \glanthompson[glanthompsongap=0.2, glanthompsonsize=2 1](A)(B)
  \optplane[angle=90](2,0)\optplane[angle=90](2,2)
  \addtopsstyle{Beam}{ArrowInside=->, arrowscale=1.5}
  \drawbeam[beampos=0.1](A){1-2}
  \drawbeam[beampos=0.1, linecolor=red](B){1}{3}
\end{pspicture}
```

Bsp. 11 enthält ein weiterführendes Beispiel mit einem Glan-Thompson-Prisma.

4.14. Polarisation

\polarization[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}

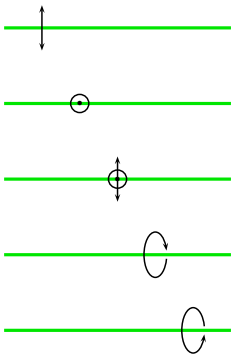
```
\begin{pspicture}(3,1)
  \polarization[beam](0,0.5)(3,0.5)
\end{pspicture}
```

`polsize`=⟨*num*⟩ default: `0.6`

Die Größe des Polarisationszeichens, der Kreis für `perp` und `polmisc` ist halb so groß.

`poltype`=`parallel`, `perp`, `misc`, `lcirc`, `rcirc` default: `parallel`

Dieser Parameter wählt die Polarizationsart aus.



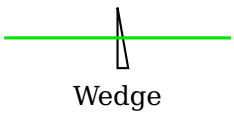
```
\begin{pspicture}(0,-0.3)(3,4.3)
  \psset[optexp]{beam}
  \begin{optexp}
  \polarization[poltype=parallel, abspos=0.5](0,4)(3,4)
  \polarization[poltype=perp, abspos=1](0,3)(3,3)
  \polarization[poltype=misc, abspos=1.5](0,2)(3,2)
  \polarization[poltype=lcirc, abspos=2](0,1)(3,1)
  \polarization[poltype=rcirc, abspos=2.5](0,0)(3,0)
  \end{optexp}
\end{pspicture}
```

`Polarization` ⟨*psstyle*⟩ default: `arrowscale=0.8, linewidth=0.7\pslinewidth, dotsize=3\pslinewidth`

Bestimmt das Aussehen der Polarisationssymbole.

4.15. Keil

`\optwedge`[⟨*options*⟩]`(`⟨*in*⟩`)(`⟨*out*⟩`){`⟨*label*⟩`}`




```
\begin{pspicture}(3,2)
  \optwedge[beam](0,1)(3,1){Wedge}
\end{pspicture}
```

`wedgeheight`=⟨*num*⟩ default: `0.8`

Höhe des Keils.

`wedgeangleright`=⟨*num*⟩ default: `10`

Der obere Winkel der rechten Grenzfläche bezogen auf die Vertikale.

`wedgeangleleft`=⟨*num*⟩ default: `0`

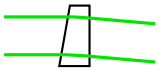
Der obere Winkel der linken Grenzfläche bezogen auf die Vertikale.

wedgeangles=⟨*left*⟩[⟨*right*⟩]

Erlaubt beide Winkel gleichzeitig zu setzen. Wird nur ein Wert angegeben, dann wird dieser für beide Winkel verwendet.

wedgewidth=⟨*num*⟩ default: 0

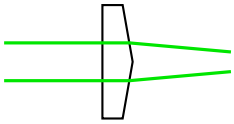
Ist der Wert größer als die berechnete Grundbreite, dann wird der Keil auf die angegebene Breite vergrößert.



```
\begin{pspicture}(2,1)
  \optwedge[wedgeangles=10 0, wedgewidth=0.4](0,0.5)(2,0.5)
  \drawwidebeam[beamwidth=0.5](0,0.5){}(2,0.5)
\end{pspicture}
```

4.16. Axikon

\axicon[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \axicon[label=1](0,1)(3,1){Axicon}
  \drawwidebeam[beamwidth=0.5](0,1){}(3,1)
\end{pspicture}
```

axiconheight=⟨*num*⟩ default: 1.5

Die Höhe des Axikons.

axiconwidth=⟨*num*⟩ default: 0.4

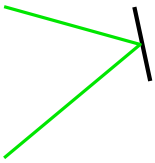
Die Breite des Axikons.

axiconangle=⟨*num*⟩ default: 10

Der obere Winkel der rechten Grenzfläche bezogen auf die Vertikale.

4.17. Spiegel

`\mirror`[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \mirror[beam](0,0)(1.8,1.5)(0,2){M}
\end{pspicture}
```

`mirrorwidth`=⟨*num*⟩ default: `1`

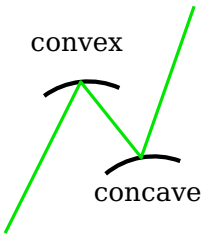
Die Breite des Spiegels.

`mirrorlinewidth`=⟨*num*⟩ or ⟨*dimen*⟩ default: `2\pslinewidth`

Die Linienbreite des Spiegels. Diese könnte ebenfalls über den `linewidth` Parameter gesetzt werden. Mit diesem Parameter kann aber die Linienbreite aller Spiegel global eingestellt werden.

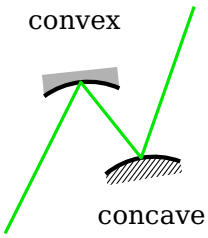
`mirrorradius`=⟨*radius*⟩[0] default: `0`

Diese Parameter setzt die Krümmung des Spiegels. Null ergibt einen ebenen, ein negativer Radius einen konvexen und ein positiver Radius einen konkaven Spiegel.



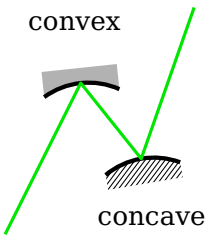

```
\begin{pspicture}(3,3)
  \psset[optexp]{labeloffset=0.5}
  \mirror[mirrorradius=1](0,0)(1,2)(1.8,1){convex}
  \mirror[mirrorradius=-1](1,2)(1.8,1)(2.5,3){concave}
  \drawbeam(0,0){-}(2.5,3)
\end{pspicture}
```

Wird kein zweiter Wert angegeben, so ist die Krümmung der zweiten Fläche eines `extended` oder `semitrans` Spiegels die Gleiche wie bei der Hauptgrenzfläche (siehe vorheriges Beispiel). Wird die Null als zweiter Wert angegeben (andere Werte werden nicht unterstützt), so ist die zweite Fläche flach:




```
\begin{pspicture}(3,3)
  \mirror[mirrorradius=1 0,
      mirrortype=semitrans](0,0)(1,2)(1.8,1){convex}
  \mirror[mirrorradius=-1 0,
      mirrortype=extended](1,2)(1.8,1)(2.5,3){concave}
  \drawbeam(0,0){-}(2.5,3)
\end{pspicture}
```

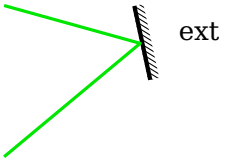
Die Angabe von mirrordepth bezieht sich immer auf die Mindesttiefe des Spiegels.




```
\begin{pspicture}(3,3)
  \psset[optexp]{mirrordepth=0.2}
  \mirror[mirrorradius=1 0,
      mirrortype=semitrans](0,0)(1,2)(1.8,1){convex}
  \mirror[mirrorradius=-1 0,
      mirrortype=extended](1,2)(1.8,1)(2.5,3){concave}
  \drawbeam(0,0){-}(2.5,3)
\end{pspicture}
```

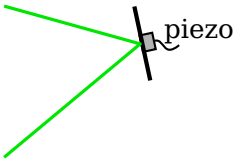
mirrortype=plain, piezo, extended, semitrans default: plain

Diese Parameter wählt die Spiegelart aus. Das Aussehen wird mit PiezoMirror, ExtendedMirror und SemitransMirror gesteuert.



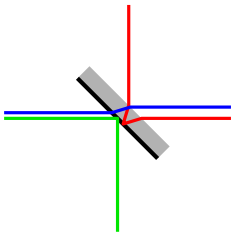

```
\begin{pspicture}(3,2)
  \mirror[mirrortype=extended,
          beam](0,0)(1.8,1.5)(0,2){ext}
\end{pspicture}
```

Beachten Sie, dass der Anschlussdraht für den Piezospiegel nicht gezeichnet wird, falls extnode verwendet wird.

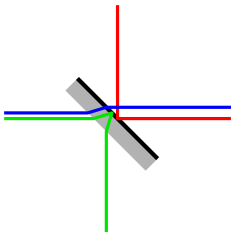



```
\begin{pspicture}(3,2)
\mirror[mirrortype=piezo,
        beam](0,0)(1.8,1.5)(0,2){piezo}
\end{pspicture}
```

Ein halbdurchlässiger Spiegel (semitrans) hat, im Gegensatz zu den anderen Spiegeltypen, zwei Grenzflächen, deren Abstand und Position vom Wert und Vorzeichen von mirrordepth abhängt.



```
\psset{unit=1.5}
\begin{pspicture}(2,2)
  \mirror[mirrortype=semitrans](0,1)(1,1)(1,0)
  \drawbeam(0,1){}(1,0)
  \drawbeam[linecolor=red](2,1){}(1,2)
  \drawbeam[beampos=0.05, linecolor=blue](0,1){}(2,1)
\end{pspicture}
```

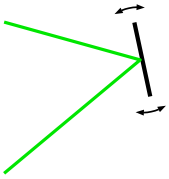


```
\psset{unit=1.5}
\begin{pspicture}(2,2)
  \mirror[mirrortype=semitrans, mirrordepth=-0.15](0,1)(1,1)(1,0)
  \drawbeam(0,1){}(1,0)
  \drawbeam[linecolor=red](2,1){}(1,2)
  \drawbeam[beampos=0.05, linecolor=blue](0,1){}(2,1)
\end{pspicture}
```

Eine negative Spiegeltiefe `mirrordepth` ist äquivalent zu einer positiven Tiefe kombiniert mit `angle=180`.

`variable=true, false` default: `false`

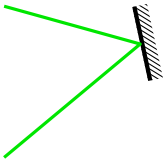
Zeichnet einen verstellbaren Spiegel, der mit zwei zusätzlichen gekrümmten Pfeilen an beiden Seiten angedeutet wird. Das Aussehen, inklusive der Pfeile, wird von `VariableStyle` vorgegeben.




```
\begin{pspicture}(3,2)
  \mirror[beam, variable](0,0)(1.8,1.5)(0,2){var}
\end{pspicture}
```

`mirrordepth=`⟨*num*⟩ default: `0.1`

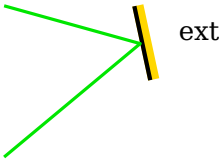
Die Gesamttiefe eines breiten Spiegels.



```
\begin{pspicture}(3,2)
  \mirror[mirrortype=extended, mirrordepth=0.2,
          beam](0,0)(1.8,1.5)(0,2)
\end{pspicture}
```

`ExtendedMirror` ⟨*psstyle*⟩ default: `linestyle=none, hatchwidth=0.5\pslinewidth, hatchsep=1.4\pslinewidth, fillstyle=hlines`

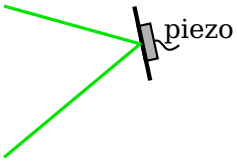
Der Stil für den `extended` Spiegel.




```
\begin{pspicture}(3,2)
  \newpsstyle{ExtendedMirror}{fillstyle=solid,
                              fillcolor=Gold,
                              linestyle=none}
  \mirror[mirrortype=extended,
          beam](0,0)(1.8,1.5)(0,2){ext}
\end{pspicture}
```

PiezoMirror ⟨*psstyle*⟩ default: `fillstyle=solid, fillcolor=black!30`

Der Stil für den `piezo` Spiegel. Dieser kann auch dazu verwendet werden, die Größe des Piezos zu verändern, wie in dem folgenden Beispiel zu sehen ist.




```
\begin{pspicture}(3,2)
  \addtopsstyle{PiezoMirror}{xunit=2}
  \mirror[mirrortype=piezo,
          beam](0,0)(1.8,1.5)(0,2){piezo}
\end{pspicture}
```

SemitransMirror ⟨*psstyle*⟩ default: `linestyle=none, fillstyle=solid, fillcolor=black!30`

Der Stil für den `semitrans` Spiegel.

4.18. Parabolspiegel

`\parabolicmirror[`⟨*options*⟩`](`⟨*in*⟩`)(`⟨*out*⟩`){`⟨*label*⟩`}`

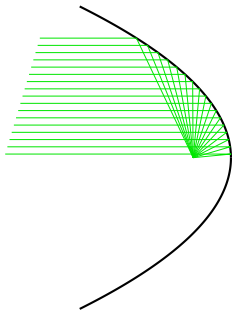
```
\begin{pspicture}(3,2)
  \parabolicmirror(0,1)(2,1){PM}
\end{pspicture}
```

parmirrorwidth=⟨*num*⟩ default: 1

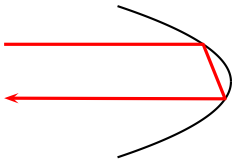
Die Breite des Parabolspiegels.

parmirrorheight=⟨*num*⟩ default: 1

Die Höhe des Parabolspiegels.



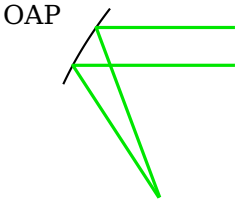
```
\begin{pspicture}(0,-2)(3,2)
  \parabolicmirror[parmirrorheight=4, parmirrorwidth=2](0,0)(3,0)
  \addtopsstyle{Beam}{beamalign=abs, linewidth=0.2\pslinewidth,
      beaminsidelast, stopinsidecount=1}
  \multido{\r=-1.55+0.1}{33}{%
    \drawbeam[beampos=\r](0,0){1}}
\end{pspicture}
```



```
\begin{pspicture}(0,-1)(3,1)
  \optplane(0,0)
  \parabolicmirror[parmirrorheight=2, parmirrorwidth=1.5](0,0)(3,0)
  \addtopsstyle{Beam}{arrows=->, beamalign=abs}
  \drawbeam[beampos=0.5, linecolor=red]{1}{2}{1}
\end{pspicture}
```

4.19. Off-axis-Parabolspiegel

`\oapmirror`[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*focus*⟩){⟨*label*⟩}



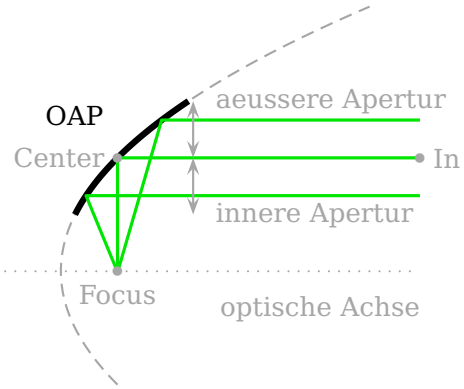

```
\begin{pspicture}(3,2.5)
  \oapmirror(3,2)(1,2)(2,0){OAP}
  \drawwidebeam[beamwidth=0.5](3,2){1}(2,0)
\end{pspicture}
```

`oapmirroraperture`=⟨*num*⟩ or ⟨*inner*⟩ ⟨*outer*⟩ default: 1

Apertur des Spiegels bezogen auf die Eingangsrichtung. Wird eine Zahl angegeben, dann gibt diese die gesamte Apertur an, die vertikal und symmetrisch zur Verbindung zwischen ⟨*in*⟩ und ⟨*center*⟩ gemessen wird. Bei zwei Zahlen geben diese die innere und äußere Apertur an.

Die Konstruktion des Spiegels anhand der vorgegebenen Punkte geschieht wie folgt: Die Gerade von Knoten ⟨*in*⟩ zu ⟨*center*⟩ liegt parallel zur optischen Achse, ⟨*focus*⟩ gibt den Fokuspunkt der Parabel an. Die Apertur des Spiegels wird mittels des Parameters `oapmirroraperture` angegeben, gemessen senkrecht zum einfallenden Strahl.

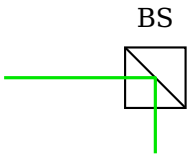
Die unterschiedlichen Parameter und die zugrundeliegende ganze Parabel sind in der folgenden Skizze dargestellt:




```
\begin{pspicture}(6,5)
\pnodes(5.5,3){In}(1.5,3){Center}(1.5,1.5){Focus}
\oapmirror[oapmirroraperture=1.5,
    linewidth=3\pslinewidth,
    plotpoints=200](In)(Center)(Focus){OAP}
\multido{\r=-0.5+0.5}{3}{%
  \drawbeam[beampos=\r](In){}(Focus)}
\end{pspicture}
```

4.20. Strahlteiler

`\beamsplitter`[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*out*⟩){⟨*label*⟩}



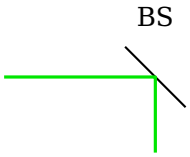

```
\begin{pspicture}(3,2)
  \beamsplitter[beam](0,1)(2,1)(2,0){BS}
\end{pspicture}
```

`bssize`=⟨*num*⟩ default: `0.8`

Die Größe des Strahlteilers.

`bsstyle`=`cube`, `plate` default: `cube`

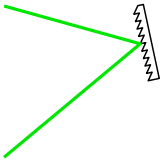
Wählt zwischen zwei Strahlteilertypen: einem Strahlteilerwürfel (`cube`) und einem halbdurchlässigen Spiegel (`plate`). Dieser halbdurchlässige Spiegel ist äquivalent zu `\mirror` mit `mirrortype`=`semitrans` für `mirrordepth`=`0`.




```
\begin{pspicture}(3,2)
  \beamsplitter[bsstyle=plate, beam](0,1)(2,1)(2,0){BS}
\end{pspicture}
```

4.21. Optisches Gitter

\optgrating[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \optgrating[beam](0,2)(1.8,1.5)(0,0){G}
\end{pspicture}
```

gratingwidth=⟨*num*⟩ default: 1

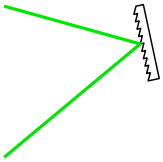
Die Breite des Gitters.

gratingheight=⟨*num*⟩ default: 0.15

Die Gesamthöhe des Gitters.

gratingdepth=⟨*num*⟩ default: 0.075

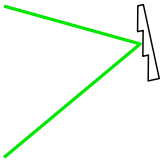
Die absolute Modulationstiefe der Gitterstruktur. Die Gesamthöhe wird an diesen Wert angepasst falls sie kleiner als die Modulationtiefe ist, andernfalls wird sie nicht beinflusst.




```
\begin{pspicture}(3,2)
  \optgrating[gratingdepth=0.05,
              beam](0,2)(1.8,1.5)(0,0){G}
\end{pspicture}
```

gratingcount=⟨*int*⟩ default: 10

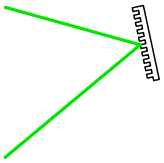
Die Anzahl der Gitterlinien.




```
\begin{pspicture}(3,2)
  \optgrating[gratingcount=3,
              beam](0,2)(1.8,1.5)(0,0){G}
\end{pspicture}
```

`gratingtype`=`blazed`, `binary` default: `blazed`

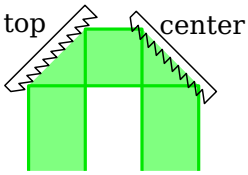
Wählt zwischen einem binären Gitter (`binary`) und einem Blazegitter (`blazed`).




```
\begin{pspicture}(3,2)
  \optgrating[gratingtype=binary,
              beam](0,2)(1.8,1.5)(0,0){G}
\end{pspicture}
```

`gratingalign`=`t`, `top`, `c`, `center` default: `top`

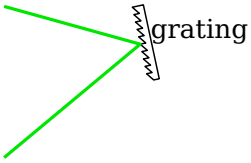
Wählt, ob die Reflektionsebene auf den Gitterspitzen (`top`), oder in der Mitte des Gitters (`center`) liegt. Wählt man `center` aus, macht es Sinn die `optexp`-Umgebung oder eine halbtransparente Strahlfüllung zu verwenden.




```
\begin{pspicture}(3,2)
\pnodes(0.5,0){A}(0.5,1.5){B}(2,1.5){C}(2,0){D}
\psset{unit=1.5, labeloffset=0.4}
\begin{optexp}
  \optgrating[gratingalign=top](A)(B)(C){top}
  \optgrating[gratingalign=center](B)(C)(D){center}
  \drawwidebeam[fillstyle=solid, fillcolor=green!50,
      beamwidth=0.5](A){-}(D)
\end{optexp}
\end{pspicture}
```

`reverse`=`true`, `false` default: `false`

Invertiert die Steigung des Blazegitters.



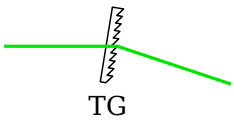

```
\begin{pspicture}(3,2)
  \optgrating[reverse, beam](0,2)(1.8,1.5)(0,0){grating}
\end{pspicture}
```

`gratinglinewidth`=⟨*num*⟩ or ⟨*dimen*⟩ default: `0.7\pslinewidth`

Die Linienbreite des Gitters. Diese könnte ebenfalls über den `linewidth` Parameter gesetzt werden. Mit diesem Parameter kann aber die Linienbreite aller Gitter global eingestellt werden.

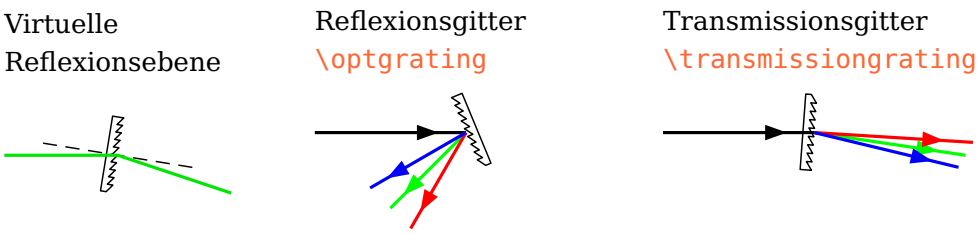
4.22. Transmissionsgitter

\transmissiongrating[⟨options⟩](⟨in⟩)(⟨center⟩)(⟨out⟩){⟨label⟩}

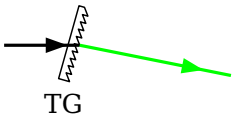



```
\begin{pspicture}(3,2)
  \transmissiongrating[beam](0,1)(1.5,1)(3,0.5){TG}
\end{pspicture}
```

Das Transmissiongitter unterstützt alle Optionen von \optgrating. Der Strahlengang wird so berechnet, dass der Strahl an einer Ebene senkrecht zum dargestellten Gitter reflektiert wird. Diese Ebene ist im folgenden Beispiel dargestellt:



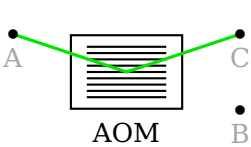
Ist dieses Verhalten nicht gewünscht, so kann man z.B. die Komponente mit angle weiter drehen, und den Strahlwinkel mit beamangle entsprechend korrigieren.




```
\begin{pspicture}(3,1.4)
  \pnodes(0,1){A}(1,1){B}(3,0.6){C}
  \transmissiongrating[angle=-10, reverse](A)(B)(C){TG}
  \addtopsstyle{Beam}{ArrowInside=->, ArrowInsidePos=0.8,
      arrowinset=0, arrowscale=1.5}
  \drawbeam[linecolor=black](A){1}
  \drawbeam[linecolor=green, loadbeam, beamangle=20]{1}(C)
\end{pspicture}
```

4.23. Akustooptischer Modulator

\optaom[⟨options⟩](⟨in⟩)(⟨trans⟩)(⟨diff⟩){⟨label⟩}




```
\begin{pspicture}(3,2)
  \pnodes(0,1.5){A}(3,0.5){B}(3,1.5){C}
  \optaom[beam](A)(B)(C){AOM}
\end{pspicture}
```

Dieses Makro stellt einen akustooptischen Modulator dar. Dieser kann, ähnlich einem \beamsplitter entweder transmittierend oder reflektierend sein. Beachten Sie, dass die Knoten nicht so angeordnet sind, wie bei den üblichen Tripolen wie z. B. dem \optgrating. Der mittlere Knoten gibt die Richtung eines transmittierten, und der dritte Knoten die Richtung der ersten Beugungsordnung an.

Diese erste Umsetzung des AOM in Version 4.8 ist noch etwas experimentell, da ich nicht genügend Testfälle hatte um alle Eventualitäten zu prüfen. Es könnten sich also noch Änderungen ergeben, die nicht abwärts kompatibel sind.

aomheight=⟨*num*⟩ default: 1

Die Höhe des Modulators.

aomwidth=⟨*num*⟩ default: 1.5

Die Breite des Modulators.

aomsize=⟨*width*⟩ ⟨*height*⟩

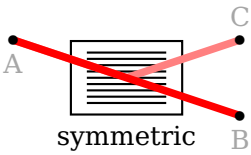
Die Breite und Höhe des Modulators, äquivalent zum Aufruf von aomwidth und aomheight.

aomgratingcount=⟨*int*⟩ default: 9

Gibt die Anzahl der Gitterlinien an. Werden weitere Änderungen des Gitters oder des Modulators allgemein gewünscht, dann muss der Parameter aomcomp verwendet werden.

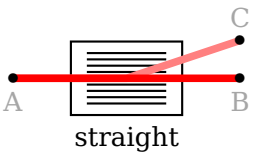
aomalign=symmetric, straight default: symmetric

Die Ausrichtung eines \optaom bezüglich der drei Referenzknoten kann mit diesem Parameter eingestellt werden. Die symmetrische (symmetric) Ausrichtung ist im folgenden Beispiel zu sehen:




```
\begin{pspicture}(3,2)
  \pnodes(0,1.5){A}(3,0.5){B}(3,1.5){C}
  \optaom[aomalign=symmetric](A)(B)(C){symmetric}
  \addtopsstyle{Beam}{fillstyle=solid, linestyle=none, beamwidth=0.1}
  \drawwidebeam[fillcolor=red!50](A){}(C)
  \drawwidebeam[fillcolor=red](A){}(B)
\end{pspicture}
```

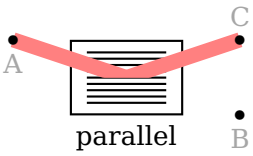
In der geraden (straight) Ausrichtung sieht der Strahlengang wie folgt aus:



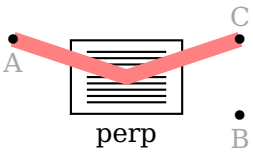

```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(3,1){B}(3,1.5){C}
  \optaom[aomalign=straight](A)(B)(C){straight}
  \addtopsstyle{Beam}{fillstyle=solid, linestyle=none, beamwidth=0.1}
  \drawwidebeam[fillcolor=red!50](A){}(C)
  \drawwidebeam[fillcolor=red](A){}(B)
\end{pspicture}
```

`aomreflalign`=`perp`, `parallel` default: `parallel`

Die Ausrichtung der reflektiven Grenzfläche kann entweder `parallel` oder senkrecht (`perp`) zur Komponente liegen. Diese Option ist lediglich aus technischer Sicht notwendig, die Richtung des reflektierten Strahls wird dadurch nicht verändert.

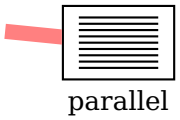



```
\begin{pspicture}(3,2)
  \pnodes(0,1.5){A}(3,0.5){B}(3,1.5){C}
  \optaom[aomreflalign=parallel](A)(B)(C){parallel}
  \addtopsstyle{Beam}{fillstyle=solid, linestyle=none, beamwidth=0.2}
  \drawwidebeam[fillcolor=red!50](A){}(C)
\end{pspicture}
```




```
\begin{pspicture}(3,2)
  \pnodes(0,1.5){A}(3,0.5){B}(3,1.5){C}
  \optaom[aomreflalign=perp](A)(B)(C){perp}
  \addtopsstyle{Beam}{fillstyle=solid, linestyle=none, beamwidth=0.2}
  \drawwidebeam[fillcolor=red!50](A){}(C)
\end{pspicture}
```

Das folgende Beispiel zeigt einen Fall, der mit `aomreflalign`=`parallel` nicht funktioniert, da der Einfallswinkel so flach ist, dass die Schnittpunkte mit der reflektierenden Grenzfläche außerhalb der Komponente liegen:




```
\begin{pspicture}(3,1.5)
  \pnodes(0,1.1){A}(3,0.8){B}(3,1.1){C}
  \optaom[aomreflalign=parallel](A)(B)(C){parallel}
  \addtopsstyle{Beam}{fillstyle=solid, linestyle=none, beamwidth=0.2}
  \drawwidebeam[fillcolor=red!50](A){}(C)
\end{pspicture}
```

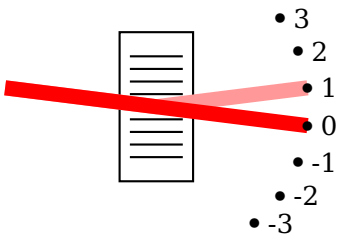
`aomcomp`=`default`, ⟨*macro*⟩ default: `default`

Tauscht die Gitterlinien durch eine beliebige Zeichung aus.

`diffractionorders`=⟨*int*⟩ default: 2

Die Anzahl der Beugungsordnungen (positive und negative), die mit `beamdiffractionorder` ausgewählt werden können, also $-\langle num\rangle, -\langle num\rangle+1, \ldots, \langle num\rangle$.

Gleichzeitig werden entsprechende Knoten `\oenode{DO-1}{<comp>}`, `\oenode{DO0}{<comp>}`, `\oenode{DO1}{<comp>}` usw. definiert.



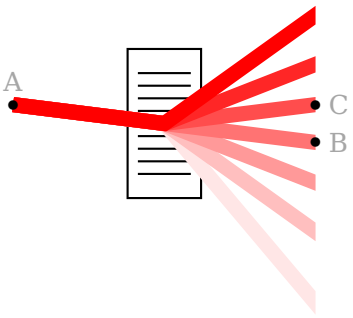

```
\begin{pspicture}(0,0.6)(4.3,3.6)
  \pnodes(0,2.5){A}(4,2){B}(4,2.5){C}
  \optaom[aomsize=1 2, aomreflalign=perp,
      diffractionorders=3](A)(B)(C)
  \addtopsstyle{Beam}{beamwidth=0.2, linestyle=none,
      fillstyle=solid}
  \drawwidebeam[fillcolor=red!40](A){}(C)
  \drawwidebeam[fillcolor=red](A){}(B)
  \multido{\i=-3+1}{7}{%
    \psdot(\oenode{DO\i}{})\uput[0](\oenode{DO\i}{}){\i}}%
\end{pspicture}%
```

`beamdiffractionorder`=⟨*int*⟩ default: {}

Die Beugungsordnung, die für einen reflektierten Strahl durch den AOM verwendet wird. Wenn der Wert leer ist, {} dann wird die Option ignoriert.

Mit Verwendung dieser Option wird unmittelbar das Verhalten der Grenzfläche festgelegt, hängt also nicht von der relativen Position von AOM und der folgenden Komponente ab. Für `beamdiffractionorder`$\neq 0$ wird der Strahl reflektiert, für den Wert 0 hingegen transmittiert. Der Parameter `beammode` wird dann ignoriert.

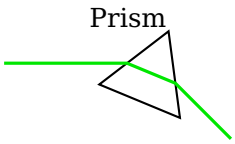
Es können nur die Beugungsordnungen verwendet werden, die für die entsprechende Komponente mit `diffractionorders` definiert wurden. Wird eine nicht-definierte Ordnung verwendet, dann wird die nächstliegende verwendet und eine Postscript-Warnung (siehe `pswarning`) ausgegeben.




```
\begin{pspicture}(0,-0.2)(4.4,3.8)
  \pnodes(0,2.5){A}(4,2){B}(4,2.5){C}
  \optaom[aomsize=1 2, aomreflalign=perp, diffractionorders=3](A)(B)(C)
  \optplane(B)
  \addtopsstyle{Beam}{beamwidth=0.2, linestyle=none, fillstyle=solid}
  \multido{\i=-3+1,\r=10+15}{7}{%
    \drawwidebeam[fillcolor=red!\r, beamdiffractionorder=\i](A){-}}
\end{pspicture}%
```

4.24. Prisma

\optprism[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \optprism[beam](0,1)(2,1)(3,0){Prism}
\end{pspicture}
```

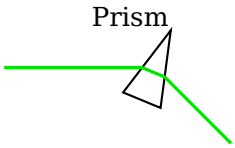
Das Prisma wird immer symmetrisch bezüglich der ⟨*in*⟩ und ⟨*out*⟩ Knoten ausgerichtet. Für asymmetrischen Strahlengang siehe Kap. 9.

prismsize=⟨*num*⟩ default: 1

Die Höhe des Prismas.

prismangle=⟨*num*⟩ default: 60

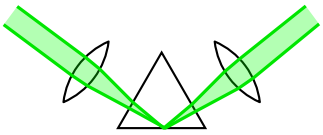
Der obere Winkel des Prismas.




```
\begin{pspicture}(3,2)
  \optprism[prismangle=30, beam](0,1)(2,1)(3,0){Prism}
\end{pspicture}
```

`prismtype`=`transmittive`, `reflective` default: `transmittive`

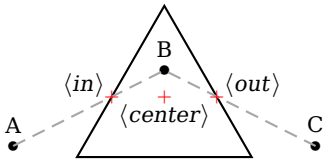
Ist dieser Parameter auf `reflective` gesetzt, so wird die Grundfläche als zusätzliche, reflektierende Grenzfläche verwendet.



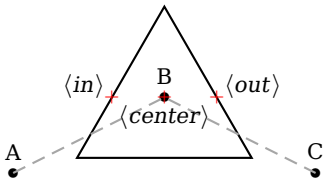
```
\begin{pspicture}(1.5,0.6)(5.1,2.2)
  \optprism[prismtype=reflective](0,1)(3,0.9)(6,1)
  \lens(1,2)(\oenodeIfc{2}{1})
  \lens(\oenodeIfc{2}{1})(5,2)
  \drawwidebeam[beamwidth=0.3, fillstyle=solid,
      fillcolor=green, opacity=0.3](1,2){2}{1}{3}(5,2)
\end{pspicture}
```

`prismalign`=`auto`, `center` default: `auto`

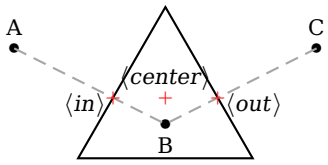
Wählt die vertikale Ausrichtung des Prisma bezüglich des „Reflektionsknoten“ ⟨*center*⟩. Ist der Wert `auto`, so wird das Prisma so verschoben, dass die Grenzflächenknoten auf der Verbindungslinie zwischen dem entsprechenden Referenzknoten und dem „Reflektionsknoten“ liegen. Für `center` fällt der „Reflektionsknoten“ mit der Komponentenmitte zusammen. Siehe die folgende Beispiele zur weiteren Erklärung.



`prismalign=auto`
`prismtype=transmittive`



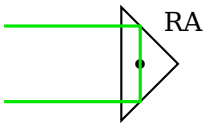
`prismalign=center`
`prismtype=transmittive`



`prismalign=auto`
`prismtype=reflective`

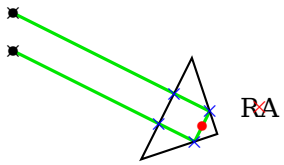
4.25. Umkehrprisma

`\rightangleprism`[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \psdot(1.8,1)
  \rightangleprism[beam](0,1.5)(1.8,1)(0,0.5){RA}
\end{pspicture}
```

Das Umkehrprisma wird so ausgerichtet, dass der einfallende und der reflektierte Strahl parallel sind und der ⟨*center*⟩ Knoten vertikal zentriert in dem Prisma liegt.




```
\begin{pspicture}(3.5,2)
  \pnodes(0,2){A}(2.5,0.5){G}(0,1.5){B}
  \begin{optexp}
  \rightangleprism[beam, showifcnodes, showoptdots](A)(G)(B){RA}
  \end{optexp}
\end{pspicture}
```

raprismsize=⟨*num*⟩ default: 1.5

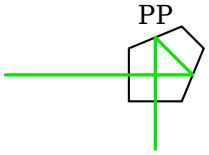
Die Länge der Eingangsfläche.

raprismalign=auto, center default: auto

Schaltet die automatische Positionierung der Komponente vertikal zur Eingangsfläche ein oder aus:

4.26. Pentaprisma

\pentaprism[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \pentaprism[beam](0,1)(2,1)(2,0){PP}
\end{pspicture}
```

pentaprismsize=⟨*num*⟩ default: 0.7

Die Länge der Eingangs- und Ausgangsfläche.

5. Faserkomponenten

In diesem Kapitel werden alle Faserkomponenten und deren Parameter beschrieben. Diese werden automatisch über Fasern mit ihren Referenzknoten verbunden und unterstützen kein Raytracing. Anstelle der automatischen Faserverbindungen können die Komponenten auch manuell mit Fasern verbunden werden, siehe Kap. 9.6.

Die meisten Komponenten benötigen nur zwei Referenzknoten und werden wie die Freistrahlzweipole behandelt. Manche spezielle Komponenten wie Koppler (5.10) und Zirkulatoren (5.9) werden anders ausgerichtet.

`usefiberstyle=true, false`

Bei manchen Komponenten (z.B. `\optfilter` oder `\optmzm`) kann es vorteilhaft sein interne Faserteile hervorzuheben. Mit diesem Parameter werden z.B. die durchlässigen Teile eines Filter angezeigt und mit dem `Fiber`-Stil gezeichnet. In der Dokumentation ist dieses Verhalten angeschaltet um die betroffenen Bereiche hervorzuheben. Hat Vorrang vor der Option `usewirestyle`, sind beide Optionen gesetzt, dann wird eine Faser gezeichnet.

`usewirestyle=true, false`

Kann verwendet werden, um interne Verbindungen als `Wire` darzustellen. Setzt `usefiberstyle` auf `false`, daher kann es sinnvoll sein, z.B. einen elektrischen Filter zu definieren:

```
\newpsobject{elecfilter}{optfilter}{usewirestyle}
```

5.1. Optische Faser

`\optfiber[`⟨*options*⟩`](`⟨*in*⟩`)(`⟨*out*⟩`){`⟨*label*⟩`}`

```
\begin{pspicture}(3,1.5)
  \optfiber[label=0.3](0,0.5)(3,0.5){SSMF}
\end{pspicture}
```

`fiberloops`=⟨*int*⟩ default: `3`

Anzahl der Faserschleifen.

```
\begin{pspicture}(3,1.5)
  \optfiber[fiberloops=2, label=0.3](0,0.5)(3,0.5){SSMF}
\end{pspicture}
```

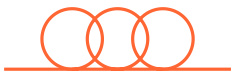
`fiberloopradius`=⟨*num*⟩ default: `0.4`

Radius der Faserschleifen.

```
\begin{pspicture}(3,1.5)
  \optfiber[fiberloopradius=0.2, label=0.3](0,0.5)(3,0.5){SSMF}
\end{pspicture}
```

`fiberloopsep`=⟨*num*⟩ default: `0.2`

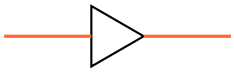
Abstand zwischen zwei aufeinanderfolgenden Faserschleifen.




```
\begin{pspicture}(3,1.5)
  \optfiber[fiberloopsep=0.6, label=0.3](0,0.5)(3,0.5){SSMF}
\end{pspicture}
```

5.2. Optischer Verstärker

`\optamp`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}



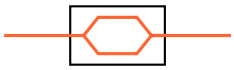

```
\begin{pspicture}(3,1.5)
  \optamp(0,1)(3,1){EDFA}
\end{pspicture}
```

`optampsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.8`

Eine einzelne Zahl gibt die Seitenlänge des Verstärkers an, zwei Zahlen die Breite und Höhe. Beachten Sie, dass `optampsize=1` und `optampsize=1 1` nicht dasselbe Ergebnis liefern. Alternativ kann das Verhältnis zwischen Höhe und Breite mit `xunit` und `yunit` geändert werden.

5.3. Mach-Zehnder-Modulator

`\optmzm`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}



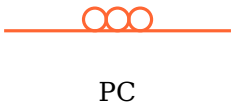

```
\begin{pspicture}(3,1.5)
  \optmzm(0,1)(3,1){MZM}
\end{pspicture}
```

`optmzmsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.8`

Eine einzelne Zahl gibt die Höhe des Modulators an, die Breite ist `1.6` mal die Höhe. Zwei Zahlen geben Breite und Höhe des Modulators direkt an.

5.4. Polarisationssteller

`\polcontrol`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}



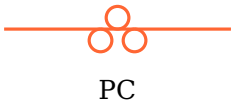

```
\begin{pspicture}(3,1.5)
  \polcontrol(0,1)(3,1){PC}
\end{pspicture}
```

`polcontrolsize`=⟨*num*⟩ default: `0.15`

Der Radius der Polarisationssteller-Kreise.

`polcontroltype`=`linear`, `triangle` default: `linear`

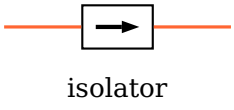
Der Typ des Polarisationsstellers.




```
\begin{pspicture}(3,1.5)
  \polcontrol[polcontroltype=triangle](0,1)(3,1){PC}
\end{pspicture}
```

5.5. Isolator

`\optisolator`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}



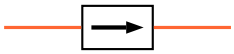

```
\begin{pspicture}(3,1.5)
  \optisolator(0,1)(3,1){isolator}
\end{pspicture}
```

`isolatorsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.6`

Eine einzelne Zahl gibt die Höhe des Isolators an, die Breite ist `1.6` mal die Höhe. Zwei Zahlen geben Breite und Höhe des Isolators direkt an.

`IsolatorArrow` ⟨*psstyle*⟩ default: `linewidth=2\pslinewidth, arrowinset=0`

Der Stil für den Isolatorpfeil. Das kann insbesondere nützlich sein um die Länge des Pfeils zu verändern.

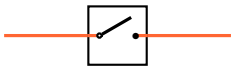


isolator

```
\begin{pspicture}(3,1.5)
  \addtopsstyle{IsolatorArrow}{xunit=1.2}
  \optisolator(0,1)(3,1){isolator}
\end{pspicture}
```

5.6. Optischer Schalter

`\optswitch`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}



opened switch

```
\begin{pspicture}(3,1.5)
  \optswitch(0,1)(3,1){opened switch}
\end{pspicture}
```

`switchsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.8`

Wird nur eine Zahl angegeben, so ist das die Seitenlänge des Schalters, andernfalls können Breite und Höhe getrennt angegeben werden.

`switchstyle`=`opened`, `closed` default: `opened`

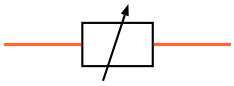
Der Zustand des Schalters kann geschlossen (`closed`) oder geöffnet (`opened`) sein.

closed switch

```
\begin{pspicture}(3,1.5)
  \optswitch[switchstyle=closed](0,1)(3,1){closed switch}
\end{pspicture}
```

5.7. Faserverzögerungstrecke

`\fiberdelayline[`⟨*options*⟩`](`⟨*in*⟩`)(`⟨*out*⟩`){`⟨*label*⟩`}`




```
\begin{pspicture}(3,1.5)
  \fiberdelayline(0,1)(3,1){delay line}
\end{pspicture}
```

`fdlsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.6`

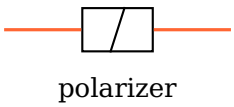
Eine einzelne Zahl gibt die Höhe der Komponente an, die Breite ist `1.6` mal die Höhe. Zwei Zahlen geben Breite und Höhe der Komponente direkt an.

`FdlArrow` ⟨*psstyle*⟩ default: `arrowinset=0, arrows=->`

Siehe `ArrowCompStyle`.

5.8. Polarisator

`\optfiberpolarizer[`⟨*options*⟩`](`⟨*in*⟩`)(`⟨*out*⟩`){`⟨*label*⟩`}`



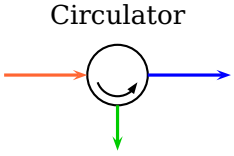

```
\begin{pspicture}(3,1.5)
  \optfiberpolarizer(0,1)(3,1){polarizer}
\end{pspicture}
```

`fiberpolsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.6`

Eine einzelne Zahl gibt die Höhe der Komponente an, die Breite ist `1.6` mal die Höhe. Zwei Zahlen geben Breite und Höhe der Komponente direkt an.

5.9. Optischer Zirkulator

`\optcirculator(`⟨*left*⟩`)(`⟨*right*⟩`)(`⟨*bottom*⟩`){`⟨*label*⟩`}`




```
\begin{pspicture}(3,2)
  \addtopsstyle{Fiber}{arrows=->}
  \addtopsstyle{FiberOut1}{linecolor=blue}
  \addtopsstyle{FiberOut2}{linecolor=green!80!black}
  \optcirculator(0,1)(3,1)(1.5,0){Circulator}
\end{pspicture}
```

`optcircsize`=⟨*num*⟩ default: `0.8`

Der Durchmesser des Zirkulators.

`optcircangleA`=⟨*num*⟩ default: `-160`

Der Startwinkel des internen Pfeils.

`optcircangleB`=⟨*num*⟩ default: `-20`

Der Endwinkel des internen Pfeils.

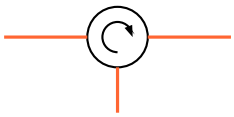
`optcircangle`=⟨*num*⟩ ⟨*num*⟩

Kurzschreibweise um beide Winkel gleichzeitig zu setzen.

```
\begin{pspicture}(3,1.5)
  \optcirculator[optcircangle=0 -90](0,1)(3,1)(1.5,0)
\end{pspicture}
```

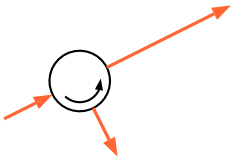
`OptCircArrow` ⟨*psstyle*⟩ default: `unit=0.7, arrows=->, arrowinset=0`

Der Stil für den internen Pfeil. Damit wird sowohl die Richtung des Pfeils als auch die Größe des Bogens bestimmt.



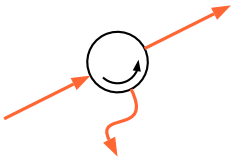
```
\begin{pspicture}(3,1.5)
  \newpsstyle{OptCircArrow}{unit=0.5, arrows=<-, arrowinset=0}
  \optcirculator[optcircangle=0 -90](0,1)(3,1)(1.5,0)
\end{pspicture}
```

Der Zirkulator wird so positioniert, dass Eingangs- und Ausgangsverbindung senkrecht zueinander stehen, wie das folgende Beispiel verdeutlicht:



```
\begin{pspicture}(3,2)
  \addtopsstyle{Fiber}{arrowscale=1.3, arrows=->, arrowinset=0}
  \optcirculator(0,0.5)(3,2)(1.5,0)
\end{pspicture}
```

Die Positionierungsparameter (Kap. 3.2) beziehen sich wie üblich auf die Referenzknoten ⟨*left*⟩ und ⟨*right*⟩, und nicht auf die automatisch bestimmte Position. Das bedeutet auch, dass die automatische Position i.a. nicht mit `position=0.5` übereinstimmt.

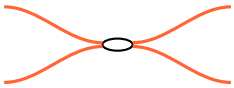


```
\begin{pspicture}(3,2)
  \addtopsstyle{Fiber}{arrowscale=1.3, arrows=->, arrowinset=0,
      ncurv=1.5}
  \optcirculator[position=0.5](0,0.5)(3,2)(1.5,0)
\end{pspicture}
```

5.10. Faserkoppler

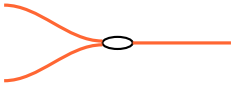
Es stehen drei Faserkoppler zur Verfügung, die alle die gleichen Formen und Parameter verwenden.

`\optcoupler`(⟨*tl*⟩)(⟨*bl*⟩)(⟨*tr*⟩)(⟨*br*⟩){⟨*label*⟩}


optcoupler

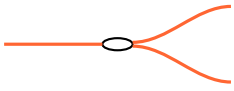
```
\begin{pspicture}(0,-0.4)(3,1)
  \optcoupler(0,1)(0,0)(3,1)(3,0){optcoupler}
\end{pspicture}
```

`\wdmcoupler`(⟨*tl*⟩)(⟨*...*⟩)(⟨*bl*⟩)(⟨*r*⟩){⟨*label*⟩}


wdmcoupler

```
\begin{pspicture}(0,-0.4)(3,1)
  \wdmcoupler(0,1)(0,0)(3,0.5){wdmcoupler}
\end{pspicture}
```

`\wdmsplitter`(⟨*l*⟩)(⟨*tr*⟩)(⟨*...*⟩)(⟨*br*⟩){⟨*label*⟩}


wdmsplitter

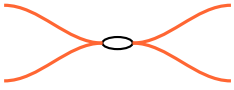
```
\begin{pspicture}(0,-0.4)(3,1)
  \wdmsplitter(0,0.5)(3,1)(3,0){wdmsplitter}
\end{pspicture}
```

`couplersize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.2`

Wird eine Zahl angegeben, so ist die Breite des Kopplers doppelt so groß, die Höhe entspricht `0.8` mal dem Wert. Mit zwei Zahlen werden Breite und Höhe direkt angegeben.

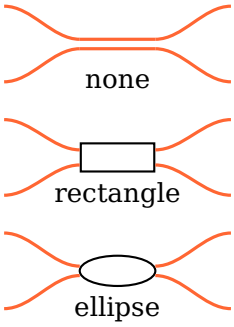
`couplersep`=⟨*num*⟩ default: `0.05`

Der vertikale Abstand zwischen zwei Faserports.


coupler

```
\begin{pspicture}(0,-0.5)(3,1)
  \optcoupler[couplersep=0](0,1)(0,0)(3,1)(3,0){coupler}
\end{pspicture}
```

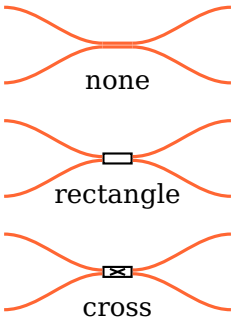
Wird dieser Wert auf {} gesetzt, dann wird der Abstand automatisch berechnet, abhängig von `couplertype` und `couplersize`.




```
\begin{pspicture}(3,4.5)
  \psset{couplersep={}, couplersize=0.5, labeloffset=0.5}
  \optcoupler[couplertype=none]%
      (0,4.5)(0,3.5)(3,4.5)(3,3.5){none}
  \optcoupler[couplertype=rectangle]%
      (0,3)(0,2)(3,3)(3,2){rectangle}
  \optcoupler[couplertype=ellipse]%
      (0,1.5)(0,0.5)(3,1.5)(3,0.5){ellipse}
\end{pspicture}
```

`couplertype`=`none`, `ellipse`, `rectangle`, `cross` default: `ellipse`

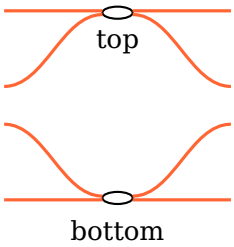
Wählt zwischen unterschiedlichen Kopplertypen.




```
\begin{pspicture}(3,4.5)
  \psset{labeloffset=0.5}
  \optcoupler[couplertype=none]%
      (0,4.5)(0,3.5)(3,4.5)(3,3.5){none}
  \optcoupler[couplertype=rectangle]%
      (0,3)(0,2)(3,3)(3,2){rectangle}
  \optcoupler[couplertype=cross]%
      (0,1.5)(0,0.5)(3,1.5)(3,0.5){cross}
\end{pspicture}
```

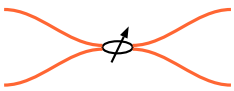
`coupleralign`=`t`, `top`, `b`, `bottom`, `c`, `center` default: `center`

Die Ausrichtung des Kopplers bezüglich der Referenzknoten. Beachten Sie, dass die Position der Komponentenmitte und damit der Beschriftung von dieser Einstellung abhängt (siehe 8.3).




```
\begin{pspicture}(3,3)
  \psset{labeloffset=0.4}
  \optcoupler[coupleralign=top]%
              (0,3)(0,2)(3,3)(3,2){top}
  \optcoupler[coupleralign=bottom]%
              (0,1.5)(0,0.5)(3,1.5)(3,0.5){bottom}
\end{pspicture}
```

`variable`=`true`, `false` default: `true`



```
\begin{pspicture}(3,1)
  \optcoupler[variable](0,1)(0,0)(3,1)(3,0)
\end{pspicture}
```

VariableCoupler ⟨*psstyle*⟩ default: `arrowinset=0, arrows=->`

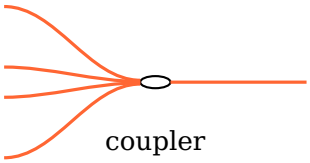
Der Stil des Pfeiles des verstellbaren Kopplers.

5.10.1. Variable Knotenanzahl

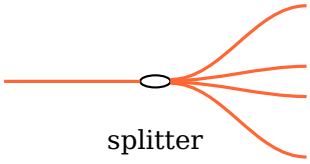
Jeder Koppler hat eine Variante, die nur zwei Knoten benötigt, einen Eingangs- und einen Ausgangsknoten. \optcoupler verwendet jeden dieser Knoten zweimal, \wdmsplitter verwendet den Knoten ⟨*out*⟩ zweimal und \wdmcoupler verwendet ⟨*in*⟩ zweimal. Das kann sehr praktisch sein, wenn man für einen Koppler die automatischen Verbindungen nicht verwendet, sondern die Komponente manuell verbindet.

```
\begin{pspicture}(4,2)
  \pnodes(0,1){A}(4,1){B}
  \psset[optexp]{fiber=none}
  \wdmsplitter[position=0.2, coupleralign=b](A)(B)
  \wdmcoupler[position=0.8, coupleralign=t](A)(B)
  \drawfiber{(A)}{1}{2}{(B)}
  \drawfiber{1}{([offset=0.5, Xnodesep=1]\oenodeIfc{2}{1})}
  \drawfiber{([offset=-0.5, Xnodesep=-1]\oenodeIfc{2}{2})}{2}
\end{pspicture}
```

Des Weiteren kann ein \wdmcoupler mehr als zwei Eingangsknoten haben, ein \wdmsplitter mehr als zwei Ausgangsknoten.

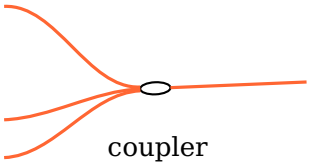



```
\begin{pspicture}(4,2)
  \pnodes(0,2){A}(0,1.2){B}(0,0.8){C}(0,0){D}(4,1){E}
  \wdmcoupler(A)(B)(C)(D)(E){coupler}
\end{pspicture}
```

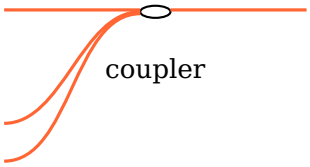



```
\begin{pspicture}(4,2)
  \pnodes(0,1){A}(4,2){B}(4,1.2){C}(4,0.8){D}(4,0){E}
  \wdmsplitter(A)(B)(C)(D)(E){splitter}
\end{pspicture}
```

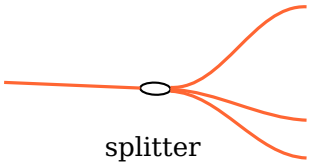
Die Ausrichtung der Komponente wird anhand des Mittelpunkts aller Eingangs- (\wdmcoupler) bzw. Ausgangsknoten (\wdmsplitter) berechnet, falls nicht coupleralign verwendet wird.



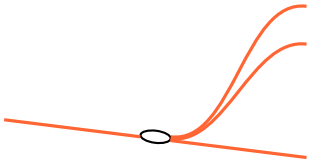

```
\begin{pspicture}(4,2)
  \pnodes(0,2){A}(0,0.5){B}(0,0){C}(4,1){D}
  \wdmcoupler(A)(B)(C)(D){coupler}
\end{pspicture}
```




```
\begin{pspicture}(4,2)
  \pnodes(0,2){E}(0,0.5){F}(0,0){G}(4,2){H}
  \wdmcoupler[coupleralign=t](E)(F)(G)(H){coupler}
\end{pspicture}
```



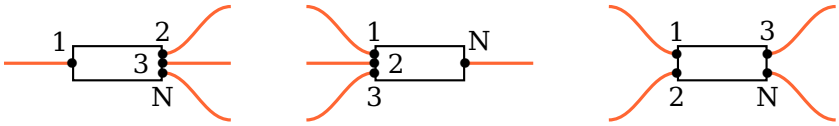

```
\begin{pspicture}(4,2)
  \pnodes(0,1){A}(4,2){B}(4,0.5){C}(4,0){D}
  \wdmsplitter(A)(B)(C)(D){splitter}
\end{pspicture}
```




```
\begin{pspicture}(4,2)
  \pnodes(0,0.5){E}(4,2){F}(4,1.5){G}(4,0){H}
  \wdmsplitter[coupleralign=b](E)(F)(G)(H){splitter}
\end{pspicture}
```

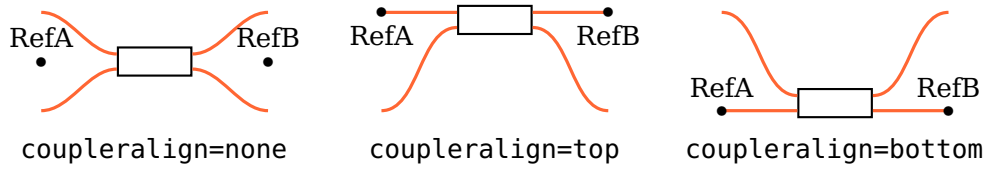
5.10.2. Eingangs- und Ausgangsknoten

Die Definition der Eingangs- und Ausgangsknoten aus Kap. 8.6 ist für Koppler nicht anwendbar. Hier werden die Knoten einfach von `1` (links oben) bis `N` (rechts unten) durchnummeriert.



5.10.3. Referenzknoten

Die Definition der Referenzknoten aus Kap. 8.2 ist für Koppler nicht anwendbar und hängt von `coupleralign` ab.



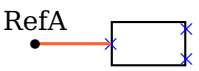
5.11. Faserbox

\fiberbox(⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}

\fiberbox(⟨*tl*⟩)(⟨*bl*⟩)(⟨*tr*⟩)(⟨*br*⟩){⟨*label*⟩}

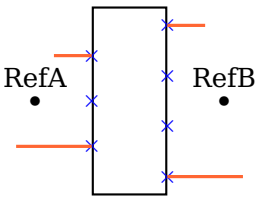
Eine \fiberbox kann entweder mit zwei, oder mit vier Knoten positioniert werden. Die Wahl beeinflusst die Position der Komponente und der Komponentenknoten und die automatischen Faserverbindungen.

Werden nur zwei Knoten verwendet, dann sind das die Referenzknoten und der Abstand zwischen zwei benachbarten Eingangs- oder Ausgangsknoten muss manuell angegeben werden (siehe fiberboxsepin und fiberboxsepout). Nur die Seiten der Komponente wird automatische mit Fasern verbunden, die nur einen einzigen Knoten hat (siehe fiberboxcount und das folgende Beispiel).




```
\begin{pspicture}(3,2)
  \fiberbox[fiberboxcount=1x2, showifcnodes](0,1)(3,1)
\end{pspicture}
```

Werden vier Knoten verwendet, dann wird die Lage der Komponentenknoten und deren Abstand automatisch bestimmt. Die Komponente wird zwischen der Mitte der beiden Eingangsknoten und der Mitte der beiden Ausgangsknoten positioniert. Der erste Eingangsknoten wird an dem ⟨*tl*⟩-Knoten ausgerichtet, der letzte Eingangsknoten an dem ⟨*bl*⟩-Knoten. Die übrigen Eingangsknoten werden dazwischen gleichverteilt. Die Ausgangsknoten werden entsprechend behandelt. Die vier angegebenen Knoten werden automatisch mit den jeweils zugeordneten Komponentenknoten verbunden.



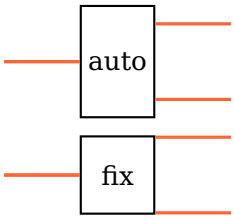

```
\begin{pspicture}(3,2)
  \fiberbox[fiberboxcount=3x4,
      showifcnodes](0.5,1.6)(0,0.4)(2.5,2)(3,0)
\end{pspicture}
```

fiberboxwidth=⟨*num*⟩ default: 1

Die Breite der Faserbox.

fiberboxheight=⟨*num*⟩ default: 0

Die Höhe der Faserbox. Ist die Höhe kleiner als der Platz der von den Eingangs- oder Ausgangsknoten benötigt wird, dann wird die Höhe automatisch berechnet. Die berechnete Höhe ist $\max(\text{sepin} \times (\text{N}-1), \text{sepout} \times (\text{M}-1))$.




```
\begin{pspicture}(3,2.8)
  \psset[optexp]{innerlabel, fiberboxwidth=1, fiberboxcount=1x2}
  \fiberbox(0,2)(0,2)(3,2.5)(3,1.5){auto}
  \fiberbox[fiberboxheight=1.05](0,0.5)(0,0.5)(3,1)(3,0){fix}
\end{pspicture}
```

fiberboxsize=⟨*width*⟩ ⟨*height*⟩

Die Breite und Höhe der Faserbox, ist äquivalent zum Aufruf von fiberboxwidth und fiberboxheight.

fiberboxsepin=⟨*num*⟩ default: 0.2

Der Abstand zwischen zwei Eingangsknoten. Dieser Wert wird nur verwendet, falls die Faserbox mit zwei Knoten positioniert wird. Ansonsten wird der Abstand automatisch ermittelt wird.

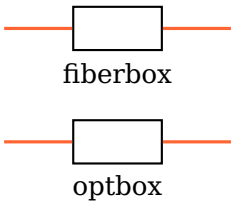
fiberboxsepout=⟨*num*⟩ default: 0.2

Analog zu fiberboxsepin, aber für die Ausgangsknoten.

fiberboxcount=⟨*N*⟩x⟨*M*⟩ default: 2x2

Anzahl der Eingangs- (⟨*N*⟩) und Ausgangsknoten (⟨*M*⟩).

Für bestimmte Parameterkombinationen ist eine \fiberbox äquivalent zu einer \optbox.

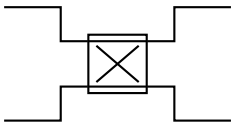



```
\begin{pspicture}(0,0.3)(3,2.6)
  \psset[optexp]{label=0.6}
  \fiberbox[fiberboxsize=1.2 0.6,
      fiberboxcount=1x1](0,2.5)(3,2.5){fiberbox}
  \optbox[optboxsize=1.2 0.6, fiber](0,1)(3,1){optbox}
\end{pspicture}
```

6. Elektrische Komponenten

6.1. Koppler

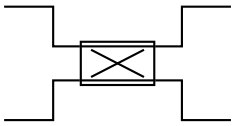
\eleccoupler(⟨*tl*⟩)(⟨*bl*⟩)(⟨*tr*⟩)(⟨*br*⟩){⟨*label*⟩}



```
\begin{pspicture}(3,1.5)
  \eleccoupler(0,1.5)(0,0)(3,1.5)(3,0)
\end{pspicture}
```

eleccouplersize=⟨*size*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: 0.8

Die Breite und Höhe des Kopplers, wird nur ein Wert angegeben dann ist die Komponente quadratisch.



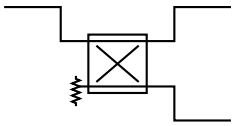
```
\begin{pspicture}(3,1.5)
  \eleccoupler[eleccouplersize=1 0.6](0,1.5)(0,0)(3,1.5)(3,0)
\end{pspicture}
```

eleccouplersep=⟨*num*⟩ default: {}

Der vertikale Abstand zwischen den beiden Eingangs- bzw. Ausgangsanschlüssen. Ist der Wert leer, bzw. gleich {}, dann wird der Abstand auf 75% der Höhe gesetzt.

eleccouplertype=standard, directional default: standard

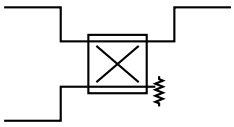
Wählt zwischen unterschiedlichen Kopplertypen.



```
\begin{pspicture}(3,1.5)
  \eleccoupler[eleccouplertype=directional](0,1.5)(0,0)(3,1.5)(3,0)
\end{pspicture}
```

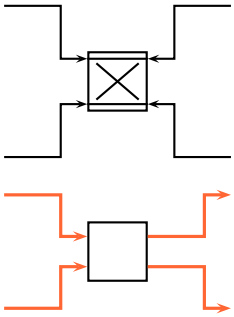
eleccouplerinput=left, right default: left

Gibt beim direktionalen Koppler die Eingangsseite an.



```
\begin{pspicture}(3,1.5)
  \eleccoupler[eleccouplertype=directional,
      eleccouplerinput=right](0,1.5)(0,0)(3,1.5)(3,0)
\end{pspicture}
```

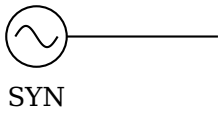
Die automatischen Verbindungen des elektrischen Kopplers sind, anders als beim optischen Koppler, alle auf die Komponente gerichtet. Das ergibt ein symmetrisches Layout der automatischen Verbindungen des elektrischen Kopplers bei Verwendung der Vorgabe `wirestyle=angle`.



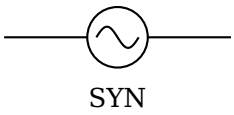
```
\begin{pspicture}(3,4)
  \addtopsstyle{Wire}{arrows=->}
  \addtopsstyle{Fiber}{arrows=->, fiberstyle=angle}
  \eleccoupler(0,4)(0,2)(3,4)(3,2)
  \optcoupler[couplersize=0.8 0.8, couplersep={},
      couplertype=rectangle](0,1.5)(0,0)(3,1.5)(3,0)
\end{pspicture}
```

6.2. Synthesizer

`\elecsynthesizer(`⟨*in*⟩`)(`⟨*out*⟩`){`⟨*label*⟩`}`




```
\begin{pspicture}(3,1.5)
  \elecsynthesizer(1,1)(3,1){SYN}
\end{pspicture}
```



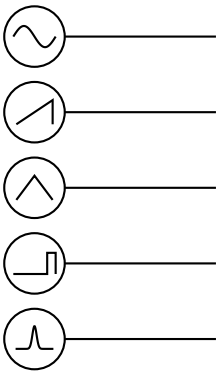

```
\begin{pspicture}(3,1.5)
  \elecsynthesizer[position=0.5](0,1)(3,1){SYN}
\end{pspicture}
```

`synthsize`=⟨*size*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.8`

Die Größe der Komponente, ist diese rund und wurden zwei Werte angegeben, dann wird der kleinere als Durchmesser verwendet.

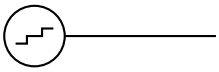
`synthtype`=`sine`, `pulse`, `sawtooth`, `rectangle`, `triangle`, `custom` default: `sine`

Wählt zwischen unterschiedlichen Typen.



```
\begin{pspicture}(0,0.6)(3,5.4)
  \elecsynthesizer[synthtype=sine](1,5)(3,5)
  \elecsynthesizer[synthtype=sawtooth](1,4)(3,4)
  \elecsynthesizer[synthtype=triangle](1,3)(3,3)
  \elecsynthesizer[synthtype=rectangle](1,2)(3,2)
  \elecsynthesizer[synthtype=pulse](1,1)(3,1)
\end{pspicture}
```

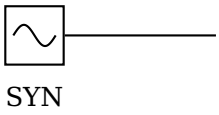
Es kann auch eine frei wählbare Wellenform gewählt werden.



```
\begin{pspicture}(0,0)(3,2)
  \makeatletter
  \def\elecsynthesizer@custom{%
    \psline(-0.25,-0.1)(-0.1,-0.1)(-0.1,0)%
          (0.1,0)(0.1,0.1)(0.25,0.1)}
  \makeatother
  \elecsynthesizer[synthtype=custom](1,1)(3,1)
\end{pspicture}
```

`synthshape`=`circle`, `rectangle` default: `circle`

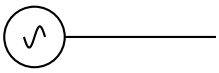
Wählt zwischen kreisförmiger und rechteckiger Außenform.




```
\begin{pspicture}(3,1.5)
  \elecsynthesizer[synthshape=rectangle](1,1)(3,1){SYN}
\end{pspicture}
```

`SynthStyle` ⟨*psstyle*⟩

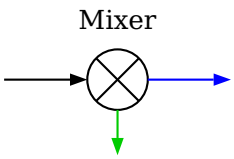
Beinflusst das Aussehen der internen Symbole.



```
\begin{pspicture}(3,1)
  \newpsstyle{SynthStyle}{xunit=0.5, yunit=-1}
  \elecsynthesizer(1,0.5)(3,0.5)
\end{pspicture}
```

6.3. Mixer

`\elecmixer`(⟨*left*⟩)(⟨*right*⟩)(⟨*bottom*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \newpsstyle{Wire}{arrowscale=1.5, arrowinset=0, arrows=->}
  \addtopsstyle{WireOut1}{linecolor=blue}
  \addtopsstyle{WireOut2}{linecolor=green!80!black}
  \elecmixer(0,1)(3,1)(1.5,0){Mixer}
\end{pspicture}
```

`elecmixersize`=⟨*num*⟩ default: `0.8`

Der Durchmesser des Mixers.

Weitere Details bezüglich der Referenzknoten und Positionierung finden Sie unter `\optcirculator`.

7. Hybridkomponenten

Dieses Kapitel beschreibt die Komponenten, die sowohl für Freistrahl- als auch Faseroptik zu verwenden sind, sich von diesen aber etwas abgrenzen.

Der `\optfilter` wird mit den Voreinstellungen zwar als Faserkomponente gehandhabt, kann jedoch im Gegensatz zu den reinen Faserkomponenten (Kap. 5) auch als Freistrahlkomponente verwendet werden. Der `\fibercollimator` hat nur eine Faser, die andere Verbindung ist Freistrahl und der `\optdetector` hat eine Freistrahl- und eine elektrische Verbindung.

7.1. Optischer Filter

`\optfilter[`⟨*options*⟩`](`⟨*in*⟩`)(`⟨*out*⟩`){`⟨*label*⟩`}`

bandpass

```
\begin{pspicture}(3,1.5)
  \optfilter(0,1)(3,1){bandpass}
\end{pspicture}
```

`filtersize=`⟨*num*⟩ default: `0.8`

Die Größe des Filters.

`filtertype=bandpass, bandstop, lowpass, highpass` default: `bandpass`

Wähle zwischen unterschiedlichen Filtertypen.

bandstop

```
\begin{pspicture}(3,1.5)
  \optfilter[filtertype=bandstop](0,1)(3,1){bandstop}
\end{pspicture}
```

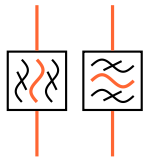
lowpass

```
\begin{pspicture}(3,1.5)
  \optfilter[filtertype=lowpass](0,1)(3,1){lowpass}
\end{pspicture}
```

```
\begin{pspicture}(3,1.5)
  \optfilter[filtertype=highpass](0,1)(3,1){highpass}
\end{pspicture}
```

`filterangle`=⟨*num*⟩ default: `0`

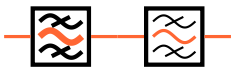
Dreht das „Innere“ des Filters relativ zum seinem Rahmen. Anstattdessn kann auch `innercompalign` verwendet werden.



```
\begin{pspicture}(2,2)
  \optfilter(0.5,0)(0.5,2)
  \optfilter[filterangle=90](1.5,0)(1.5,2)
\end{pspicture}
```

`FilterStyle` ⟨*psstyle*⟩

Beinflusst das Aussehen der internen Filterlinien.



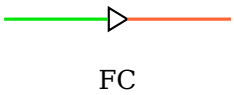
```
\begin{pspicture}(3,1.5)
  \newpsstyle{OptComp}{linewidth=2\pslinewidth}
  \optfilter(0,1)(1.5,1)
  \newpsstyle{FilterStyle}{linewidth=0.5\pslinewidth}
  \optfilter(1.5,1)(3,1)
\end{pspicture}
```

Verwendung als Freistrahlkomponenten, `allowbeaminside` ist auf `false` voreingestellt:

```
\begin{pspicture}(3,1.5)
  \optfilter[fiber=none](0,1)(3,1)
  \addtopsstyle{Beam}{beamwidth=0.3,
     fillstyle=solid, fillcolor=green, opacity=0.2}
  \drawwidebeam(0,1){}(3,1)
\end{pspicture}
```

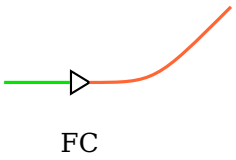
7.2. Faserkollimator

`\fibercollimator`(⟨*in*⟩)(⟨*A*⟩)(⟨*B*⟩)(⟨*out*⟩){⟨*label*⟩}



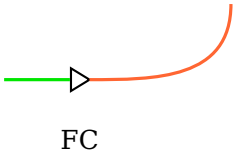

```
\begin{pspicture}(3,1.5)
  \fibercollimator[beam](0,1)(3,1){FC}
\end{pspicture}
```

Der Faserkollimator kann mit zwei, drei oder vier Punkten verwendet werden. Bei zwei Punkten wird der Kollimator wie jeder andere Zweipol zwischen ⟨*in*⟩ und ⟨*out*⟩ Knoten platziert. Bei drei Punkten wird eine `\psbezier` Kurve gezeichnet, wobei der mittlere Punkt doppelt verwendet wird. Die Positionierungsparameter (siehe Kap. 3.2) können verwendet werden um den Kollimator zwischen dem ⟨*in*⟩ und ⟨*A*⟩ Knoten auszurichten.




```
\begin{pspicture}(3,2)
  \fibercollimator[beam](0,1)(2,1)(3,2){FC}
\end{pspicture}
```

Bei vier Knoten wird eine `\psbezier` Kurve mit allen vier Knoten gezeichnet. Die Positionierungsparameter (siehe Kap. 3.2) können verwendet werden um den Kollimator zwischen den ersten beiden Knoten (⟨*in*⟩ und ⟨*A*⟩) zu verschieben.



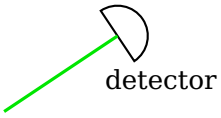

```
\begin{pspicture}(3,2)
  \fibercollimator[beam](0,1)(2,1)(3,1)(3,2){FC}
\end{pspicture}
```

`fibercolsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.3`

Eine einzelne Zahl gibt die Seitenlänge des Kollimators an, zwei Zahlen die Breite und Höhe. Beachten Sie, dass `fibercolsize=1` und `fibercolsize=1 1` nicht dasselbe Ergebnis liefern. Alternativ kann das Verhältnis zwischen Höhe und Breite mit `xunit` und `yunit` geändert werden.

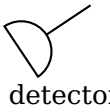
7.3. Detektor

`\optdetector`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.2)
  \optdetector[beam](0,0)(1.5,1){detector}
\end{pspicture}
```

Der Detektor wird in der Voreinstellung ans Ende der Referenzlinie platziert (`position`=`end`). Wird dieser Parameter überschrieben, dann wird die ausgehende Verbindung wie bei einer elektrischen Komponente behandelt.



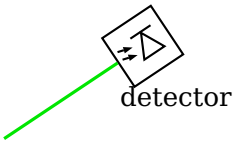

```
\begin{pspicture}(3,1.2)
  \optdetector[position=0.5](0,0)(1.5,1){detector}
\end{pspicture}
```

`detsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.8`

Wenn eine einzelne Zahl angegeben ist, dann ist das die Seitenlänge (`diode`) bzw. der Durchmesser (`round`) des Detektors. Werden zwei Zahlen angegeben, so sind das Breite und Höhe des Detektors. Beachten Sie, dass `detsize=1` und `detsize=1 1` für `dettype=round` nicht dasselbe ist.

`dettype`=`round`, `diode` default: `round`

Dieser Parameter bestimmt den Typ und damit das Aussehen des Detektors.




```
\begin{pspicture}(3,1.5)
  \optdetector[beam, dettype=diode](0,0)(1.5,1){detector}
\end{pspicture}
```

`DetectorStyle` ⟨*psstyle*⟩

Der Stil der Diode für `dettype=diode`.

```
\begin{pspicture}(3,1.5)
  \newpsstyle{OptComp}{linewidth=3\pslinewidth}
  \newpsstyle{DetectorStyle}{linewidth=0.333\pslinewidth}
  \optdetector[dettype=diode](0,1)(2,1)
\end{pspicture}
```

8. Spezielle Knoten

Jedes `pst-optexp`-Objekt stellt mehrere spezielle Knoten zur Verfügung, die mit dessen Geometrie und Positionierung zusammenhängen. Diese Knoten stehen für weitere Verwendung zur Verfügung.

Sie sollten immer die dafür vorgesehenen Makros verwenden um auf die Knotennamen zuzugreifen.

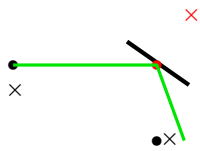
`\oenode`{⟨*node*⟩}{⟨*comp*⟩}

Das ist das grundlegende Makro mit dem auf die Knotennamen einer Komponente zugegriffen werden kann. Das erste Argument ⟨*node*⟩ ist der Bezeichner des angeforderten Knotens. Das zweite Argument ⟨*comp*⟩ ist der Name der Komponente (gemäß Kap. 8.1). Ist dieses leer so wird das zuletzt definierte Objekt verwendet.

Für viele der Knoten wird ein eigenes Makro bereitgestellt, das Sie dann auch verwenden sollten, da sich die Namenskonventionen ändern könnten. Die Makros stellen sicher, dass Sie immer die passenden Knotennamen erhalten. Daher sind die verfügbaren Bezeichner auch nicht aufgelistet.

`showoptdots`=`true`, `false` default: `false`

Markiert einige der speziellen Komponentenknoten: die schwarzen Punkte sind die normalen und die schwarzen Kreuze die transformierten Referenzknoten (Kap. 8.2), der rote Punkt ist der Mittelpunktknoten (Kap. 8.3) und das rote Kreuz der Beschriftungsknoten (Kap. 8.4).



```
\begin{pspicture}(3,2)
\mirror[showoptdots, angle=10, beam](0,1)(1.9,1)(1.9,0)
\end{pspicture}
```

8.1. Komponenten-Bezeichner

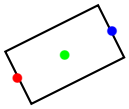
Alle Komponenten einer Aufbauskizze werden in Reihenfolge ihrer Definition im Code aufsteigend nummeriert, angefangen bei 1. Die Komponenten und deren Spezialknoten können immer über diese Nummer (ID) referenziert werden.

IDs können auch relativ zur aktuellen ID über die Syntax <⟨*num*⟩ angegeben werden: <1 ist die vorletzte ID, <2 die vorvorletzte ID usw. Die vorletzte ID kann abgekürzt auch mit < erhalten werden, und die aktuelle ID mit <0, was äquivalent zu einem leeren Parameter ist.

`compname`=⟨*string*⟩

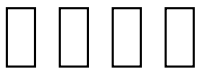
Weist einer Komponente einen Bezeichner zu. Die Komponente kann nun sowohl über diesen Bezeichner als auch über die ID referenziert werden. Dieser Parameter kann nur im optionalen Argument einer `pst-optexp` Komponente verwendet werden. Der Bezeichner sollte innerhalb einer `pspicture`-Umgebung eindeutig sein.

Beispiele für unterschiedliche Methoden auf ein und dieselbe Komponente zuzugreifen.



```
\begin{pspicture}(2,2)
  \optbox[compname=MyBox](0,1)(2,2)
  \psdot[linecolor=red](\oenodeIn{MyBox}) % Verwende den Namen
  \psdot[linecolor=blue](\oenodeOut{1}) % Verwende die ID
  \psdot[linecolor=green](\oenodeCenter{})% Nimm die letzte Komponente
\end{pspicture}
```

Die Verwendung von relativen IDs ist u.a. nützlich, um eine neue Komponente relativ zur vorangegangenen zu definieren. Die Verwendung von z.B. `\oenodeOut{}` in den Referenzknoten ist nicht möglich, da hiermit zum Zeitpunkt der Auswertung der Koordinaten schon auf die neue, nicht fertige Komponente referenziert wird. Ein Aufruf der Art `\optbox(\oenodeOut{})(B)` ist daher nicht möglich, und führt zu einem Fehler. Mit relativen IDs kann das allerdings mit `\optbox(\oenodeOut{<})(B)` erreicht werden:



```
\begin{pspicture}(4,2)
  \psset{optboxsize=0.4 0.8, abspos=0.5}
  \optbox(0,1)(4,1)
  \multido{}{3}{\optbox(\oenodeOut{<})(4,1)}
\end{pspicture}
```

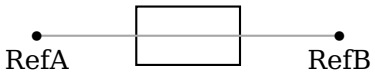
8.2. Referenzknoten

\oenodeRefA{⟨comp⟩}

\oenodeRefB{⟨comp⟩}

Die Eingangs- und Ausgangs-Referenzknoten.

Das sind die ursprünglichen Knoten, die für die Positionierung der Komponente benutzt wurden, \oenodeRefA ist der erste und \oenodeRefB der letzte Knoten. Diese Zuordnung ist nicht gültig für Faserkoppler (5.10.3) und Zirkulatoren (5.9).



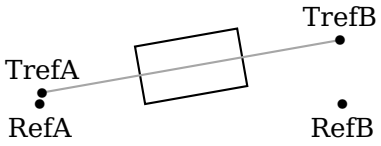

```
\begin{pspicture}(5,1)
  \pnodes(0.5,0.5){A}(4.5,0.5){B}
  \optbox(A)(B)
  \psline[style=Refline](\oenodeRefA{})(\oenodeRefB{})
  \psdot(\oenodeRefA{})\uput[-90](\oenodeRefA{}){RefA}
  \psdot(\oenodeRefB{})\uput[-90](\oenodeRefB{}){RefB}
\end{pspicture}
```

\oenodeTrefA{⟨comp⟩}

\oenodeTrefB{⟨comp⟩}

Die transformierten Eingangs- und Ausgangs-Referenzknoten.

Das sind die Referenzknoten nachdem sie zusammen mit der Komponente gemäß der compshift und angle Parameter transformiert wurden.

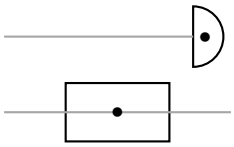



```
\begin{pspicture}(5,1.8)
  \pnodes(0.5,0.5){A}(4.5,0.5){B}
  \optbox[compshift=0.5, angle=10](A)(B)
  \psdot(\oenodeRefA{})\uput[-90](\oenodeRefA{}){RefA}
  \psdot(\oenodeRefB{})\uput[-90](\oenodeRefB{}){RefB}
  \psdot(\oenodeTrefA{})\uput[90](\oenodeTrefA{}){TrefA}
  \psdot(\oenodeTrefB{})\uput[90](\oenodeTrefB{}){TrefB}
\end{pspicture}
```

8.3. Mittelpunktknoten

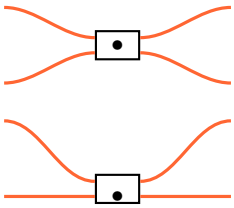
\oenodeCenter{⟨comp⟩}

Dieser Knoten liegt, bis auf wenige Ausnahmen, wie z.B. \optdetector und \optcoupler (siehe unten), im Mittelpunkt der Komponente.



```
\begin{pspicture}(3,2)
  \optbox(0,0.5)(3,0.5)
  \optdetector(0,1.5)(2.5,1.5)
\end{pspicture}
```

Bei \optcoupler hängt die Position der Komponentenmitte von den coupleralign Einstellungen ab.

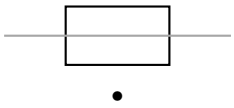


```
\begin{pspicture}(0,0.3)(3,3)
  \psset[optexp]{couplersize=0.6 0.4, couplersep=0.2,
     couplertype=rectangle}
  \optcoupler(0,3)(0,2)(3,3)(3,2)
  \psdot(\oenodeCenter{})
  \optcoupler[coupleralign=b](0,1.5)(0,0.5)(3,1.5)(3,0.5)
  \psdot(\oenodeCenter{})
\end{pspicture}
```

8.4. Beschriftungsknoten

\oenodeLabel{⟨comp⟩}

Auf diesen Knoten wird die Beschriftung platziert. Der Knoten ist auch verfügbar wenn keine Beschriftung angegeben wurde. Für labeloffset=0 ist dieser Knoten identisch mit dem Mittelpunktsknoten (Kap. 8.3).



```
\begin{pspicture}(3,1.5)
  \optbox(0,1)(3,1)
\end{pspicture}
```

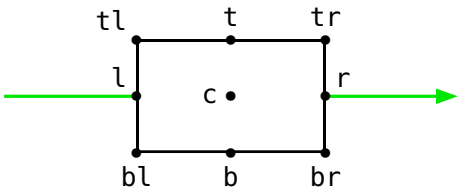
8.5. Externe Knoten

`\oenodeExt{`⟨*comp*⟩`}`

Ein externer Knoten kann an unterschiedliche Positionen entlang des Komponentenrandes gesetzt werden. Dieser wird nur für den externen Zugriff definiert, und beeinflusst in keiner Weise die Komponente.

`extnode=`⟨*refpoint*⟩

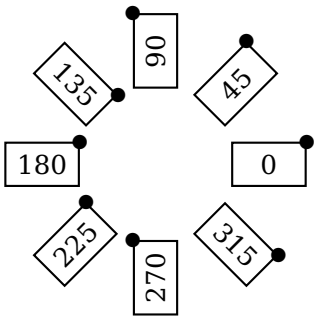
Bestimmt die Position des externen Knotens. Analog zum Referenzpunkt von `\rput`, kann jede Kombination von `c` (mittig), `t` (oben), `b` (unten), `l` (links) und `r` (rechts) sein. Nicht jede Komponente unterstützt alle möglichen Kombinationen, die möglichen Positionen jeder Komponente sind in Kap. 12.2.1 zusammengefasst.



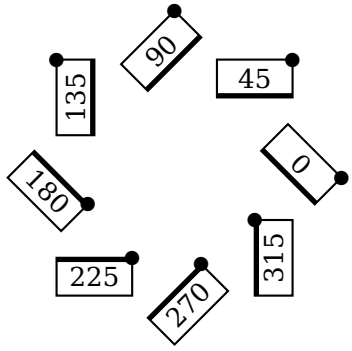
`extnodealign=rel, relative, abs, absolute` default: `abs`

Die Bezeichnung von „oben“ (`t`) und den anderen Positionierungsparametern von `extnode` können absolut oder relativ zur Komponente betrachtet werden.

In dem folgenden Beispiel ist der externe Knoten immer „oben rechts“ (`tr`) platziert, unabhängig von der Reihenfolge der Referenzknoten (`extnodealign=abs`). Das Verhalten ist identisch zu `labelref=relative` für die Ausrichtung der Beschriftung.

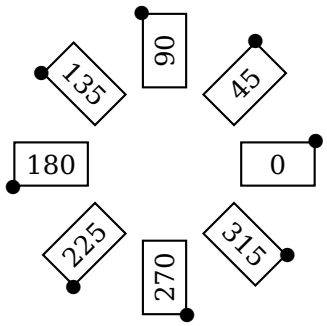



```
\begin{pspicture}(-2,-2)(2,2)
  \psset{endbox, optboxsize=1 0.6, dotscale=1.5}
  \psset{extnodealign=abs, extnode=tr}
  \multido{\i=0+45}{8}{%
    \optbox[innerlabel](0,0)(1;\i){\i}
    \psdot(\oenodeExt{})
  }%
\end{pspicture}
```

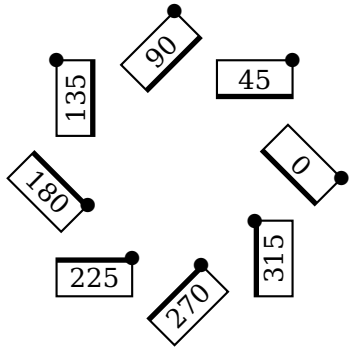



```
\begin{pspicture}(-2.2,-2.2)(2.2,2.2)
  \psset{dotscale=1.5, mirrortype=extended,
      mirrordepth=0.5, label=0.25 . . relative,
      extnodealign=abs, extnode=tr}
  \newpsstyle{ExtendedMirror}{}
  \multido{\i=0+45,\ii=-45+45}{8}{%
    \mirror(0,0)(1.5;\i)(2.1212;\ii){\i}
    \psdot(\oenodeExt{})
  }%
\end{pspicture}
```

In dem darauffolgenden Beispiel bezieht sich die Positionierung relativ zur Verbindung zwischen Eingangs- und Ausgangsreferenzknoten (`extnodealign=rel`).



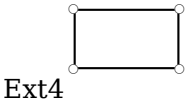

```
\begin{pspicture}(-2,-2)(2,2)
  \psset{endbox, optboxsize=1 0.6, dotscale=1.5}
  \psset{extnodealign=rel, extnode=tr}
  \multido{\i=0+45}{8}{%
    \optbox[innerlabel](0,0)(1;\i){\i}
    \psdot(\oenodeExt{})
  }%
\end{pspicture}
```




```
\begin{pspicture}(-2.2,-2.2)(2.2,2.2)
  \psset{dotscale=1.5, mirrortype=extended,
      mirrordepth=0.5, label=0.25 . . relative,
      extnodealign=abs, extnode=tr}
  \newpsstyle{ExtendedMirror}{}
  \multido{\i=0+45,\ii=-45+45}{8}{%
    \mirror(0,0)(1.5;\i)(2.1212;\ii){\i}
    \psdot(\oenodeExt{})
  }%
\end{pspicture}
```

`extnodes`=⟨*list*⟩

Erlaubt die Definition mehrerer externer Knoten, ⟨*list*⟩ ist eine Liste von ⟨*refpoint*⟩, wie sie von `extnode` verwendet werden. Der erste Knoten kann wie gewohnt mit `\oenodeExt` angesprochen werden (ist äquivalent zu `\oenode{Ext}` und `\oenode{Ext1}`). Für alle weiteren Knoten muss `\oenode` mit den Knotenbezeichnungen `Ext`⟨*num*⟩ verwendet werden: `\oenode{Ext1}` für den ersten, `\oenode{Ext2}` für den zweiten usw.




```
\begin{pspicture}(3,2)
  \optbox[extnodes={tl,tr,br,bl}](0,1)(3,1)
  \multido{\i=1+1}{4}{\psdot[dotstyle=o](\oenode{Ext\i}{})}%
\end{pspicture}
```

8.6. Grenzflächenknoten

`\oenodeIfc{`⟨*num*⟩`}{`⟨*comp*⟩`}`

Der Grenzflächenknoten ⟨*num*⟩, wobei ⟨*num*⟩ eine positive Ganzzahl oder der Buchstabe `N` ist. Der letzte Knoten ist immer `N`, kann aber auch über die entsprechende Zahl angesprochen werden.

`\oenodeIn{`⟨*comp*⟩`}`

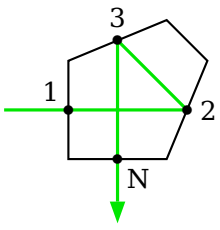
Der Knoten ⟨*1*⟩ ist der Eingangsknoten und sollte immer über `\oenodeIn` angesprochen werden.

`\oenodeOut{`⟨*comp*⟩`}`

Der Knoten `N` ist der Ausgangsknoten und sollte immer über `\oenodeOut` angesprochen werden.

„Eingang“ und „Ausgang“ können nur anhand einer relativen Orientierung definiert werden. Per Definition ist der Eingangsknoten derjenige zu dem ein Lichtstrahl ausgehend vom Referenzknoten `\oenodeRefA` verläuft. Analog dazu ist die Definition ds Ausgangsknotens. Diese Bezeichnung kann nur für Faserkoppler (siehe Kap. 5.10.2) und Strahlteiler (4.20) nicht angewendet werden.

`\oenodeIn` ist äquivalent zu `\oenodeIfc{1}` und `\oenodeOut` ist äquivalent zu `\oenodeIfc{N}`. Die Eingangs- und Ausgangsknoten sollten über die explizit bereitgestellten Makros angesprochen werden, `\oenodeIfc` sollte nur für die weiteren Grenzflächenknoten, sofern vorhanden, verwendet werden.




```
\begin{pspicture}(2.8,3)
  \pentaprism[pentaprismsize=1.3](0,1.5)(1.5,1.5)(1.5,0)
  \drawbeam[arrows=->, arrowinset=0, arrowscale=1.5](0,1.5){}(1.5,0)
  \psdot(\oenodeIn{})\uput[135](\oenodeIn{}){1}
  \psdot(\oenodeIfc{2}{})\uput[0](\oenodeIfc{2}{}){2}
  \psdot(\oenodeIfc{3}{})\uput[90](\oenodeIfc{3}{}){3}
  \psdot(\oenodeOut{})\uput[-45](\oenodeOut{}){N}
\end{pspicture}
```

In Kap. 12.2.2 finden Sie eine vollständige Liste aller Komponenten mit ihren Grenzflächenknoten.

8.7. Referenzknoten für die Rotation

`\oenodeRotref`{⟨*comp*⟩}

Der Referenzknoten um den eine Komponente mit `angle` gedreht wird. Die Position des Knotens wird mit dem `rotateref` Parameter definiert und kann dieselben Werte annehmen wie `extnode`. Eine vollständige Liste der möglichen Positionen finden Sie in Kap. 12.2.1.

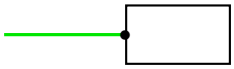
```
\begin{pspicture}(0,0.1)(12,1.8)
  \optbox[angle=20](0.5,1)(4.5,1)
  \psdot(\oenodeRotref{})\uput[-90](\oenodeRotref{}){Rotref}
  \optbox[angle=20, rotateref=bl](7.5,1)(11.5,1)
  \psdot(\oenodeRotref{})\uput[-90](\oenodeRotref{}){Rotref}
\end{pspicture}
```

8.8. Strahlknoten

Die Endknoten eines `\drawbeam` oder `\drawwidebeam` Kommandos können wieder verwendet werden, wenn `savebeampoints` gesetzt ist.

`\oenodeBeam`{⟨*num*⟩}

Damit wird der Endknoten eines vorangegangenen `\drawbeam` Kommandos angesprochen. Wird ⟨*num*⟩ leer gelassen so wird 1 genommen, was in der Regel dem letzten Strahlengang entspricht. Auf welchen Strahl genau Bezug genommen wird, hängt i.a. von den gewählten Einstellungen von `savebeampoints` und entsprechender Wahl von ⟨*num*⟩ ab.

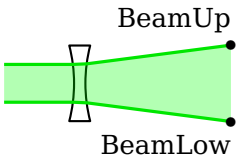


```
\begin{pspicture}(0,0)(3,2)
  \pnodes(0,1){A}(1.6,1){B}
  \optbox[position=end](A)(B)
  \drawbeam[beaminsidelast=false](A){1}
  \psdot(\oenodeBeam{})
\end{pspicture}
```

\oenodeBeamUp{⟨*num*⟩}

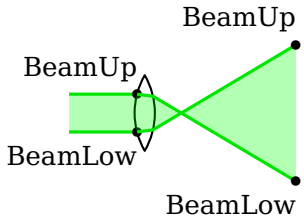
\oenodeBeamLow{⟨*num*⟩}

Damit wird der obere („upper“) oder untere („lower“) Endknoten eines vorangegangenen \drawwidebeam Kommandos angesprochen. Wird ⟨*num*⟩ leer gelassen so wird 1 genommen, womit man sich in der Regel auf den letzten Strahlengang bezieht. Auf welchen Strahl genau Bezug genommen wird, hängt i.a. von den gewählten Einstellungen von savebeampoints und entsprechender Wahl von ⟨*num*⟩ ab.



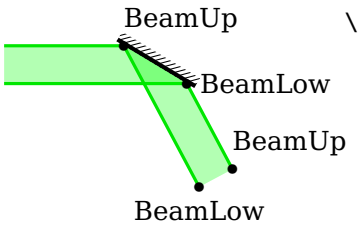

```
\begin{pspicture}(0,0)(3,2)
  \pnodes(0,1){A}(3,1){B}
  \lens[abspos=1, lensradius=-2](A)(B)
  \addtopsstyle{Beam}{%
    fillstyle=solid, fillcolor=green, opacity=0.3}
  \drawwidebeam[beamwidth=0.5](A){}(B)
\end{pspicture}
```

Beachten Sie, dass „oben“ („upper“) und „unten“ („lower“) bezüglich der Ausbreitungsrichtung des Strahls definiert ist. Ein Randstrahl der als „upper“ startet can zu „lower“ werden, wenn er den anderen Randstrahl kreuzt. Das wird in dem folgenden Beispiel erläutert:




```
\begin{pspicture}(0,0)(3,2)
  \pnodes(0,1){A}(3,1){B}
  \lens[abspos=1, n=2](A)(B)
  \addtopsstyle{Beam}{%
    fillstyle=solid, fillcolor=green, opacity=0.3}
  \drawwidebeam[beamwidth=0.5](A){}
  \drawwidebeam[loadbeam, beaminsidefirst]{}(B)
\end{pspicture}
```

Bei einer Reflektion können ebenfalls die Rollen vertauscht werden.




```
\begin{pspicture}(0,-1)(4,2)
  \pnodes(0,1.5){A}(2,1.5){B}(2.8,0){C}
  \mirror[mirrorwidth=1.2, mirrortype=extended](A)(B)(C)
  \addtopsstyle{Beam}{%
    fillstyle=solid, fillcolor=green, opacity=0.3}
  \drawwidebeam[beamwidth=0.5](A){}
  \drawwidebeam[loadbeam]{}(C)
\end{pspicture}
```

Beachten Sie, dass sich die Orientierung von „oben“ und „unten“ in Version 3.3 geändert hat, siehe Kap. 12.3.5.

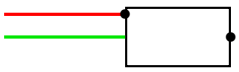
`\oeBeamCenter{`⟨*num*⟩`}`

Damit erhält man die Mitte zwischen oberem und unterem Endknoten, die Koordinaten sind nur aus Postscript-Ebene verfügbar, im Gegensatz zu `\oenodeBeam*`, daher auch die an `\oeBeamVec*` angelehnte Nomenklatur. Ein Anwendungsbeispiel ist Bsp. 8.9.

Die Strahlknoten können nur innerhalb der `pspicture`-Umgebung verwendet werden, in der sie definiert wurden. Um auf bestimmte Strahlknoten von außerhalb zugreifen zu können müssen diese vorher explizit umdefiniert werden:

```
\pnode(\oenodeBeam{}){MyBeamNode}
```

Desweiteren werden die Endknoten durch `stopinside` beeinflusst. Siehe Kap. 9.5 für weitere Details.



```
\begin{pspicture}(0,0)(3,2)
  \pnodes(0,1){A}(1.6,1){B}
  \optbox[position=end](A)(B)
  \psset{beaminsidelast=false}
  \drawbeam[linecolor=red, beampos=0.3](A){1}
  \psdot(\oenodeBeam{})
  \drawbeam[stopinside](A){1}
  \psdot(\oenodeBeam{})
\end{pspicture}
```

8.9. Strahlvektor

Zusätzlich zu den Strahlknoten (siehe Kap. 8.8) kann auf Postscript-Ebene auf die normierten Strahlvektoren der letzten gespeicherten Strahlen (siehe `savebeam`) zugegriffen werden.

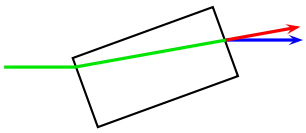
Auch hier wird zwischen Einzelstrahlen, und oberem bzw. unterem Strahl unterschieden. ⟨*num*⟩ hat dieselbe Bedeutung wie bei \oenodeBeam.

\oeBeamVec{⟨*num*⟩}

\oeBeamVecUp{⟨*num*⟩}

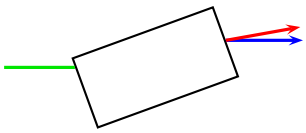
\oeBeamVecLow{⟨*num*⟩}

Bei den gespeicherten Werten handelt es sich um den normierten Richtungsvektor des letzten Strahlsegments, dieser ist nur auf Postscript-Ebene verfügbar. Dieser Richtungsvektor ist im folgenden Beispiel in rot zu sehen. Im Gegensatz dazu wird mit savebeam der Richtungsvektor gespeichert, der nach der letzten Grenzfläche gelten würde, also für einen nachfolgenden Strahl verwendet würde. Dieser ist im Beispiel in blau eingezeichnet.



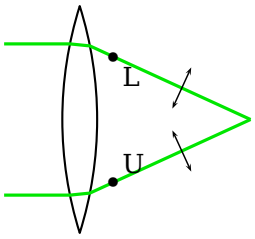
```
\begin{pspicture}(4,2)
  \optbox[optboxsize=2 1, angle=20, n=2](0,1)(4,1)
  \drawbeam[beaminsidelast, beamalign=abs](0,1){1}
  \pnode(\oenodeBeam{}){Out}
  \nodexn{(Out)+(!\oeBeamVec{})}{C}
  \drawbeam[linecolor=blue, arrows=->, loadbeam]{1}(C)
  \drawbeam[linecolor=red, arrows=->](Out)(C)
\end{pspicture}
```

Das Verhalten ist unabhängig davon, ob der letzte Strahl nur berechnet, oder auch gezeichnet wurde. Das nächste Beispiel verwendet stopinside anstelle vom Parameter beaminsidelast im vorangegangenen Beispiel; die Strahlvektoren bleiben davon unbeeinflusst.



```
\begin{pspicture}(4,2)
  \optbox[optboxsize=2 1, angle=20, n=2](0,1)(4,1)
  \drawbeam[stopinside, beamalign=abs](0,1){1}
  \pnode(\oenodeBeam{}){Out}
  \nodexn{(Out)+(!\oeBeamVec{})}{C}
  \drawbeam[linecolor=blue, arrows=->, loadbeam]{1}(C)
  \drawbeam[linecolor=red, arrows=->](Out)(C)
\end{pspicture}
```

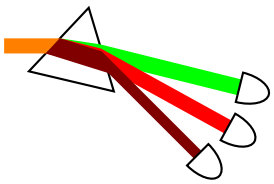
Das kann dazu genutzt werden, um z.B. die Polarisation eines Strahls zu kennzeichnen. Im folgenden Beispiel werden die neuen Knoten ⟨*U*⟩ und ⟨*L*⟩definiert, indem mit \nodexn die Endpunkte des oberen bzw. unteren Strahls entlang ihrer Trajektorie zurück verschoben werden.




```
\begin{pspicture}(3,3)
  \pnodes(0,1.5){A}(3,1.5){B}
  \lens[abspos=1, lens=5 5 3, n=2.0](A)(B)
  \optplane[angle=90](B)
  \drawwidebeam[beamwidth=2](A){1-2}
  \nodexn{(\oenodeBeamUp{})-2(!\oeBeamVecUp{})}{U}
  \nodexn{(\oenodeBeamLow{})-2(!\oeBeamVecLow{})}{L}
  \polarization(U)(\oenodeBeamUp{})
  \polarization(L)(\oenodeBeamLow{})
\end{pspicture}
```

\oeBeamVecMedian{⟨*num*⟩}Der Mittelwert aus oberem und unterem Strahlvektor.

Bsp. 8.1

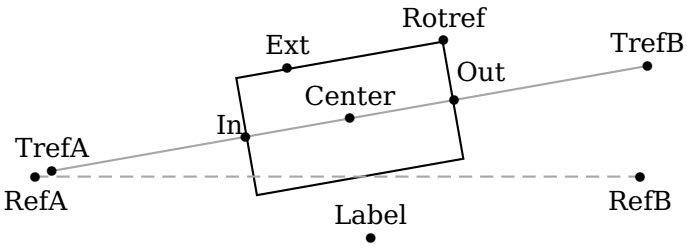


```
\begin{pspicture}(3.5,2.6)
  \pnodes(0,2){A}(1,2){B}(3,1){C}
  \optprism(A)(B)(C)
  \newpsstyle{Beam}{linestyle=none, fillstyle=solid,
      beamwidth=0.2, beamnodealign=vector}
  \drawwidebeam[fillcolor=orange](A){}
  \psset[optexp]{beampathskip=1}
  \drawwidebeam[fillcolor=green, n=1.2, savebeam=2](A){}(C)
  \drawwidebeam[fillcolor=red, n=1.4, savebeam=3](A){}(C)
  \drawwidebeam[fillcolor=red!50!black, n=1.57, savebeam=4](A){}(C)
  \multido{\i=2+1}{3}{%
    \backlayer{%
      \pnode(!\oeBeamCenter{\i}){Det\i}
      \nodexn{(Det\i)-(!\oeBeamVecMedian{\i})}{Det\i'}}
    \optdetector[detsize=0.4 0.4](Det\i')(Det\i)}
\end{pspicture}
```

Weitere Anwendungsbeispiele sind Bsp. 11 und Bsp. 11.

8.10. Knotenübersicht

Dies ist eine Übersicht über alle verfügbaren Komponentenknoten.




```
\begin{pspicture}(0.3,0.1)(4.8,1.6)
  \pnodes(0.5,0.5){A}(4.5,0.5){B}
  \optbox[compshift=0.5, angle=10, rotateref=tr, extnode={-0.5,1}](A)(B)
\end{pspicture}
```

9. Verbinden von Komponenten

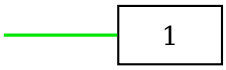
Das `pst-optexp`-Paket stellt unterschiedlichen Methoden bereit, Komponenten vollautomatisch oder manuell, mit Lichtstrahlen, Fasern oder Drähten zu verbinden.

- In Kap. 9.1 wird beschrieben wie Komponenten und Knoten für Verbindungen angesprochen werden.
- Strahlverbindungen im Allgemeinen und das Verhalten von Einzelstrahlen wird in Kap. 9.2 beschrieben.
- Kap. 9.3 erweitert diese Beschreibung von Strahlverbindungen auf ausgedehnte Strahlen und deren Eigenheiten.
- In Kap. 9.4 wird gezeigt, wie fehlerhafte Strahlverbindungen (verfehlen von Grenzflächen) gehandhabt wird).
- Kap. 9.5 erläutert, wie bestimmte, stückweise definierte Strahlengänge, wie z.B. bei eine Teleskop, einfach realisiert werden können.
- Manuelle Faserverbindungen werden in Kap. 9.6 behandelt, manuelle elektrischen Verbindungen, die sehr ähnlich funktionieren, in Kap. 9.7.
- Automatische Verbindungen werden in Kap. 9.8 erklärt.
- Kap. 9.9 erklärt das Konzept der „front“ und „back“-Ebene.

9.1. Zugriff auf Komponenten

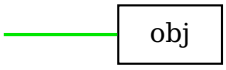
Alle Makros für Verbindungen akzeptieren entweder eine ID, ein `compname` Bezeichner (siehe Kap. 8.1) oder einen PSTricks-Knoten als Argument, die als $\langle obj_1 \rangle$, $\langle obj_2 \rangle$, ... bezeichnet werden. Um zwischen Knoten und Komponenten unterscheiden zu können, müssen Knoten entweder in runde Klammern innerhalb der geschweiften Klammern eingeschlossen werden oder dürfen nur runde Klammern haben: `\drawbeam{(node)}{comp}`, oder `\drawbeam(node){comp}`.

1. Auf alle Komponenten kann immer unter Verwendung der ID zugegriffen werden.



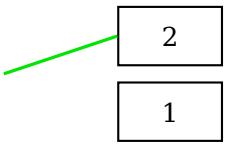

```
\begin{pspicture}(3,1.5)
  \pnodes(0,1){A}(1.5,1){B}
  \optbox[position=end, labeloffset=0](A)(B){1}
  \drawbeam(A){1}
\end{pspicture}
```

2. Wenn eine Komponente mit `compname` einen zusätzlichen Namen erhalten hat, so kann dieser anstelle der ID verwendet werden.




```
\begin{pspicture}(3,2)
  \psset{position=end, labeloffset=0}
  \optbox[compname=obj](0,1)(1.5,1){obj}
  \drawbeam(0,1){obj}
\end{pspicture}
```

3. Wird das Argument leer gelassen, dann greift man auf die zuletzt definierte Komponente zu.




```
\begin{pspicture}(3,2)
  \psset{position=end, labeloffset=0}
  \optbox(0,0.5)(1.5,0.5){1}
  \optbox(0,1.5)(1.5,1.5){2}
  \drawbeam(0,1){}
\end{pspicture}
```

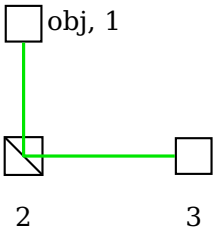
4. Es können auch Zahlintervalle verwendet werden, gültige Intervallangaben sind

   `x-y` Von x bis y, ist x größer als y, so wird heruntergezählt. Ist der gesamte Bereich nicht definiert, dann wird ein Fehler ausgegeben.

   `x-` Von x bis zur letzten Komponente.

   `-y` Von der ersten Komponente bis y. Ist y außerhalb des definierten Bereichs, dann wird eine Warnung ausgegeben.

   `-` Verbinde alle Komponenten.




```
\begin{pspicture}(3,3)
  \psset{position=end, optboxsize=0.5 0.5, bssize=0.5}
  \optbox[compname=obj](0.5,0.5)(0.5,2.5){obj, 1}
  \beamsplitter(0.5,2.5)(0.5,1)(2.5,1){2}
  \optbox(0.5,1)(2.5,1){3}
  \drawbeam{1-3}
\end{pspicture}
```

9.2. Strahlen zeichnen

Es gibt mehrere Möglichkeiten Komponenten und Knoten mit Strahlen zu verbinden: einfache Linien oder ausgedehnte Strahlen. Die meisten Parameter haben für beide Typen dieselbe Bedeutung, daher beschränken sich die entsprechenden Beispiele auf die einfachen Strahlen. Die Parameter die nur ausgedehnte Strahlen betreffen, werden in Kap. 9.3 beschrieben.

`\drawbeam`[⟨*options*⟩]{⟨obj_1⟩}{⟨obj_2⟩}...

`\drawwidebeam`[⟨*options*⟩]{⟨obj_1⟩}{⟨obj_2⟩}...

Diese beiden Makros sind für die Strahlverbindungen zuständig, sie akzeptieren eine variable Anzahl an Argumenten (mindestens zwei), die entweder eine ID, ein ID-Intervall, ein `compname` oder ein PSTricks-Knoten sein können (siehe Kap. 9.1. Um zwischen Knoten und Komponenten unterscheiden zu können, müssen Knoten entweder in runde Klammern innerhalb der geschweiften Klammern eingeschlossen werden oder dürfen nur runde Klammern haben: `\drawbeam{(node)}{comp}`, oder `\drawbeam(node){comp}`.

9.2.1. Raytracing

Der Strahlengang durch die Komponenten kann mit zwei unterschiedlichen Methoden ermittelt werden: über den Brechungsindex und das Snelliussche Brechungsgesetz, oder indem die Komponenten unabhängig von ihren optischen Eigenschaften einfach verbunden werden.

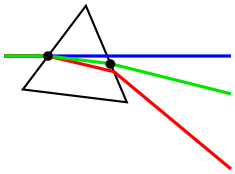
Behalten Sie immer im Hinterkopf, dass dieses Paket für *Skizzen* optischer Aufbauten gedacht ist und daher keine vollständige Umgebung zur Strahlverfolgung bereitstellt. Abweichungen vom physikalisch richtigen Weg können in vielen Fällen erwünscht sein: Winkel und Divergenzen werden häufig extremer dargestellt um bestimmte Effekte hervorzuheben. Daher sollten Sie den Brechungsindex (siehe Kap. 9.2.2) lediglich as Mittel zur Optimierung des gewünschten Lichtweges sehen und nicht zu sehr an den physikalisch richtigen Werten festhalten.

`raytrace`=`true`, `false` default: `true`

Wählt die Methode der Strahlverfolgung, mit `true` wird der Brechungsindex („raytracing“) verwendet, `false` verbindet die Komponente lediglich („connect“ Variante).

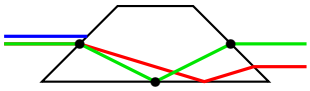
In vielen Fällen ist „connect“ identisch mit „raytracing“ für n=1, außer für \optprism oder \doveprism bei denen die Grenzflächenknoten (Kap. 8.6) dann einfach nur verbunden werden. Die folgenden beiden Beispiele zeigen die Unterschiede der beiden Methoden für diese Komponenten. Die Grenzflächenknoten sind zum besseren Verständnis hervorgehoben.

Der „raytracing“ Strahl mit n=1 (blau) wird durch die Komponente transmittiert ohne die Richtung zu verändern, für n=1.5 (rot) wird die Brechung an den Grenzflächen berechnet, und der „connect“ Strahl (grün) durchläuft genau die Grenzflächenknoten.



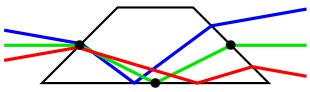
```
\begin{pspicture}(3,2.2)
  \pnodes(0,1.5){A}(1,1.5){B}(3,1){C}
  \optprism[prismsize=1.2](A)(B)(C)
  \optplane(C)
  \drawbeam[n=1, linecolor=blue](A){1}{2}
  \drawbeam[n=1.5, linecolor=red](A){1}{2}
  \drawbeam[raytrace=false, linecolor=DGreen](A){1}{2}
\end{pspicture}
```

Das \doveprism wird im Prinzip genauso behandelt wie das \optprism, der „raytracing“ Strahl mit n=1 (blau) trifft wegen fehlender Brechung die zweite Grenzfläche gar nicht, und endet daher schon an der ersten Grenzflächen (siehe Kap. 9.4). Der grüne „connect“ Strahl verläuft wieder durch die Grenzflächenknoten.

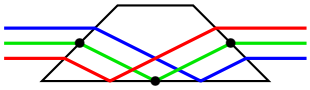


```
\begin{pspicture}(4,2)
  \pnodes(0,1){A}(4,1){B}
  \doveprism[doveprismsize=1](A)(B)
  \drawbeam[n=1, linecolor=blue, beampos=0.1](A){1}(B)
  \drawbeam[n=1.5, linecolor=red](A){1}(B)
  \drawbeam[raytrace=false, linecolor=DGreen](A){1}(B)
\end{pspicture}
```

Für „connect“ können auch unterschiedliche Anfangsbedingungen (9.2.3) verwendet werden.



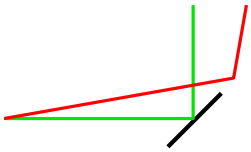
```
\begin{pspicture}(3.7,2)
  \pnodes(0,1){A}(4,1){B}
  \doveprism[doveprismsize=1](A)(B)
  \psset[optexp]{raytrace=false}
  \drawbeam[linecolor=blue, beampos=0.2, beamangle=-10](A){1}(B)
  \drawbeam[raytrace=false, linecolor=DGreen](A){1}(B)
  \drawbeam[linecolor=red, beampos=-0.2, beamangle=10](A){1}(B)
\end{pspicture}
```



```
\begin{pspicture}(3.7,2)
  \pnodes(0,1){A}(4,1){B}
  \doveprism[doveprismsize=1](A)(B)
  \psset[optexp]{raytrace=false}
  \drawbeam[linecolor=blue, beampos=0.2](A){1}(B)
  \drawbeam[linecolor=DGreen](A){1}(B)
  \drawbeam[linecolor=red, beampos=-0.2](A){1}(B)
\end{pspicture}
```

useNA=true, false default: true

Ein Strahlengang wird abgebrochen, falls eine Komponente nicht getroffen wird. Für manche Zwecke, z.B. zum Ermitteln des richtigen Brechungsindex, kann es sinnvoll sein, wenn die numerische Apertur eine Komponente nicht berücksichtigt wird. In dem folgenden Beispiel wird der rote Strahl gezeichnet, obwohl er außerhalb der gezeichneten Spiegelfläche liegt.



```
\begin{pspicture}(3,2)
  \pnodes(0,0.5){A}(2.5,0.5){B}(2.5,2){C}
  \mirror(A)(B)(C)
  \psset{useNA=false}
  \drawbeam(A){}(C)
  \drawbeam[linecolor=red, beamangle=10](A){}(C)
\end{pspicture}
```

9.2.2. Brechungsindex

Das Raytracing wird durch den Brechungsindex bestimmt, der für jede Komponente einzeln gesetzt werden kann. Der Wert ist relativ zum Hintergrund, für den ein konstanter Wert für die gesamte Skizze angenommen wird.

n=⟨*code*⟩ default: 1.5

Setzt den Brechungsindex relativ zum Hintergrund. Diese Option hat zwei Anwendungsbereiche: zum einen kann der Index einer Komponente gesetzt werden, zum anderen der Brechungsindex für einen Strahlengang:

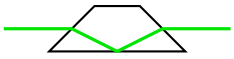
1. Wird n als globaler Parameter oder als Komponentenparameter verwendet, so wird der Index für alle betroffenen Komponenten gesetzt. Ohne weitere Änderungen wird jeder Lichtstrahl diese Werte verwenden.

2. Als Parameter zu \drawbeam oder \drawwidebeam kann der Brechungsindex der Komponenten für einzelne Strahlengänge überschrieben oder geändert werden. Das kann sehr nützlich sein, um z.B. chromatische Dispersion zu illustrieren. Soll die Änderung global für alle oder mehrere Lichtstrahlen wirksam sein, so muss der gewünschte Wert dem Beam-Stil hinzugefügt werden, da globale Definitionen von n nur die Komponenten betreffen.

Der ⟨*code*⟩ kann jeglicher Postscript-Kode sein, der zu einer Zahl ausgewertet werden kann, also z.B. `1.5` oder `5 sqrt`. Beginnt ⟨*code*⟩ mit einem ∗, so wird der Code als algebraischer Ausdruck[1] ausgewertet, d.h. `*sqrt(5)` ist äquivalent zu `5 sqrt`. Innerhalb von ⟨*code*⟩ kann mit `n` auf den vordefinierten Brechungsindex der Komponenten zugegriffen werden, es kann also eine Änderung relativ zum ursprünglichen Wert erzielt werden. Mit `n=n 1.01 mul` kann `n` z.B. um ein Prozent geändert werden.

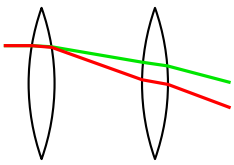
In den folgenden Beispielen werden diese unterschiedlichen Möglichkeiten von n gezeigt.

1. Setze den Brechungsindex einer Komponente. Interessanterweise ist für die Standardform des \doveprism ein Brechungsindex von $\sqrt{5}$ der ideale Wert so dass der Ausgangsstrahl parallel zum Eingangsstrahl verläuft.



```
\begin{pspicture}(3,1.5)
  \pnodes(0,1){A}(3,1){B}
  \doveprism[n=*sqrt(5)](A)(B)
  \drawbeam(A){}(B)
\end{pspicture}
```

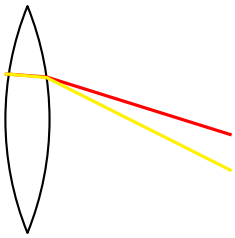
2. Ein Strahl (grün) verwendet die voreingestellten Werte der Komponenten, der rote überschreibt diese mit 2.



```
\begin{pspicture}(3,3)
  \pnodes(0,1){A}(3,1){B}
  \optplane(A)
  \psset{lens=3 3 2, n=1.5}
  \lens[abspos=0.5](A)(B)
  \lens[abspos=2](A)(B)
  \optplane(B)
  \addtopsstyle{Beam}{beampos=0.5}
  \drawbeam{-}
  \drawbeam[linecolor=red, n=2]{-}
\end{pspicture}
```

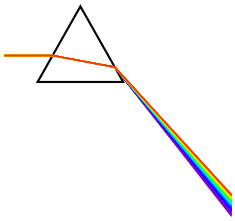
[1] http://mirror.ctan.org/graphics/pstricks/base/doc/pst-news08.pdf

3. Der rote Strahl verwendet die voreingestellten Werte, der grüne ändert diese um `0.5`.



```
\begin{pspicture}(3,3)
  \pnodes(0,1.5){A}(3,1.5){B}
  \lens[lens=4 4 3, n=2, abspos=0.3](A)(B)
  \optplane[compname=Plane](B)
  \psset{beampos=0.6}
  \drawbeam[linecolor=red](A){1}{Plane}
  \drawbeam[linecolor=yellow, n=*(n+0.5)](A){1}{Plane}
\end{pspicture}
```

4. Mit dynamischer Anpassung von `n` kann chromatische Dispersion sehr gut emuliert werden. Hier wird die Dispersion eines Prisma gezeigt, der Brechungsindex wird mit der Sellmaier Gleichung[2] für die unterschiedlichen Wellenlängen berechnet.



```
\begin{pspicture}(3,3)
  \pnodes(-1,0){A}(1,2.2){B}(3,0){C}
  \optplane(0,2.15)
  \optprism[prismalign=center, prismangle=59](A)(B)(C)
  \optplane(C)
  \definecolor[ps]{bl}{rgb}{%
    tx@addDict begin tx@OptexpDict begin
      Wavelength wavelengthToRGB
      Red Green Blue
    end end }%
  \addtopsstyle{Beam}{linecolor=bl,
      linewidth=0.4\pslinewidth, beamalign=abs}
  \multido{\iLambda=400+4}{60}{%
    \pstVerb{/Wavelength \iLambda\space def }%
    \drawbeam[n=Wavelength Sellmaier]{-}%
  }%
\end{pspicture}
```

5. Die numerische Apertur einer Lochblende als spektraler Filter.

[2] http://en.wikipedia.org/wiki/Sellmeier_equation

```
\begin{pspicture}(0.3,0)(3.3,5.2)
  \pnodes(0.3,1){A}(2,1){B}(3,2){C}(2.5,5){D}
  \optplane(A)
  \optgrating[reverse](A)(B)(C)
  \pinhole[phwidth=-0.1, innerheight=0.03, abspos=0.5](D)(B)
  \optplate[position=end, plateheight=1.5](B)(D)
  \definecolor[ps]{bl}{rgb}{%
    tx@addDict begin Red Green Blue end}%
  \drawbeam[linecolor=red]{1-2}
  \addtopsstyle{Beam}{linecolor=bl, linewidth=0.4\pslinewidth}
  \multido{\i=0+1}{60}{%
    \pstVerb{%
      \i\space 650 400 sub 59 div mul 400 add
      tx@addDict begin wavelengthToRGB end }%
    \drawbeam[beamangle=\i\space 0.1 mul 3 sub]{2-}%
  }%
\end{pspicture}
```

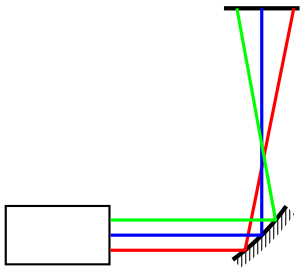
refractiveindex=⟨*code*⟩

Dieser Parameter ist seit Version 3.0 veraltet, verwenden Sie stattdessen n.

9.2.3. Anfangsbedingungen

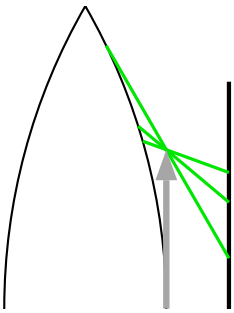
beampos=[⟨x⟩]⟨y⟩ default: 0

Die Startposition (⟨x⟩, ⟨y⟩) des Strahls an der ersten Grenzfläche. Beide Zahlen sind vom Typ ⟨*psnum*⟩. Wird eine einzelne Zahl angegeben, wird die x-Koordinate auf 0 gesetzt.



```
\begin{pspicture}(4,3.8)
  \pnodes(1.5,0.5){A}(3.5,0.5){B}(3.5,3.5){C}
  \optbox[position=end](B)(A)
  \mirror[mirrortype=extended, mirrorradius=3](A)(B)(C)
  \optplate[position=end](B)(C)
  \drawbeam[linecolor=red, beampos=-0.2]{1-3}
  \drawbeam[linecolor=blue, beampos=0]{1-3}
  \drawbeam[linecolor=green, beampos=0.2]{1-3}
\end{pspicture}
```

Die Startposition ist relativ zum entsprechenden Grenzflächenknoten, wie im folgenden Beispiel gezeigt wird. Die Startposition ist fest, abhängig z.B. von beamangle kann der eigentliche Schnittpunkt mit der ersten Grenzfläche jedoch anders sein.

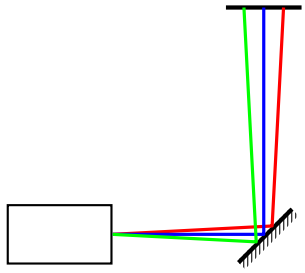


```
\begin{pspicture*}(0,2)(3.05,6)
  \lens[lens=8 8 8, abspos=0.6](0.5,2)(3,2)
  \optplate[position=1, plateheight=6](0.5,2)(3,2)
  \psset{beampos=2.1}
  \drawbeam[beamangle=-60]{1}{2}
  \drawbeam[beamangle=-40]{1}{2}
  \drawbeam[beamangle=-20]{1}{2}
\end{pspicture*}
```

An welcher Grenzfläche der ersten Komponente gestartet wird hängt von `beaminsidefirst` und `startinside` ab.

`beamangle`=⟨*psnum*⟩ default: `0`

Der Anfangswinkel des Strahls. Dieser ist in der Regel relativ zu der Verbindung von der ersten zur zweiten Komponente, mit `beamalign` kann dieses Verhalten verändert werden.

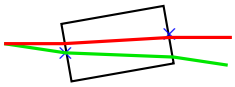


```
\begin{pspicture}(4,4)
  \pnodes(1.5,0.5){A}(3.5,0.5){B}(3.5,3.5){C}
  \optbox[position=end](B)(A)
  \mirror[mirrortype=extended](A)(B)(C)
  \optplate[position=end](B)(C)
  \drawbeam[linecolor=red, beamangle=3]{-3}
  \drawbeam[linecolor=blue, beamangle=0]{1-}
  \drawbeam[linecolor=green, beamangle=-3]{-}
\end{pspicture}
```

`beamalign`=`rel`, `relative`, `abs`, `absolute`, `firstcomp` default: `relative`

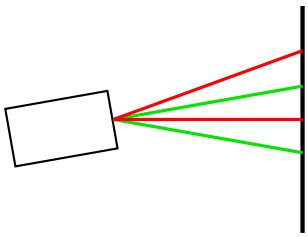
Gibt vor, ob der Winkel `beamangle` relativ zu der Verbindung zwischen den ersten beiden Komponenten ist, ob es sich um einen absoluten Winkel handelt, oder ob der Winkel durch die Ausrichtung der ersten Komponente vorgegeben wird.

Ein absoluter Winkel kann sehr hilfreich sein, falls eine der beiden ersten Komponenten gedreht oder verschoben wurde. Dabei werden die Grenzflächenknoten, auf die sich ein relativer Winkel bezieht, ebenfalls transformiert (siehe Kap. 3.3). In dem folgenden Beispiel verläuft daher der grüne Strahl mit der relativen Ausrichtung nicht horizontal.



```
\begin{pspicture}(3,1.5)
  \pnodes(0,0.75){A}(3,0.75){B}
  \optbox[angle=10, showifcnodes](A)(B)
  \drawbeam(A){}(B)
  \drawbeam[linecolor=red, beamalign=abs](A){}(B)
\end{pspicture}
```

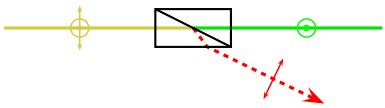
Im folgenden Beispiel liegen beide Komponenten auf der horizontalen Linie $\overline{AB}$. Der grüne Strahl, mit beamalign=rel verläuft daher auch horizontal. Der rote Strahl ist hingegen mit beamalign=firstcomp entsprechend der Drehung der ersten Komponente ausgerichtet.



```
\begin{pspicture}(4,3)
  \pnodes(1.5,1.5){A}(4,1.5){B}
  \optbox[position=start, rotateref=r, angle=10](A)(B)
  \optplate[position=end, plateheight=3](A)(B)
  \addtopsstyle{Beam}{beamdiv=20}
  \drawwidebeam{-}
  \drawwidebeam[beamalign=firstcomp, linecolor=red]{-}
\end{pspicture}
```

beampathskip=⟨*num*⟩ default: 0

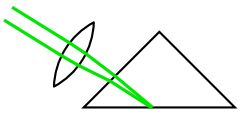
Überspringt ⟨*num*⟩ Segmente am Anfang des Strahlengangs. Diese Option beeinflusst lediglich ab welchem Segment der Strahlgang gezeichnet wird, das Raytracing beginnt immer noch an der ersten Grenzfläche. Um das zu ändern können Sie startinsidecount verwenden.



```
\begin{pspicture}(5,2)
\pnodes(0,1){A}(5,1){B}(2,0){C}
\begin{optexp}
  \glanthompson(A)(B)
  \optplane(B)\optplane[angle=90](C)
  \polarization[abspos=1, poltype=misc, linecolor=yellow!80!black](A)(B)
  \drawbeam[beaminsidelast, linecolor=yellow!80!black](A){1}
  \drawbeam[beampathskip=2](A){1-2}
  \drawbeam[beampathskip=2, savebeam, linestyle=dashed, dash=2pt 2pt,
      linecolor=red,arrowscale=1.5, arrows=->](A){1}{3}
\end{optexp}
\nodexn{(\oenodeBeam{})-1.5(!\oeBeamVec{})}{D}
\polarization[poltype=parallel, linecolor=red](D)(\oenodeBeam{})
\polarization[abspos=4, poltype=perp, linecolor=green, dotscale=1.5](A)(B)
\end{pspicture}
```

beampathcount=⟨*num*⟩ default: -1

Anzahl der Strahlsegmente, die gezeichnet werden sollen. Werden am Anfang Segmente mit beampathskip übersprungen, so werden diese nicht mitgezählt.



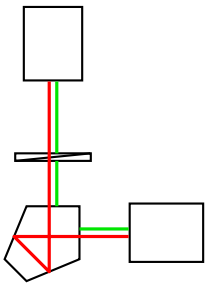
```
\begin{pspicture}(3,2)
  \pnodes(0,2){A}(2,0.8){B}(4,2){C}
  \optprism[compname=P, prismtype=reflective, prismangle=90](A)(B)(C)
  \lens[abspos=1.33](\oenodeIfc{2}{P})(A)%
  \drawwidebeam[beamwidth=0.2, beaminsidelast,
      beampathcount=4](A){2}{1}
\end{pspicture}
```

9.2.4. Interner Strahlengang

Ein Strahlengang setzt sich aus Strahlen zwischen den Komponenten und innerhalb der Komponenten zusammen. Ob diese internen Strahlen gezeichnet werden, kann mit mehreren Parametern festgelegt werden. Die erste, letzte und die übrigen Komponenten werden separat gehandhabt.

`beaminside`=`true`, `false` default: `true`

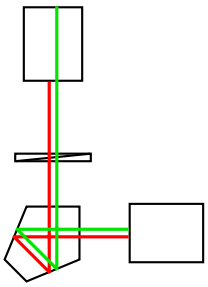
Zeichnet den Strahlengang innerhalb aller Komponenten mit Ausnahme der ersten und letzten, die von dieser Option nicht beeinflusst.



```
\begin{pspicture}(0.3,0.3)(3,4)
  \psset{optboxwidth=1}
  \optbox[position=end](1,1)(1,3)
  \optretplate(1,1)(1,3)
  \pentaprism(1,3)(1,1)(2,1)
  \optbox[position=end](1,1)(2,1)
  \drawbeam[beampos=-0.05, linecolor=red]{-}
  \drawbeam[beampos=0.05, beaminside=false]{-}
\end{pspicture}
```

`beaminsidefirst`=`true`, `false` default: `false`

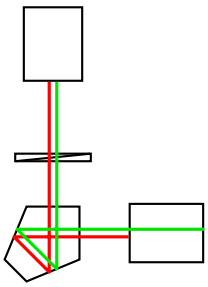
Zeichne den Strahlengang innerhalb der ersten Komponenten.



```
\begin{pspicture}(0.3,0.3)(3,4)
  \psset{optboxwidth=1}
  \optbox[position=end](1,1)(1,3)
  \optretplate(1,1)(1,3)
  \pentaprism(1,3)(1,1)(2,1)
  \optbox[position=end](1,1)(2,1)
  \drawbeam[beampos=-0.05, linecolor=red]{-}
  \drawbeam[beampos=0.05, beaminsidefirst]{-}
\end{pspicture}
```

`beaminsidelast`=`true`, `false` default: `false`

Zeichne den Strahlengang innerhalb der letzten Komponenten.

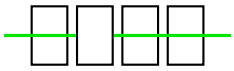


```
\begin{pspicture}(0.3,0.3)(3,4)
  \psset{optboxwidth=1}
  \optbox[position=end](1,1)(1,3)
  \optretplate(1,1)(1,3)
  \pentaprism(1,3)(1,1)(2,1)
  \optbox[position=end](1,1)(2,1)
  \drawbeam[beampos=-0.05, linecolor=red]{-}
  \drawbeam[beampos=0.05, beaminsidelast]{-}
\end{pspicture}
```

`allowbeaminside`=`true`, `false` default: `true`

Damit kann bestimmt werden, ob der Strahlengang innerhalb einer Komponente gezeichnet werden darf. Dieser Parameter wirkt nur auf Komponenten und

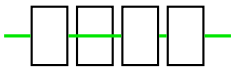
überschreibt, falls auf `false` gesetzt, jegliche Strahleinstellungen mit `beaminside`, `beaminsidefirst` und `beaminsidelast`. Damit kann der Strahlengang in einzelnen Komponenten unterdrückt werden, für die es keinen Sinn macht (z.B. `\optfilter` und `\optdiode`, was sonst nicht möglich ist.



```
\begin{pspicture}(3,2)
  \psset{optboxwidth=0.5}
  \pnodes(0,1){A}(3,1){B}
  \optbox[position=0.2](A)(B)
  \optbox[position=0.4, allowbeaminside=false](A)(B)
  \optbox[position=0.6](A)(B)
  \optbox[position=0.8](A)(B)
  \drawbeam(A){-}(B)
\end{pspicture}
```

`forcebeaminside`=`true`, `false` default: `false`

Wenn auf `true` gesetzt, wird der Strahlengang innerhalb der betroffenen Komponenten immer gezeichnet wird, unabhängig von allen anderen Strahleinstellungen mit `allowbeaminside`, `beaminside`, `beaminsidefirst` und `beaminsidelast`. Damit kann der Strahl innerhalb einzelner Komponenten gezeichnet werden, auch wenn alle anderen Komponenten in dem Strahlengang keinen internen Strahl haben sollen.



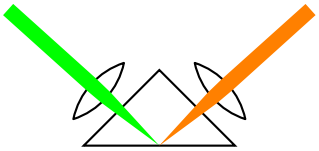
```
\begin{pspicture}(3,2)
  \psset{optboxwidth=0.5}
  \pnodes(0,1){A}(3,1){B}
  \optbox[position=0.2](A)(B)
  \optbox[position=0.4, forcebeaminside](A)(B)
  \optbox[position=0.6](A)(B)
  \optbox[position=0.8](A)(B)
  \drawbeam[beaminside=false](A){-}(B)
\end{pspicture}
```

`startinsidecount`=⟨*num*⟩ default: `-1`

Anzahl der Segmente, die innerhalb der ersten Komponente verwendet werden. Diese Option hat nur eine Wirkung, wenn `beaminsidefirst` oder `startinside` auf `true` gesetzt sind.

`stopinsidecount`=⟨*num*⟩ default: `-1`

Anzahl der Segmente, die innerhalb der letzten Komponente verwendet werden. Diese Option hat nur eine Wirkung, wenn `beaminsidelast` oder `stopinside` auf `true` gesetzt sind.



```
\begin{pspicture}(4,2)
  \pnodes(0,2){A}(2,0){B}(4,2){C}
  \optprism[compname=P, n=1, prismtype=reflective,
      prismangle=90](A)(B)(C)
  \lens[abspos=1.08](\oenodeIfc{2}{P})(A)
  \lens[abspos=1.08](\oenodeIfc{2}{P})(C)
  \newpsstyle{Beam}{linestyle=none, fillstyle=solid}
  \drawwidebeam[fillcolor=green, beamwidth=0.2,
      beaminsidelast, stopinsidecount=1](A){2}{1}
  \drawwidebeam[fillcolor=orange, beaminsidefirst,
      startinsidecount=1, loadbeam]{1}{3}(C)
\end{pspicture}
```

beammode=refl, trans, reflective, transmittive, auto default: auto

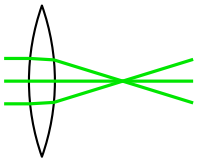
Mit dieser Option kann das Strahlverhalten bei Komponenten wie z. B. dem \beamsplitter auf reflektierend oder transmittierend festgelegt werden, unabhängig von den verwendeten Knoten und deren relative Position bezüglich der Komponente.

9.2.5. Verbinden mit Knoten

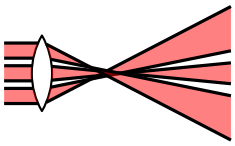
Wie vorher schon einmal erwähnt können Komponenten auch mit Knoten verbunden werden. Für Raytracing wird in der Regel eine Ebene benötigt, die auf unterschiedliche Weise automatisch oder manuell erstellt werden kann.

beamnodealign=vec, conn, vector, connection default: connection

Dieser Parameter bestimmt die Ausrichtung der automatisch erstellen Grenzflächen. In der Voreinstellung (connection) wird die Ebene vertikal zur direkten Verbindung zwischen dem Knoten und dem vorangegangenen Grenzflächenknoten ausgerichtet.

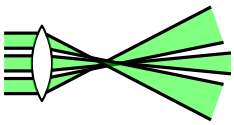


```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(2.5,1){B}
  \lens[lens=3 3 2, n=2.5, abspos=0.5](A)(B)
  \multido{\r=-0.3+0.3}{3}{%
    \drawbeam[beampos=\r](A){}(B)}
\end{pspicture}
```



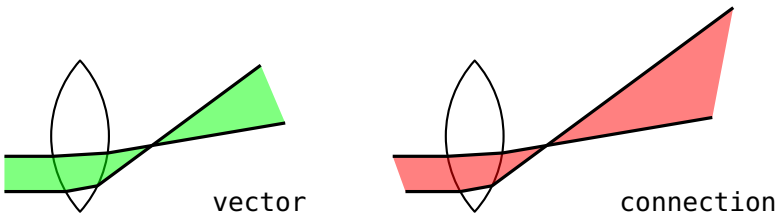
```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(3,1){B}
  \lens[abspos=0.5](A)(B)
  \addtopsstyle{Beam}{linecolor=black, fillstyle=solid,
      beamwidth=0.2, fillcolor=red, opacity=0.5, beaminside=false}
  \multido{\r=-0.3+0.3}{3}{\drawwidebeam[beampos=\r](A){}(B)}
\end{pspicture}
```

Ist der Wert auf `vector` gesetzt, dann wird die Ebene vertikal zum einfallenden Strahlvektor ausgerichtet. Bei weiten Strahlen wird der gemittelte Strahlenvektor (siehe `\oeBeamVecMedian`) als Referenz verwendet. Beispiele in dem das sehr praktisch ist, sind Bsp. 8.9 und Bsp. 11.



```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(3,1){B}
  \lens[abspos=0.5](A)(B)
  \addtopsstyle{Beam}{linecolor=black, fillstyle=solid,
      beamwidth=0.2, beamnodealign=vector, fillcolor=green,
      opacity=0.5, beaminside=false}
  \multido{\r=-0.3+0.3}{3}{\drawwidebeam[beampos=\r](A){}(B)}
\end{pspicture}
```

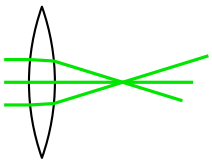
Die automatische Ausrichtung ist in vielen Fällen sehr praktisch, kann aber wiederum unerwünschte Ergebnisse bei gedrehten und verschobenen Komponenten ergeben und die Ebenenausrichtung kann nicht beliebig gesetzt werden. Im folgenden Beispiel ist beim grünen Strahl `beamnodealign` auf `vector` gesetzt, bei dem roten Strahl hingegen auf `connection`:



Das kann mit einer „unsichtbaren“ Ebene erreicht werden, die mit `\optplane` definiert wird.

`\optplane`(⟨*center*⟩)

Die Ebene wird durch lediglich einen Knoten definiert, die Drehung darum kann mit `angle` angegeben werden. Der Winkel ist relativ zur ursprünglichen, vertikalen Ausrichtung.



```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(2.5,1){B}
  \lens[lens=3 3 2, n=2.5, abspos=0.5](A)(B)
  \optplane[angle=-30](B)
  \multido{\r=-0.3+0.3}{3}{%
    \drawbeam[beampos=\r](A){-}}
\end{pspicture}
```

9.2.6. Automatische Verbindungen

Lichtstrahlen können auch zusammen mit der Komponentendefinition gezeichnet werden. Das kann für einfache Strahlengänge sehr praktisch sein, ist aber sehr unflexibel.

`beam=true, false` default: `false`

Kann als Option für jede Komponente verwendet werden, ist äquivalent zu

```
\drawbeam[raytrace=false](\oenodeRefA{}){}(\oenodeRefB{})
```

womit ein Strahl vom ersten Referenzknoten über die Komponente zum zweiten Referenzknoten gezeichnet wird.

9.2.7. Strahlaussehen

`Beam` ⟨*psstyle*⟩ default: `linecolor=green!90!black, linejoin=1`

Das Aussehen der Lichtstrahlen kann mit diesem Stil gesteuert werden. Das optionale Argument von `\draw*beam` kann ebenfalls dafür verwendet werden und kann die Einstellungen des `Beam`-Stils überschreiben.

`addtoBeam=`⟨*list*⟩

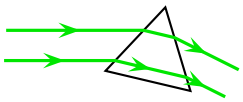
Der vorhandene `Beam`-Stil wird lokal um die Parameter in ⟨*list*⟩ erweitert, ⟨*list*⟩ muss mit geschweiften Klammern gekapselt werden.

`newBeam=`⟨*list*⟩

Ähnlich wie `addtoBeam`, nur wird der `Beam`-Stil mit den neuen Parametern überschrieben.

ArrowInsideMinLength=⟨*psnum*⟩ default: 0.2

\draw∗beam verwendet auch die Option ArrowInside aus pstricks-add. In dem Fall wird für alle Strahlsegmente ein Pfeil gezeichnet, was häufig für die internen, kurzen Strahlengänge unerwünscht ist. Mit ArrowInsideMinLength kann angegeben werden ab welcher Mindestlänge eines Strahlsegmentes die Pfeil gezeichnet werden. In dem folgenden Beispiel ist der Wert so gewählt, dass nur der untere interne Strahl einen Pfeil hat (siehe auch Bsp. 11).

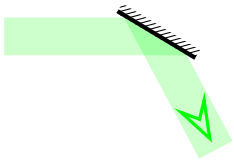


```
\begin{pspicture}(3,2)
  \optprism(0,1)(2,1)(3,0.5)
  \drawwidebeam[beamwidth=0.4, ArrowInside=->,
      ArrowInsideMinLength=0.5, arrowscale=1.5,
      raytrace=false](0,1){}(3,0.5)
\end{pspicture}
```

ArrowInsideMaxLength=⟨*psnum*⟩ default: -1

Gibt analog zu ArrowInsideMinLength die maximale Länge eines Segments an, für das Pfeile gezeichnet werden. Ist der Wert negativ, dann gibt es keine obere Grenze.

ArrowInside kann auch verwendet werden ohne dass ein Linientyp definiert ist.



```
\begin{pspicture}(0,0)(3.5,2)
  \pnodes(0,1.5){A}(2,1.5){B}(2.8,0){C}
  \mirror[mirrorwidth=1.2, mirrortype=extended](A)(B)(C)
  \newpsstyle{Beam}{linecolor=green, linestyle=none,
      fillstyle=solid, fillcolor=green, opacity=0.2}
  \drawwidebeam[beamwidth=0.5](A){}(C)
  \drawbeam[ArrowInside=->, arrowscale=2.5, ArrowFill=false,
      linewidth=1.5\pslinewidth, ArrowInsidePos=0.9,
      ArrowInsideMaxLength=1.8](A){}(C)
\end{pspicture}
```

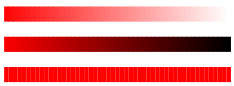
fade ⟨*linestyle*⟩

Erzeugt einen Farbverlauf von der aktuellen linecolor zu einer anderen Farbe, definiert durch fadeto. Der Linienstil funktioniert momentan nur vernünftig mit Linien, nicht jedoch mit Kurven. Der gesamte Pfad wird in fadepoints−1 Segmente unterteilt.

```
\begin{pspicture}(3,1)
  \psline[linewidth=0.2, linecolor=red, linestyle=fade,
      fadepoints=50](0,0.5)(3,0.5)
\end{pspicture}
```

fadeto=white, black, transparency default: white

Die Endfarbe des Farbverlaufs, hängt auch von fadefuncname und fadefunc ab.



```
\begin{pspicture}(3,1)
  \psset{linewidth=0.2, linecolor=red, linestyle=fade, fadepoints=50}
  \psline[fadeto=white](0,0.9)(3,0.9)
  \psline[fadeto=black](0,0.5)(3,0.5)
  \psline[fadeto=transparency](0,0.1)(3,0.1)
\end{pspicture}
```

fadepoints=⟨*num*⟩ default: 200

Anzahl der Punkte, die für den Linienstil fade verwendet werden.

fadefuncname=gauss, linear, squared, exp, custom default: linear

Ein paar vordefinierte Funktionen, die den Farbverlauf bestimmen. Wird custom angegeben, dann wird der Postscript-Code aus fadefunc verwendet.

fadefunc=⟨*PS code*⟩

Eine benutzerdefinierte Funktion für den Farbverlauf des Linienstils fade. Der ⟨*PS code*⟩ muss eine Zahl auf dem Stack im Intervall $[0,1]$ (0: Pfadanfang, 1: Pfadende) auf das Intervall $[0,1]$ abbilden (0: fadeto-Farbe, 1: aktuelle Linienfarbe). Die Funktionswerte werden auf das Intervall $[0,1]$ abgeschnitten.

```
\begin{pspicture}(3,1)
  \psset{linewidth=0.2, linecolor=red, linestyle=fade, fadepoints=50}
  \psline[fadefuncname=gauss](0,1)(3,1)
  \psline[fadefuncname=exp](0,0.7)(3,0.7)
  \psline[fadefuncname=custom, fadefunc={360 mul cos 1 add 0.5
      mul}](0,0.4)(3,0.4)
  \psline[fadefuncname=custom, fadefunc={neg 1 add 2
      mul}](0,0.1)(3,0.1)
\end{pspicture}
```

Ein schönes Beispiel ist Bsp. 11.

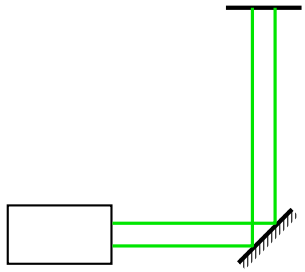
9.3. Aufgeweitete Strahlen

Alle Informationen über einfache Strahlen (siehe Kap. 9.2) betreffen auch aufgeweitete Strahlen. Hier werden nur zusätzliche Einstellmöglichkeiten aufgeweiteter Strahlen behandelt.

\drawwidebeam[⟨*options*⟩]{⟨obj_1⟩}{⟨obj_2⟩}...

beamwidth=⟨*psnum*⟩ default: 0

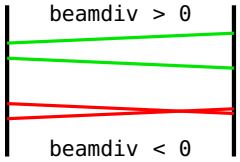
Die Anfangsstrahlbreite an der mit beampos angegebenen Position, gemessen vertikal zur Verbindungslinie zur nächsten Komponente.



```
\begin{pspicture}(4,4)
  \pnodes(1.5,0.5){A}(3.5,0.5){B}(3.5,3.5){C}
  \optbox[position=end](B)(A)
  \mirror[mirrortype=extended](A)(B)(C)
  \optplate[position=end](B)(C)
  \drawwidebeam[beamwidth=0.3]{-}
\end{pspicture}
```

beamdiv=⟨*psnum*⟩ default: 0

Die Strahldivergenz an der Startposition, angegeben in Grad. Positive Werte ergebenen einen expandierenden Strahl, negative einen kontrahierenden.



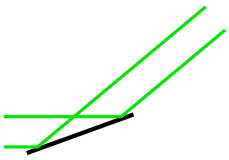

```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(3,1){B}
  \psset{plateheight=2}
  \optplate[position=start](A)(B)
  \optplate[position=end](A)(B)
  \addtopsstyle{Beam}{beamwidth=0.2}
  \drawwidebeam[beampos=0.4, beamdiv=5]{-}
  \drawwidebeam[beampos=-0.4, beamdiv=-5, linecolor=red]{-}
\end{pspicture}
```

9.3.1. Strahlaussehen

Das Aussehen aufgeweiteter Strahlen kann genauso wie bei einfachen Strahlen über Beam oder das optionale Argument vorgegeben werden (siehe Kap. 9.2.7).

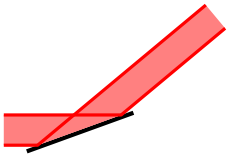
Breite Strahlen bestehen aus zwei Randstrahlen, die gemäß der Linieneinstellungen gezeichnet werden, und können zusätzlich gefüllt werden.

1. Nur die Randstrahlen.



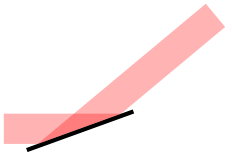
```
\begin{pspicture}(3,2)
  \pnodes(0,0.5){A}(1,0.5){B}(2.8,2){C}
  \mirror[mirrorwidth=1.5](A)(B)(C)
  \drawwidebeam[beamwidth=0.4](A){}(C)
\end{pspicture}
```

2. Randstrahlen, und Füllstil.



```
\begin{pspicture}(3,2)
  \pnodes(0,0.5){A}(1,0.5){B}(2.8,2){C}
  \mirror[mirrorwidth=1.5](A)(B)(C)
  \addtopsstyle{Beam}{fillstyle=solid,
      fillcolor=red!50!white, linecolor=red}
  \drawwidebeam[beamwidth=0.4](A){}(C)
\end{pspicture}
```

3. Nur Füllstil.

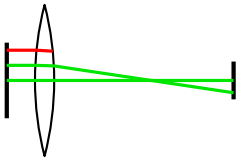


```
\begin{pspicture}(3,2)
  \pnodes(0,0.5){A}(1,0.5){B}(2.8,2){C}
  \mirror[mirrorwidth=1.5](A)(B)(C)
  \addtopsstyle{Beam}{fillstyle=solid, fillcolor=red,
      opacity=0.3, linestyle=none}
  \drawwidebeam[beamwidth=0.4](A){}(C)
\end{pspicture}
```

9.4. Fehlerbehandlung

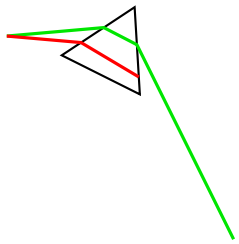
Alle Strahlen werden direkt mit Postscript berechnet und gezeichnet, so dass eine umfassende Fehlerbehandlung zur Zeit der Kompilation nicht möglich ist. Um jedoch trotzdem fehlerfreie Bilder zu bekommen, wird der Strahlengang abgebrochen, sobald eine Situation eintritt, die nicht unterstützt wird. Das kann eintreten, falls der Strahl eine Grenzfläche nicht trifft, Totalreflektion auftritt oder wenn die folgende Komponente keine Freistrahlverbindungen unterstützt. Die folgenden Beispiele zeigen Situationen, in denen diese Fehler auftreten:

1. Ein Strahl trifft eine Grenzfläche nicht. Der rote Strahl in dem folgenden Beispiel verpasst die letzte Grenzfläche und endet daher an der vorhergehenden.



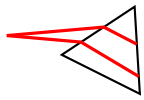
```
\begin{pspicture}(3,2.7)
  \pnodes(0,1){A}(3,1){B}
  \optplate[position=start](A)(B)
  \lens[lens=4 4 2, abspos=0.5, n=2.5](A)(B)
  \optplate[position=end, plateheight=0.5](A)(B)
  \drawbeam{-}
  \drawbeam[beampos=0.2]{-}
  \drawbeam[beampos=0.4, linecolor=red]{-}
\end{pspicture}
```

2. Totale interne Reflektion tritt an einer Grenzfläche auf, die nicht dafür vorgesehen ist. Der rote Strahl im folgenden Beispiel würde reflektiert werden, aber das wird für diese Komponente nicht unterstützt und der Strahl endet an der total reflektierenden Grenzfläche.



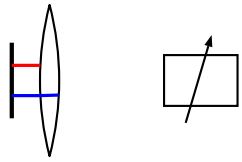
```
\begin{pspicture}(3,2.5)
  \pnodes(0,2){A}(3,0){B}
  \optplane(A)
  \optprism[n=1.8](A)([Xnodesep=1.5]A)(B)
  \optplane(B)
  \drawbeam[beamangle=5]{-}
  \drawbeam[beamangle=-5, linecolor=red]{-}
\end{pspicture}
```

Dasselbe gilt ebenfalls für aufgeweitete Strahlen (siehe Kap. 9.3). Bei diesen wird der Strahlengang abgebrochen, falls einer der beiden Randstrahlen einen Fehler aufweist.



```
\begin{pspicture}(3,2.5)
  \pnodes(0,2){A}(3,0){B}
  \optplane(A)
  \optprism[n=1.8](A)([Xnodesep=1.5]A)(B)
  \optplane(B)
  \drawwidebeam[beamdiv=10, linecolor=red]{-}
\end{pspicture}
```

3. Eine Komponente soll verbunden werden, die jedoch keine Freistrahlverbindungen unterstützt. In diesem Fall wird die falsche Komponente und alle Folgenden ignoriert.



```
\begin{pspicture}(3,2.7)
  \pnodes(0,1){A}(2,1){B}
  \optplate[position=start](A)(B)
  \lens[lens=4 4 2, abspos=0.5, n=2.5](A)(B)
  \fiberdelayline[fdlsize=1 0.7, position=end, fiber=none](A)(B)
  \drawbeam[beampos=0.2, linecolor=red]{-}
  \drawbeam[beampos=-0.2, linecolor=blue, beaminsidelast]{-}
\end{pspicture}
```

pswarning=true, false default: false

Um eine Fehlersuche zu vereinfachen kann ein Hinweis ausgegeben werden sobald ein Strahlengang vorzeitig beendet. Die Fehlermeldung wird auf die Standardausgabe geschrieben, wenn die Datei mit einem Postscript-Interpreter gelesen wird, also mit einem Betrachter geöffnet oder z.B. mit ps2pdf konvertiert wird.

9.5. Angepasste Strahlen

Im Prinzip können alle Strahlengänge mit passender Wahl des Brechungsindex und der Komponentenparameter (z.B. der Linsenkrümmung) realisiert werden, was jedoch sehr aufwendig werden kann. Um das zu vereinfachen, besteht die Möglichkeit die Endpunkte eines Strahls als Anfangspunkte für folgende Strahlen zu nutzen, so dass man abschnittsweise die Divergenz und die Ausbreitungsrichtung definieren kann.

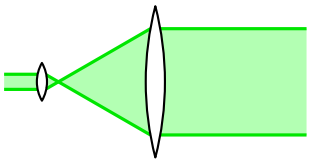
savebeampoints=true, false, ⟨*int*⟩ default: true

Speichert die Endpunkte eines \draw*beam Makros, vorangegangene Punkte werden überschrieben. \drawbeam und \drawwidebeam werden getrennt behandelt. Wird hier eine Zahl $\langle int \rangle > 0$ angegeben, so können beliebig viele Endpunkte separat gespeichert werden, siehe z.B. Bsp. 11. In der Regel reicht an- und abschalten vollkommen aus, was den Zahlen 1 und 0 entspricht.

loadbeampoints=true, false, ⟨*int*⟩ default: false

Verwende die Endpunkte eines vorangegangenen Strahls als Startpunkte. Wird keine Zahl ⟨*int*⟩ angegeben, dann wird 1 verwendet.

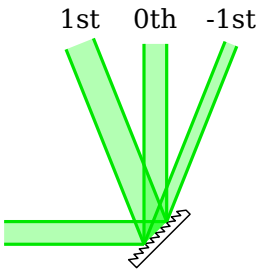
Als Beispiel nehmen wir ein Teleskop, die Strahldivergenz am Eingang und am Ausgang ist Null. Anstatt die Linsenparameter und den Brechungsindex genau einzustellen, so dass die Ausgangsdivergenz Null ist, zeichnen wir einfach die drei Strahlenteile mit den gewünschten Divergenzen, und verwenden die Endpunkte des jeweils vorangegangenen Strahls als neue Ausgangspunkte.



```
\begin{pspicture}(4,2)
  \pnodes(0,1){A}(4,1){B}
  \begin{optexp}
    \lens[lens=0.5 0.5 0.5, abspos=0.5](A)(B)
    \lens[lens=4 4 2, abspos=2](A)(B)
    \addtopsstyle{Beam}{%
      fillstyle=solid, fillcolor=green, opacity=0.3}
    \psset{loadbeampoints}
    \drawwidebeam[beamwidth=0.2, stopinside](A){1}
    \drawwidebeam[beamdiv=-60]{1}{2}
    \drawwidebeam{2}(B)
  \end{optexp}
\end{pspicture}
```

Als weiteres Beispiel betrachten wir die Beugung an einem Gittern in die nullte und erste Beugungsordnung. Zuerst wird der Strahl von dem Eingang bis zum Gitter gezeichnet, die Endpunkte dieses Strahls werden nun für die Strahlen in beide Beugungsordnungen verwendet.

Bsp. 9.1




```
\begin{pspicture}(3.5,3.5)
  \pnodes(0,0.5){A}(2,0.5){B}(2,3){C}
  \nodexn{(C)-(1,0)}{C'}
  \nodexn{(C)(1,0)}{C''}
  \optgrating(A)(B)(C)
  \addtopsstyle{Beam}{%
    fillstyle=solid, fillcolor=green, opacity=0.3}
  \drawwidebeam[beamwidth=0.3](A){1}
  \psset{savebeampoints=false, loadbeampoints}
  \drawwidebeam{1}(C)
  \drawwidebeam{1}(C')
  \drawwidebeam{1}(C'')
\end{pspicture}
```

savebeam=true, false, ⟨*int*⟩ default: true

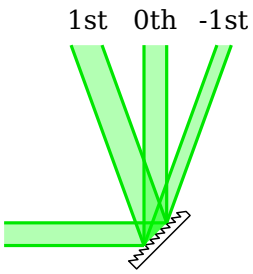
Speichert die Endzustände, d.h. Endpunkte und Ausgangsvektoren, eines \draw∗beam Makros, vorangegangene Werte werden überschrieben. \drawbeam und \drawwidebeam

werden getrennt behandelt. Wird hier eine Zahl $\langle int \rangle > 0$ angegeben, so können beliebig viele Strahlen separat gespeichert werden. In der Regel reicht an- und abschalten vollkommen aus.

`loadbeam`=`true`, `false`, $\langle int \rangle$ default: `false`

Verwende den Endzustand (Punkte und Richtung) eines vorangegangenen Strahls als Anfangszustand. Der Parameter `beamangle` kann verwendet werden um die Richtung relativ zu der geladenen zu verändern (siehe folgendes Beispiel).

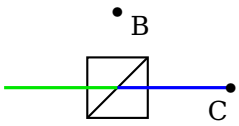
Bsp. 9.2



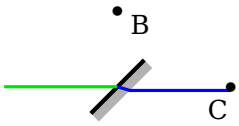

```
\begin{pspicture}(3.5,3.5)
  \pnodes(0,0.5){A}(2,0.5){B}(2,3){C}
  \optgrating(A)(B)(C)
  \addtopsstyle{Beam}{%
    fillstyle=solid, fillcolor=green, opacity=0.3}
  \drawwidebeam[savebeam, beamwidth=0.3](A){1}
  \psset{savebeam=false, loadbeam}
  \drawwidebeam[beamangle=-20]{1}(C)
  \drawwidebeam{1}(C)
  \drawwidebeam[beamangle=20]{1}(C)
\end{pspicture}
```

Beachten Sie, dass `loadbeam` und `savebeam` nicht immer funktionieren, wenn der Strahlengang an einem halbdurchlässigen Spiegel oder einem Strahlteiler endet. In diesen Fällen wird nur der transmittierte Strahlvektor gespeichert, soll der Strahlengang über den reflektierenden Weg fortgesetzt werden, so ist das nicht möglich.

In den beiden folgenden Beispielen wird demnach der rote Strahl, der von der Komponente zum Knoten $\langle B \rangle$ fortgesetzt werden soll, nicht gezeichnet.




```
\begin{pspicture}(3,1.5)
  \pnodes(0,0.5){A}(1.5,1.5){B}(3,0.5){C}
  \beamsplitter(A)(1.5,0.5)(B)
  \drawbeam[beaminsidelast, savebeam](A){}
  \drawbeam[linecolor=red, beaminsidefirst, loadbeam]{}(B)
  \drawbeam[linecolor=blue, beaminsidefirst, loadbeam]{}(C)
\end{pspicture}
```

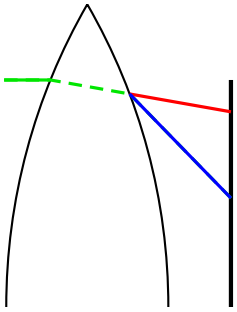



```
\begin{pspicture}(3,1.5)
  \pnodes(0,0.5){A}(1.5,1.5){B}(3,0.5){C}
  \mirror[mirrortype=semitrans](A)(1.5,0.5)(B)
  \drawbeam[beaminsidelast, savebeam](A){}
  \drawbeam[linecolor=red, beaminsidefirst, loadbeam]{}(B)
  \drawbeam[linecolor=blue, beaminsidefirst, loadbeam]{}(C)
\end{pspicture}
```

startinside=true, false default: false

Mit dem Parameter beaminsidefirst kann eingestellt werden, ob der interne Strahlengang in der erste Komponente gezeichnet wird. Für angepasste Strahlengänge kann es aber notwendig sein dem internen Strahlengang in der ersten Komponente zu folgen ohne diesen zu zeichnen, z.B. wenn der vorangegangene Strahl an der ersten Grenzfläche schon endet.

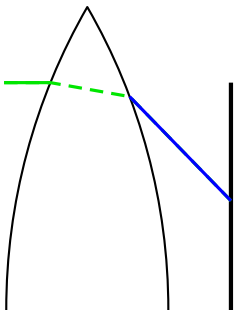
Im folgenden Beispiel wird der einfallende grüne Strahl falsch durch den roten Strahl fortgesetzt, da der grüne an der linken Grenzfläche endet, der rote aber fälschlicherweise annimmt, dass der Endzustand sich auf die rechte Grenzfläche bezieht. Dem blauen Strahl wird nun mit startinside mitgeteilt, dass der Endzustand sich auf die linke Grenzfläche bezieht.



```
\begin{pspicture*}(0,2)(3.05,6)
  \optplane(0,2)
  \lens[lens=8 8 8, abspos=0.6, n=1.8](0.5,2)(3,2)
  \optplate[position=1, plateheight=6](0.5,2)(3,2)
  \psset{savebeam=false}
  \drawbeam[beampos=3, linestyle=dashed]{-}
  \drawbeam[beampos=3, savebeam]{1-2}
  \drawbeam[loadbeam, linecolor=red]{2-3}
  \drawbeam[loadbeam, startinside, linecolor=blue]{2-3}
\end{pspicture*}
```

stopinside=true, false default: false

Äquivalent zu startinside, bezieht sich aber auf die letzte Komponenten.



```
\begin{pspicture*}(0,2)(3.05,6)
  \optplane(0,2)
  \lens[lens=8 8 8, abspos=0.6, n=1.8](0.5,2)(3,2)
  \optplate[position=1, plateheight=6](0.5,2)(3,2)
  \psset{savebeam=false}
  \drawbeam[beampos=3, linestyle=dashed]{-}
  \drawbeam[beampos=3, savebeam, stopinside]{1-2}
  \drawbeam[loadbeam, linecolor=blue]{2-3}
\end{pspicture*}
```

9.6. Faserverbindungen

Alle Komponenten können mit Fasern verbunden werden, die Faserkomponenten in Kap. 5 werden automatisch verbunden.

\drawfiber[⟨options⟩]{⟨obj$_1$⟩}{⟨obj$_2$⟩}...

Dieses Makro zeichnet Faserverbindungen und akzeptiert eine variable Anzahl an Argumenten (mindestens zwei), die entweder eine ID, ein ID-Intervall, ein compname oder ein PSTricks-Knoten sein können (siehe Kap. 9.1. Um zwischen Knoten und Komponenten unterscheiden zu können, müssen Knoten entweder in runde Klammern innerhalb der geschweiften Klammern eingeschlossen werden oder dürfen nur runde Klammern haben: \drawbeam{(node)}{comp}, oder \drawbeam(node){comp}. Werden mehr als zwei Argumente mitgegeben, werden die Verbindungen $1 \rightarrow 2$, $2 \rightarrow 3$, usw. gezeichnet.

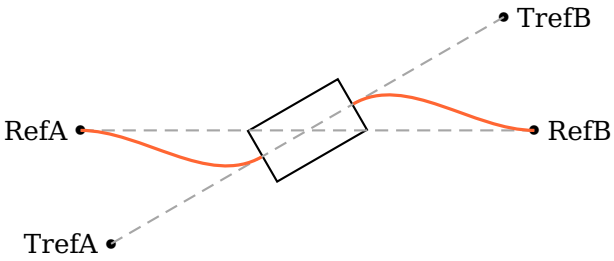
Die automatischen Faserverbindungen werden intern als \drawfiber von den Referenzknoten zur Komponente gezeichnet.

9.6.1. Faserwinkel

Üblicherweise werden Fasern als \nccurve-Verbindungen gezeichnet, die Winkel an beiden Kurvenenden werden automatisch anhand der Komponentenausrichtung bestimmt und hängen davon ab, ob eine Komponente mit einer anderen Komponente oder einem Knoten verbunden wird.

Komponente mit Knoten

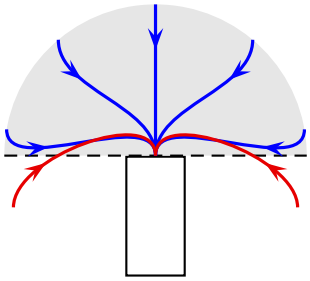
Eine Verbindung zwischen einer Komponente und einem Knoten ist das, was bei automatischen Verbindungen verwendet wird (siehe Kap. 9.8). Folgendes Beispiel erläutert die Ausrichtung der Kurve:




```
\begin{pspicture}(-1,-0.7)(7,2.7)
  \pnodes(0,1){A}(6,1){B}
  \optbox[angle=30](A)(B)
  \drawfiber(A){}(B)
\end{pspicture}
```

Der Winkel an der Komponente entspricht dem der transformierten Referenzlinie. Der Winkel an dem Knoten entspricht dem der ursprünglichen Referenzlinie der Komponente zu der die Verbindung geht. Die Faser ist in Richtung des entsprechenden Komponentenknotens orientiert.

Das führt dazu, dass im folgenden Beispiel die Anfangswinkel der roten Fasern um 180° gegenüber den blauen Fasern gedreht sind, die Winkel an der Komponente bleiben unverändert.

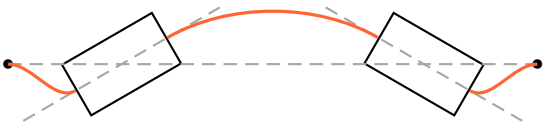


```
\begin{pspicture}(-2,-1.6)(2,2)
  \optbox[position=end](0,1)(0,0)
  \addtopsstyle{Fiber}{linecolor=blue}
  \multido{\i=10+40}{5}{%
    \drawfiber(2;\i){}
  }
  \addtopsstyle{Fiber}{linecolor=red!90!black}
  \drawfiber(2;-20){}
  \drawfiber(2;200){}
\end{pspicture}
```

Komponente mit Komponente

Diese Variante ist sehr nützlich um glattere Verbindungen zwischen Komponenten zu erreichen, die nicht auf derselben Referenzlinie positioniert sind, sondern z.B. in einer Schleife angeordnet werden, siehe Bsp. 4 und Bsp. 9.6.2.

In diesem Fall werden die Kurvenwinkel von der Komponente bestimmt, mit der das jeweilige Ende verbunden ist. Die Fasern zeigen immer von der Komponentenmitte weg.



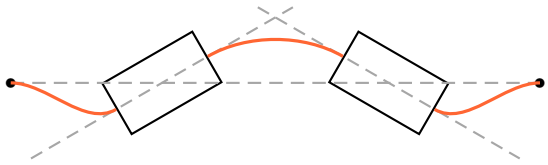
```
\begin{pspicture}(0,-0.8)(7,0.8)
  \pnodes(0,0){A}(3,0){B}(4,0){C}(7,0){D}
  \optbox[angle=30](A)(B)
  \optbox[angle=-30](C)(D)
  \drawfiber(A){1}{2}(D)
\end{pspicture}
```

Diese automatisch berechneten Winkel können vielfältig manipuliert werden, die möglichen Parameter sind im Folgenden aufgelistet.

`fiberalign`=`rel`, `relative`, `center`, `abs`, `absolute` default: `relative`

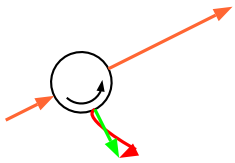
Wählt die Referenz für die Faserwinkel.

`relative` Die Faserwinkel sind relativ zur Komponente. Wird ein Knoten mit der Komponente verbunden, so entspricht der Winkel am Knoten dem der Referenzlinie der Komponente. Der Winkel an der Komponente entspricht dem der transformierten Referenzlinie.



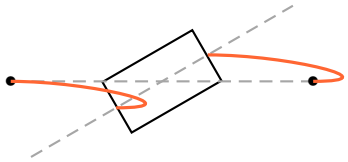
```
\begin{pspicture}(1,-1.7)(7,1.7)
  \pnodes(0,0){A}(4,0){B}(3,0){C}(7,0){D}
  \optbox[angle=30](A)(B)
  \optbox[angle=-30](C)(D)
  \drawfiber(A){1}{2}(D)
\end{pspicture}
```

center Die Winkel sowohl am Knoten als auch an der Komponente sind relativ zu der Verbindung zwischen dem betroffenen Komponentenknoten und der Komponentenmitte. Meistens ist das äquivalent zu `relative`, wird aber z.B. für `\optcirculator` und `\elecmixer` benötigt. In dem Beispiel hat die grüne Faser die korrekte Ausrichtung, während die rote `relative` ausgerichtet wird.



```
\begin{pspicture}(1,0)(3,2)
  \addtopsstyle{Fiber}{arrowscale=1.3, arrows=->, arrowinset=0}
  \optcirculator[fiber=t](0,0.5)(3,2)(1.5,0)
  \drawfiber[linecolor=red]{}(1.5,0)
  \drawfiber[fiberalign=center, linecolor=green]{}(1.5,0)
\end{pspicture}
```

absolute Die Faserwinkel werden als absolute Werte angegeben, die automatischen Winkel werden nicht berücksichtigt.



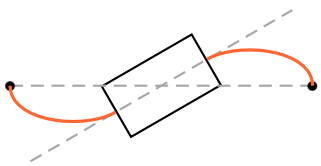
```
\begin{pspicture}(1,-1)(4.5,1)
  \pnodes(0,0){A}(4,0){B}
  \optbox[angle=30](A)(B)
  \drawfiber[fiberalign=abs, ncurv=1.5](A){}(B)
\end{pspicture}
```

fiberangleA=⟨*num*⟩ default: 0

Ein additiver Wert für den Anfangswinkel, der gesamte Winkel hängt von `fiberalign` ab. Ist dieser auf `absolute` eingestellt, so ist `fiberangleA` der absolute Winkel, andernfalls wird der Wert zu dem automatisch berechneten Winkel hinzuaddiert.

fiberangleB=⟨*num*⟩ default: 0

Wie `fiberangleB`, aber für den Endwinkel.



```
\begin{pspicture}(0,-1)(4,1)
  \pnodes(0,0){A}(4,0){B}
  \optbox[angle=30](A)(B)
  \drawfiber[fiberangleA=-90](A){}
  \drawfiber[fiberangleB=-90]{}(B)
\end{pspicture}
```

`startnode`=`auto`, `N`, `1`, `2`, `...` default: `auto`

Für die Verbindungen mit `\drawfiber` werden die Knoten genommen, die sich am nächsten sind. Das passt häufig, aber nicht immer. Hiermit kann der Startknoten explizit gewählt werden.

`stopnode`=`auto`, `N`, `1`, `2`, `...` default: `auto`

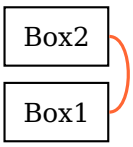
Genau wie `startnode`, aber für den Stopknoten.

Die folgenden Beispiele zeigen Anwendungsmöglichkeiten für `startnode` und `stopnode`.

1. In diesem Fall ist der nächstgelegene Knoten auch der richtige.

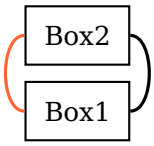
```
\begin{pspicture}(3,2)
  \pnodes(-0.5,1){A}(1.5,1){B}(3.5,1){C}
  \optbox(A)(B){Box1}\optbox(B)(C){Box2}
  \drawfiber{1}{2}
\end{pspicture}
```

2. Hier ist der kürzeste Abstand nicht eindeutig definiert, da es zwei Verbindungen mit der gleichen Länge gibt.




```
\begin{pspicture}(2,2)
  \pnodes(0,0.5){A}(2,0.5){B}([offset=1]B){C}(A|C){D}
  \optbox(A)(B){Box1}\optbox(C)(D){Box2}
  \drawfiber{1}{2}
\end{pspicture}
```

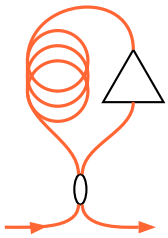
3. Um die gewünschte Verbindung zu wählen ist es meistens ausreichend entweder `startnode` oder `stopnode` zu setzen. Ist ein Knoten fest vorgegeben, wird der anderen wieder als der mit dem kürzesten Abstand gewählt. Nur in Ausnahmefällen sollte es notwendig sein beide Knoten explizit anzugegeben.




```
\begin{pspicture}(2,2)
  \psset{label=0}
  \pnodes(0,0.5){A}(2,0.5){B}([offset=1]B){C}(A|C){D}
  \optbox(A)(B){Box1}\optbox(C)(D){Box2}
  \drawfiber[startnode=1]{1}{2}
  \drawfiber[startnode=N, linecolor=black]{1}{2}
\end{pspicture}
```

4. Ein weiteres Beispiel, bei dem `startnode` vorgegeben werden muss.

Bsp. 9.3



```
\begin{pspicture}(2.1,3)
  \pnodes(0,0){A}(2,0){B}(0.3,3){C}(1.7,3){D}
  \psset{fiber=none, arrowscale=1.2, arrowinset=0}
  \optcoupler[fiber=l,
              addtoFiberIn1={angleA=0, ArrowInside=->},
              addtoFiberIn2={angleA=180, arrows=<-},
              abspos=0.5, compname=Cpl](A)(B)(C)(D)
  \optfiber[compname=Hnlf, abspos=1](C)(C|A)
  \optamp[abspos=2, compname=Amp](D|B)(D)
  \drawfiber{Cpl}{Hnlf}
  \drawfiber[startnode=1, ncurv=1.2]{Hnlf}{Amp}
  \drawfiber{Cpl}{Amp}
\end{pspicture}
```

9.6.2. Faseraussehen

`Fiber` ⟨*psstyle*⟩

Das Aussehen der Fasern wird durch diesen Stil vorgegeben. Das optionale Argument von `\drawfiber` kann ebenfalls verwendet werden, die entsprechenden Parameter können die Einstellungen von `Fiber` überschreiben.

`addtoFiber`=⟨*list*⟩

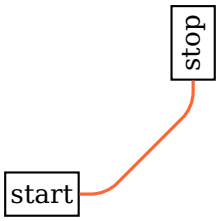
Der vorhandene `Fiber`-Stil wird lokal um die Parameter in ⟨*list*⟩ erweitert, ⟨*list*⟩ muss mit geschweiften Klammern gekapselt werden. Siehe z.B. Bsp. 9.6.2.

`newFiber`=⟨*list*⟩

Ähnlich wie `addtoFiber`, nur wird der `Fiber`-Stil mit den neuen Parametern überschrieben.

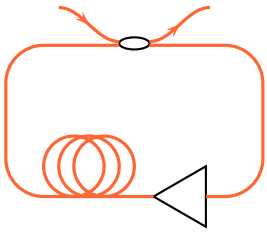
`fiberstyle`=⟨*string*⟩ default: `curve`

Die Fasern werden in der Regel als `\nccurve` gezeichnet, aber eine beliebige andere Knotenverbindung `\nc`⟨*string*⟩ des `pst-node` Paketes kann ausgewählt werden, lesen Sie die entsprechende Dokumentation[3] für mögliche Werte. In den folgenden Beispielen werden Anwendungsmöglichkeiten gezeigt, eine weitere ist in Bsp. 11 zu sehen.




```
\begin{pspicture}(3,3)
  \pnodes(1, 0.5){A}(2.5,0.5){B}(2.5,2){C}
  \psset{optboxsize=1 0.6, innerlabel}
  \optbox[position=start](A)(B){start}
  \optbox[position=end](B)(C){stop}
  \drawfiber[fiberstyle=diag, linearc=0.5]{1}{2}
\end{pspicture}
```

Bsp. 9.4



```
\begin{pspicture}(4,2.5)
  \pnodes(1,2.5){In}(3,2.5){Out}(0,2){A}(4,2){B}(4,0){C}(0,0){D}
  \optcoupler[fiber=t, addtoFiber={ArrowInside=->},
      coupleralign=b](In)(A)(Out)(B)
  \psset{fiber=none}
  \optamp[position=0.35](C)(D)
  \optfiber[position=0.35](D)(C)
  \drawfiber{2}{3}
  \addtopsstyle{Fiber}{fiberstyle=bar, linearc=0.5, armA=1.5}
  \drawfiber[stopnode=1]{1}{2}
  \drawfiber[stopnode=1]{1}{3}
\end{pspicture}
```

9.7. Elektrische Verbindungen

Alle Komponenten können mit Drähten verbunden werden, die elektrischen Komponenten in Kap. 6 werden automatisch verbunden. Die `wire`∗-Parameter sind äquivalent zu den entsprechenden `fiber`∗-Parametern, die detailliertere Informationen enthalten.

`\drawwire`[⟨*options*⟩]{⟨obj_1⟩}{⟨obj_2⟩}...

Dieses Makro akzeptiert eine variable Anzahl an Argumenten (mindestens zwei), die entweder eine ID, ein ID-Intervall, ein `compname` oder ein PSTricks-Knoten sein

[3] http://mirror.ctan.org/help/Catalogue/entries/pst-node.html

können (siehe Kap. 9.1. Um zwischen Knoten und Komponenten unterscheiden zu können, müssen Knoten entweder in runde Klammern innerhalb der geschweiften Klammern eingeschlossen werden oder dürfen nur runde Klammern haben: `\drawbeam{(node)}{comp}`, oder `\drawbeam(node){comp}`. Werden mehr als zwei Argumente mitgegeben, werden die Verbindungen $1 \to 2$, $2 \to 3$, usw. gezeichnet.

Die automatischen Verbindungen werden intern als `\drawwire` von den Referenzknoten zur Komponente gezeichnet.

`wirealign`=`rel`, `relative`, `center`, `abs`, `absolute` default: `relative`

Siehe `fiberalign`.

`wireangleA`=⟨*num*⟩ default: `0`

Siehe `fiberangleA`.

`wireangleB`=⟨*num*⟩ default: `0`

Siehe `fiberangleB`.

`wirestyle`=⟨*string*⟩ default: `angle`

Siehe `fiberstyle`, der Vorgabewert ist jedoch `angle`.

`addtoWire`=⟨*list*⟩

Der vorhandene `Wire`-Stil wird lokal um die Parameter in ⟨*list*⟩ erweitert, ⟨*list*⟩ muss mit geschweiften Klammern gekapselt werden.

`newWire`=⟨*list*⟩

Ähnlich wie `addtoWire`, nur wird der `Wire`-Stil mit den neuen Parametern überschrieben.

`Wire` ⟨*psstyle*⟩

Das Aussehen der elektrischen Verbindungen wird durch diesen Stil vorgegeben, analog zu Kap. 9.6.2. Das optionale Argument von `\drawwire` kann ebenfalls verwendet werden, die entsprechenden Parameter können die Einstellungen von `Wire` überschreiben.

9.8. Automatische Faser- und Drahtverbindungen

Alle Faserkomponenten, die in Kap. 5 beschrieben werden, werden automatisch über `\drawfiber` mit ihren Referenzknoten verbunden, das Verhalten ist in Kap. 9.6.1 beschrieben. Entsprechend dazu werden die elektrischen Komponenten (Kap. 6) mit `\drawwire` mit Drähten verbunden. Die automatischen Verbindungen können umfangreich kontrolliert werden, ohne diese Makros explizit zu verwenden. Das Aussehen jeder Verbindung kann ebenfalls separat konfiguriert werden, siehe Kap. 9.8.1.

`fiber`=`[*+]none`, `all`, `i`, `o`, ⟨*refpoint*⟩ default: `all`

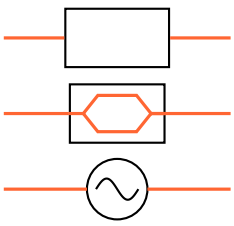
Dieser Parameter gibt an, welche Faserverbindungen gezeichnet werden. Der Parameter kann nur die Faserkomponenten betreffen (falls ein `*` vorangestellt ist), nur die elektrischen und Freistrahlkomponenten (`+` wird vorangestellt), oder alle Komponententypen. Typische Freistrahlkomponenten, die häufig als Faserkomponenten verwendet werden, sind `\optbox` und `\optdetector`. Beachten Sie, dass elektrische Komponenten nicht automatisch mit Fasern verbunden werden können, Sie müssen dann explizit `\drawfiber` verwenden.

Es können alle Verbindungen gezeichnet werden (`all`), keine (`none`), oder es können bestimmte ausgewählt werden. Die Werte `i` (`l` ist äquivalent) zeichnet nur die Eingangsfasern, `o` (bzw. `r`) alle ausgehenden Fasern. Diese Angaben sind ausschließend, d.h. mit `io` werden gar keine Verbindungen gezeichnet.

Komponenten, die über mehr als einen Eingang oder Ausgang verfügen (Koppler und `\optcirculator`) erlauben weitergehende Auswahl mit `t` und `b`, möglicherweise in Verbindung mit `i`, `o`, `l` und `r`.

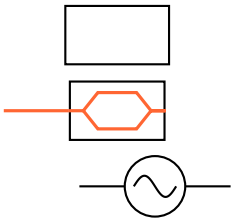
Die folgenden Beispiele zeigen einige Möglichkeiten.

1. Zeichne alle Fasern für alle Komponenten.



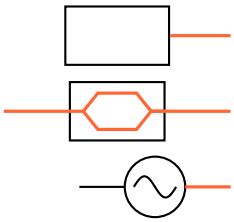
```
\begin{pspicture}(3,3)
  \psset{fiber}
  \optbox(0,2.5)(3,2.5)
  \optmzm(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](0,0.5)(3,0.5)
\end{pspicture}
```

2. Nur die Eingangsverbindung einer Faserkomponente (`\optmzm`) wird gezeichnet, die anderen Komponenten bleiben unbeeinflusst.



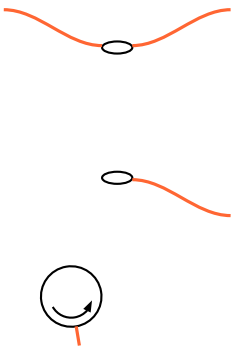
```
\begin{pspicture}(3,3)
  \psset{fiber=*i}
  \optbox(0,2.5)(3,2.5)
  \optmzm(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](1,0.5)(3,0.5)
\end{pspicture}
```

3. Von der Faserkomponente werden alle Verbindungen gezeichnet, die anderen Komponenten erhalten nur eine Ausgangsverbindung.



```
\begin{pspicture}(3,3)
  \psset{fiber=+o}
  \optbox(0,2.5)(3,2.5)
  \optmzm(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](1,0.5)(3,0.5)
\end{pspicture}
```

4. Die Werte `t` und `b` betreffen nur die Komponenten mit mehr als zwei Verbindungen, also `\optcirculator` und die Koppler (Kap. 5.10).



```
\begin{pspicture}(3,1)
  \optcoupler[fiber=t](0,1)(0,0)(3,1)(3,0)
\end{pspicture}
```

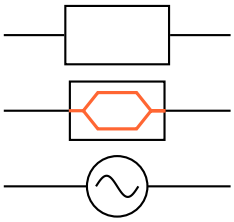
```
\begin{pspicture}(3,1)
  \wdmsplitter[fiber=br](0,0.5)(3,1)(3,0)
\end{pspicture}
```

```
\begin{pspicture}(3,1)
  \optcirculator[fiber=b](0,0.5)(3,1)(1,0)
\end{pspicture}
```

`wire`=`[*+]none`, `all`, `i`, `o`, ⟨*refpoint*⟩ default: `all`

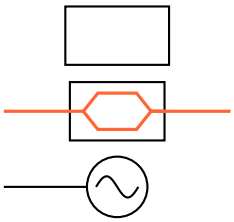
Dieser Parameter gibt an, welche elektrische Verbindungen gezeichnet werden. Der Parameter kann nur die elektrischen Komponenten betreffen (falls ein `*` vorangestellt ist), nur die Faser- und Freistrahlkomponenten (`+` wird vorangestellt), oder alle Komponenten.

1. Zeichne alle Drähte aller Komponenten.



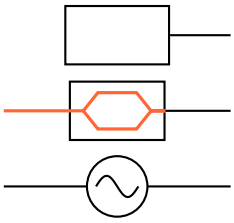
```
\begin{pspicture}(3,3)
  \psset{wire}
  \optbox(0,2.5)(3,2.5)
  \optmzm(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](0,0.5)(3,0.5)
\end{pspicture}
```

2. Nur die Eingangsverbindung einer elektrischen Komponente (`\elecsynthesizer`) wird gezeichnet, die anderen Komponenten bleiben unbeeinflusst.



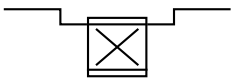
```
\begin{pspicture}(3,3)
  \psset{wire=*i}
  \optbox(0,2.5)(3,2.5)
  \optmzm(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](0,0.5)(3,0.5)
\end{pspicture}
```

3. Von der Freistrahl- und Faserkomponente wird nur die Ausgangsverbindung gezeichnet, die elektrischen Komponente erhält jedoch alle Verbindungen.

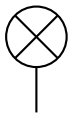


```
\begin{pspicture}(3,3)
  \psset{wire=+o}
  \optbox(0,2.5)(3,2.5)
  \optmzm(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](0,0.5)(3,0.5)
\end{pspicture}
```

4. Die Werte `t` und `b` betreffen nur die Komponenten mit mehr als zwei Verbindungen, also `\elecmixer` und `\eleccoupler`.



```
\begin{pspicture}(3,1)
  \eleccoupler[wire=t](0,1)(0,0)(3,1)(3,0)
\end{pspicture}
```



```
\begin{pspicture}(3,1.5)
  \elecmixer[wire=b](0,1)(3,1)(1,0)
\end{pspicture}
```

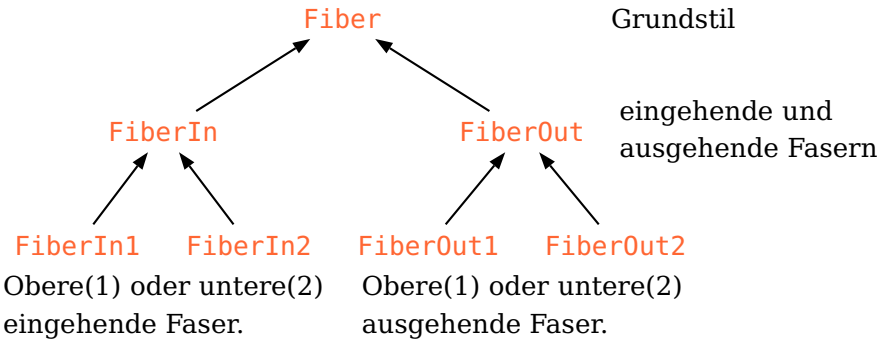


Abbildung 9.1.: Vererbungsdiagramm der PS-Stile für die automatischen Faserverbindungen. Sie sollten diese Stile mit `\addtopsstyle` oder den entsprechenden `addto`⟨*style*⟩ Parametern ändern um die Vererbungslinie beizubehalten. Eine identisches Vererbungsdiagramm besteht für die entsprechenden `Wire...` Stile.

9.8.1. Aussehen der automatischen Verbindungen

Die Stile aller automatischen Faserverbindungen können separat konfiguriert werden, der zugrundeliegende Stil ist immer `Fiber`, der in Kap. 9.6.2 beschrieben ist.

`FiberIn` `FiberIn1` `FiberIn2` `FiberOut` `FiberOut1` `FiberOut2`

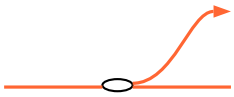
Der Stil `FiberIn` bezieht sich auf alle Eingangsfasern und erbt alle Eigenschaften von `Fiber`. Hat eine Komponente zwei davon, so können diese mit `FiberIn1` und `FiberIn2` angesprochen werden, die alles von `FiberIn` übernehmen. Entsprechend bezieht sich `FiberOut` auf alle Ausgangsfasern, falls es davon zwei gibt dann können diese mit `FiberOut1` und `FiberOut2` geändert werden. Siehe Abb. 9.1 für ein Vererbungsdiagramm der Faserstile. Beachten Sie, dass diese Vererbung nur erhalten bleibt, falls die Stile nur mit `\addtopsstyle` oder den entsprechenden `addto`⟨*style*⟩ Parametern verändert werden.

`WireIn` `WireIn1` `WireIn2` `WireOut` `WireOut1` `WireOut2`

Der Stil `WireIn` bezieht sich auf alle eingehenden elektrischen Verbindungen und erbt alle Eigenschaften von `Wire`. Hat eine Komponente zwei davon, so können diese mit `WireIn1` und `WireIn2` angesprochen werden, die alles von `WireIn` übernehmen. Entsprechend bezieht sich `WireOut` auf alle ausgehenden elektrischen Verbindungen, falls es davon zwei gibt dann können diese mit `WireOut1` und `WireOut2` geändert werden. Siehe Abb. 9.1 für ein Vererbungsdiagramm der Stile. Beachten Sie, dass diese Vererbung nur erhalten bleibt, falls die Stile nur mit `\addtopsstyle` oder den entsprechenden `addto`⟨*style*⟩ Parametern verändert werden.

new⟨*style*⟩
addto⟨*style*⟩

Für jeden Faserstil werden zwei Parameter new⟨*style*⟩ und addto⟨*style*⟩ bereitgestellt, mit denen die Stile für einzelne Komponenten verändert werden können. Damit können neuen Komponenten mit anderen voreingestellten Verbindungen definiert werden (Kap. 10.1), oder explizite Gruppierung kann vermieden werden.

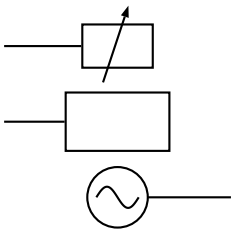


99/1

```
\begin{pspicture}(3,2)
  \newpsobject{tapcoupler}{wdmsplitter}{%
    coupleralign=b,
    addtoFiberOut1={arrows=->, arrowscale=1.2, arrowinset=0}}
  \tapcoupler(0,0.5)(3,1.5)(3,0.5){99/1}
\end{pspicture}
```

9.8.2. Bekannte Einschränkungen

Die Kombination von fiber und wire erlaubt in der derzeitigen Implementierung (Version v6.1) nicht die beliebige Kombination aller Parametervarianten. Die meisten funktionieren, aber das folgende Beispiel zeigt eine Variante, die nicht korrekt funktioniert (der \elecsynthesizer sollte eine Eingangsfaser haben):



```
\begin{pspicture}(3,3)
  \psset[optexp]{fiber=i, wire=+i}
  \fiberdelayline(0,2.5)(3,2.5)
  \optbox(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](0,0.5)(3,0.5)
\end{pspicture}
```

In solchen speziellen Fällen muss auf manuelle Verbindungen zurückgegriffen werden.

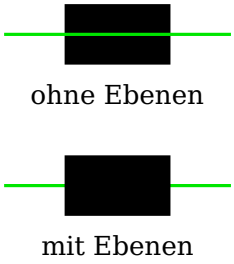
9.9. Zeichenebenen

Die Reihenfolge der Konstruktion einer Skizze ist die, dass zuerst die Komponenten definiert werden müssen, bevor diese verbunden werden können. Das beinhaltet aber, dass alle Verbindungen über die Komponenten gezeichnet werden, was nicht erwünscht sein muss.

Um das zu vermeiden, wird die optexp-Umgebung bereitgestellt, innerhalb derer alle Verbindungen hinter die Komponenten gelegt werden.

\begin{optexp}

\end{optexp}




```
\begin{pspicture}(3,4)
  \psset{beam, fillstyle=solid, fillcolor=black}
  \optbox(0,3.5)(3,3.5){ohne Ebenen}
  \begin{optexp}
    \optbox(0,1.5)(3,1.5){mit Ebenen}
  \end{optexp}
\end{pspicture}
```

Beachten Sie, dass das erreicht wird, indem der gesamte Code innerhalb der Umgebung doppelt ausgeführt wird. Das funktioniert mit allen `pst-optexp`-Makros, da diese sich intern darum kümmern nichts doppelt sondern nur in geänderter Reihenfolge zu zeichnen. Alle anderen Zeichnungen und Texte werden zweimal gezeichnet, was zu unschönen Ergebnissen aufgrund von geändertem Antialiasing in dem Betrachter führen kann.

Um zu verhindern, dass anderer Code doppelt ausgeführt wird, muss dieser entweder außerhalb der optexp-Umgebung definiert werden, oder durch eins der beiden folgenden Makros auf eine Ebene beschränkt werden.

\backlayer{⟨*code*⟩}

Führe ⟨*code*⟩ nur beim ersten Durchgang aus.

\frontlayer{⟨*code*⟩}

Führe ⟨*code*⟩ nur beim zweiten Durchgang aus.

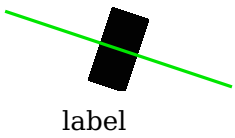
In diesem Beispiel wird der Text über dem Rechteck angezeigt, obwohl der Text zuerst definiert wird:

```
\begin{pspicture}(3,1)
\begin{optexp}
  \frontlayer{\rput(1.5,0.5){front}}
  \backlayer{\psframe*[linecolor=DOrange!50](1,0)(2,1)}
\end{optexp}
\end{pspicture}
```

10. Benutzerdefinierte Komponenten

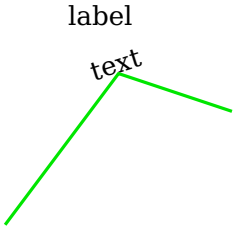
Das pst-optexp-Paket stellt zwei Makros zur Verfügung um eigene Komponenten zu zeichnen. Diese können z.B. aus Bildern oder eigenen Zeichnungen bestehen.

\optdipole[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*comp*⟩}{⟨*label*⟩}




```
\begin{pspicture}(3,3)
  \optdipole[labeloffset=1, beam](0,2)(3,1){%
    \rput(0,0){\rule{0.5cm}{1cm}}%
  }{label}
\end{pspicture}
```

\opttripole[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*out*⟩){⟨*comp*⟩}{⟨*label*⟩}

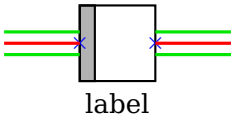



```
\begin{pspicture}(3,3)
  \opttripole[beam](3,1.5)(1.5,2)(0,0){%
    \rput[b](0,0){text}%
  }{label}
\end{pspicture}
```

Diese Komponenten können sowohl mit Fasern als auch mit Strahlen verbunden werden, können aber nicht deren volles Potential ausschöpfen. \opttripole definiert eine einzelne reflektive Grenzfläche, \optdipole kann zwei transmittive Grenzflächen haben:

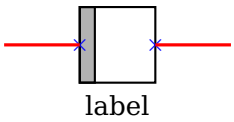
optdipolesize=⟨*width*⟩[⟨*height*⟩] default: 0 0

Hiermit können zwei transmittierende Grenzflächen für \optdipole definiert werden, die sich bei $(\pm 0.5\langle width\rangle, 0)$ befinden.




```
\begin{pspicture}(3,2)
\pnodes(0,1){A}(3,1){B}
\optdipole[optdipolesize=1, showifcnodes](A)(B){%
  \psframe[fillstyle=solid,fillcolor=black!30](-0.5,-0.5)(-0.3,0.5)
  \psframe(-0.5,-0.5)(0.5,0.5)}{label}
\drawwidebeam[beamwidth=0.3, beaminside=false](A){}(B)
\drawbeam[beaminside=false, linecolor=red](A){}(B)
\end{pspicture}
```

Wenn zusätzlich eine Höhe ⟨*height*⟩ ungleich Null angegeben wird, so wird diese als Apertur für das Raytracing genommen (siehe useNA). Im folgenden Beispiel ist die Höhe gleich `0.2` und damit kleiner als der aufgeweitete Strahl der deshalb anders als im vorangegangenen Beispiel nicht gezeichnet wird.

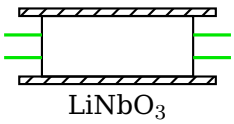



```
\begin{pspicture}(3,2)
\pnodes(0,1){A}(3,1){B}
\optdipole[optdipolesize=1 0.2, showifcnodes](A)(B){%
  \psframe[fillstyle=solid,fillcolor=black!30](-0.5,-0.5)(-0.3,0.5)
  \psframe(-0.5,-0.5)(0.5,0.5)}{label}
\drawwidebeam[beamwidth=0.3, beaminside=false](A){}(B)
\drawbeam[beaminside=false, linecolor=red](A){}(B)
\end{pspicture}
```

Weitere Beispiel sind Bsp. 11 und Bsp. 11.

optdipolecomp=⟨*macros*⟩

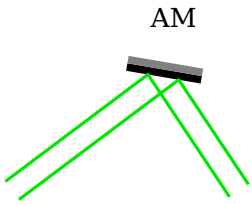
Mit dieser Option kann die Zeichnung einer Komponente definiert werden, um dann mit `\newpsobject` basierend auf \optdipole eine neuen Komponente zu definieren.




```
\begin{pspicture}(3,2)
\def\phasemodulatorcomp{%
  \psframe(-1,-0.4)(1,0.4)
  \psframe[fillstyle=hlines](-1.3,-0.5)(1.3,-0.4)
  \psframe[fillstyle=hlines](-1.3,0.5)(1.3,0.4)}
\newpsobject{phasemodulator}{optdipole}{optdipolesize=2 0.8,
    optdipolecomp={\phasemodulatorcomp}}
\pnodes(0,1){A}(3,1){B}
\phasemodulator(A)(B){LiNbO$_3$}
\drawwidebeam[beamwidth=0.3, beaminside=false](A){}(B)
\end{pspicture}
```

opttripolecomp=⟨*macros*⟩

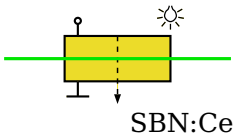
Mit dieser Option kann die Zeichnung einer Komponente definiert werden, um dann mit \newpsobject basierend auf \opttripole eine neuen Komponente zu definieren.



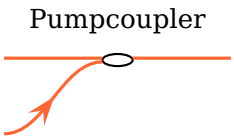

```
\begin{pspicture}(3,2)
\def\ampmodcomp{%
  \psframe*[linestyle=none](-0.5,0)(0.5,0.1)
  \psframe*[linestyle=none, linecolor=gray](-0.5,0.1)(0.5,0.2)}
\newpsobject{ampmod}{opttripole}{opttripolecomp={\ampmodcomp}}
\pnodes(0,0){A}(2,1.5){B}(3,0){C}
\ampmod(A)(B)(C){AM}
\drawwidebeam[beamwidth=0.3](A){}(C)
\end{pspicture}
```

10.1. Benutzerdefinierte Version existierender Komponenten

Sie können eine benutzerdefinierte Version einer existierenden Komponente mit \newpsobject definieren, die z.B. andere Voreinstellungen oder und ein anderes Aussehen hat.

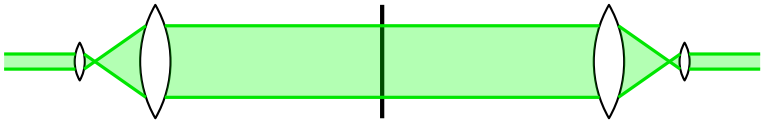



```
\begin{pspicture}(3,1.7)
\newpsobject{sbn}{crystal}{%
  voltage, lamp, %
  fillstyle=solid, fillcolor=yellow!90!black}
\sbn[label=1.2 45, beam](0,1)(3,1){SBN:Ce}
\end{pspicture}
```




```
\begin{pspicture}(3,1.7)
\newpsobject{pumpcoupler}{wdmcoupler}{%
  coupleralign=top, addtoFiberIn2={ArrowInside=->, arrowscale=2}}
\pumpcoupler[label=0.5 180](0,1)(0,0)(3,1){Pumpcoupler}
\end{pspicture}
```

Bsp. 10.1



```
\begin{pspicture}(10,2)
  \newpsobject{MOLensIn}{lens}{lens=0.5 0.5 0.5}
  \newpsobject{MOLensOut}{lens}{lens=1.5 1.5 1.5}
  \pnodes(0,1){A}(10,1){B}
  \MOLensIn[abspos=1](A)(B)\MOLensOut[abspos=2](A)(B)
  \optplate[plateheight=1.5](A)(B)
  \MOLensOut[abspos=8](A)(B)\MOLensIn[abspos=9](A)(B)
  \addtopsstyle{Beam}{n=1, fillstyle=solid, fillcolor=green, opacity=0.3}
  \psset{loadbeampoints}
  \drawwidebeam[beamwidth=0.2, stopinside](A){1}
  \drawwidebeam[beamdiv=-70]{1}{2}
  \drawwidebeam[stopinside]{2-4}
  \drawwidebeam[beamdiv=-70]{4}{5}
  \drawwidebeam{5}(B)
\end{pspicture}
```

10.2. Neue Objekte definieren

`pst-optexp` stellt einige Benutzermakros zur Verfügung, mit denen sehr bequem neue, eigene Komponenten definiert werden können.

Beachten Sie, dass dieser Teil der Dokumentation nicht vollständig ist. Die notwendigen Schritte zur Definition neuer Komponenten seit Version 3.0 werden grob beschrieben, können sich aber noch ändern und sind nicht zwangsläufig abwärtskompatibel.

`\newOptexpDipole`[⟨*fixopt*⟩]{⟨*name*⟩}{⟨*dftopt*⟩}

`\newOptexpTripole`[⟨*fixopt*⟩]{⟨*name*⟩}{⟨*dftopt*⟩}

`\newOptexpFiberDipole`[⟨*fixopt*⟩]{⟨*name*⟩}{⟨*dftopt*⟩}

`\newOptexpElecDipole`[⟨*fixopt*⟩]{⟨*name*⟩}{⟨*dftopt*⟩}

Diese Makros erzeugen den Code für die Positionierung, Drehung, Beschriftung und die Ebenenverwaltung der neuen Komponente \⟨*name*⟩. Im einfachsten Fall müssen Sie lediglich die eigentliche Zeichnung der Komponente erstellen.

Mit dem Argument ⟨*fixopt*⟩ können Optionen voreingestellt werden, die später über das optionale Argument der Komponente nicht verändert werden können, ⟨*dftopt*⟩ erlaubt Voreinstellungen zu machen, die überschrieben werden können. Beide Argumente ⟨*fixopt*⟩ und ⟨*dftopt*⟩ sind optional und können auch ganz weggelassen werden.

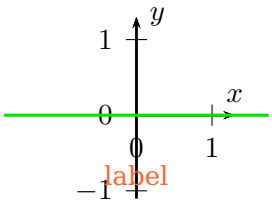
Die Definition der neuen Komponenten kann in unterschiedliche Schritte eingeteilt werden, je nach Komplexität und Anforderung des Benutzers:

1. Die Zeichnung (siehe Kap. 10.2.1), das ist der einzige erforderliche Schritt.
2. Unterstützung von Raytracing (optional).
3. Unterstützung für `rotateref`, `extnode` und `position=start|end` (optional).

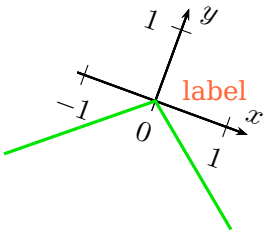
Die folgenden Abschnitte enthalten die Entwicklung einer einfachen Beispielkomponente `\testcomp`.

10.2.1. Die Zeichnung der Komponente

Sie müssen ein Makro `\...@comp` definieren, welches die Zeichnung enthält. In den folgenden Beispielen sehen Sie die Ausrichtung und Position des Koordinatensystems für einen Zweipol und einen Dreipol.

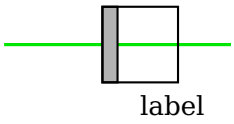



```
\begin{pspicture}(3.5,2.6)
\newOptexpDipole{mydipole}
\makeatletter
\def\mydipole@comp{%
  \psaxes[arrows=->](0,0)(0,-1.1)(1.3,1.3)[$x$,90][$y$,0]
}%
\makeatother
\mydipole(0,1)(3.5,1){\color{spot}label}
\drawbeam(0,1){}(3.5,1)
\end{pspicture}
```




```
\begin{pspicture}(3.5,3.7)
\newOptexpTripole{mytripole}{labelangle=-60}
\makeatletter
\def\mytripole@comp{%
  \psaxes[arrows=->](0,0)(-1.1,0)(1.3,1.3)[$x$,90][$y$,0]
}%
\makeatother
\mytripole(0,1)(2,1.7)(3,0){\color{spot}label}
\drawbeam(0,1){}(3,0)
\end{pspicture}
```

Unsere Beispielkomponente und ihre Zeichnung wird also wie folgt definiert:




```
\begin{pspicture}(3, 2)
\newOptexpDipole{testcomp}
\makeatletter
\def\testcomp@comp{%
  \psframe[fillstyle=solid, fillcolor=black!30](-0.5,-0.5)(-0.3,0.5)
  \psframe(-0.5,-0.5)(0.5,0.5)
}%
\makeatother
\begin{optexp}
\testcomp[position=0.6, labelalign=l](0,1)(3,1){label}
\drawbeam[beaminside=false](0,1){}(3,1)
\end{optexp}
\end{pspicture}
```

Die Positionierung der Komponente und der Beschriftung, sowie die Ebenen funktioniert also schon.

10.2.2. Unterstützung von Raytracing

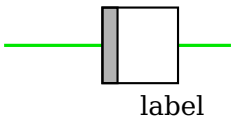
Damit die Komponente Raytracing unterstützt, müssen die Grenzflächen mit ihren Eigenschaften definiert werden. Die Definition der Ebenen werden in einenm Makro `\⟨name⟩@nodes` definiert, damit sie auch in den unterschiedlichen Ebenen verfügbar sind.

Die Definition der Ebenen erfolgt dann über das Makro `\newOptexpComp`

```
\def\testcomp@nodes{%
  \newOptexpComp{%
    % Definition der ersten Grenzflaeche (GF)
```

```
    {-0.5 0} % x- und y-Koordinate der GF-Mitte
    {0 1}   % Richtungsvektor der GF (x und y)
    -0.5 0.5 % Grenzen fuer die NA, gesehen als Faktoren des GF-Vektor
    trans   % Verhalten der GF, kann 'trans', 'refl' oder 'abs' sein
    {PlainIfc} % Art der GF
    % Dasselbe fuer die rechte GF, nur die Mitte ist anders
    {0.5 0} {0 1} -0.5 0.5 trans {PlainIfc}
    \POE@key@n % Brechungsindex, koennte jede Zahl sein
  }%
}%
```

Unsere Komponent sieht also folgendermaßen aus, und der Strahl verläuft nicht mehr innerhalb der Komponente:



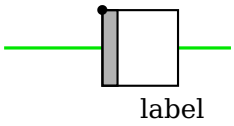

```
\begin{pspicture}(3, 2)
\newOptexpDipole{testcomp}
\makeatletter
\def\testcomp@comp{%
  \psframe[fillstyle=solid, fillcolor=black!30](-0.5,-0.5)(-0.3,0.5)
  \psframe(-0.5,-0.5)(0.5,0.5)}%
\def\testcomp@nodes{%
  \newOptexpComp{%
    {-0.5 0} {0 1} -0.5 0.5 trans {PlainIfc}
    {0.5 0} {0 1} -0.5 0.5 trans {PlainIfc} \POE@key@n }}%
\makeatother
\begin{optexp}
\testcomp[position=0.6, labelalign=l](0,1)(3,1){label}
\drawbeam[beaminside=false](0,1){}(3,1)
\end{optexp}
\end{pspicture}
```

10.2.3. Unterstützung von externen Knoten

Damit nun, als letzter Schritt auch externe Knoten mit `extnode`, Positionierung an den Anfang oder das Ende mit `position=start|end`, oder auch das Setzen eines anderen Drehpunktes mit `rotateref` möglich wird, müssen die externen Dimensionen der Komponente bekannt gemacht werden. Dafür muss ein weiteres Makro mit dem Name `\`⟨*name*⟩`@ref` definiert werden, in dem auf Postscript-Ebene die Werte `@@x` (halbe Breite), `@@y` (halbe Höhe), und optional auch `@@x0` und `@@y0` (Verschiebung der Mitte) definiert werden müssen.

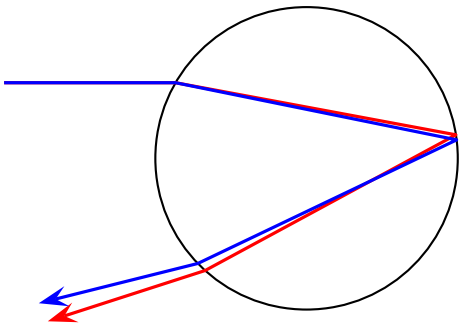
```
\def\testcomp@ref{%
  \POE@setref{ /@@x 0.5 def /@@y 0.5 def }%
}%
```

Der Code für unsere Beispielkomponente `\testcomp` ist damit vollständig, und sieht folgendermaßen aus:




```
\begin{pspicture}(3, 2)
\newOptexpDipole{testcomp}
\makeatletter
\def\testcomp@comp{%
  \psframe[fillstyle=solid, fillcolor=black!30](-0.5,-0.5)(-0.3,0.5)
  \psframe(-0.5,-0.5)(0.5,0.5)}%
\def\testcomp@nodes{%
  \newOptexpComp{%
    {-0.5 0} {0 1} -0.5 0.5 trans {PlainIfc}
    {0.5 0} {0 1} -0.5 0.5 trans {PlainIfc} \POE@key@n }}%
\def\testcomp@ref{%
  \POE@setref{ /@@x 0.5 def /@@y 0.5 def }}%
\makeatother
\begin{optexp}
\testcomp[position=0.6, labelalign=l, extnode=tl](0,1)(3,1){label}
\drawbeam[beaminside=false](0,1){}(3,1)
\psdot(\oenodeExt{})
\end{optexp}
\end{pspicture}
```

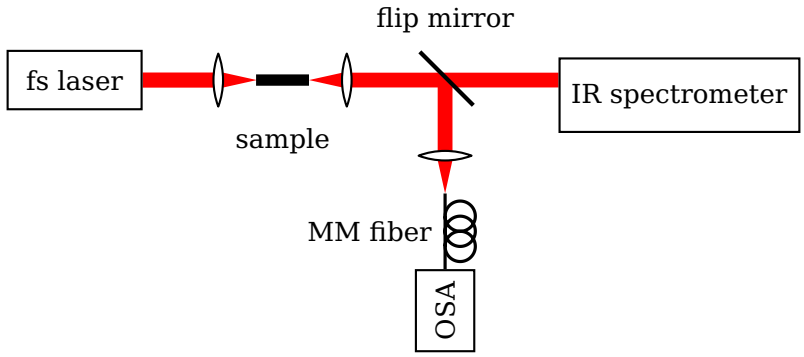
10.2.4. Beispiele für benutzerdefinierte Komponenten



```
\begin{pspicture}(6,4)
  \newOptexpTripole{raindrop}
  \makeatletter
  \def\raindrop@nodes{%
    \newOptexpComp{%
      {0 -2} { 0 2 } -2 2 trans {CurvedIfc}
      {0 2} {0 -2} -2 2 refl {CurvedIfc}
      {0 -2} { 0 2 } -2 2 trans {CurvedIfc}
      \POE@key@n }}%
  \def\raindrop@comp{\pscircle(0,0){2}}%
  \raindrop(0,4)(4,2)(0,0)
  \psset{beamalign=abs, arrows=->, arrowscale=2}
  \drawbeam[linecolor=red](0,3){}(0,1)
  \drawbeam[n=*n*1.05, linecolor=blue](0,3){}(0,1)
\end{pspicture}
```

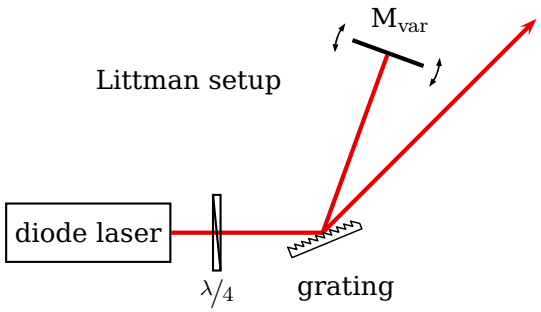
11. Beispiele

Bsp. 11.1



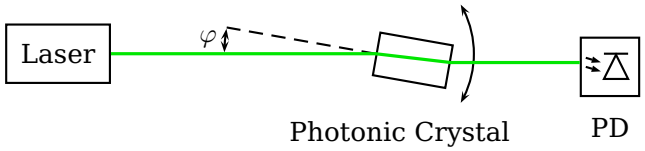

```
\begin{pspicture}(0.1,0.4)(10,4.5)
  \psset[optexp]{lens=1 1 0.7, loadbeampoints}
  \pnodes(2,4){L}([Xnodesep=4]L){M}([Xnodesep=1.5]M){IRSpec}([offset=-2.5]M){OSA}
  \begin{optexp}
  \optbox[position=start, innerlabel, optboxwidth=1.8](L)(M){fs laser}
  \lens[abspos=1](L)(M)
  \optbox[abspos=1.85, optboxsize=0.7 0.15, fillstyle=solid,
      fillcolor=black](L)(M){sample}
  \lens[abspos=2.7](L)(M)
  \mirror[labelangle=45](L)(M)(OSA){flip mirror}
  \optbox[position=end, optboxsize=3.2 1, innerlabel, compshift=-0.2](M)(IRSpec){IR
      spectrometer}
  \lens[abspos=1](M)(OSA)
  \optplane[angle=90]([offset=-1.5]M)
  \optfiber[fiberloopradius=0.2, label=1,
      newFiber={linewidth=1.5\pslinewidth}]([offset=-1.5]M)(OSA){MM fiber}
  \optbox[position=end, innerlabel, optboxwidth=1.1](M)(OSA){OSA}
  \newpsstyle{Beam}{linestyle=none, fillcolor=red, fillstyle=solid, raytrace=false}
  \drawwidebeam[beamwidth=0.2, stopinside]{1-2}
  \drawwidebeam[beamdiv=-25]{2-3}
  \drawwidebeam[loadbeampoints=false, beamdiv=25]{3-4}
  \drawwidebeam[startinside]{4-5}
  \drawwidebeam[savebeampoints=false, beamalign=abs]{5-6}
  \drawwidebeam[stopinside]{5}{7}
  \drawwidebeam[beamdiv=-25]{7-8}
\end{optexp}
\end{pspicture}
```

Bsp. 11.2



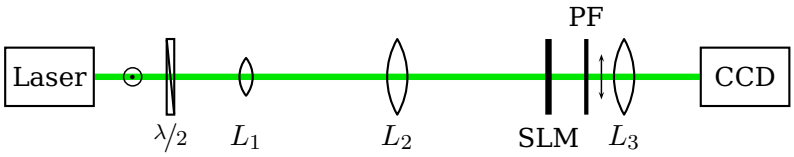

```
\begin{pspicture}(-4.2,-1)(3,3)
\pnodes(-2,0){LOut}(0,0){Grat}(4;45){Out}(2.5;70){Mvar}
\newpsstyle{Beam}{linewidth=2\pslinewidth, linecolor=red!90!black}
\begin{optexp}
  \optbox[optboxwidth=2.2, labeloffset=0, position=start](LOut)(Grat){diode laser}
  \mirror[variable, label=0.5](Grat)(Mvar)(Grat){M$_\mathrm{var}$}
  \optretplate[position=0.3](LOut)(Grat){$\nicefrac{\lambda}{4}$}
  \optgrating(LOut)(Grat)(Out){grating}
  \drawbeam[arrows=->]{1}{3}{4}(Out)
  \drawbeam{2}{4}
\end{optexp}
\rput[l](-3,2){Littman setup}
\end{pspicture}
```

Bsp. 11.3



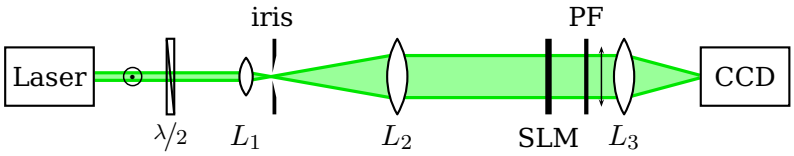

```
\begin{pspicture}(8.5,1.9)
\pnodes(1.4,1.3){Laser}(7.6,1.3){Diode}
\optbox[position=start, labeloffset=0](Laser)(Diode){Laser}%
\optbox[abspos=4, optboxsize=1 0.6, labeloffset=1, n=3,
       compname=PC, angle=-10, rotateref=l](Laser)(Diode){Photonic Crystal}
\optdetector[dettype=diode](\oenodeOut{PC})(Diode|\oenodeOut{PC}){PD}
\nodexn{(\oenodeIn{PC}) + (2;170)}{Angle1}
\psline[linestyle=dashed](\oenodeIn{PC})(Angle1)
\psarc{<->}(\oenodeIn{PC}){1.3}{330}{30}
\psarc[arcsep=1pt]{<->}(\oenodeIn{PC}){2}{170}{180}
\uput{2.1}[175](\oenodeIn{PC}){\small $\varphi$}
\drawbeam{-}
\end{pspicture}
```

Bsp. 11.4



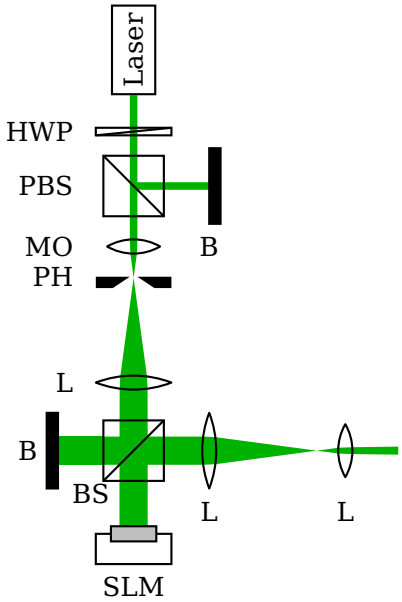

```
\begin{pspicture}(10.4,1.7)
\pnodes(1.2,1){Start}(9.2,1){CCD}
\begin{optexp}
\optbox[position=end, label=0, optboxwidth=1.2](CCD)(Start){Laser}
\optbox[position=end, label=0, optboxwidth=1.2](Start)(CCD){CCD}
\polarization[poltype=perp,abspos=0.5](Start)(CCD)
\optretplate[abspos=1](Start)(CCD){$\nicefrac{\lambda}{2}$}
\lens[lens=0.4 0.4 0.5,abspos=2](Start)(CCD){$L_1$}
\lens[abspos=4](Start)(CCD){$L_2$}
\optplate[abspos=6, platelinewidth=3\pslinewidth](Start)(CCD){SLM}
\optplate[abspos=6.5, labelangle=180](Start)(CCD){PF}
\polarization[abspos=6.7](Start)(CCD)
\lens[abspos=7](Start)(CCD){$L_3$}
\drawbeam[linewidth=2\pslinewidth]{1}{2}
\end{optexp}
\end{pspicture}
```

Bsp. 11.5



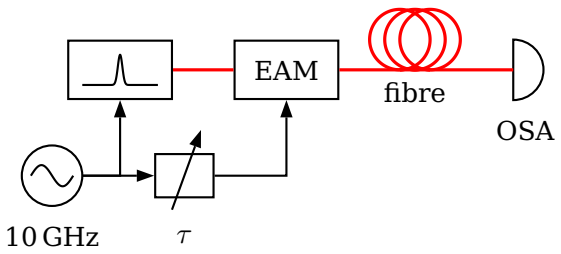

```
\begin{pspicture}(10.4,1.7)
\pnodes(1.2,1){Start}(9.2,1){CCD}
\begin{optexp}
\optbox[position=start, label=0, optboxwidth=1.2](Start)(CCD){Laser}
\polarization[poltype=perp,abspos=0.5](Start)(CCD)
\optretplate[abspos=1](Start)(CCD){$\nicefrac{\lambda}{2}$}
\lens[lens=0.4 0.4 0.5,abspos=2](Start)(CCD){$L_1$}
\pinhole[phwidth=0.05, abspos=2.35, labelangle=180](Start)(CCD){iris}
\lens[abspos=4](Start)(CCD){$L_2$}
\optplate[abspos=6,platelinewidth=3\pslinewidth](Start)(CCD){SLM}
\optplate[abspos=6.5,labelangle=180](Start)(CCD){PF}
\lens[abspos=7](Start)(CCD){$L_3$}
\optbox[position=end, label=0, optboxwidth=1.2](Start)(CCD){CCD}
\polarization[abspos=6.7, polsize=0.8](Start)(CCD)
\addtopsstyle{Beam}{fillstyle=solid, fillcolor=green!40!white, loadbeampoints}
\drawwidebeam[beamwidth=0.1, stopinside]{1}{4}
\drawwidebeam[beamdiv=-20]{4}{6}
\drawwidebeam[stopinside]{6-9}
\drawwidebeam[beamdiv=-30]{9-10}
\end{optexp}
\end{pspicture}
```

Bsp. 11.6: Angepasst von arXiv:1112.5270v2



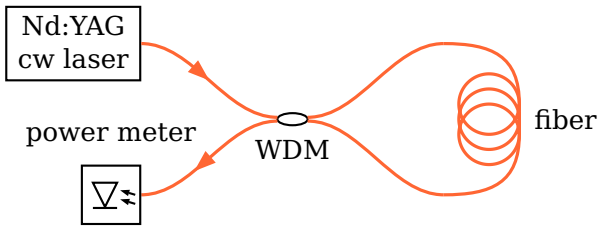

```
\begin{pspicture}(0.3,0.4)(5.5,8.3)
\pnodes(2,7){L}(2,5.8){Pbs}([Xnodesep=1]Pbs){Blk1}(2,2.3){Bs}
\pnodes([Xnodesep=-1]Bs){Blk2}([Xnodesep=3.5]Bs){Out}(2,1.3){Slm}
\begin{optexp}
  \optbox[position=end, innerlabel, optboxsize=1.2 0.6](Pbs)(L){Laser}
  \psset{labelalign=r, mirrortype=extended}
  \newpsstyle{ExtendedMirror}{fillstyle=solid, fillcolor=black}
  \optretplate[position=0.4](L)(Pbs){HWP}
  \beamsplitter[labelangle=-90](L)(Pbs)(Blk1){PBS}
  \mirror[label=. -90 c](Pbs)(Blk1)(Pbs){B}
  \lens[abspos=0.8, lens=0.7 0.7 0.7, n=2](Pbs)(Bs){MO}
  \pinhole[phwidth=0.15, abspos=1.2](Pbs)(Bs){PH}
  \lens[abspos=2.6, lensradius=1.4](Pbs)(Bs){L}
  \psset{labelalign=c}
  \beamsplitter[labelangle=-135](Pbs)(Bs)(Blk2){BS}
  \mirror[labeloffset=0.4](Bs)(Blk2)(Bs){B}
  \lens[abspos=1, lensradius=1.4](Bs)(Out){L}
  \lens[abspos=2.8, lens=0.7 0.7 0.7, n=2](Bs)(Out){L}
  \opttripole(Bs)(Slm)(Bs){%
    \psframe[fillstyle=solid,fillcolor=gray!50](-0.3,0)(0.3,0.2)
    \psline(-0.3,0.1)(-0.5,0.1)(-0.5,0.5)(0.5,0.5)(0.5,0.1)(0.3,0.1)}{SLM}
  \addtopsstyle{Beam}{linestyle=none, fillstyle=solid, fillcolor=green!70!black}
  \drawwidebeam[beamwidth=0.1]{1-4}
  \drawwidebeam[beamwidth=0.1, beaminsidefirst, beaminsidelast]{3}{5-8}
  \psset{savebeampoints=false, loadbeampoints}
  \drawwidebeam{8-11}(Out)\drawwidebeam[beaminsidefirst]{8}{12}
\end{optexp}
\end{pspicture}
```

Bsp. 11.7



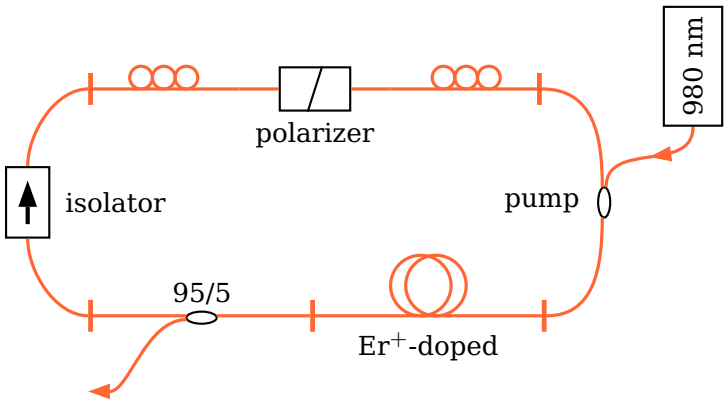

```
\begin{pspicture}(6.4,3.2)
\addtopsstyle{Fiber}{linecolor=red}\psset{fiber=none}
\pnodes(2.3,2.3){Lin}([Xnodesep=4.5]Lin){Det}
\optbox[label=0.2, position=start, compname=L, extnode=b](Lin)(Det){%
  \psGauss[yunit=0.03,sigma=0.03]{-0.5}{0.5}}
\optbox[label=0, compname=EAM, extnode=b, abspos=1.5](Lin)(Det){EAM}
\optfiber[labeloffset=0.3, abspos=3.2](Lin)(Det){fibre}
\optdetector(Lin)(Det){OSA}
\pnode([Xnodesep=-0.5,offset=-1]\oenodeExt{L}){Osc}
\elecsynthesizer[wire=none](Osc)(\oenodeExt{EAM}|Osc){10\,GHz}
\optarrowcomp[arrowcompsize=0.8 0.6, arrowscale=1.5](Osc)(\oenodeExt{EAM}|Osc){$\tau$}
\drawfiber{1-4}
\addtopsstyle{Wire}{arrows=->, arrowinset=0, arrowscale=1.5}
\drawwire{5}{6}\psset[optexp]{wireangleB=90}
\drawwire{6}(\oenodeExt{EAM})\drawwire{5}(\oenodeExt{L})
\end{pspicture}
```

Bsp. 11.8



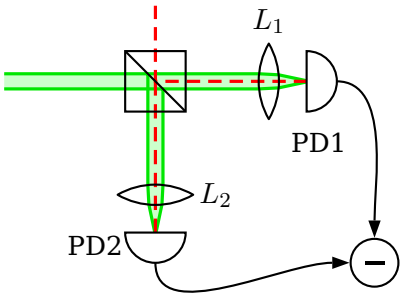

```
\begin{pspicture}(0.2,0.6)(8.2,3.5)
\pnodes(2,3){Laser}(2,1){PwMeter}(6,3){CplTop}(6,1){CplBot}
\psset{arrowscale=1.5, arrowinset=0}
\optbox[position=start, optboxsize=1.8 1, labeloffset=0](Laser)([Xnodesep=0.1]Laser){%
  \begin{tabular}{@{}c@{}}Nd:YAG\\[-0.4ex]cw laser\end{tabular}}
\optcoupler[addtoFiberIn1={ArrowInside=->}, addtoFiberIn2={ArrowInside=-<},
          labeloffset=0.4](Laser)(PwMeter)(CplTop)(CplBot){WDM}
\optfiber[addtoFiberOut={ncurv=1, angleB=0}, addtoFiberIn={ncurv=1, angleA=0},
        compshift=-1, label=0.2 . l](CplBot)(CplTop){fiber}
\optdetector[dettype=diode]([Xnodesep=0.1]PwMeter)(PwMeter){power meter}
\end{pspicture}
```

Bsp. 11.9



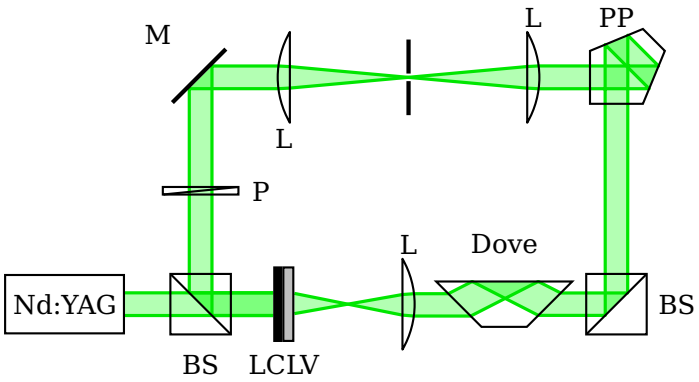

```
\begin{pspicture}(0.9,0.9)(10.4,6.1)
\psset{arrowscale=1.5, arrowinset=0}
\pnodes(2,5){PC1in}(4,5){PC1out}(6,5){PC2in}(8,5){PC2out}(2,2){CplSig}
\pnodes(5,2){CplIn}(2,1){CplOut}(10,4.5){Pump}(8,2){PumpSig}
\optisolator[compshift=0.8, addtoFiberIn={angleA=180},
           addtoFiberOut={angleB=180}, label=0.5 . l]%
           (CplSig)(PC1in){isolator}
\polcontrol[addtoFiberIn={arrows=|-}](PC1in)(PC1out)
\optfiberpolarizer[labeloffset=0.6](PC1out)(PC2in){polarizer}
\polcontrol[addtoFiberOut={arrows=-|}](PC2in)(PC2out)
\wdmsplitter[labeloffset=0.3, coupleralign=bottom, addtoFiberIn={arrows=|-},
           addtoFiberOut1={arrows=->}, addtoFiberOut2={arrows=-|}]%
           (CplIn)(CplOut)(CplSig){95/5}
\wdmcoupler[addtoFiberIn1={ArrowInside=->}, addtoFiberIn2={angleA=0},
          addtoFiberOut={angleB=0,arrows=-|}, ncurv=0.9,
          coupleralign=bottom, compshift=0.8](Pump)(PC2out)(PumpSig){pump}
\optbox[position=start, innerlabel, optboxwidth=1.6](Pump)([offset=-0.1]Pump){980~nm}
\optfiber[fiberloops=2, labeloffset=0.4](CplIn)(PumpSig){Er$^+$-doped}
\end{pspicture}
```

Bsp. 11.10



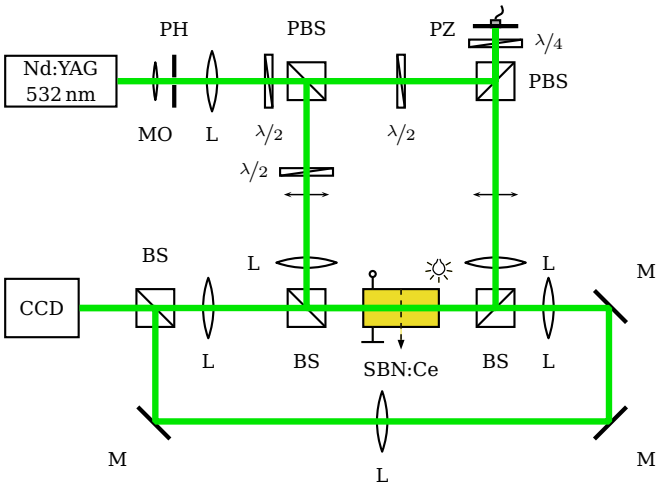

```
\begin{pspicture}(1,1.2)(6.2,5)
\pnodes(1,4){SigIn}(3,4){BS}(3,5){LO}(5,4){Det1}(3,2){Det2}
\begin{optexp}
  \optplane[compname=LO, angle=90](LO)
  \beamsplitter[compname=BS](SigIn)(BS)(Det2)
  \lens[abspos=0.5, compname=L2, n=2.1](Det2)(BS){$L_2$}
  \lens[abspos=0.5, compname=L1, n=2.1](Det1)(BS){$L_1$}
  \psset[optexp]{extnodealign=rel, extnode=r}
  \optdetector[compname=Det1](BS)(Det1){PD1}
  \optdetector[compname=Det2](BS)(Det2){PD2}
  \addtopsstyle{Beam}{beamwidth=0.2, fillstyle=solid,
                      fillcolor=green, opacity=0.2}
  \drawwidebeam(SigIn){BS}{L2}{Det2}
  \drawwidebeam[beaminsidefirst]{BS}{L1}{Det1}
  \newpsstyle{Beam}{linecolor=red, linestyle=dashed, linewidth=1.5\pslinewidth}
  \drawbeam{LO}{BS}{L1}{Det1}
  \drawbeam[beaminsidefirst]{BS}{L2}{Det2}
\end{optexp}
\cnodeput[framesep=5pt]([Xnodesep=0.5,
    offset=-0.5]\oenodeExt{Det1}|\oenodeExt{Det2}){M}{\rule{2.5mm}{1pt}}
\psset{arrows=<-, arrowscale=1.5, arrowinset=0}
\nccurve[angleA=90]{M}{\oenodeExt{Det1}}
\nccurve[angleA=180, angleB=-90]{M}{\oenodeExt{Det2}}
\end{pspicture}
```

Bsp. 11.11



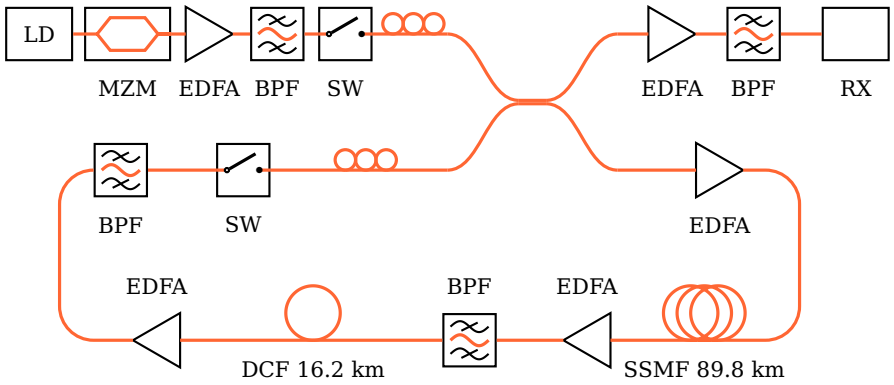

```
\begin{pspicture}(-0.2,0)(9,5)
\psset[optexp]{lens=1.2 0 1.2, n=1.72}
\addtopsstyle{Beam}{fillstyle=solid, fillcolor=green, opacity=0.3}
\pnodes(2.4,1){BS1}([offset=3]BS1){M1}([Xnodesep=5.5]M1){PP}(PP|BS1){BS2}
\begin{optexp}
  \optbox[label=0, position=start, optboxwidth=1.6]([Xnodesep=-1]BS1)(BS1){Nd:YAG}
  \beamsplitter[compname=BS](BS2)(BS1)(M1){BS}
  \optdipole[position=0.2, compname=LCLV, optdipolesize=0.28 1](BS1)(BS2){%
    \psframe[fillstyle=solid,fillcolor=black,dimen=outer](-0.14,-0.5)(-0.02,0.5)
    \psframe[fillstyle=solid,fillcolor=gray!50,dimen=outer](-0.02,-0.5)(0.14,0.5)
  }{LCLV}
  \optretplate(BS1)(M1){P}
  \mirror(BS1)(M1)(PP){M}
  \lens[position=0.2](M1)(PP){L}
  \pinhole(M1)(PP)
  \lens[position=0.2](PP)(M1){L}
  \pentaprism(M1)(PP)(BS2){PP}
  \beamsplitter(PP)(BS2)(BS1){BS}
  \doveprism[compname=Dove, position=0.27, n=2.3](BS2)(BS1){Dove}
  \lens[n=2.5](BS2)(BS1){L}
  \drawwidebeam[beamwidth=0.3]{1-3}
  \drawwidebeam[loadbeampoints]{3}{2}{4-}{3}
\end{optexp}
\end{pspicture}
```

Bsp. 11.12



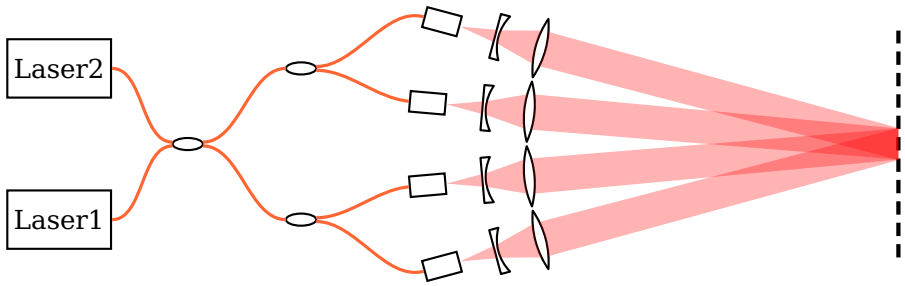

```
\begin{pspicture}(0,-0.2)(8.6,5.6)
\pnodes(1.5,5){Laser}(4,5){PBS}(6.5,5){PBS2}(6.5,5.7){piezo}(4,2){BSFwd}
\pnodes(6.5,2){BSBwd}(2,2){BS4f}(2,0.5){M4f3}(8,2){M4f1}(8,0.5){M4f2}(1,2){CCD}
\psset{mirrorwidth=0.6, plateheight=0.7, outerheight=0.7, labeloffset=0.7,
    labelstyle=\scriptsize, lens=1.2 1.2 0.8, bssize=0.5}
\optbox[position=start, optboxsize=1.5 0.7, innerlabel]%
  (Laser)(PBS){\parbox{1.5cm}{\centering Nd:YAG\\ 532\,nm}}
\lens[lensheight=0.5, position=0.2](Laser)(PBS){MO}
\pinhole[position=0.3,labelangle=180](Laser)(PBS){PH}
\lens[position=0.5](Laser)(PBS){L}
\optretplate[position=0.8](Laser)(PBS){$\nicefrac{\lambda}{2}$}
\beamsplitter(Laser)(PBS)(BSFwd){PBS}
\optretplate[position=0.4](PBS)(BSFwd){$\nicefrac{\lambda}{2}$}
\polarization(PBS)(BSFwd)\polarization(PBS2)(BSBwd)
\lens[position=0.8](PBS)(BSFwd){L}
\optretplate(PBS)(PBS2){$\nicefrac{\lambda}{2}$}
\beamsplitter(PBS)(PBS2)(piezo){PBS}
\optretplate[abspos=0.5](PBS2)(piezo){$\nicefrac{\lambda}{4}$}
\mirror[mirrortype=piezo,labelangle=90](PBS2)(piezo)(PBS2){PZ}
\lens[position=0.8,labelangle=180](PBS2)(BSBwd){L}
\crystal[crystalsize=1 0.5, voltage, lamp, fillstyle=solid,
    fillcolor=yellow!90!black, labeloffset=0.8](BSFwd)(BSBwd){SBN:Ce}
\beamsplitter(PBS)(BSFwd)(BSBwd){BS}
\beamsplitter[labelangle=-90](PBS2)(BSBwd)(BSFwd){BS}
\mirror(BSBwd)(M4f1)(M4f2){M}\mirror(M4f1)(M4f2)(M4f3){M}
\lens[labelangle=180](M4f2)(M4f3){L}\mirror(M4f2)(M4f3)(BS4f){M}
\beamsplitter(M4f3)(BS4f)(CCD){BS}
\optbox[position=end, label=0, optboxwidth=1](BS4f)(CCD){CCD}
\lens[abspos=0.7](BS4f)(BSFwd){L}\lens[abspos=0.7](BSBwd)(M4f1){L}
\addtopsstyle{Beam}{linewidth=2\pslinewidth, linejoin=2}
\drawbeam{1-6}{11-14}{13}{12}{15}{18}{16}{17}{25}{23}{24}
\drawbeam[beaminsidefirst]{6}{7}{10}{17}{16}{18}{26}{19-24}
\end{pspicture}
```

Bsp. 11.13



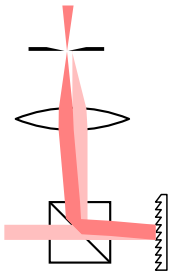

```
\begin{pspicture}(13,5.5)
\psset{optboxwidth=1, labelstyle=\footnotesize, fiber=none}
\pnodes(1,5){LD}([Xnodesep=5.5]LD){CPLin1}([offset=-2]CPLin1){CPLin2}
\pnodes([Xnodesep=2.5]CPLin1){CPLout1}([Xnodesep=2.5]CPLin2){CPLout2}
\pnode([Xnodesep=3]CPLout1){RX}
\optbox[position=start, label=0](LD)(CPLin1){LD}
\optmzm[abspos=0.8](LD)(CPLin1){MZM}
\optamp[abspos=2](LD)(CPLin1){EDFA}
\optfilter[abspos=3](LD)(CPLin1){BPF}
\optswitch[abspos=4](LD)(CPLin1){SW}
\polcontrol[abspos=5, fiber=out](LD)(CPLin1)
\drawfiber{1-6}
\optcoupler[couplertype=none, fiber](CPLin1)(CPLin2)(CPLout1)(CPLout2)
\optamp[abspos=0.8, fiber=in](CPLout1)(RX){EDFA}
\optfilter[abspos=2](CPLout1)(RX){BPF}
\optbox[position=end](CPLout1)(RX){RX}
\drawfiber{8-10}
\pnodes([Xnodesep=-0.5]LD|CPLin2){TL}(RX|TL){TR}([offset=-2.5]TR){BR}(TL|BR){BL}
\optamp[fiber=i](CPLout2)(TR){EDFA}
\optfiber[label=0.3 . t, position=0.85](BL)(BR){SSMF 89.8~km}
\drawfiber[fiberstyle=angle, arm=1.2, linearc=0.5, startnode=N]{11}{12}
\optamp[position=0.3](BR)(BL){EDFA}
\optfilter[position=0.55, labelangle=180](BL)(BR){BPF}
\optfiber[fiberloops=1, label=0.3 . t, position=0.35](BL)(BR){DCF 16.2~km}
\optamp[position=0.85](BR)(BL){EDFA}
\optfilter[position=0.2](TL)(CPLin2){BPF}
\optswitch(TL)(CPLin2){SW}
\polcontrol[position=0.8, fiber=out](TL)(CPLin2)
\drawfiber{12-16}\drawfiber{17-19}
\drawfiber[fiberstyle=angle, arm=0.5, linearc=0.5, stopnode=1]{16}{17}
\end{pspicture}
```

Bsp. 11.14



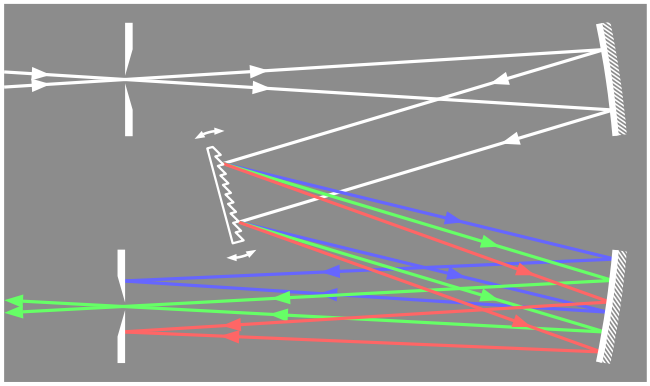

```
\begin{pspicture}(12, 4)
  \psset{labeloffset=0.3, fiber=none}
  \pnodes(1.6,2){In}(12, 2){Ref}
  \optplate[linestyle=dashed, plateheight=3, position=1, compname=RefPlane](In)(Ref)
  \addtopsstyle{Beam}{linestyle=none, beamdiv=20, beaminside=false, fillstyle=solid,
      fillcolor=red, opacity=0.3}
  \multido{\i=1+1, \ii=165+10}{4}{%
    \rput(Ref){\pnode(6;\ii){A\i}}
    \optbox[optboxsize=0.5 0.3, position=end, compname=Box\i, extnode=l](Ref)(A\i)
    \lens[lens=0 -0.6 0.6, abspos=0.5, compname=L\i](A\i)(Ref)
    \lens[lens=1.2 1.2 0.8, abspos=1.1, n=1.65, compname=LC\i](A\i)(Ref)
    \drawwidebeam{Box\i}{L\i}{LC\i}{RefPlane}
  }%
  \optcoupler[abspos=1](In)(In)(Ref)(Ref)
  \wdmsplitter[abspos=2.5, compshift=1](In)(Ref)(Ref)
  \wdmsplitter[abspos=2.5, compshift=-1](In)(Ref)(Ref)
  \optbox[compshift=-1, position=start, label=0](In)(Ref){Laser1}
  \optbox[compshift=1, position=start, label=0](In)(Ref){Laser2}
  \drawfiber{17}{14}{15}{Box1}\drawfiber{15}{Box2}
  \drawfiber{18}{14}{16}{Box3}\drawfiber{16}{Box4}
\end{pspicture}
```

Bsp. 11.15



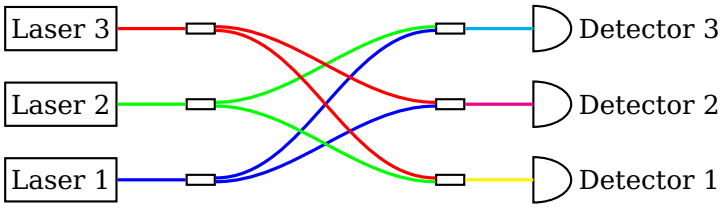
```
\begin{pspicture}(0.5,0)(2.2,4)
  \pnodes(0,1){A}(1,1){BS}(2,1){G}(1,4){B}
  \beamsplitter(G)(BS)(B) \optgrating(BS)(G)(BS)
  \lens[lens=2 2 1.5, compshift=0.1, n=2.25](BS)(B)
  \pinhole[phwidth=0.05, innerheight=0.05, position=0.8,
      compshift=0.18](BS)(B)
  \optplane[angle=90](B)
  \addtopsstyle{Beam}{linestyle=none, beamwidth=0.2, fillstyle=solid}
  \drawwidebeam[fillcolor=red!25!white](A){1-2}{1}{3-5}
  \drawwidebeam[fillcolor=red!50!white, beamangle=-5]{2}{1}{3-5}
\end{pspicture}
```

Bsp. 11.16: Angepasst von Wikipedia



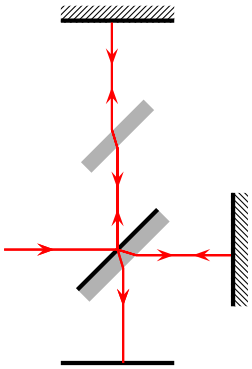
```
\begin{pspicture}(0,-1)(8.5,4)
\psframe[fillstyle=solid,fillcolor=gray!90,linestyle=none](0,-1)(8.5,4)
\addtopsstyle{OptComp}{linecolor=white}
\newpsstyle{Beam}{ArrowInside=->, linejoin=2, arrowscale=1.3, arrowinset=0}
\addtopsstyle{ExtendedMirror}{hatchcolor=white, hatchsep=0.5\pslinewidth}
\pnodes(0,3){A}(8,3){B}(3,1.5){C}(8,0){D}(0,0){E}%
\psset{linewidth=1.5\pslinewidth, mirrorradius=13, mirrorwidth=1.5,
    gratingwidth=1.3, mirrortype=extended, phwidth=0.1, outerheight=1.5}%
\begin{optexp}
  \pinhole[position=0.2](A)(B)%
  \mirror(A)(B)(C)
  \optgrating[reverse, angle=15, variable](B)(C)(D)
  \mirror(C)(D)(E)
  \pinhole[position=0.8](D)(E)
  \drawwidebeam[ArrowInsidePos=0.3, beamwidth=0.2, linecolor=white,
      beamdiv=-7.15](A){1-3}
  \psset{loadbeampoints, savebeampoints=false}%
  \drawwidebeam[ArrowInsidePos=0.6, beamangle=3, linecolor=blue!60!white]{3-5}(E)
  \drawwidebeam[ArrowInsidePos=0.7, linecolor=green!60!white, arrows=->,
      ArrowInsideMinLength=2]{3-5}(E)
  \drawwidebeam[ArrowInsidePos=0.8, beamangle=-3, linecolor=red!60!white]{3-5}(E)
\end{optexp}
\end{pspicture}
```

Bsp. 11.17



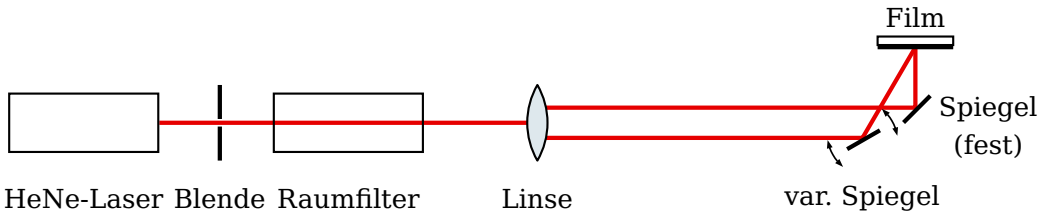

```
\begin{pspicture}(0,0.7)(9.4,3.3)
  \psset[optexp]{couplertype=rectangle, fiber=none, detsize=0.5 0.6}
  \multido{\i=1+1}{3}{%
    \optbox[optboxsize=1.5 0.6, innerlabel,position=start](1.5,\i)(7,\i){Laser \i}
    \wdmsplitter[position=0.2](1.5,\i)(7,\i)(7,\i)
    \wdmcoupler[position=0.8](1.5,\i)(1.5,\i)(7,\i)
    \optdetector[label=0.4 90 l](1.5,\i)(7,\i){Detector \i}
  }%
  \drawfiber[linecolor=blue, startnode=N]{1}{2}{7}
  \drawfiber[linecolor=blue]{2}{11}
  \drawfiber[linecolor=green, stopnode=1]{5}{6}{11}
  \drawfiber[linecolor=green, stopnode=2]{6}{3}
  \drawfiber[linecolor=red]{9}{10}{3}
  \drawfiber[linecolor=red, startnode=2]{10}{7}
  \drawfiber[linecolor=cyan]{11}{12}
  \drawfiber[linecolor=magenta]{7}{8}
  \drawfiber[linecolor=yellow]{3}{4}
\end{pspicture}
```

Bsp. 11.18



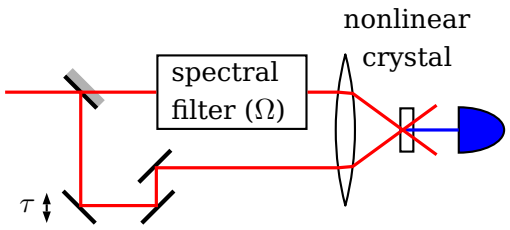
```
\psset{unit=1.5}
\begin{pspicture}(0,1)(2.2,4.2)
  \pnodes(0,2){In}(1,2){G}(1,4){M1}(2,2){M2}(1,1){S}
  \mirror[mirrortype=semitrans](In)(G)(M1)
  \optbox[optboxwidth=0.15, angle=45, style=SemitransMirror](G)(M1)
  \mirror[mirrortype=extended](G)(M1)(G)
  \mirror[mirrortype=extended](G)(M2)(G)
  \optplate[position=end](G)(S)
  \newpsstyle{Beam}{linecolor=red, ArrowInside=->, ↩
      ArrowInsidePos=0.4, arrowscale=1.5}
  \drawbeam(In){1}{4}{1}{5}
  \drawbeam(In){1-3}{2}{1}{5}
\end{pspicture}
```

Bsp. 11.19: Angepasst von semibyte.de



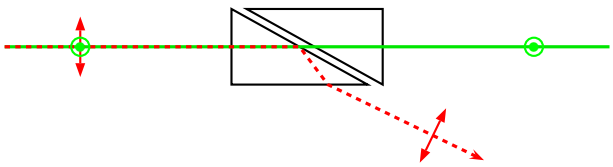

```
\begin{pspicture}(13.6,2.6)
  \psset[optexp]{labeloffset=1, mirrorwidth=0.5, optboxwidth=2}
  \pnodes(2,1){Laser}(12,2){Film}(Film|Laser){Ref}
  \begin{optexp}
    \optbox[position=start](Laser)(Ref){HeNe-Laser}
    \pinhole[abspos=0.8](Laser)(Ref){Blende}
    \optbox[abspos=2.5](Laser)(Ref){Raumfilter}
    \lens[fillstyle=solid, fillcolor={[rgb]{0.87,0.91,0.93}}](Laser)(Ref){Linse}
    \mirror[label=1 30]([offset=0.2]Laser)([offset=0.2]Ref)(Film){%
      \begin{tabular}{c}Spiegel\\(fest)\end{tabular}}
    \opttripole[label=0.4](Ref)(Film)(Ref){%
      \psframe(0.5,1\pslinewidth)(-0.5,5\pslinewidth)
      \psline[linewidth=2\pslinewidth](0.5,0)(-0.5,0)
    }{Film}
    \newpsstyle{Beam}{linewidth=2\pslinewidth, linecolor=red!90!black,
       beamalign=absolute}
    \drawbeam{1-4}
    \drawbeam[beampos=0.2]{4-6}
    \mirror[variable, label=0.8 -30]([offset=-0.2]Laser)([offset=-0.2,
       Xnodesep=-0.7]Ref)(Film){var. Spiegel}
    \drawbeam[beampos=-0.2]{4}{7}{6}
  \end{optexp}
\end{pspicture}
```

Bsp. 11.20



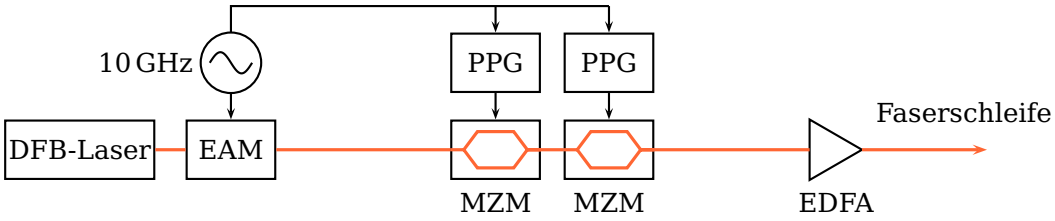

```
\begin{pspicture}(0,0.2)(6.6,3.1)
  \psset{mirrorwidth=0.6, useNA=false, beaminsidefirst}
  \pnodes(0,2){In}(1,2){BS}(1,0.5){M1}(2,1){M2}(6,1.5){Det}
  \mirror[mirrortype=semitrans, n=1](In)(BS)(M1)
  \mirror[label=0.6 180 . absolute, compname=M](BS)(M1)(M2|M1)
  \mirror(M1)(M2|M1)(M2)
  \mirror(M2|M1)(M2)([Xnodesep=1]M2)
  \optbox[optboxsize=2 1, abspos=2, label=0, ↵
     allowbeaminside=false](BS)([offset=0.5]Det){%
    \parbox{1.6\psxunit}{spectral\\filter ($\Omega$)}}
  \lens[lens=4 4 2, n=3.3]([Xnodesep=-3]Det)(Det)
  \optbox[optboxsize=0.2 0.6, n=1, label=-1.2](\oenodeOut{6})(Det){%
    \begin{tabular}{@{}c@{}}nonlinear\\ crystal\end{tabular}}
  \optplate[linestyle=none](\oenodeOut{7})(Det)
  \optdetector[detsize=0.6 0.6, fillstyle=solid, fillcolor=blue](\oenodeOut{6})(Det)
  \newpsstyle{Beam}{linewidth=1.5\pslinewidth}
  \drawbeam[linecolor=blue]{7-9}
  \addtopsstyle{Beam}{linecolor=red}
  \drawbeam(In){1-4}{6-8}
  \drawbeam{1}{5-8}
  \rput[r](\oenodeLabel{M}){$\tau$
    \psline[arrows=<->, arrowinset=0](0.15,0.25)(0.15,-0.15)}
\end{pspicture}
```

Bsp. 11.21



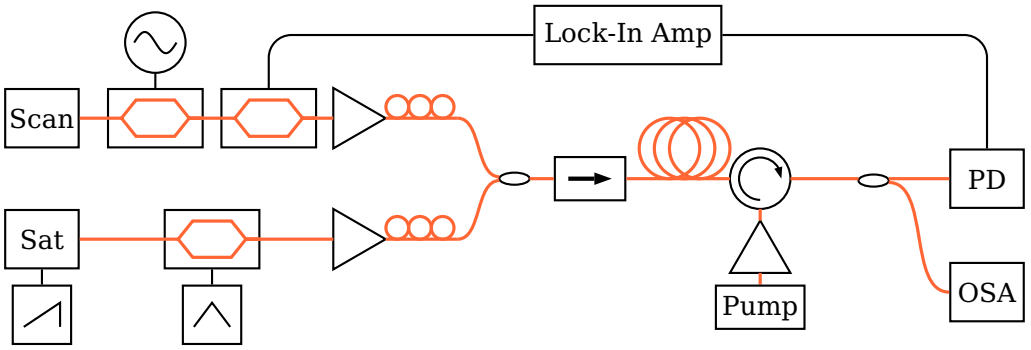
```
\begin{pspicture}(-1,-0.5)(8,1.5)
  \newpsobject{perppol}{polarization}{poltype=perp, linecolor=green, polsize=1}
  \newpsobject{parpol}{polarization}{poltype=parallel, linecolor=red, polsize=0.8}
  \newpsstyle{Polarization}{arrowinset=0, arrowscale=1.2}
  \pnodes(0,1){A}(8,1){B}(4,-0.5){C}
  \glanthompson[glanthompsongap=0.2, glanthompsonsize=2 1](A)(B)
  \optplane(B)\optplane[angle=90](C)
  \drawbeam(A){1-2}
  \drawbeam[linestyle=dashed, dash=2pt 2pt, linecolor=red, arrows=->](A){1}{3}
  \parpol[abspos=1](A)(B)\perppol[abspos=1](A)(B)
  \nodexn{(\oenodeBeam{})-1.5(!\oeBeamVec{})}{D}
  \perppol[abspos=7](A)(B)\parpol(D)(\oenodeBeam{})
\end{pspicture}
```

Bsp. 11.22



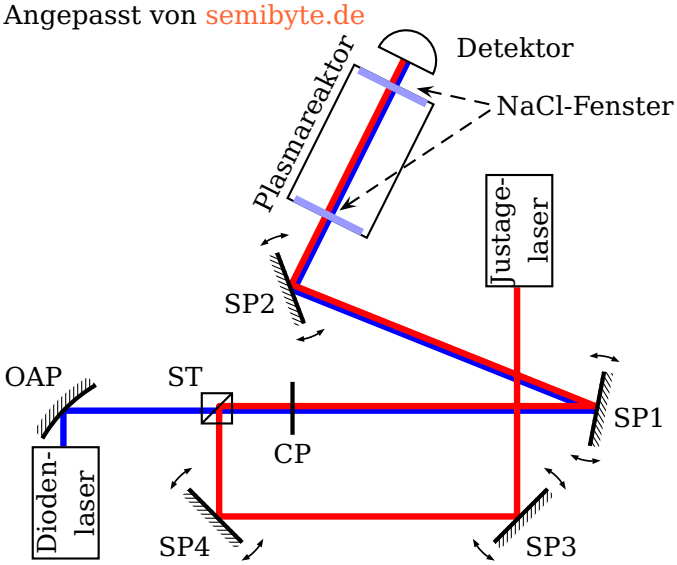

```
\begin{pspicture}(13.7,2.9)
\newpsstyle{Wire}{arm=0.1, arrows=->}
\psset[optexp]{optboxsize=1.2 0.8, optmzmsize=1.2 0.8, fiber=none, label=0.7}
\pnodes(2,1){L}([Xnodesep=11]L){A}
\optbox[position=start, optboxwidth=2, label=0](L)(A){DFB-Laser}
\optbox[abspos=1, label=0, compname=Eam, extnode=t](L)(A){EAM}
\optmzm[abspos=4.5, compname=Mzm1, extnode=t](L)(A){MZM}
\optmzm[abspos=6, compname=Mzm2, extnode=t](L)(A){MZM}
\optamp[abspos=9](L)(A){EDFA}
\drawfiber{1-5}\drawfiber[arrows=->]{}(A)
\pnode([offset=1.5]\oenodeExt{Eam}){Osci}
\elecsynthesizer[compname=Syn, position=0.5, wire=o, label=0.5 .
    r](Osci)(\oenodeExt{Eam}){10\,GHz}
\psset[optexp]{label=0, wire, optboxsize=0.8 1.2}
\optbox[compname=Ppg1](\oenodeExt{Mzm1}|Osci)(\oenodeExt{Mzm1}){PPG}
\optbox[compname=Ppg2](\oenodeExt{Mzm2}|Osci)(\oenodeExt{Mzm2}){PPG}
\drawwire[arrows=-, wireangleB=-90]{Syn}(\oenodeExt{Mzm2}|Osci)
\rput([offset=0.5, Xnodesep=-0.3]A){Faserschleife}
\end{pspicture}
```

Bsp. 11.23: Angepasst von DOI:10.1364/OL.37.000930



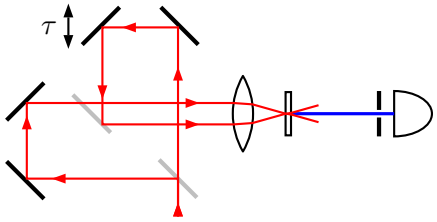

```
\begin{pspicture}(0,0.2)(13.5,4.7)
\psset[optexp]{optboxsize=1 0.8, label=0, fiber=none}
\pnodes(1,3.2){Scan}\pnode([offset=-1.6]Scan){Sat}
\pnode([Xnodesep=5]Scan){ScanCpl}\pnode(ScanCpl|Sat){SatCpl}
\pnode([Xnodesep=1.5,offset=-0.8]ScanCpl){Cpl}\pnode([Xnodesep=5]Cpl){PD}
\pnode([Xnodesep=2.5, offset=-1.4]Cpl){Pump}
\optbox[position=start](Scan)(ScanCpl){Scan}
\optmzm[abspos=1, compname=M1, extnode=t](Scan)(ScanCpl)
\optmzm[abspos=2.5, compname=M2, extnode=t](Scan)(ScanCpl)
\optamp[abspos=3.7](Scan)(ScanCpl)
\polcontrol[abspos=4.5, fiber=o](Scan)(ScanCpl)
\drawfiber{1-5}
\optbox[position=start, compname=S, extnode=b](Sat)(SatCpl){Sat}
\optmzm[abspos=1.75, compname=M3, extnode=b](Sat)(SatCpl)
\optamp[abspos=3.7](Sat)(SatCpl)
\polcontrol[abspos=4.5, fiber=o](Sat)(SatCpl)
\drawfiber{6-9}
\wdmcoupler[fiber=i](ScanCpl)(SatCpl)(Cpl)
\optisolator[position=0.1](Cpl)(Pump|Cpl)
\optfiber[position=0.6](Cpl)(Pump|Cpl)
\addtopsstyle{OptCircArrow}{arrows=<-}
\optcirculator[optcircangle=0 -90](Cpl)(PD)(Pump)
\wdmsplitter[position=0.8, coupleralign=t](Cpl)(PD)
\optbox[position=end, compname=Pd, extnode=t](Cpl)(PD){PD}
\optbox[position=end, compshift=-1.5](Cpl)(PD){OSA}
\optamp[fiber](Pump)(\oenodeIfc{2}{13})
\optbox[position=start, optboxsize=0.6 1.2](Pump)(Pump|Cpl){Pump}
\drawfiber{10-15}\drawfiber{14}{16}
\optbox[compshift=0.7, optboxsize=2.5
    0.8](\oenodeExt{M2})(\oenodeExt{Pd}|\oenodeExt{M2}){Lock-In Amp}
\addtopsstyle{Wire}{arrm=0.1}
\drawwire[wireangleA=90, linearc=0.3](\oenodeExt{M2}){}
\drawwire[wireangleB=90, linearc=0.3]{}(\oenodeExt{Pd})
\elecsynthesizer([offset=0.2]\oenodeExt{M1})(\oenodeExt{M1})
\psset[optexp]{synthshape=rectangle}
\elecsynthesizer[synthtype=sawtooth](\oenodeExt{S}|Pump)(\oenodeExt{S})
\elecsynthesizer[synthtype=triangle](\oenodeExt{M3}|Pump)(\oenodeExt{M3})
\end{pspicture}
```

Bsp. 11.24: Angepasst von semibyte.de



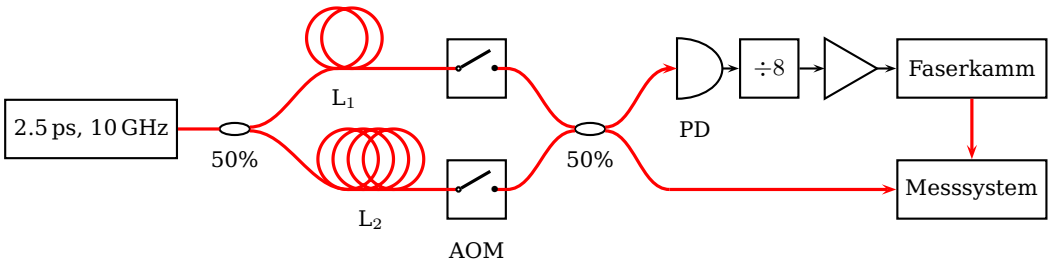

```
\begin{pspicture}(0.2,0.5)(9,7.4)
  \pnode(1,2){DL}\pnode([offset=0.5]DL){OAP}\pnode([Xnodesep=2]OAP){ST}
  \pnode([offset=-1.4]ST){SP4}\pnode([Xnodesep=4]SP4){SP3}
  \pnode([offset=3]SP3){JL}\pnode([Xnodesep=1]SP3|ST){SP1}
  \pnode([Xnodesep=1]ST|JL){SP2}\pnode([Xnodesep=1.5,offset=3]SP2){Det}
  \psset[optexp]{optboxsize=1.5 0.9, labeloffset=0.6}
  \begin{optexp}
    \optbox[position=start, innerlabel](DL)(OAP){\parbox{1.5\psunit}{\centering
       Dioden-\\laser}}
    \psset[optexp]{mirrortype=extended}
    \mirror[mirrorradius=2](DL)(OAP)(ST){OAP}
    \psset[optexp]{variable}
    \beamsplitter[bssize=0.4, labelangle=-45](SP4)(ST)(SP1){ST}
    \optplate[abspos=1, plateheight=0.7](ST)(SP1){CP}
    \mirror(ST)(SP1)(SP2){SP1}\mirror(SP1)(SP2)(Det){SP2}
    \optdipole[label=-0.7 . b relative, optdipolesize=2, position=0.6](SP2)(Det){%
      \psframe[dimen=outer](-1,-0.6)(1,0.6)
      \psset{fillstyle=solid, fillcolor=blue!40, linestyle=none}
      \psframe(-1,-0.5)(-0.9,0.5)\psframe(0.9,-0.5)(1,0.5)}{Plasmareaktor}
    \optdetector[label=. 30 l](SP2)(Det){Detektor}
    \optbox[position=end, innerlabel](SP3)(JL){\parbox{1.5\psunit}{\centering
       Justage-\\laser}}
    \mirror(SP4)(SP3)(JL){SP3}\mirror(ST)(SP4)(SP3){SP4}
    \psset{linewidth=2\pslinewidth}
    \drawbeam[linecolor=blue, beampos=0.03]{1-2}
    \drawbeam[beamalign=abs, loadbeampoints, linecolor=blue]{2-8}
    \drawbeam[linecolor=red, beampos=-0.03]{9-11}{3-8}
  \end{optexp}
  \pnode([Xnodesep=1.2,offset=-0.6]\oenodeIn{8}){NaCl}
  \psset{linestyle=dashed, arrows=->, arrowscale=1.5}
  \ncline[nodesepA=0.1, nodesepB=0.2]{NaCl}{\oenodeIn{7}}
  \ncline[nodesepA=0.1, nodesepB=0.3]{NaCl}{\oenodeOut{7}}
  \rput[l](NaCl){NaCl-Fenster}
\end{pspicture}
```

Bsp. 11.25: Angepasst von DOI:10.1364/AOP.1.000308, Abb. 8



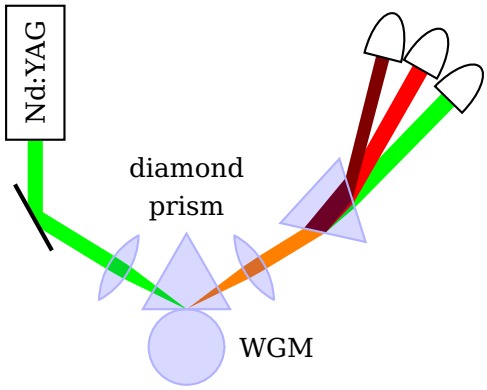

```
\begin{pspicture}(5.7,2.8)
\psset{mirrorwidth=0.7, bsstyle=plate, bssize=0.5, arrowinset=0, arrowscale=1.2}
\newpsobject{splitterplate}{beamsplitter}{linewidth=2\pslinewidth, linecolor=gray!50}
\pnodes(2.3,0){In}([offset=0.5]In){STM1}([Xnodesep=-2]STM1){M1}
\pnodes([offset=1]M1){M2}([offset=2]STM1){M3}([Xnodesep=-1]M3){M4}(M4|M2){STM2}
\splitterplate(In)(STM1)(M1)
\mirror(STM1)(M1)(M2)
\mirror(M1)(M2)(M3|M2)
\mirror(STM1)(M3)(M4)
\mirror[label=0.6 180 . absolute](M3)(M4)(STM2)
\rput[r](\oenodeLabel{}){$\tau$
  \psline[arrows=<->](0.15, 0.4)(0.15, -0.2)}
\splitterplate[compoffset=0.2, compname=STM2](M4)(STM2)([Xnodesep=1]STM2)
\pnode(\oenodeCenter{STM2}){STM2C}
\pnode([Xnodesep=4]STM2C){Out}
\lens[n=2](STM2C)(Out)
\optbox[position=0.65, optboxsize=0.1 0.6](STM2C)(Out)
\optplate[linestyle=none, position=0.75](STM2C)(Out)
\pinhole[position=0.95, outerheight=0.6](STM2C)(Out)
\optdetector[detsize=0.5 0.6](STM2C)(Out)
\drawbeam[linecolor=blue]{8-}
\newpsstyle{Beam}{ArrowInside=->, linecolor=red, ArrowInsideMinLength=1, arrows=>-}
\drawbeam[ArrowInsidePos=0.835](In){1-3}{7-9}
\drawbeam[ArrowInsidePos=0.74](In){1}{4-9}
\end{pspicture}
```

Bsp. 11.26



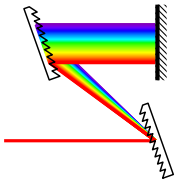

```
\begin{pspicture}(13.8,3.8)
  \pnode(2.3,2){Signal}
  \pnode([Xnodesep=1.5,offset=0.8]Signal){UpIn}
  \pnode([Xnodesep=1.5, offset=-0.8]Signal){LowIn}
  \pnode([Xnodesep=3.5]UpIn){UpOut}
  \pnode(UpOut|LowIn){LowOut}
  \pnode([Xnodesep=1.5]UpOut){UpIn2}
  \pnode([Xnodesep=3]UpIn2){UpOut2}
  \pnode([Xnodesep=1.5]LowOut){LowIn2}
  \pnode(UpOut2|LowIn2){LowOut2}
  \psset{labelstyle=\footnotesize}
  \psset{fiber=none}\addtopsstyle{Fiber}{linecolor=red}
  \psset{wire=none}\addtopsstyle{Wire}{arm=0.1, arrows=->}
  \optbox[position=start, label=0, optboxsize=2.3 ↵
      0.8](Signal)([Xnodesep=0.1]Signal){2.5\,ps, 10\,GHz}
  \wdmsplitter[label=0.4](Signal)(UpIn)(LowIn){50\%}
  \optfiber[fiberloops=4, label=0.4, position=0.3](LowIn)(LowOut){L$_2$}
  \optswitch[position=0.7](LowIn)(LowOut){AOM}
  \optfiber[fiberloops=2, label=0.4, position=0.2](UpIn)(UpOut){L$_1$}
  \optswitch[position=0.7](UpIn)(UpOut)
  \optcoupler[label=0.4, position=0.3](UpOut)(LowOut)(UpIn2)(LowIn2){50\%}
  \optbox[optboxsize=2 0.8, position=end, label=0, ↵
      extnode=t](LowIn2)(LowOut2){Messsystem}
  \drawfiber{1-4}{7}(LowIn2)\drawfiber[arrows=->](LowIn2){8}
  \optdetector[abspos=0.4, detsize=0.6 0.8](UpIn2)(UpOut2){PD}
  \optbox[optboxsize=0.8 0.8, label=0, position=0.44](UpIn2)(UpOut2){$\div 8$}
  \optamp[position=0.8](UpIn2)(UpOut2)
  \optbox[position=end, extnode=b, label=0, optboxsize=2 0.8](UpIn2)(UpOut2){Faserkamm}
  \drawfiber{2}{5-7}(UpIn2)\drawfiber[arrows=->](UpIn2){9}
  \drawwire{9-12}
  \drawfiber[arrows=->](\oenodeExt{12})(\oenodeExt{8})
\end{pspicture}
```

Bsp. 11.27: Angepasst von DOI:10.1103/PhysRevLett.105.263904



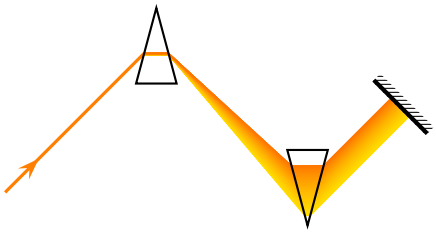

```
\begin{pspicture}(0.6,2.8)(6.8,7.8)
\pnodes(1,6){Laser}(1,5){M}(3,4){PR}(5,5){PT}(6,7){D1}
\newpsstyle{Glass}{fillcolor=blue, fillstyle=solid, opacity=0.15, ↩
    linecolor=blue!30!white}
\begin{optexp}
  \optbox[position=start, innerlabel, optboxwidth=1.8](Laser)(M){Nd:YAG}
  \optprism[style=Glass, compname=P, n=1, prismtype=reflective, prismangle=60, ↩
      labeloffset=1.2](M)(PR)(PT){\begin{tabular}{@{}c@{}}diamond\\ prism\end{tabular}}
  \mirror(Laser)(M)(\oenodeIfc{2}{P})%
  \lens[abspos=1.05, style=Glass](\oenodeIfc{2}{P})(M)%
  \optprism[compname=PDet, style=Glass](\oenodeIfc{2}{P})(PT)(D1)%
  \lens[abspos=1.05, style=Glass](\oenodeIfc{2}{P})(\oenodeOut{PDet})%
  \newpsstyle{Beam}{linestyle=none, fillstyle=solid}%
  \drawwidebeam[fillcolor=green, beamwidth=0.2, beaminsidelast, ↩
      beampathcount=5]{1}{3}{4}{2}%
  \drawwidebeam[fillcolor=orange, beaminsidefirst, startinsidecount=1, loadbeam, ↩
      beaminsidelast]{2}{6}%
  \psset{savebeam=false, loadbeam, beamnodealign=vector}%
  \drawwidebeam[fillcolor=orange]{6}{5}%
  \drawwidebeam[fillcolor=green, n=1.2, beampathskip=1, savebeam=2]{6}{5}(D1)
  \drawwidebeam[fillcolor=red, beampathskip=1, n=1.4, savebeam=3]{6}{5}(D1)
  \drawwidebeam[fillcolor=red!50!black, beampathskip=1, n=1.57, savebeam=4]{6}{5}(D1)
  \multido{\i=2+1}{3}{%
    \backlayer{%
      \pnode(!\oeBeamCenter{\i}){Det\i}
      \nodexn{(Det\i)-(!\oeBeamVecMedian{\i})}{Det\i'}}
    \optdetector[detsize=0.6 0.5](Det\i')(Det\i)}
\end{optexp}
\pscircle[style=Glass]([offset=-0.5]\oenodeIfc{2}{P}){0.5}
\rput[l]([offset=-0.5,Xnodesep=0.7]\oenodeIfc{2}{P}){WGM}
\end{pspicture}
```

Bsp. 11.28



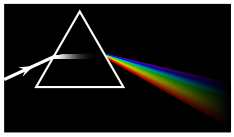
```
\begin{pspicture}(2.2,2.2)
\definecolor[ps]{bl}{rgb}{tx@addDict begin Red Green Blue end}%
\pnode(0,0.5){In}\pnode(2,0.5){G1}
\pnode([Xnodesep=-1.5, offset=1.3]G1){G2}\pnode(G1|G2){MG}
\begin{optexp}
  \optgrating[angle=40, gratingalign=c](In)(G1)(G2)
  \optgrating[reverse, angle=40, gratingalign=c](G1)(G2)(MG)
  \mirror[mirrorwidth=1, mirrortype=extended](G2)(MG)(G2)
  \drawbeam[linecolor=red](In){1}
  \multido{\i=0+1}{40}{%
    \pstVerb{\i\space 650 400 sub 39 div mul 400 add
           tx@addDict begin wavelengthToRGB end }%
    \drawbeam[linecolor=bl, beamangle=\i\space 16 sub 0.2 mul]{1-2}%
    \drawbeam[linecolor=bl, loadbeampoints]{2-3}
  }%
\end{optexp}
\end{pspicture}
```

Bsp. 11.29



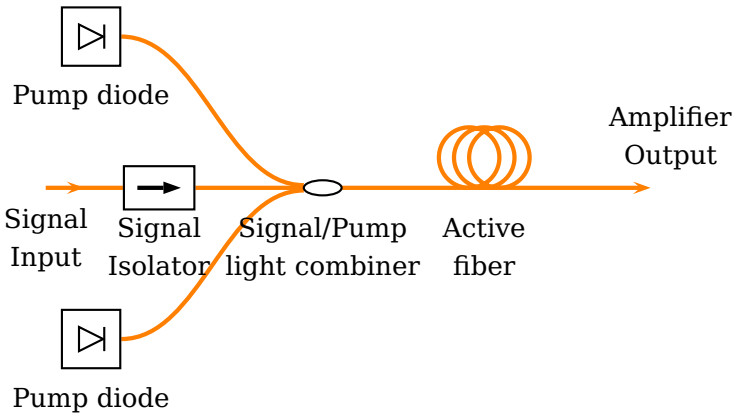
```
\begin{pspicture}(5.6,3)
\pnodes(0,0.5){S1}(2,2.5){S2}(3,1.5){S3}(4,0.5){S4}(4.5,1){S5}
\psset{prismangle=30, mirrortype=extended}
\definecolor[ps]{bl}{rgb}{tx@addDict begin Red Green Blue end}%
\begin{optexp}
  \optprism(S1)(S2)(S3)\optprism(S3)(S4)(S5)
  \mirror[compshift=-0.07](\oenodeIn{2})([offset=1,Xnodesep=1]\oenodeIn{2})(\oenodeIn{2})
  \drawbeam[linecolor=orange, ArrowInside=->, ArrowInsidePos=0.2, arrowscale=1.5](S1){1}
  \addtopsstyle{Beam}{linecolor=bl, linewidth=0.35\pslinewidth, beampathskip=1}
  \multido{\r=0+0.5}{120}{%
    \pstVerb{\r\space 620 590 sub 59 div mul 590 add
           tx@addDict begin wavelengthToRGB end }%
    \drawbeam[n=3.42 -0.002 \r\space mul add, beampos=\r\space 60 sub 0.0002
        mul](S1){1-3}
  }%
\end{optexp}
\end{pspicture}
```

Bsp. 11.30



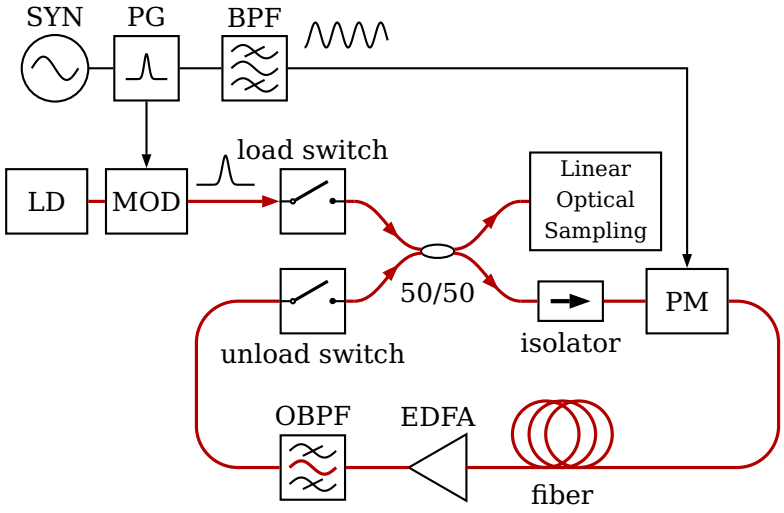
```
\begin{pspicture}(3,1.7)
\pnodes(-1,0){A}(1,1){B}(3,0){C}
\psframe*(0,0)(3,1.7)
\begin{optexp}
  \optplane(0,0.7)
  \optprism[prismalign=center, linecolor=white](A)(B)(C)
  \optplane(C)
  \addtopsstyle{Beam}{linecolor=white}
  \drawbeam[ArrowInside=->]{1-2}
  \definecolor[ps]{bl}{rgb}{%
    tx@addDict begin Red Green Blue end}%
  \addtopsstyle{Beam}{linestyle=fade, beaminsidelast, fadeto=black}
  \multido{\i=0+1}{60}{%
    \drawbeam[n=1.5 \i\space 0.003 mul add, linecolor=white, beampathskip=1,
        fadefuncname=gauss]{1-2}
    \pstVerb{\i\space 650 400 sub 59 div mul 400 add
      tx@addDict begin wavelengthToRGB end }%
    \drawbeam[n=1.5 \i\space 0.003 mul add, linecolor=bl, beampathskip=2,
        linewidth=0.3\pslinewidth]{-}%
  }%
\end{optexp}
\end{pspicture}
```

Bsp. 11.31

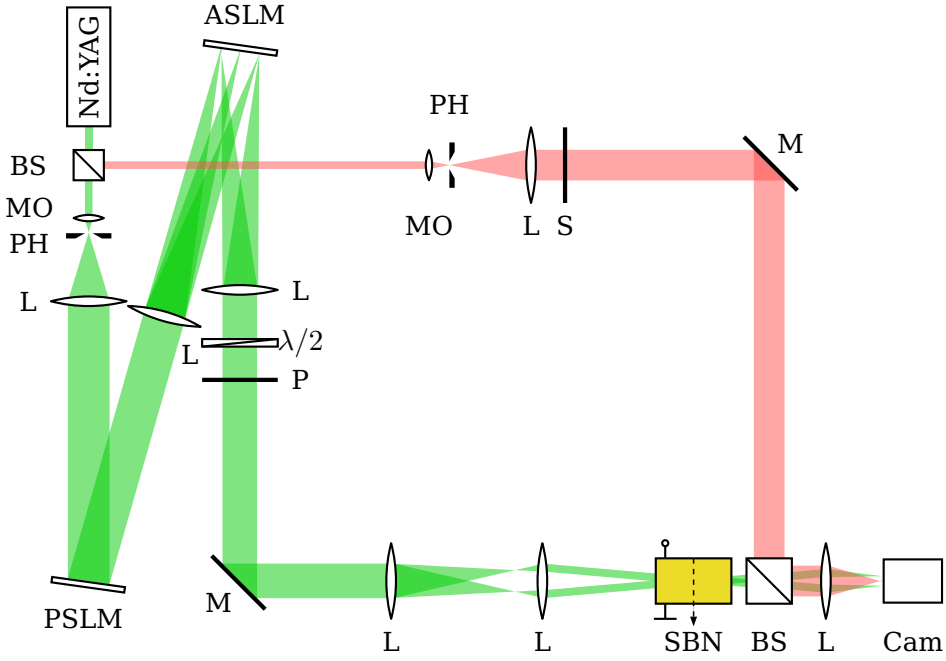



```
\begin{pspicture}(10,6)
  \begin{optexp}
    \psset[optexp]{usefiberstyle, couplersize=0.25, couplersep=0.07}
    \newpsstyle{Fiber}{linecolor=orange, linewidth=2\pslinewidth}
    \pnodes(2,1){PumpDiode}(2,5){UpperPumpDiode}(1,3){SignalIsolatorIn}(4, ↩
        3){SignalIsolatorOut}(1,3){SignalIn}(10,3){SignalCombinerOut}
    \pnodes(7,3){FiberIn}(9,3){FiberOut}(9,3){AmpOut}
    \optdiode[compname=PumpDiode, ↩
        position=start](PumpDiode)([Xnodesep=1]PumpDiode){Pump diode}
    \optdiode[compname=UpperPumpDiode, ↩
        position=start](UpperPumpDiode)([Xnodesep=1]UpperPumpDiode){Pump diode}
    \optisolator[compname=SignalIsolator, ↩
        fiber=none](SignalIsolatorIn)(SignalIsolatorOut)%
        {\begin{tabular}{@{}c@{}}Signal\\Isolator\end{tabular}}
    \wdmcoupler[compname=SignalPumpCombiner](UpperPumpDiode)(\oenodeOut{SignalIsolator})(PumpDi
        {\begin{tabular}{@{}c@{}}Signal/Pump\\light combiner\end{tabular}}
    \optfiber[
        compname=ActiveFiber,
        position=start,
        addtoFiberOut={arrows=->}](FiberIn)(FiberOut)%
        {\begin{tabular}{@{}c@{}}Active\\fiber\end{tabular}}
    \drawfiber[ArrowInside=->](SignalIn){SignalIsolator}
  \end{optexp}
  \nput{75}{AmpOut}{\begin{tabular}{@{}c@{}}Amplifier\\Output\end{tabular}}
  \nput{-90}{SignalIn}{\begin{tabular}{@{}c@{}}Signal\\Input\end{tabular}}
\end{pspicture}
```

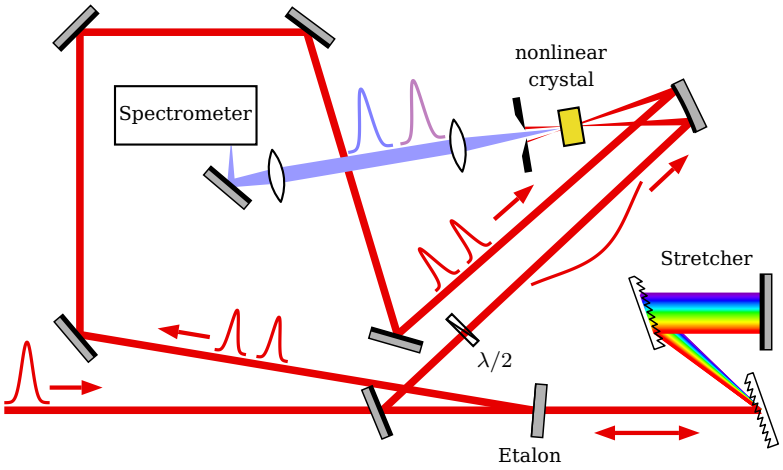
Bsp. 11.32



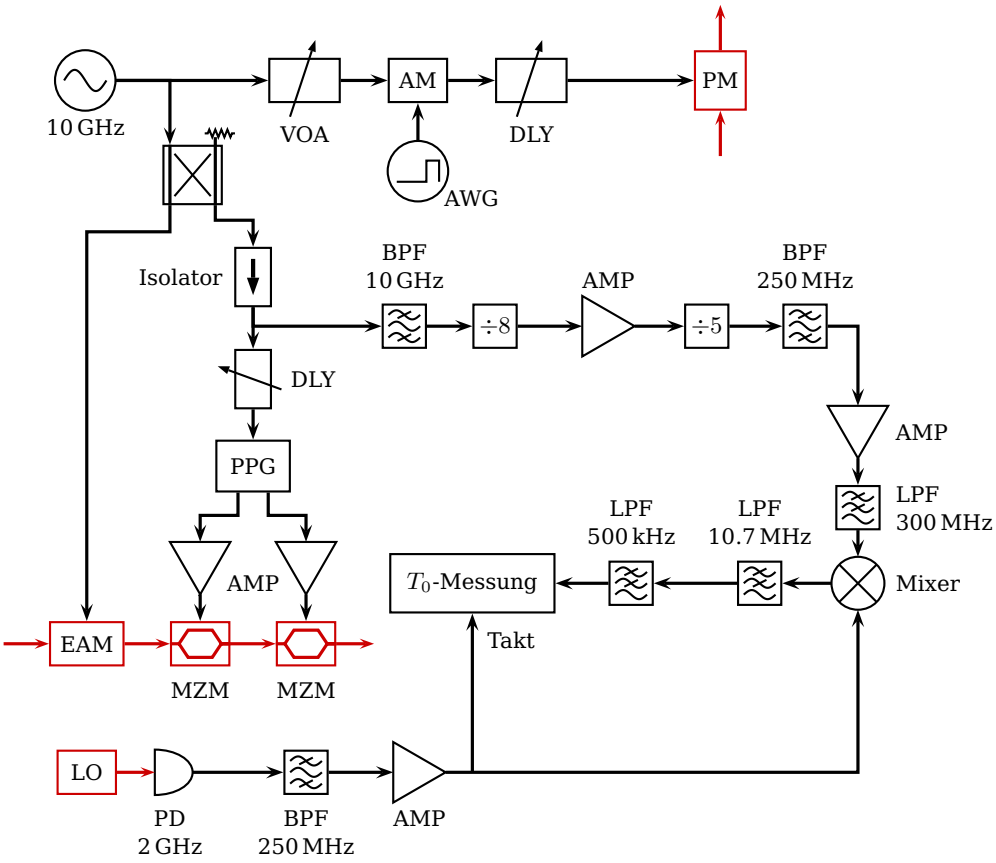
Bsp. 11.33: Angepasst von DOI:10.1364/OL.37.000797



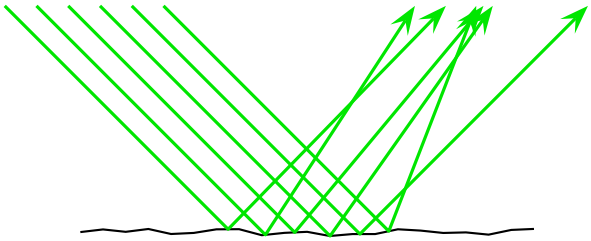
Bsp. 11.34: Angepasst von DOI:10.1364/AOP.1.000308, Abb. 37



Bsp. 11.35



Bsp. 11.36



```
\begin{pspicture}(10,3)
\makeatletter
\newOptexpTripole{roughsurface}
\def\roughsurface@nodes{%
  \pssavepath[linestyle=none, arrows=-,ArrowInside=-]{\oenode@Path{A}}{%
    \code{0 srand}
    \moveto(!-3 Rand 0.5 sub 0.1 mul)
    \multido{\r=-2.7+0.3}{20}{%
      \lineto(!\r\space Rand 0.5 sub 0.1 mul)}%
  }%
  \newOptexpComp{%
    {0 0} tx@IntersectDict /\PIT@name{\oenode@Path{A}} get 0 0 refl {PathIfc}
    1 }%
}%
\def\roughsurface@comp{%
  \pstracecurve{\oenode@Path{A}}
}%
\makeatother
\optplane[angle=90](1,3)
\roughsurface(0,3)(5,0)(10,3)
\addtopsstyle{Beam}{beamalign=abs, beamangle=-45, arrows=->, arrowscale=2}
\multido{\r=0+0.3}{6}{%
\drawbeam[beampos=\r]{1}{2}{1}}
\end{pspicture}
```

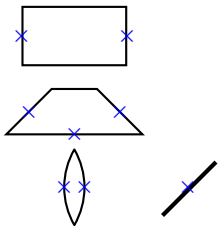
12. Zusatzinformationen

12.1. Interne Struktur der Komponenten

`showifcnodes`=`true`, `false` default: `false`

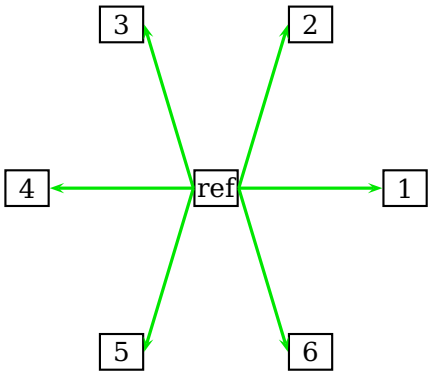
`IfcNodeStyle`⟨*psstyle*⟩ default: `dotstyle=x, dotscale=1.5, linecolor=blue`

Jede Komponente besteht aus Grenzflächen, die über einen Knoten auf der optischen Achse, einen Ebenevektor bzw. einen Krümmungsradius, und ihre optische Eigenschaft (reflektiv oder transmittiv, oder beides für z.B. `\beamsplitter`) definiert ist. Die Grenzflächenknoten können mit `showifcnodes` angezeigt werden.



```
\begin{pspicture}(4,3)
  \psset{showifcnodes}
  \optbox(0,2.5)(4,2.5)
  \doveprism(0,1.5)(4,1.5)
  \lens(0,0.5)(4,0.5)
  \mirror(0,0.5)(3.5,0.5)(3.5,3)
\end{pspicture}
```

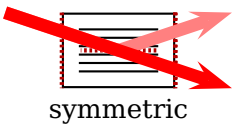
`\draw*beam` berechnet den Abstand zwischen den Grenzflächenknoten zweier Komponenten und verbindet die beiden nächstgelegenen.



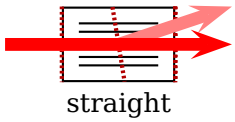

```
\begin{pspicture}(-3,-3)(3,3)
  \psset{optboxsize=0.6 0.5, labeloffset=0}
  \multido{\i=0+60,\ii=1+1}{6}{%
    \pnode(2.5;\i){A}
    \optbox([Xnodesep=-1]A)([Xnodesep=1]A){\ii}
  }
  \optbox[compname=ref](-1,0)(1,0){ref}
  \addtopsstyle{Beam}{arrows=->}
  \multido{\i=1+1}{6}{\drawbeam{ref}{\i}}
\end{pspicture}
```

showinterfaces=true, false default: false

Zeigt alle Grenzflächen einer Komponente an.




```
\begin{pspicture}(3,1.5)
  \psset[optexp]{showinterfaces, aomgratingcount=6}
  \addtopsstyle{IfcStyle}{linewidth=2\pslinewidth}
  \pnodes(0,1.5){A}(3,0.5){B}(3,1.5){C}
  \optaom[aomalign=symmetric](A)(B)(C){symmetric}
  \addtopsstyle{Beam}{arrows=->, linewidth=4\pslinewidth}
  \drawbeam[linecolor=red!50](A){}(C)
  \drawbeam[linecolor=red](A){}(B)
\end{pspicture}
```




```
\begin{pspicture}(3,1.5)
  \psset[optexp]{showinterfaces, aomgratingcount=6}
  \addtopsstyle{IfcStyle}{linewidth=2\pslinewidth}
  \pnodes(0,1){A}(3,1){B}(3,1.5){C}
  \optaom[aomalign=straight](A)(B)(C){straight}
  \addtopsstyle{Beam}{arrows=->, linewidth=4\pslinewidth}
  \drawbeam[linecolor=red!50](A){}(C)
  \drawbeam[linecolor=red](A){}(B)
\end{pspicture}
```

IfcStyle⟨*psstyle*⟩ default: linestyle=dashed, dash=1pt 1pt, linecolor=red!70!black

Bestimmt das Aussehen der mit showinterfaces angezeigten Grenzflächen.

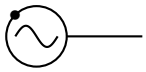
12.2. Übersicht der Spezialknoten

In diesem Abschnitt finden sie Übersichten für alle Spezialknoten, die von allen Komponenten bereitgestellt werden: die möglichen Positionen für externe und Rotationsknoten, und die Grenzflächenknoten.

12.2.1. Externe und Rotationsreferenzknoten

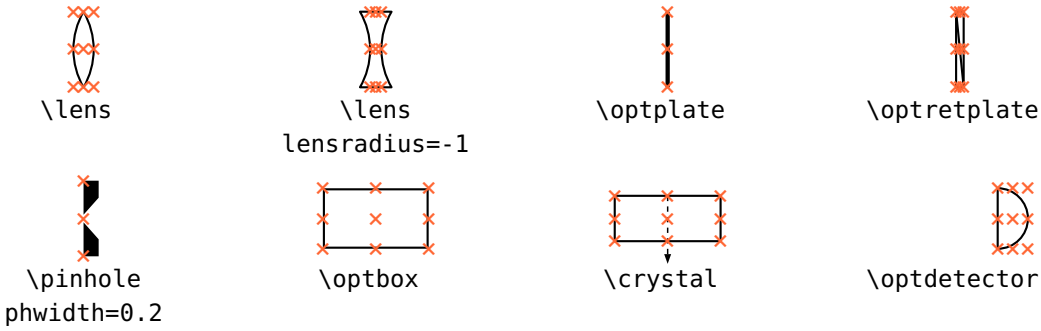
Hier finden Sie eine Übersicht über alle möglichen externen Knoten (Kap. 8.5) und Rotationsknoten (Kap. 8.7) aller Komponenten. Angezeigt werden die Positionen für alle möglichen Kombinationen von `t`, `c` oder `b` mit `l`, `c` oder `r`. Es sind nicht immer alle Möglichkeiten vorhanden, so können `t` und `b` ignoriert werden (z.B. bei `\mirror`), oder auch `r` und `l` (z.B. bei `\optplate`).

Anstelle dieser Werte können auch Bruchteile davon verwendet werden: `t` entspricht $+1\cdot\Delta y$, `b` entspricht $-1\cdot\Delta y$, `l` entspricht $-1\cdot\Delta x$ und `r` entspricht $+1\cdot\Delta x$. Das kann gerade im Fall einer runden Außenform sehr hilfreich sein, wie das folgende Beispiel zeigt.



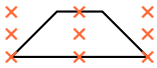
```
\begin{pspicture}(2,1)
  \elecsynthesizer[extnode={-0.707,0.707}](1,0.5)(2,0.5)
  \psdot(\oenodeExt{})
\end{pspicture}
```

Freistrahlkomponenten



\optdetector
dettype=diode

\optdiode


\doveprism

\glanthompson
glanthompsongap=0.15

\mirror
mirrortype=piezo

\mirror
mirrortype=extended

\mirror

\mirror
mirrortype=semitrans

\optprism

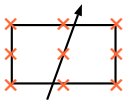
\optgrating

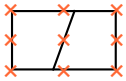
\beamsplitter

\beamsplitter
bsstyle=plate

\rightangleprism

\pentaprism


\optarrowcomp

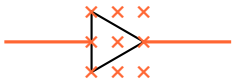

\optbarcomp

\optdipole

\transmissiongrating

Faserkomponenten

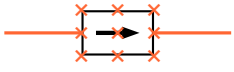
\optfiber

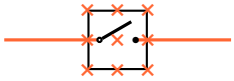

\optamp

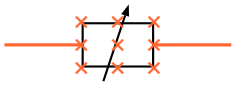
\optmzm

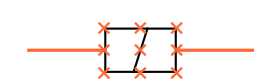
\optfilter

\polcontrol


\optisolator


\optswitch

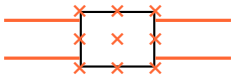

\fiberdelayline


\optfiberpolarizer

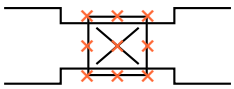
\fibercollimator

\optcirculator

\optcoupler
couplersize=0.5 0.3


\fiberbox

Elektrische Komponenten


\eleccoupler

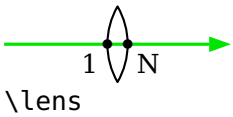
\elecmixer

\elecsynthesizer

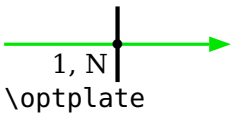
12.2.2. Grenzflächenknoten

Hier sind alle Komponenten und deren Grenzflächenknoten aufgelistet. Ist ein Knoten mit „1, N“ beschriftet, dann fallen beide Knoten zusammen.

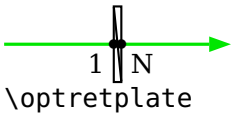
Freistrahlkomponenten



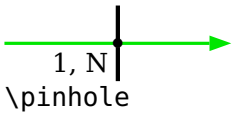
\lens



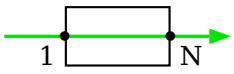
\optplate



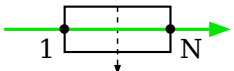
\optretplate



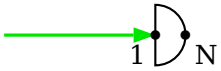
\pinhole



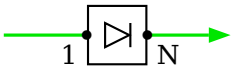
\optbox



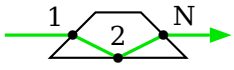
\crystal



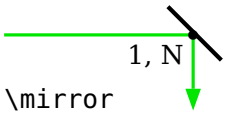
\optdetector



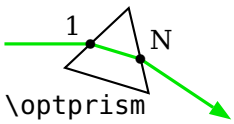
\optdiode



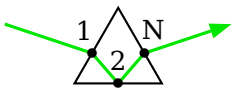
\doveprism



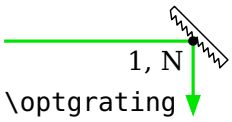
\mirror



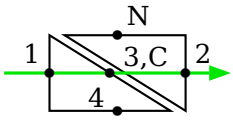
\optprism



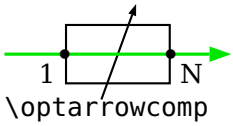
\optprism
prismtype=reflective



\optgrating



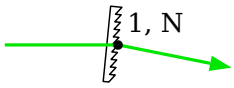
\glanthompson



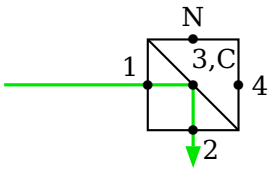
\optarrowcomp

\optbarcomp

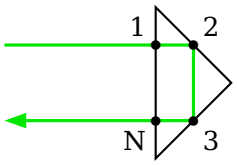
\optdipole



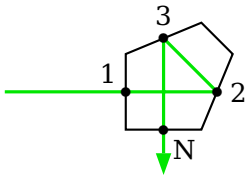
\transmissiongrating



\beamsplitter

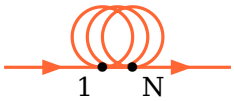


\rightangleprism

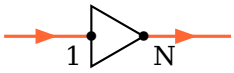


\pentaprism

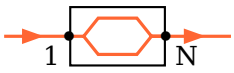
Faserkomponenten



\optfiber



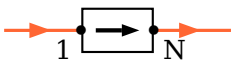
\optamp



\optmzm

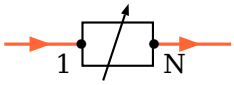
\optfilter

\polcontrol

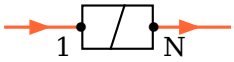


\optisolator

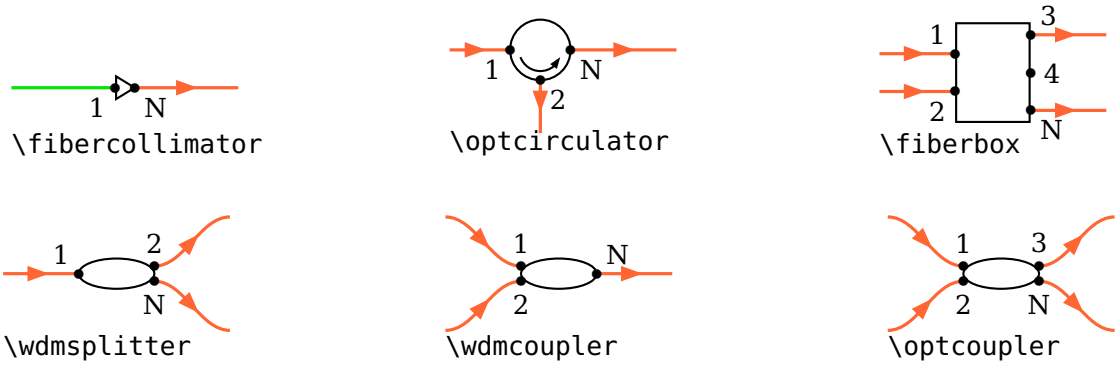
\optswitch



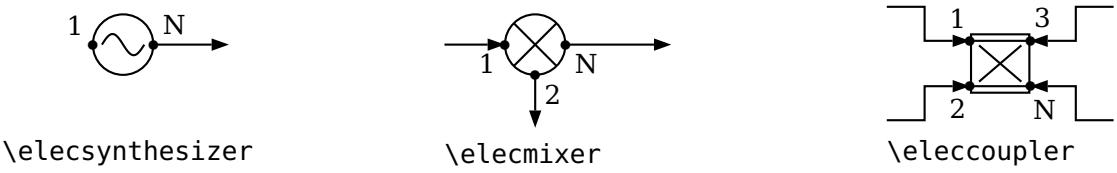
\fiberdelayline



\optfiberpolarizer



Elektrische Komponenten



12.3. Abwärtskompatibilität

In diesem Abschnitt finden Sie eine umfangreiche Auflistung aller Änderungen in einer Version bezüglich der Vorgängerversion, die nicht abwärts kompatibel sind. Sie finden ebenfalls Informationen was Sie bei einem Wechsel zur neuen Version beachten müssen, und was sie bei älteren Dokumenten gegebenenfalls ändern müssen um mit der neuen Version korrekt zu funktionieren.

12.3.1. Version 4.3a

`prismtype` Die Option `prismtype` war immer noch „zusammengeschustert“ und die automatische Positionierung war recht willkürlich. Dies sollte nun endlich korrigiert sein, unter Option `prismalign` ist ein entsprechendes Beispiel zu sehen.

`\optprism` Die Positionierung der Beschriftung war nicht konsistent mit anderen Komponenten, der Bezugspunkt war nicht der Mittelpunkt und auch nicht fest. Das ist jetzt behoben, könnte aber leichte Korrekturen an bestehenden Skizzen erfordern.

12.3.2. Version 4.3

`\rightangleprism` Die Ausrichtung des `\rightangleprism` und die Positionierung der Beschriftung war falsch. Das kommt aber nur zum tragen, wenn Eingangs- und Ausgangsvektor sehr unterschiedliche Längen haben.

12.3.3. Version 4.2

`prismtype` Die in Version 4.1 eingeführte Option `prismtype` benötigte immer eine zusätzliche Drehung der Komponente um 180°. Das entfällt nun, da die Komponentenausrichtung korrigiert wurde, erfordert aber gegebenenfalls eine Korrektur von Skizzen, die mit Version 4.1 erstellt wurden.

12.3.4. Version 4.0

Die Option `conn` und das alte, fehlerhafte Verfahren Strahlverbindungen zu zeichnen, wurde nun entfernt. Zur Migration älterer Dokumente siehe Kap. 12.3.6. Damit wurde auch die Option `connjoin` überflüssig und wurde entfernt.

Das Aussehen von `\optfiber` hat sich etwas geändert, da in der vorherigen Implementierung der Abstand zwischen zwei Schleifen von der Anzahl der Schleifen abhing (siehe das Beispiel unten). Beachten Sie, dass die Voreinstellung für `fiberloopsep` von 0.3 auf 0.2 geändert wurde um ein ähnliches Aussehen bei Verwendung der Voreinstellungen zu erhalten. Dadurch sitzen die Eingangs- und Ausgangsknoten jedoch etwas näher zusammen (siehe unten).

Das Makro `\newOptexpDipoleNolabel` wurde entfernt und kann ohne weitere Änderungen durch `\newOptexpDipole` ersetzt werden.

The alte Syntax mit `\mycomp@iii` zur Definition eigener Komponenten wird nicht mehr unterstützt.

12.3.5. Version 3.3

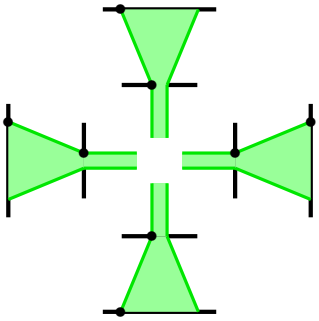
`\opttripole` Der Ausgangswinkel für die Beschriftung des `\opttripole` wurde um 180° geändert um konsistent zu den anderen Dreipolen zu sein.

Strahlknoten

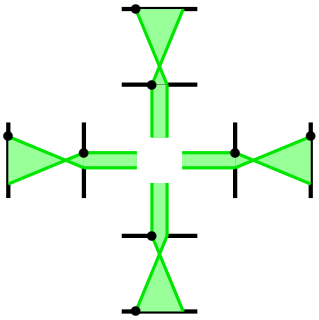
Vor Version 3.3 sind the Strahlknoten `\oenodeBeamUp` und `\oenodeBeamLow` nicht korrekt ausgerichtet gewesen, „upper“ und „lower“ war immer relativ zur Strahlrichtung (analog zu `extnodealign=relative`).

Das konnte zu Problemen führen wenn `loadbeampoints` zusammen mit `beamdiv` verwendet wurde, da das Vorzeichen von `beamdiv` nicht eindeutig kontrahierenden bzw. expandierenden Strahlen zugeordnet war.

Seit Version 3.3 werden die Strahlknoten analog zu `extnodealign=absolute` definiert und die Zuordnung eines negativen `beamdiv` mit kontrahierenden Strahlen sowie positiven `beamdiv` mit expandierenden Strahlen ist immer gültig. In den folgenden beiden Beispielen ist immer der `\oenodeBeamUp` gekennzeichnet.



```
\begin{pspicture}(-2,-2)(2,2)
  \addtopsstyle{Beam}{fillstyle=solid, fillcolor=green!40}
  \multido{\i=0+90,\ia=1+2,\ib=2+2}{4}{%
    \optplate[position=end](0.3;\i)(1;\i)
    \optplate[position=end, plateheight=1.5](1;\i)(2;\i)
    \drawwidebeam[beamwidth=0.2](0.3;\i){\ia}
    \pnode(\oenodeBeamUp{}){Up}
    \drawwidebeam[loadbeampoints, beamdiv=45]{\ia-\ib}
    \psdot(Up)\psdot(\oenodeBeamUp{})
  }
\end{pspicture}
```



```
\begin{pspicture}(-2,-2)(2,2)
  \addtopsstyle{Beam}{fillstyle=solid, fillcolor=green!40}
  \multido{\i=0+90,\ia=1+2,\ib=2+2}{4}{%
    \optplate[position=end](0.3;\i)(1;\i)
    \optplate[position=end](1;\i)(2;\i)
    \drawwidebeam[beamwidth=0.2](0.3;\i){\ia}
    \pnode(\oenodeBeamUp{}){Up}
    \drawwidebeam[loadbeampoints, beamdiv=-45]{\ia-\ib}
    \psdot(Up)\psdot(\oenodeBeamUp{})
  }
\end{pspicture}
```

12.3.6. Version 3.0

Die Änderungen in Version 3.0 gegenüber Version 2.1, die nicht abwärtskompatibel sind.

Allgemeine Parameter

`namingscheme`=old, new default: new

In Version 2.1 mussten spezielle Komponentenknoten über ihren expliziten Namen angesprochen werden. Wenn Sie das alte Namensschema benötigen, da Sie direkt auf die Knoten zugegriffen haben, so müssen Sie `namingscheme=old` verwenden. Seit Version 3.0 werden Makros für den Zugriff auf die Komponentenknoten bereitgestellt, so dass das eigentlichen Namensschema unerheblich ist, siehe Kap. 8

`endbox` `angle` `rotateref` Diese Parameter wirken sich jetzt auf alle Komponenten aus.

`extnode` Einigen Komponenten stellen nun mehr mögliche Positionen für `extnode` zur Verfügung. So muss war z.B. für `\optdetector` nur ein Knoten möglich, der für jeden Wert von `extnode` zugänglich war, jetzt aber nur mit `extnode=r`. Alle anderen Werte setzen den Knoten an andere Stellen.

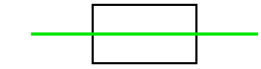
`\optbox` Eine `\optbox` hat immer einen Eingangs- und Ausgangsknoten an den entsprechenden Grenzflächen. In Version 2.1 fielen Eingangs- und Ausgangsknoten zusammen falls `endbox` auf `true` gesetzt war.

\fibercollimator Der \fibercollimator hat in den Voreinstellungen nur eine Faserverbindung, der Strahl muss manuell oder durch Angabe des beam Parameters gezeichnet werden.

Strahlverbindungen

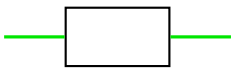
In Version 2.1 konnten Strahlverbindungen intern sein, d.h. die Verbindung wurde zusammen mit der Komponenten definiert (z.B. mit beam, oder conn) oder extern über einen separaten Aufruf von \drawbeam. Der Hauptgrund für die internen Verbindungen war der ästhetische Aspekt, zuerst die Verbindung und dann die Komponente darüber zu zeichnen, aber Bequemlichkeit spielte auch eine Rolle. Seit Version 3.0 kann die Reihenfolge von Verbindungen und Komponenten über Ebenen geregelt werden (Kap. 9.9).

Manche Varianten für interne Strahlverbindungen können auch in 3.0 verwendet werden:



conn=o-i, conn=i-o

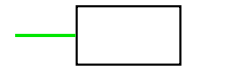
```
\begin{pspicture}(3,2)
  \pnode(0,1){A}\pnode(3,1){B}
  \optbox[beam](A)(B)
\end{pspicture}
```



conn=o-o

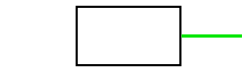
```
\begin{pspicture}(3,2)
  \pnode(0,1){A}\pnode(3,1){B}
  \optbox[beam, beaminside=false](A)(B)
\end{pspicture}
```

Andere Verbindungen können nur über einen zusätzlichen \drawbeam Aufruf erreicht werden, falls die Verbindung dann hinter der Komponente sein soll, musst der beteiligte Code in eine optexp-Umgebung geschachtelt werden. Außer diesem Aspekt sind die separaten Verbindungen äquivalent:



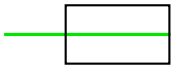
conn=o-, conn=a-

```
\begin{pspicture}(3,2)
  \pnode(0,1){A}\pnode(3,1){B}
  \optbox(A)(B)
  \drawbeam(A){}
\end{pspicture}
```



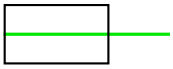
conn=-o, conn=-b

```
\begin{pspicture}(3,2)
  \pnode(0,1){A}\pnode(3,1){B}
  \optbox(A)(B)
  \drawbeam{}(B)
\end{pspicture}
```



conn=i-, A-

```
\begin{pspicture}(3,2)
  \pnode(0,1){A}\pnode(3,1){B}
  \begin{optexp}
    \optbox(A)(B)
    \drawbeam[beaminsidelast](A){}
  \end{optexp}
\end{pspicture}
```



conn=-i, conn=-B

```
\begin{pspicture}(3,2)
  \pnode(0,1){A}\pnode(3,1){B}
  \begin{optexp}
    \optbox(A)(B)
    \drawbeam[beaminsidefirst]{}(B)
  \end{optexp}
\end{pspicture}
```

Dokumentationsindex

A

abspos, 9, 10, 13–15, 17, **22**, 23, 31, 41, 79, 86, 89, 96–99, 101, 104–106, 111–113, 115, 120, 132, 138–142, 144, 146–148, 151–155, 157, 158
addtoBeam, 106, **106**
addtoFiber, 120, **120**, 121
addtoFiberIn, 127, 142, 143
addtoFiberIn1, 120, 127, 142, 143
addtoFiberIn2, 120, 127, 131, 142, 143
addtoFiberOut, 127, 142, 143, 161
addtoFiberOut1, 127, 143
addtoFiberOut2, 127, 143
addtoOptComp, 25, 26, **26**
\addtopsstyle, 25, 126
addtoWire, 122, **122**
addtoWireIn, 127
addtoWireIn1, 127
addtoWireIn2, 127
addtoWireOut, 127
addtoWireOut1, 127
addtoWireOut2, 127
allowbeaminside, 38, 75, **102**, 103, 152
angle, 23, **23**, 40, 45, 51, 71, 78, 80, 85, 88–90, 100, 101, 105, 106, 116–119, 138, 139, 144, 148–150, 153, 159, 164
aomalign, 52, **52**, 53, 166
aomcomp, 52, **53**
aomgratingcount, **52**, 166
aomheight, 52, **52**
aomreflalign, 53, **53**, 54, 55
aomsize, **52**, 54, 55
aomwidth, 52, **52**
arrowcompangle, **34**

arrowcompheight, 34, **34**
arrowcompshape, 34, **34**
arrowcompsize, **34**, 142
ArrowCompStyle, 34, **34**, 62
arrowcompwidth, 34, **34**
ArrowInside, 107
ArrowInsideMaxLength, 107, **107**
ArrowInsideMinLength, **106**, 107, 149, 156
asphere coefficients, **29**, 30, 31
asphereconstant, **29**, 30, 31
asphereheight, **29**, 30, 31
aspherradiusleft, **29**, 30, 31
aspherradiusright, **29**, 30
aspherewidth, **29**, 30
\aspheriklens, **29**, 30, 31
\axicon, 42, **42**
axiconangle, **42**
axiconheight, **42**
axiconwidth, **42**

B

\backlayer, 89, 128, 158
barcompangle, 35, **35**
barcompheight, 35, **35**
barcompshape, 35, **35**
barcompsize, **35**
BarCompStyle, 35, **35**
barcompwidth, 35, **35**
Beam, 15, 18, 40, 46, 47, 51–55, 75, 86, 87, 89, 96–98, 100, 104–106, **106**, 107, 109, 110, 113, 114, 132, 138–141, 144–146, 148–152, 156, 158–160, 164, 166, 173, 174
beam, 21, 25–28, 31–33, 37–41, 43–46, 48–51, 55–57, 75, 76, 78, **106**, 129, 131, 175
beamalign, 36, 46, 47, 88, 97, 99, **99**, 100, 137, 138, 151, 155, 164
beamangle, 11, 36, 51, 94, 95, 98, 99, **99**, 111, 114, 148, 149, 159, 164
beamdiffractionorder, 54, **54**, 55
beamdiv, 36, 100, 109, **109**, 111, 113, 132, 138, 140, 148, 149, 173, 174
beaminside, 11, **101**, 102, 103, 105, 130, 134–136, 148, 175
beaminsidefirst, 86, 99, 102, **102**, 103, 115, 141, 144, 146, 152, 176
beaminsidelast, 86–88, 102, **102**, 103, 112, 141, 158, 176

beammode, 54, **104**
beamnodealign, 89, **104**, 105, 158
beampathcount, 101, **101**, 158
beampathskip, 89, **100**, 101, 158–160
beampos, 11, 30, 31, 40, 44–48, 87, 94–98, **98**, 99, 102, 104–106, 109, 111, 112, 115, 151, 155, 159, 164
\beamsplitter, 18, 20, 48, **48**, 49, 52, 92, 104, 114, 141, 144–146, 148, 155, 156, 165
beamwidth, 11, 15, 18, 36, 42, 47, 50, 52–56, 75, 86, 87, 89, 101, 104, 105, 107, 109, **109**, 110, 113, 114, 130–132, 138, 140, 141, 144, 145, 148, 149, 158, 173, 174
bssize, **48**, 92, 146, 155, 156
bsstyle, 18, **48**, 49, 156

C

caxisinv, **37**
caxislength, 37, **37**
compname, 18, 79, **79**, 91–93, 97, 101, 104, 116, 120, 121, 139, 142, 144, 145, 148, 152–154, 156, 158, 161, 166
compoffset, 24, **24**, 156
compshift, 17, **23**, 24, 80, 90, 138, 142, 143, 148, 154, 159
conn, 172, 175
connjoin, 172
coupleralign, 65, **65**, 66, 67, 81, 121, 127, 131, 143, 154
couplersep, 64, **64**, 65, 71, 81, 161
couplersize, 64, **64**, 65, 71, 81, 161
couplertype, 64, 65, **65**, 71, 81, 147, 150
\crystal, 37, **37**, 38, 131, 146, 152
CrystalCaxis, 38, **38**
crystalheight, 37, **37**
CrystalLamp, **38**
crystalsize, 37, **37**, 146
crystalwidth, 37, **37**

D

DetectorStyle, 77, **77**
detsize, **77**, 89, 150, 152, 156–158
dettype, 13, 18, 77, **77**, 139, 142
diffractionorders, **53**, 54, 55

\doveprism, 39, **39**, 94–96, 145, 165
doveprismsize, 39, **39**, 94, 95
\drawbeam, 7, 10, 11, 13, 20, 30, 31, 36, 39, 40, 43–48, 51, 84–88, 91–104, 106, 107, 111–116, 122, 130, 133–137, 139, 140, 144, 146, 150–153, 155, 156, 159, 160, 164–166, 175, 176
\drawfiber, 12, 17, 66, 115–121, 123, 142, 147, 148, 150, 153, 154, 157, 161
\drawwidebeam, 11, 15, 18, 36, 42, 47, 50, 52–56, 75, 85–87, 89, 93, 96, 100, 101, 104, 105, 107–114, 130–132, 138, 140, 141, 144, 145, 148, 149, 158, 165, 173, 174
\drawwire, 121–123, 142, 153, 154, 157

E

\eleccoupler, 70, **70**, 71, 125
eleccouplerinput, **70**, 71
eleccouplersep, **70**
eleccouplersize, 70, **70**
eleccouplertype, 70, **70**, 71
\elecmixer, **72**, 73, 118, 125
elecmixersize, **73**
\elecsynthesizer, 24, 71, **71**, 72, 123–125, 127, 142, 153, 154, 167
endbox, **23**, 174
environ, 6
ExtendedMirror, 25, 44, 45, **45**, 83, 141, 149
extnode, 44, 82, **82**, 83, 85, 90, 133, 135, 136, 142, 144, 148, 153, 154, 157, 167, 174
extnodealign, 82, **82**, 83, 144, 173
extnodes, **83**, 84

F

fade, **107**, 108
fadefunc, 108, **108**
fadefuncname, 108, **108**, 160
fadepoints, 107, 108, **108**
fadeto, 107, 108, **108**, 160
FdlArrow, **62**
fdlsize, **62**, 112
Fiber, 58, 62–64, 71, 117, 118, 120, **120**, 121, 126, 142, 157, 161
fiber, 15–17, 25, 34, 35, 66, 69, 75, 112, 118, 120, 121, 123, **123**, 124, 127, 138, 142, 147, 148, 150, 153, 154, 157, 161

fiber∗, 121
fiberalign, **117**, 118, 122
fiberangleA, 118, **118**, 119, 122
fiberangleB, 118, **118**, 119, 122
\fiberbox, 68, **68**, 69
fiberboxcount, 68, 69, **69**
fiberboxheight, 69, **69**
fiberboxsepin, 68, 69, **69**
fiberboxsepout, 68, **69**
fiberboxsize, 69, **69**
fiberboxwidth, 69, **69**
\fibercollimator, 74, 75, **75**, 76, 175
fibercolsize, **76**
\fiberdelayline, 34, 62, **62**, 112, 127
FiberIn, 126, 161
FiberIn1, 126
FiberIn2, 126
fiberloopradius, 59, **59**, 138
fiberloops, 59, **59**, 143, 147, 157
fiberloopsep, 59, **59**, 172
FiberOut, 126, 161
FiberOut1, 62, 126
FiberOut2, 62, 126
fiberpolsize, **62**
fiberstyle, 12, 71, **120**, 121, 122, 147
fillstyle, 11
filterangle, 75, **75**
filtersize, **74**
FilterStyle, 75, **75**
filtertype, 74, **74**, 75
forcebeaminside, 103, **103**
\frontlayer, 128

G

\glanthompson, 39, **39**, 40, 101, 153
glanthompsongap, **39**, 40, 153
glanthompsonheight, 39, **39**
glanthompsonsize, **39**, 40, 153
glanthompsonwidth, 39, **39**

gratingalign, 50, **50**, 159
gratingcount, 49, **49**
gratingdepth, 49, **49**
gratingheight, **49**
gratinglinewidth, **50**
gratingtype, **49**, 50
gratingwidth, **49**, 149

I
IfcNodeStyle, **165**
IfcStyle, 166, **166**
innercompalign, 24, **24**, 75
innerheight, **32**, 98, 148
innerlabel, **21**, 23, 82, 83, 121, 146, 155
IsolatorArrow, 61, **61**
isolatorsize, **60**

K
Komponenten
- \asphericlens, **29**, 30, 31
- \axicon, 42, **42**
- \beamsplitter, 18, 20, 48, **48**, 49, 52, 92, 104, 114, 141, 144–146, 148, 155, 156, 165
- \crystal, 37, **37**, 38, 131, 146, 152
- \doveprism, 39, **39**, 94–96, 145, 165
- \eleccoupler, 70, **70**, 71, 125
- \elecmixer, **72**, 73, 118, 125
- \elecsynthesizer, 24, 71, **71**, 72, 123–125, 127, 142, 153, 154, 167
- \fiberbox, 68, **68**, 69
- \fibercollimator, 74, 75, **75**, 76, 175
- \fiberdelayline, 34, 62, **62**, 112, 127
- \glanthompson, 39, **39**, 40, 101, 153
- \lens, **27**
- \mirror, 10, 18, 24–26, 43, **43**, 44–46, 48, 78, 83, 87, 95, 98, 99, 107, 109, 110, 114, 138, 139, 141, 145, 146, 149–152, 155, 156, 158, 159, 165, 167
- \oapmirror, 47, **47**, 48
- \optamp, 59, **59**, 120, 121, 147, 153, 154, 157
- \optaom, 51, **51**, 52–55, 166
- \optarrowcomp, 33, **33**, 34, 142

- \optbarcomp, 34, **34**, 35
- \optbox, 10–17, 19–23, 33, **33**, 69, 79–88, 90, 92, 98–100, 102, 103, 109, 116–121, 123–125, 127, 128, 138–143, 145–148, 150–158, 165, 166, 174–176
- \optcirculator, 62, **62**, 63, 64, 73, 118, 123, 124, 154
- \optcoupler, 64, **64**, 65, 66, 71, 81, 120, 121, 124, 142, 147, 148, 157
- \optdetector, 13, 16–18, 74, 76, **76**, 77, 81, 89, 123, 139, 142, 144, 150, 152, 155–158, 174
- \optdiode, 38, **38**, 103, 161
- \optdipole, 24, 129, **129**, 130, 145, 155
- \optfiber, **58**, 59, 120, 121, 138, 142, 143, 147, 154, 157, 161, 172
- \optfiberpolarizer, 35, 62, **62**, 143
- \optfilter, 24, 58, 74, **74**, 75, 103, 147
- \optgrating, 26, 49, **49**, 50–52, 98, 113, 114, 139, 148, 149, 159
- \optisolator, 60, **60**, 61, 143, 154, 161
- \optmzm, 16, 17, 58, 60, **60**, 123–125, 147, 153, 154
- \optplane, 36, 39, 40, 47, 55, 89, 94, 96–98, 101, 105, 106, 111, 115, 138, 144, 148, 153, 160, 164
- \optplate, 13, 31, **31**, 98–100, 109, 111, 112, 115, 132, 140, 148, 150, 152, 155, 156, 167, 173, 174
- \optprism, 55, **55**, 56, 89, 94, 97, 101, 104, 107, 111, 158–160
- \optretplate, 13, 32, **32**, 102, 139–141, 145, 146
- \optsource, 36, **36**
- \optswitch, 61, **61**, 147, 157
- \opttripole, 129, **129**, 131, 141, 151, 173
- \optwedge, 41, **41**, 42
- \parabolicmirror, 46, **46**, 47
- \pentaprism, 57, **57**, 84, 102, 145
- \pinhole, 32, **32**, 33, 98, 140, 141, 145, 146, 148, 149, 151, 156
- \polarization, 40, **40**, 41, 89, 101, 140, 146, 153
- \polcontrol, 60, **60**, 143, 147, 154
- \rightangleprism, 56, **56**, 57, 172
- \transmissiongrating, 51, **51**
- \wdmcoupler, 64, **64**, 66, 67, 131, 143, 150, 154, 161
- \wdmsplitter, 16, 17, 64, **64**, 66, 67, 124, 127, 143, 148, 150, 154, 157

L

label, 15–17, 19–21, **21**, 37, 38, 42, 59, 69, 83, 120, 129–131, 133–136, 138–143, 145–148, 150–157

labelalign, **19**, 20, 21, 134–136, 141

labelangle, 10, 20, **20**, 21, 24, 134, 138, 140, 141, 146, 147, 155
labeloffset, 10, 13–15, 17, 19, **19**, 20, 21, 43, 50, 65, 81, 92, 129, 139, 141–143, 146, 148, 151, 155, 158, 166
labelref, 10, 20, **20**, 21, 82
labelstyle, 16, 17, **19**, 146, 147, 157
lamp, **37**
\lens, **27**
lens, 9–11, 14, 15, 18, 22–24, 26–28, **28**, 31, 36, 56, 86, 89, 96, 97, 99, 101, 104–106, 111–113, 115, 132, 138, 140, 141, 144–146, 148, 151, 152, 156, 158, 165
lensheight, 11, 14, 15, 27, **27**, 146
lensradius, 11, 13–15, 28, **28**, 86, 141
lensradiusleft, **27**, 28
lensradiusright, **27**, 28
lenswidth, 28, **28**, 29
linestyle, 11
loadbeam, 87, 88, 104, 114, **114**
loadbeampoints, **112**, 113, 132, 138, 140, 141, 145, 159, 173

M

Makros
- \addtopsstyle, 25, 126
- \backlayer, 89, 128, 158
- \drawbeam, 7, 10, 11, 13, 20, 30, 31, 36, 39, 40, 43–48, 51, 84–88, 91–104, 106, 107, 111–116, 122, 130, 133–137, 139, 140, 144, 146, 150–153, 155, 156, 159, 160, 164–166, 175, 176
- \drawfiber, 12, 17, 66, 115–121, 123, 142, 147, 148, 150, 153, 154, 157, 161
- \drawwidebeam, 11, 15, 18, 36, 42, 47, 50, 52–56, 75, 85–87, 89, 93, 96, 100, 101, 104, 105, 107–114, 130–132, 138, 140, 141, 144, 145, 148, 149, 158, 165, 173, 174
- \drawwire, 121–123, 142, 153, 154, 157
- \frontlayer, 128
- \nccurve, 12, 116, 121
- \ncput, 22
- \newOptexpComp, 134
- \newOptexpDipole, 132–136, 172
- \newOptexpDipoleNolabel, 172
- \newOptexpElecDipole, 132
- \newOptexpFiberDipole, 132

– \newOptexpTripole, 132, 134, 137, 164
– \newpsobject, 25, 130, 131
– \newpsstyle, 8, 25
– \nodexn, 88
– \oeBeamCenter, 89, 158
– \oeBeamVec, 88, 101, 153
– \oeBeamVec*, 87
– \oeBeamVecLow, 89
– \oeBeamVecMedian, 89, 105, 158
– \oeBeamVecUp, 89
– \oenode, 54, 83, 84
– \oenodeBeam, 86–88, 101, 153
– \oenodeBeam*, 87
– \oenodeBeamLow, 89, 173
– \oenodeBeamUp, 89, 173, 174
– \oenodeCenter, 79, 81, 156
– \oenodeExt, 82, 83, 136, 142, 144, 153, 154, 157, 167
– \oenodeIfc, 56, 66, 84, 101, 104, 154, 158
– \oenodeIn, 79, 84, 139, 155, 159
– \oenodeLabel, 152, 156
– \oenodeOut, 79, 84, 139, 152, 155, 158, 161
– \oenodeRefA, 22, 80, 84, 106
– \oenodeRefB, 22, 80, 106
– \oenodeRotref, 85
– \oenodeTrefA, 80
– \oenodeTrefB, 80
– \psbezier, 76
\mirror, 10, 18, 24–26, 43, **43**, 44–46, 48, 78, 83, 87, 95, 98, 99, 107, 109, 110, 114, 138, 139, 141, 145, 146, 149–152, 155, 156, 158, 159, 165, 167
mirrordepth, 18, 44, 45, **45**, 48, 83
mirrorlinewidth, **43**
mirrorradius, 43, **43**, 44, 98, 149, 155
mirrortype, 18, 25, 43, 44, **44**, 45, 46, 48, 83, 87, 98, 99, 107, 109, 114, 141, 146, 149, 150, 152, 155, 159
mirrorwidth, **43**, 87, 107, 110, 146, 149, 151, 152, 156, 159
multido, 6

N

n, 10, 11, 39, 86, 88, 89, 94, 95, **95**, 96–98, 104, 106, 111, 112, 115, 132, 137, 139, 141, 144, 145, 148, 152, 156, 158–160
namingscheme, 174
\nccurve, 12, 116, 121
\ncput, 22
newBeam, **106**
newFiber, **120**, 138
newFiberIn, 127
newFiberIn1, 127
newFiberIn2, 127
newFiberOut, 127
newFiberOut1, 127
newFiberOut2, 127
newOptComp, 25, 26, **26**
\newOptexpComp, 134
\newOptexpDipole, 132–136, 172
\newOptexpDipoleNolabel, 172
\newOptexpElecDipole, 132
\newOptexpFiberDipole, 132
\newOptexpTripole, 132, 134, 137, 164
\newpsobject, 25, 130, 131
\newpsstyle, 8, 25
newWire, **122**
newWireIn, 127
newWireIn1, 127
newWireIn2, 127
newWireOut, 127
newWireOut1, 127
newWireOut2, 127
\nodexn, 88
npos, 22

O

\oapmirror, 47, **47**, 48
oapmirroraperture, **47**, 48
\oeBeamCenter, 89, 158
\oeBeamVec, 88, 101, 153
\oeBeamVec*, 87
\oeBeamVecLow, 89

\oeBeamVecMedian, 89, 105, 158
\oeBeamVecUp, 89
\oenode, 54, 83, 84
\oenodeBeam, 86–88, 101, 153
\oenodeBeam*, 87
\oenodeBeamLow, 89, 173
\oenodeBeamUp, 89, 173, 174
\oenodeCenter, 79, 81, 156
\oenodeExt, 82, 83, 136, 142, 144, 153, 154, 157, 167
\oenodeIfc, 56, 66, 84, 101, 104, 154, 158
\oenodeIn, 79, 84, 139, 155, 159
\oenodeLabel, 152, 156
\oenodeOut, 79, 84, 139, 152, 155, 158, 161
\oenodeRefA, 22, 80, 84, 106
\oenodeRefB, 22, 80, 106
\oenodeRotref, 85
\oenodeTrefA, 80
\oenodeTrefB, 80
\optamp, 59, **59**, 120, 121, 147, 153, 154, 157
optampsize, **59**
\optaom, 51, **51**, 52–55, 166
\optarrowcomp, 33, **33**, 34, 142
\optbarcomp, 34, **34**, 35
\optbox, 10–17, 19–23, 33, **33**, 69, 79–88, 90, 92, 98–100, 102, 103, 109, 116–121, 123–125, 127, 128, 138–143, 145–148, 150–158, 165, 166, 174–176
optboxheight, 33, **33**
optboxsize, 11, 12, 20–22, **33**, 69, 79, 82, 83, 88, 92, 121, 138, 139, 141, 142, 146, 148, 150, 152–157, 166
optboxwidth, 13, 16, 17, 20, 33, **33**, 102, 103, 138–140, 143, 145–147, 150, 151, 153, 158
optcircangle, 63, **63**, 154
optcircangleA, **63**
optcircangleB, **63**
OptCircArrow, 63, **63**, 154
optcircsize, **63**
\optcirculator, 62, **62**, 63, 64, 73, 118, 123, 124, 154
OptComp, 25, **25**, 26, 75, 77, 149
\optcoupler, 64, **64**, 65, 66, 71, 81, 120, 121, 124, 142, 147, 148, 157

\optdetector, 13, 16–18, 74, 76, **76**, 77, 81, 89, 123, 139, 142, 144, 150, 152, 155–158, 174
\optdiode, 38, **38**, 103, 161
optdiodesize, **38**
\optdipole, 24, 129, **129**, 130, 145, 155
optdipolecomp, 130, **130**
optdipolesize, **129**, 130, 145, 155
optexp, 11, 18, 50, 127, 128, 175
\optfiber, **58**, 59, 120, 121, 138, 142, 143, 147, 154, 157, 161, 172
\optfiberpolarizer, 35, 62, **62**, 143
\optfilter, 24, 58, 74, **74**, 75, 103, 147
\optgrating, 26, 49, **49**, 50–52, 98, 113, 114, 139, 148, 149, 159
optional, 26, **26**
OptionalStyle, 26, **26**
\optisolator, 60, **60**, 61, 143, 154, 161
\optmzm, 16, 17, 58, 60, **60**, 123–125, 147, 153, 154
optmzmsize, **60**, 153
\optplane, 36, 39, 40, 47, 55, 89, 94, 96–98, 101, 105, 106, 111, 115, 138, 144, 148, 153, 160, 164
\optplate, 13, 31, **31**, 98–100, 109, 111, 112, 115, 132, 140, 148, 150, 152, 155, 156, 167, 173, 174
\optprism, 55, **55**, 56, 89, 94, 97, 101, 104, 107, 111, 158–160
\optretplate, 13, 32, **32**, 102, 139–141, 145, 146
\optsource, 36, **36**
\optswitch, 61, **61**, 147, 157
\opttripole, 129, **129**, 131, 141, 151, 173
opttripolecomp, **130**, 131
\optwedge, 41, **41**, 42
outerheight, **32**, 146, 149, 156

P

Pakete
- environ, 6
- multido, 6
- pst-eucl, 6
- pst-intersect, 6
- pst-node, 6, 22, 121
- pst-optexp, 6, 25, 128, 129
- pst-plot, 6

– pstricks, 6
– pstricks-add, 6, 107
\parabolicmirror, 46, **46**, 47
Parameter
– abspos, 9, 10, 13–15, 17, **22**, 23, 31, 41, 79, 86, 89, 96–99, 101, 104–106, 111–113, 115, 120, 132, 138–142, 144, 146–148, 151–155, 157, 158
– addtoBeam, 106, **106**
– addtoFiber, 120, **120**, 121
– addtoFiberIn, 127, 142, 143
– addtoFiberIn1, 120, 127, 142, 143
– addtoFiberIn2, 120, 127, 131, 142, 143
– addtoFiberOut, 127, 142, 143, 161
– addtoFiberOut1, 127, 143
– addtoFiberOut2, 127, 143
– addtoOptComp, 25, 26, **26**
– addtoWire, 122, **122**
– addtoWireIn, 127
– addtoWireIn1, 127
– addtoWireIn2, 127
– addtoWireOut, 127
– addtoWireOut1, 127
– addtoWireOut2, 127
– allowbeaminside, 38, 75, **102**, 103, 152
– angle, 23, **23**, 40, 45, 51, 71, 78, 80, 85, 88–90, 100, 101, 105, 106, 116–119, 138, 139, 144, 148–150, 153, 159, 164
– aomalign, 52, **52**, 53, 166
– aomcomp, 52, **53**
– aomgratingcount, **52**, 166
– aomheight, 52, **52**
– aomreflalign, 53, **53**, 54, 55
– aomsize, **52**, 54, 55
– aomwidth, 52, **52**
– arrowcompangle, **34**
– arrowcompheight, 34, **34**
– arrowcompshape, 34, **34**
– arrowcompsize, **34**, 142
– arrowcompwidth, 34, **34**
– ArrowInside, 107
– ArrowInsideMaxLength, 107, **107**

– ArrowInsideMinLength, **106**, 107, 149, 156
– aspherecoefficients, **29**, 30, 31
– asphereconstant, **29**, 30, 31
– aspherеheight, **29**, 30, 31
– asphereradiusleft, **29**, 30, 31
– asphereradiusright, **29**, 30
– aspherewidth, **29**, 30
– axiconangle, **42**
– axiconheight, **42**
– axiconwidth, **42**
– barcompangle, 35, **35**
– barcompheight, 35, **35**
– barcompshape, 35, **35**
– barcompsize, **35**
– barcompwidth, 35, **35**
– beam, 21, 25–28, 31–33, 37–41, 43–46, 48–51, 55–57, 75, 76, 78, **106**, 129, 131, 175
– beamalign, 36, 46, 47, 88, 97, 99, **99**, 100, 137, 138, 151, 155, 164
– beamangle, 11, 36, 51, 94, 95, 98, 99, **99**, 111, 114, 148, 149, 159, 164
– beamdiffractionorder, 54, **54**, 55
– beamdiv, 36, 100, 109, **109**, 111, 113, 132, 138, 140, 148, 149, 173, 174
– beaminside, 11, **101**, 102, 103, 105, 130, 134–136, 148, 175
– beaminsidefirst, 86, 99, 102, **102**, 103, 115, 141, 144, 146, 152, 176
– beaminsidelast, 86–88, 102, **102**, 103, 112, 141, 158, 176
– beammode, 54, **104**
– beamnodealign, 89, **104**, 105, 158
– beampathcount, 101, **101**, 158
– beampathskip, 89, **100**, 101, 158–160
– beampos, 11, 30, 31, 40, 44–48, 87, 94–98, **98**, 99, 102, 104–106, 109, 111, 112, 115, 151, 155, 159, 164
– beamwidth, 11, 15, 18, 36, 42, 47, 50, 52–56, 75, 86, 87, 89, 101, 104, 105, 107, 109, **109**, 110, 113, 114, 130–132, 138, 140, 141, 144, 145, 148, 149, 158, 173, 174
– bssize, **48**, 92, 146, 155, 156
– bsstyle, 18, **48**, 49, 156
– caxisinv, **37**
– caxislength, 37, **37**
– compname, 18, 79, **79**, 91–93, 97, 101, 104, 116, 120, 121, 139, 142, 144, 145, 148, 152–154, 156, 158, 161, 166

– compoffset, 24, **24**, 156
– compshift, 17, **23**, 24, 80, 90, 138, 142, 143, 148, 154, 159
– conn, 172, 175
– connjoin, 172
– coupleralign, 65, **65**, 66, 67, 81, 121, 127, 131, 143, 154
– couplersep, 64, **64**, 65, 71, 81, 161
– couplersize, 64, **64**, 65, 71, 81, 161
– couplertype, 64, 65, **65**, 71, 81, 147, 150
– crystalheight, 37, **37**
– crystalsize, 37, **37**, 146
– crystalwidth, 37, **37**
– detsize, **77**, 89, 150, 152, 156–158
– dettype, 13, 18, 77, **77**, 139, 142
– diffractionorders, **53**, 54, 55
– doveprismsize, 39, **39**, 94, 95
– eleccouplerinput, **70**, 71
– eleccouplersep, **70**
– eleccouplersize, 70, **70**
– eleccouplertype, 70, **70**, 71
– elecmixersize, **73**
– endbox, **23**, 174
– extnode, 44, 82, **82**, 83, 85, 90, 133, 135, 136, 142, 144, 148, 153, 154, 157, 167, 174
– extnodealign, 82, **82**, 83, 144, 173
– extnodes, **83**, 84
– fade, 108
– fadefunc, 108, **108**
– fadefuncname, 108, **108**, 160
– fadepoints, 107, 108, **108**
– fadeto, 107, 108, **108**, 160
– fdlsize, **62**, 112
– fiber, 15–17, 25, 34, 35, 66, 69, 75, 112, 118, 120, 121, 123, **123**, 124, 127, 138, 142, 147, 148, 150, 153, 154, 157, 161
– fiber*, 121
– fiberalign, **117**, 118, 122
– fiberangleA, 118, **118**, 119, 122
– fiberangleB, 118, **118**, 119, 122
– fiberboxcount, 68, 69, **69**
– fiberboxheight, 69, **69**

– fiberboxsepin, 68, 69, **69**
– fiberboxsepout, 68, **69**
– fiberboxsize, 69, **69**
– fiberboxwidth, 69, **69**
– fibercolsize, **76**
– fiberloopradius, 59, **59**, 138
– fiberloops, 59, **59**, 143, 147, 157
– fiberloopsep, 59, **59**, 172
– fiberpolsize, **62**
– fiberstyle, 12, 71, **120**, 121, 122, 147
– fillstyle, 11
– filterangle, 75, **75**
– filtersize, **74**
– filtertype, 74, **74**, 75
– forcebeaminside, 103, **103**
– glanthompsongap, **39**, 40, 153
– glanthompsonheight, 39, **39**
– glanthompsonsize, **39**, 40, 153
– glanthompsonwidth, 39, **39**
– gratingalign, 50, **50**, 159
– gratingcount, 49, **49**
– gratingdepth, 49, **49**
– gratingheight, **49**
– gratinglinewidth, **50**
– gratingtype, **49**, 50
– gratingwidth, **49**, 149
– innercompalign, 24, **24**, 75
– innerheight, **32**, 98, 148
– innerlabel, **21**, 23, 82, 83, 121, 146, 155
– isolatorsize, **60**
– label, 15–17, 19–21, **21**, 37, 38, 42, 59, 69, 83, 120, 129–131, 133–136, 138–143, 145–148, 150–157
– labelalign, **19**, 20, 21, 134–136, 141
– labelangle, 10, 20, **20**, 21, 24, 134, 138, 140, 141, 146, 147, 155
– labeloffset, 10, 13–15, 17, 19, **19**, 20, 21, 43, 50, 65, 81, 92, 129, 139, 141–143, 146, 148, 151, 155, 158, 166
– labelref, 10, 20, **20**, 21, 82
– labelstyle, 16, 17, **19**, 146, 147, 157
– lamp, **37**

– lens, 9–11, 14, 15, 18, 22–24, 26–28, **28**, 31, 36, 56, 86, 89, 96, 97, 99, 101, 104–106, 111–113, 115, 132, 138, 140, 141, 144–146, 148, 151, 152, 156, 158, 165
– lensheight, 11, 14, 15, 27, **27**, 146
– lensradius, 11, 13–15, 28, **28**, 86, 141
– lensradiusleft, **27**, 28
– lensradiusright, **27**, 28
– lenswidth, 28, **28**, 29
– linestyle, 11
– loadbeam, 87, 88, 104, 114, **114**
– loadbeampoints, **112**, 113, 132, 138, 140, 141, 145, 159, 173
– mirrordepth, 18, 44, 45, **45**, 48, 83
– mirrorlinewidth, **43**
– mirrorradius, 43, **43**, 44, 98, 149, 155
– mirrortype, 18, 25, 43, 44, **44**, 45, 46, 48, 83, 87, 98, 99, 107, 109, 114, 141, 146, 149, 150, 152, 155, 159
– mirrorwidth, **43**, 87, 107, 110, 146, 149, 151, 152, 156, 159
– n, 10, 11, 39, 86, 88, 89, 94, 95, **95**, 96–98, 104, 106, 111, 112, 115, 132, 137, 139, 141, 144, 145, 148, 152, 156, 158–160
– namingscheme, 174
– newBeam, **106**
– newFiber, **120**, 138
– newFiberIn, 127
– newFiberIn1, 127
– newFiberIn2, 127
– newFiberOut, 127
– newFiberOut1, 127
– newFiberOut2, 127
– newOptComp, 25, 26, **26**
– newWire, **122**
– newWireIn, 127
– newWireIn1, 127
– newWireIn2, 127
– newWireOut, 127
– newWireOut1, 127
– newWireOut2, 127
– npos, 22
– oapmirroraperture, **47**, 48
– optampsize, **59**

– optboxheight, 33, **33**
– optboxsize, 11, 12, 20–22, **33**, 69, 79, 82, 83, 88, 92, 121, 138, 139, 141, 142, 146, 148, 150, 152–157, 166
– optboxwidth, 13, 16, 17, 20, 33, **33**, 102, 103, 138–140, 143, 145–147, 150, 151, 153, 158
– optcircangle, 63, **63**, 154
– optcircangleA, **63**
– optcircangleB, **63**
– optcircsize, **63**
– optdiodesize, **38**
– optdipolecomp, 130, **130**
– optdipolesize, **129**, 130, 145, 155
– optional, 26, **26**
– optmzmsize, **60**, 153
– opttripolecomp, **130**, 131
– outerheight, **32**, 146, 149, 156
– parmirrorheight, 46, **46**, 47
– parmirrorwidth, 46, **46**, 47
– pentaprismsize, **57**, 84
– phlinewidth, **32**, 33
– phwidth, **32**, 33, 98, 140, 141, 148, 149
– plateheight, **31**, 32, 98–100, 109, 111, 115, 132, 146, 148, 155, 173
– platelinewidth, **31**, 140
– platesize, **32**
– platewidth, 32, **32**
– polcontrolsize, **60**
– polcontroltype, 60, **60**
– polsize, **40**, 140, 153
– poltype, 41, **41**, 101, 140, 153
– position, 10–17, 20–22, **22**, 23, 36, 37, 63, 64, 66, 71, 76, 77, 86, 87, 92, 98–100, 102, 103, 109, 111, 112, 115, 117, 121, 123–125, 127, 133–136, 138–143, 145–151, 153–158, 161, 173, 174
– prismalign, **56**, 97, 160, 171
– prismangle, 55, **55**, 97, 101, 104, 158, 159
– prismsize, **55**, 94
– prismtype, **55**, 56, 101, 104, 158, 171, 172
– pswarning, 54, **112**
– raprismalign, **57**
– raprismsize, **57**

– raytrace, 39, **93**, 94, 95, 106, 107, 138
– refractiveindex, **98**
– reverse, **50**, 51, 98
– rotateref, 23, **23**, 85, 90, 100, 133, 135, 139
– savebeam, 87–89, **113**, 114, 115, 158
– savebeampoints, 85, 86, **112**, 113, 138, 141, 149
– showifcnodes, 68, 100, 130, 165, **165**
– showinterfaces, 166, **166**
– showoptdots, 57, **78**
– sourceheight, 36, **36**
– sourcesize, **36**
– sourcewidth, 36, **36**
– startinside, 99, 103, 115, **115**, 138
– startinsidecount, 100, **103**, 104, 158
– startnode, 119, **119**, 120, 147, 150
– stopinside, 87, 88, 103, 113, 115, **115**, 132, 138, 140
– stopinsidecount, 46, **103**, 104
– stopnode, 119, **119**, 121, 147, 150
– switchsize, **61**
– switchstyle, 61, **61**
– synthshape, 72, **72**, 154
– synthsize, **71**
– synthtype, **71**, 72, 154
– thicklens, **28**
– usefiberstyle, 58, **58**
– useNA, 95, **95**, 130, 152
– usewirestyle, 58, **58**
– variable, 18, 45, **45**, 65, 149, 155
– voltage, **37**
– wedgeangleleft, **41**
– wedgeangleright, **41**
– wedgeangles, **41**, 42
– wedgeheight, **41**
– wedgewidth, 42, **42**
– wire, 33, 34, **124**, 125, 127, 142, 153, 157
– wire∗, 121
– wirealign, **122**
– wireangleA, **122**, 154
– wireangleB, **122**, 142, 153, 154

– wirestyle, 71, **122**
parmirrorheight, 46, **46**, 47
parmirrorwidth, 46, **46**, 47
\pentaprism, 57, **57**, 84, 102, 145
pentaprismsize, **57**, 84
phlinewidth, **32**, 33
phwidth, **32**, 33, 98, 140, 141, 148, 149
PiezoMirror, 44, 46, **46**
\pinhole, 32, **32**, 33, 98, 140, 141, 145, 146, 148, 149, 151, 156
plateheight, **31**, 32, 98–100, 109, 111, 115, 132, 146, 148, 155, 173
platelinewidth, **31**, 140
platesize, **32**
platewidth, 32, **32**
Polarization, **41**, 153
\polarization, 40, **40**, 41, 89, 101, 140, 146, 153
\polcontrol, 60, **60**, 143, 147, 154
polcontrolsize, **60**
polcontroltype, 60, **60**
polsize, **40**, 140, 153
poltype, 41, **41**, 101, 140, 153
position, 10–17, 20–22, **22**, 23, 36, 37, 63, 64, 66, 71, 76, 77, 86, 87, 92, 98–100, 102, 103, 109, 111, 112, 115, 117, 121, 123–125, 127, 133–136, 138–143, 145–151, 153–158, 161, 173, 174
prismalign, **56**, 97, 160, 171
prismangle, 55, **55**, 97, 101, 104, 158, 159
prismsize, **55**, 94
prismtype, **55**, 56, 101, 104, 158, 171, 172
\psbezier, 76
pspicture, 79, 87
pst-eucl, 6
pst-intersect, 6
pst-node, 6, 22, 121
pst-optexp, 6, 25, 128, 129
pst-plot, 6
pstricks, 6
pstricks-add, 6, 107
pswarning, 54, **112**

R

raprismalign, **57**
raprismsize, **57**
raytrace, 39, **93**, 94, 95, 106, 107, 138
refractiveindex, **98**
reverse, **50**, 51, 98
\rightangleprism, 56, **56**, 57, 172
rotateref, 23, **23**, 85, 90, 100, 133, 135, 139

S

savebeam, 87–89, **113**, 114, 115, 158
savebeampoints, 85, 86, **112**, 113, 138, 141, 149
SemitransMirror, 44, **46**, 150
showifcnodes, 68, 100, 130, 165, **165**
showinterfaces, 166, **166**
showoptdots, 57, **78**
sourceheight, 36, **36**
sourcesize, **36**
sourcewidth, 36, **36**
startinside, 99, 103, 115, **115**, 138
startinsidecount, 100, **103**, 104, 158
startnode, 119, **119**, 120, 147, 150
Stile
- ArrowCompStyle, 34, **34**, 62
- BarCompStyle, 35, **35**
- Beam, 15, 18, 40, 46, 47, 51–55, 75, 86, 87, 89, 96–98, 100, 104–106, **106**, 107, 109, 110, 113, 114, 132, 138–141, 144–146, 148–152, 156, 158–160, 164, 166, 173, 174
- CrystalCaxis, 38, **38**
- CrystalLamp, **38**
- DetectorStyle, 77, **77**
- ExtendedMirror, 25, 44, 45, **45**, 83, 141, 149
- FdlArrow, **62**
- Fiber, 58, 62–64, 71, 117, 118, 120, **120**, 121, 126, 142, 157, 161
- FiberIn, 126, 161
- FiberIn1, 126
- FiberIn2, 126
- FiberOut, 126, 161
- FiberOut1, 62, 126
- FiberOut2, 62, 126

- FilterStyle, 75, **75**
- IfcNodeStyle, **165**
- IfcStyle, 166, **166**
- IsolatorArrow, 61, **61**
- OptCircArrow, 63, **63**, 154
- OptComp, 25, **25**, 26, 75, 77, 149
- OptionalStyle, 26, **26**
- PiezoMirror, 44, 46, **46**
- Polarization, **41**, 153
- SemitransMirror, 44, **46**, 150
- SynthStyle, 72, **72**
- VariableCoupler, **66**
- VariableStyle, **26**, 45
- Wire, 58, 71, 73, 122, **122**, 126, 142, 153, 154, 157
- WireIn, 126
- WireIn1, 126
- WireIn2, 126
- WireOut, 126
- WireOut1, 73, 126
- WireOut2, 73, 126

stopinside, 87, 88, 103, 113, 115, **115**, 132, 138, 140
stopinsidecount, 46, **103**, 104
stopnode, 119, **119**, 121, 147, 150
switchsize, **61**
switchstyle, 61, **61**
synthshape, 72, **72**, 154
synthsize, **71**
SynthStyle, 72, **72**
synthtype, **71**, 72, 154

T

thicklens, **28**
\transmissiongrating, 51, **51**

U

Umgebungen
- optexp, 11, 18, 50, 127, 128, 175
- pspicture, 79, 87

usefiberstyle, 58, **58**

useNA, 95, **95**, 130, 152
usewirestyle, 58, **58**

V
variable, 18, 45, **45**, 65, 149, 155
VariableCoupler, **66**
VariableStyle, **26**, 45
voltage, **37**

W
\wdmcoupler, 64, **64**, 66, 67, 131, 143, 150, 154, 161
\wdmsplitter, 16, 17, 64, **64**, 66, 67, 124, 127, 143, 148, 150, 154, 157
wedgeangleleft, **41**
wedgeangleright, **41**
wedgeangles, **41**, 42
wedgeheight, **41**
wedgewidth, 42, **42**
Wire, 58, 71, 73, 122, **122**, 126, 142, 153, 154, 157
wire, 33, 34, **124**, 125, 127, 142, 153, 157
wire*, 121
wirealign, **122**
wireangleA, **122**, 154
wireangleB, **122**, 142, 153, 154
WireIn, 126
WireIn1, 126
WireIn2, 126
WireOut, 126
WireOut1, 73, 126
WireOut2, 73, 126
wirestyle, 71, **122**

A. Versionsgeschichte

Diese Versionsgeschichte ist eine Liste von Änderungen, die für den Nutzer des Pakets von Bedeutung sind. Änderungen, die eher technischer Natur sind und für den Nutzer des Pakets nicht relevant sind und das Verhalten des Pakets nicht ändern, werden nicht aufgeführt. Wenn ein Eintrag der Versionsgeschichte ein Feature als *improved* oder *extended* bekannt gibt, so bedeutet dies, dass eine Modifikation die Syntax und das Verhalten des Pakets nicht beeinflusst, oder das es für ältere Versionen kompatibel ist. Einträge, die als *deprecated*, *modified*, *renamed*, oder *removed* deklariert sind, verlangen besondere Aufmerksamkeit. Diese bedeuten, dass eine Modifikation Änderungen in existierenden Dokumenten mit sich ziehen kann. Die Zahlen an der rechten Seite stehen für die relevante Stelle dieser Dokumentation, zusätzliche Informationen zur Migration von älteren Dokumenten sind in Kap. 12.3 zu finden.

6.1 2022-02-06

Fixed bug in `\oapmirror`

6.0 2021-02-26

Modified `\wdmcoupler` to handle an arbitrary number of input nodes 5.10
Modified `\wdmsplitter` to handle an arbitrary number of output nodes . . . 5.10
Removed deprecated `lampscale`, use style `CrystalLamp`.
Removed deprecated `pollinewidth`, use style `Polarization`.
Removed deprecated `\optgrid`, use `\optgrating`.
Removed deprecated `optgridwidth`, use `gratingwidth`.
Removed deprecated `optgridheight`, use `gratingheight`.
Removed deprecated `optgriddepth`, use `gratingdepth`.
Removed deprecated `optgridcount`, use `gratingcount`.
Removed deprecated `optgridtype`, use `gratingtype`.
Removed deprecated `optgridlinewidth`, use `gratinglinewidth`.
Removed deprecated `align`, use `coupleralign`.
Removed deprecated `namingschema`.

5.2 2014-11-26

Added component `\asphericlens` 4.2
Added component `\axicon` 4.16
Added component `\optwedge` 4.15

5.1 2014-11-19

Added component `\oapmirror` 4.19
Added support for filled beams with path interfaces.

5.0 2014-10-30

Added support for arbitrary interface shapes (experimental, requires the `pst-intersect` package and `pstricks` version 2.53).
Added component `\parabolicmirror` 4.18

4.10 2014-06-04

Added value `firstcomp` to parameter `beamalign` 9.2.3
Modified `\optsource` to handle also the beam alignment 4.9

4.9 2014-06-03

Added new component `\optsource` 4.9
Improved naming of the last component name, which can now be accessed also by its number besides using `N`.
Fixed bug related to `extnodes` together with `compname`

4.8 2014-02-28

Added component `\optaom` 4.23
Added option `beammode` 9.2.4
Added option `showinterfaces` 12.1

4.7 2014-01-12

Added syntax `<`⟨*num*⟩ for accessing relative component IDs 8.1

4.6 2013-11-27

Added option `extnodes` 8.5
Added option `optdipolecomp` 10
Added option `opttripolecomp` 10

4.5 2013-07-21

Added option `usewirestyle` 5
Added line style `fade`, with options `fadeto`, `fadepoints`, `fadefunc`, and `fadefuncname` . . . . . 9.2.7 .

4.4 2013-07-14

Added component `\transmissiongrating` 4.22
Modified `innerlabel` to explicitly set `labelalign=c`
Fixed trailing spaces in `savebeampoints` (and `savebeam`).
Allow Postscript code in `angle`.

4.3a 2013-07-02

Fixed tripole alignment bugs introduced in version 4.3.
Again fixed the alignment of reflective `\optprism` 4.24
Fixed improper alignment of `\optprism` label 12.3.1

4.3 2013-06-28

Added option `raprismalign` 4.25
Fixed alignment of `\rightangleprism` and rotation of its label.

4.2 2013-06-07

Added option `startinsidecount` 9.2.4
Added option `stopinsidecount` 9.2.4
Added option `beampathcount` 9.2.3
Added option `beamnodealign` 9.2.5
Added `\oeBeamVecMedian` 8.9
Added `\oeBeamCenter` 8.8
Modified the orientation of a reflective `\optprism` 12.3.3
Fixed bug of `\optprism` and `\rightangleprism` inside an `optexp` environment.
Fixed bug of wrong `\rightangleprism` rotation in special cases.
Fixed buggy handling of multiple saved beams 9.5

4.1 2013-05-21

Added option `prismtype` 4.24

4.0 2013-04-15

Added electrical components 6
Added wire connections with `\drawwire` 9.7
Added component `\optarrowcomp` 4.7
Added component `\optbarcomp` 4.8
Improved `\optdetector` 7.3
Extended `\drawfiber` to support component ranges 9.6
Extended `couplersep` to support automatic calculation 5.10
Fixed appearance of `\optfiber` 12.3.4
Fixed `extnodealign` to affect also multipoles 8.5
Fixed `pswarning`.
Removed old, buggy beam connection code 12.3.4
Removed option `conn` 12.3.4
Removed option `connjoin` 12.3.4
Removed `\newOptexpDipoleNolabel` 12.3.4

3.6 2013-03-20

Added option `compoffset` 3.3
Added option `beampathskip` 9.2.3
Added option `innercompalign` 3.3
Fixed some bugs in the raytracing and beam code.

3.5 2013-02-22

Added option `filterangle` 7.1
Fixed wrong output fiber style in `\optcoupler`
Fixed strange Postscript error which occured with some interpreters.

3.4 2013-02-03

Fixed bugs when using `fillstyle` for some components.
Extended option `switchsize` 5.6
Modified coupler center node to be on the base line 8.3
Extended error checking for `\drawbeam` and `\drawfiber` 9.1
Fixed some bugs in the connection code.

3.3a 2012-09-18

Fixed bug which was exposed by an update of pst-node.
Fixed trailing spaces.

3.3 2012-08-17

Extended `\opttripole` and `\optdipole` 10
Modified reference angle for label of `\opttripole` 12.3.5
Added option `optdipolesize` to support two interfaces for `\optdipole` . . . 10
Added option `gratingalign` 4.21
Added option `forcebeaminside` 9.2.4
Extended option `mirrorradius` 4.17
Improved `\drawbeam` and `\drawwidebeam` to be able to use `ArrowInside` without linestyle 9.2.7
Fixed orientation of `\oenodeBeamUp` and `\oenodeBeamLow` 8.8
Fixed a bug related to `beaminside` and ambiguous components.
Fixed some trailing spaces.
Added more examples 11

3.2 2012-07-26

Added component `\glanthompson` 4.13
Added access to beam vectors with `\oeBeamVec`, `\oeBeamVecUp` and `\oeBeamVecLow` 8.9
Fixed wrong computation of node distance in `\fiberbox`
Fixed bug in `\wdmsplitter` for `coupleralign=b` and `couplertype=none`
Fixed bug when filling components placed with `position=end/start`
Added value `absolute` to parameter `labelref` 3.1
Added examples 11

3.1 2012-07-17

Added component `\fiberbox` 5.11
Extended connection macros to not required curly braces around node parenthesis any more 8.1
Modified `\fibercollimator` to have `allowbeaminside=false` by default . . 7.2
Extended fiber couplers to allow using only two nodes 5.10
Fixed a bug concerning node expressions with `\drawfiber`

3.0 2012-07-09

Modified beam connections with `\drawbeam` to support raytracing 9.2
Added wide beams with `\drawwidebeam` 9.3
Added `\drawfiber` for fiber connections 9.6
Added `optexp` environment for layering of components and connections . . 9.9
Added german documentation.
Modified naming of component nodes 8
Modified `extnode` to work with more components 8.5
Modified `angle` and `rotateref` to affect all components 3.3
Modified `endbox` to affect all components 3.2
Extended `position` by values `start` and `end` 3.2
Extended `abspos` by values `start` and `end` 3.2
Removed deprecated lens code which used `lenswidth` and `lensheight` for construction 4.1
Added option `platesize` 4.4
Added option `phwidth` 4.5
Modified option `caxislength` 4.10
Deprecated option `lampscale` 4.10
Added style `CrystalCaxis` 4.10
Added style `CrystalLamp` 4.10
Added option `optboxsize` 4.6
Extended option `detsize` 7.3
Added style `DetectorStyle` 7.3
Removed deprecated `\detector`, use `\optdetector` 7.3
Extended option `doveprismsize` 4.12
Added style `Polarization` 4.14
Deprecated option `pollinewidth` 4.14
Removed option `polwidth` 4.14
Removed option `pol` 4.14
Added style `VariableStyle` 4.17
Added mirror type `semitrans` 4.17
Renamed `\optgrid` to `\optgrating` 4.21
Renamed `optgridwidth` to `gratingwidth` 4.21
Renamed `optgridheight` to `gratingheight` 4.21
Renamed `optgriddepth` to `gratingdepth` 4.21
Renamed `optgridcount` to `gratingcount` 4.21
Renamed `optgridtype` to `gratingtype` 4.21
Renamed `optgridlinewidth` to `gratinglinewidth` 4.21

Added option `prismalign` 4.24
Modified faulty alignment of `\rightangleprism` 4.25
Extended option `optampsize` 5.2
Extended option `optmzmsize` 5.3
Added option `polcontroltype` 5.4
Extended option `isolatorsize` 5.5
Added style `IsolatorArrow` 5.5
Extended option `fdlsize` 5.7
Added style `FdlArrow` 5.7
Extended option `fiberpolsize` 5.8
Added component `\optcirculator` 5.9
Extended option `couplersize` 5.10
Extended option `couplertype` 5.10
Renamed option `align` to `coupleralign` 5.10
Added style `VariableCoupler` 5.10
Added style `FilterStyle` 7.1
Extended option `fibercolsize` 7.2
Removed deprecated option `labelrelative` 3.1
Removed deprecated option `iwidth` 4.5
Removed deprecated option `owidth` 4.5
Removed deprecated option `bswidth` 4.20
Deprecated `\newOptexpDipoleNolabel`, use `\newOptexpDipole` 10.2
Deprecated option `refractiveindex` 9.2.2
Deprecated option `conn` 9.2
Extended option `fiber` 9.8

2.1 2009-11-05

Added component `\optfiberpolarizer` 5.8
Added option `compshift` 3.3
Added option `label` 3.1
Added option `connjoin`
Added options `addtoBeam` and `newBeam` 9.2
Added style `OptComp` and related options `addtoOptComp` and `newOptComp` . . 3.5
Added option `bsstyle` 4.20
Extended `\fibercollimator` to use up to four reference nodes 7.2
Improved `thicklens` to work also with plain lenses 4.1
Use pst-doc class for the documentation.

2.0 2008-07-27

Added fiber-optical components 5
Added component `\optdiode` 4.11
Added component `\pentaprism` 4.26
Added component `\rightangleprism` 4.25
Added component `\doveprism` 4.12
Added component `\optprism` 4.24
Added `\drawbeam` 9.2
Added component connections (options `fiber`, `conn` and `beam`) 9
Added option `compname` 8.1
Added option `extnode` 8.5
Renamed `\detector` to `\optdetector` 7.3

1.2 2008-06-17

Modified lens design to use interface curvatures 4.1
Added options `lensradiusleft` and `lensradiusright` 4.1
Added option `thicklens` 4.1
Added option `lenstype` 4.1
Added option `mirrorradius` (curved mirrors) 4.17
Added option `optgridtype` (binary gratings) 4.21
Added `\newOptexpDipole` 10.2
Added `\newOptexpDipoleNolabel` 10.2
Added `\newOptexpTripole` 10.2
Added `\newOptexpFiberDipole` 10.2
General improvements of TEX and Postscript code

1.1 2007-09-06

Improved labeling features 3.1
Added parameter `labelref` 3.1
Replaced `labelrelative` by `labelref=relative` 3.1
Renamed `\polarisation` to `\polarization` 4.14
Renamed `polwidth` to `polsize` 4.14
Renamed `pol` to `poltype` 4.14
Renamed `bswidth` to `bssize` 4.20
Renamed `iwidth` to `innerheight` 4.5
Renamed `owidth` to `outerheight` 4.5
Added support for fillstyle for all components

1.0 2007-07-18

First CTAN version